The Beginner's Textbook for Fully Homomorphic Encryption

Ronny Ko *
LG Electronics Inc.

* *Acknowledgments:*
Robin Geelen (KU Leuven)
Tianjian Yang (Peking University)
Yongwoo Lee (Inha University)
Nolan Carouge (Grenoble INP Ensimag)
Navid Abapour (University of Surrey)

Preface

Fully Homomorphic Encryption (FHE) is a cryptographic scheme that enables computations to be performed directly on encrypted data, as if the data were in plaintext. After all computations are performed on the encrypted data, it can be decrypted to reveal the result. Correct decryption requires the selected parameters and evaluated circuit to satisfy the scheme's noise and failure-probability bounds. Exact schemes compute in their specified finite plaintext ring; CKKS returns an approximation whose error must be tracked, rather than an exact real-number result.

FHE supports basic operations such as addition and multiplication on encrypted numbers. Using these fundamental operations, more complex computations can be constructed, including subtraction, division, logic gates (e.g., AND, OR, XOR, NAND, MUX), and even advanced mathematical functions such as ReLU, sigmoid, and trigonometric functions (e.g., sin, cos). The interpretation and implementation depend on the plaintext domain and allowed input range. For example, division in a finite ring requires an invertible denominator, while real-valued reciprocal and activation functions typically use a bounded-domain approximation or a suitable discretized circuit.

FHE enables privacy-preserving machine learning by allowing a server to process the client's data in its encrypted form through an ML model. Under the security assumptions and a correctly designed protocol, an evaluator without the decryption key cannot recover the encrypted inputs or outputs from the ciphertexts. Protocol metadata, deliberately released outputs, and access patterns require separate analysis. FHE can also help keep smart-contract data confidential. FHE alone does not prove that an evaluator executed the requested computation correctly; integrity and verifiability require additional protocol mechanisms. Other applications of FHE include secure outsourcing of data analytics, encrypted database queries, privacy-preserving searches, efficient multi-party computation for digital signatures, and more.

This book is designed to help the reader understand how FHE works at the mathematical level. The book comprises the following four parts:

- **Part I: Basic Math** explains necessary background concepts for FHE, such as groups, fields, orders, polynomial rings, cyclotomic polynomials, vectors and matrices, the Chinese Remainder Theorem, Taylor series, polynomial interpolation, and the Fast Fourier Transform.
- **Part II: Post-quantum Cryptography** explains well-known lattice-based cryptographic schemes, which include LWE, RLWE, GLWE, GLev, and GGSW cryptosystems.
- **Part III: Generic Fully Homomorphic Encryption** explains the generic techniques of FHE adopted by many existing schemes, such as homomorphic addition, multiplication, modulus switching, and key switching.
- **Part IV: Fully Homomorphic Encryption Schemes** explains four widely used FHE schemes: TFHE, BFV, CKKS, and BGV, as well as their RNS-variant versions.

This book is available both as an **arXiv PDF** and on an **auto-generated dynamic website** (powered by make4ht). We provide a Python Demo FHE library (TFHE, BFV, BGV, CKKS) for educational purposes, which is available at `https://github.com/fhetextbook/fhetextbook.github.io`. This bilingual fork includes a documented academic review in `audit/ACADEMIC_REVIEW.md`; confirmed corrections have been applied to both editions. The review records its scope and remaining limitations. Please report any errors regarding the book draft on the **Issues Board**.

Acknowledgments

Special thanks go to the following researchers: Robin Geelen (KU Leuven, robin.geelen@esat.kuleuven.be), for his thoughtful and dedicated feedback; Yongwoo Lee (Inha University, yongwoo@inha.ac.kr), for his general advice; Tianjian Yang (Peking University, 2300012738@stu.pku.edu.cn), for correcting numerous typos and other errors throughout the draft; Nolan Carouge (Grenoble INP Ensimag, nolan.carouge.pro@gmail.com), for proof-reading Part 1; and Navid Abapour (University of Surrey, n.abapour@surrey.ac.uk), for proof-reading Part 3 and 4.

Contents

I Basic Math **10**

A-1 Modulo Arithmetic **11**
A-1.1 Overview . . . 11
A-1.2 Modulo Arithmetic . . . 11
A-1.3 Inverse . . . 13
A-1.4 Modulo Division . . . 13
A-1.5 Centered Residue Representation . . . 13

A-2 Group **15**
A-2.1 Definitions . . . 15
A-2.2 Examples . . . 15

A-3 Field **17**
A-3.1 Definitions . . . 17
A-3.2 Examples . . . 17
A-3.3 Theorems . . . 18

A-4 Order **19**
A-4.1 Definitions . . . 19
A-4.2 Theorems . . . 19

A-5 Polynomial Ring **21**
A-5.1 Overview . . . 21
A-5.1.1 Example . . . 22
A-5.1.2 Discussion . . . 22
A-5.2 Coefficient Rotation . . . 23
A-5.2.1 Example . . . 24

A-6 Decomposition **26**
A-6.1 Number Decomposition . . . 26
A-6.1.1 Example . . . 26
A-6.2 Polynomial Decomposition . . . 26
A-6.2.1 Example . . . 27
A-6.2.2 Discussion . . . 27
A-6.3 Approximate Decomposition . . . 28
A-6.4 Gadget Decomposition . . . 28

A-7 Roots of Unity **29**
A-7.1 Definitions . . . 29
A-7.2 Theorems . . . 29

A-8 Cyclotomic Polynomial **32**
A-8.1 Definitions . . . 32

A-8.2 Theorems . . . 32

A-9 Roots of Unity and Cyclotomic Polynomial over Ring 34

A-10 Lagrange's Polynomial Interpolation 36
A-10.1 The Field Condition for Unique Interpolation . . . 37

A-11 Vector and Matrix 38
A-11.1 Vector Arithmetic . . . 38
A-11.2 Various Types of Matrix . . . 39
A-11.3 Matrix Arithmetic . . . 39
A-11.4 Projection . . . 41
A-11.5 Basis of a Polynomial Ring . . . 42
A-11.6 Isomorphism between Polynomials and Vectors over Integers (and Ring) . . . 42
A-11.6.1 Finding Appropriate Modulus t . . . 45
A-11.7 Isomorphism between Polynomials and Vectors over Complex Numbers . . . 47
A-11.7.1 Isomorphism between $\mathbb{C}^{\frac{n}{2}}$ and $\hat{\mathbb{C}}^n$ . . . 47
A-11.7.2 Isomorphism between $\hat{\mathbb{C}}^n$ and $\mathbb{R}[X]/(X^n+1)$ . . . 49
A-11.8 Transforming Basis between Polynomial Ring and Vector Space of Complex Vectors with Real Scalars . . . 50

A-12 Euler's Formula 52
A-12.1 Pythagorean Identity . . . 52
A-12.2 Imaginary Number . . . 52
A-12.3 Euler's Formula . . . 53
A-12.4 Vandermonde Matrix with Roots of Cyclotomic Polynomial over Complex Numbers 53
A-12.5 Vandermonde Matrix with Roots of Cyclotomic Polynomial over Ring ($\mathbb{Z}_p$) . . . 56

A-13 Modulo Rescaling 59
A-13.1 Rescaling Modulo of Congruence Relations . . . 59
A-13.1.1 Example . . . 59

A-14 Chinese Remainder Theorem 61
A-14.1 Application: Residue Number System (RNS) . . . 62

A-15 Taylor Series 64

A-16 Efficient Polynomial Multiplication by FFT and NTT 65
A-16.1 Background and Motivation . . . 65
A-16.2 Forward FFT (or NTT) . . . 66
A-16.2.1 High-level Idea . . . 66
A-16.2.2 Details . . . 67
A-16.3 Point-wise Multiplication . . . 68
A-16.4 Inverse FFT (or NTT) . . . 68

II Post-quantum Cryptography 71

B-1 Lattice-based Cryptography **72**
B-1.1 Overview . . . 72
B-1.2 LWE Cryptosystem . . . 74
B-1.3 RLWE Cryptosystem . . . 75

B-2 LWE Cryptosystem **76**
B-2.1 Setup . . . 76
B-2.2 Encryption . . . 76
B-2.3 Decryption . . . 76
B-2.3.1 In the Case of t not Dividing q . . . 77

B-3 RLWE Cryptosystem **80**
B-3.1 Setup . . . 80
B-3.2 Encryption . . . 80
B-3.3 Decryption . . . 81

B-4 GLWE Cryptosystem **82**
B-4.1 Setup . . . 82
B-4.2 Encryption . . . 82
B-4.3 Decryption . . . 83
B-4.3.1 Discussion . . . 83
B-4.4 An Alternative Version of GLWE . . . 84
B-4.5 Public Key Encryption . . . 85

B-5 GLev **88**
B-5.1 Encryption . . . 88
B-5.2 Decryption . . . 88
B-5.3 Lev and RLev . . . 88

B-6 GGSW **89**
B-6.1 Encryption . . . 89
B-6.2 Decryption . . . 89
B-6.3 GSW and RGSW . . . 89

III Generic Fully Homomorphic Encryption **90**

C-1 GLWE Ciphertext-to-Ciphertext Addition **91**
C-1.1 Discussion . . . 92

C-2 GLWE Ciphertext-to-Plaintext Addition **93**

C-3 GLWE Ciphertext-to-Plaintext Multiplication **94**
C-3.1 Gadget Decomposition for Noise Suppression . . . 95
C-3.1.1 Discussion . . . 96

C-4 GLWE Modulus Switching **97**

C-4.1 LWE Modulus Switching . . . 97
C-4.2 Example . . . 99
C-4.3 Discussion . . . 100
C-4.4 RLWE Modulus Switching . . . 100
C-4.5 GLWE Modulus Switching . . . 103

C-5 GLWE Key Switching **105**

IV Fully Homomorphic Encryption Schemes **109**

D-1 TFHE Scheme **110**
D-1.1 Encryption and Decryption . . . 111
D-1.2 Homomorphic Ciphertext-to-Ciphertext Addition . . . 111
D-1.3 Homomorphic Ciphertext-to-Plaintext Addition . . . 111
D-1.4 Homomorphic Ciphertext-to-Plaintext Multiplication . . . 112
D-1.5 Homomorphic Key Switching . . . 112
D-1.6 Homomorphic Ciphertext-to-Ciphertext Multiplication . . . 112
D-1.6.1 Generalization to GLWE-to-GGSW Multiplication . . . 115
D-1.7 Coefficient Extraction . . . 118
D-1.8 Noise Bootstrapping . . . 120
D-1.8.1 Overview . . . 120
D-1.8.2 Modulus Switch for Noise Bootstrapping . . . 121
D-1.8.3 Halving the Plaintext Space To be Used . . . 122
D-1.8.4 Blind Rotation . . . 123
D-1.8.5 Coefficient Extraction . . . 124
D-1.8.6 Key Switching . . . 124
D-1.8.7 Noise Bootstrapping Summary . . . 124
D-1.8.8 Example: Noise Bootstrapping . . . 125
D-1.8.9 Discussion . . . 128
D-1.8.10 Application: Gate Bootstrapping . . . 128
D-1.8.11 Application: Neural Networks Bootstrapping . . . 129
D-1.9 TFHE on a Discrete Torus . . . 130

D-2 BFV Scheme **132**
D-2.1 Single Value Encoding . . . 133
D-2.2 Batch Encoding . . . 134
D-2.2.1 $\mathsf{Encoding}_1$ . . . 134
D-2.2.2 $\mathsf{Encoding}_2$ . . . 135
D-2.2.3 $\mathsf{Decoding}_1$ . . . 135
D-2.2.4 $\mathsf{Decoding}_2$ . . . 136
D-2.2.5 Summary . . . 136
D-2.3 Encryption and Decryption . . . 137
D-2.4 Ciphertext-to-Ciphertext Addition . . . 137
D-2.4.1 Noise Bound Analysis . . . 138
D-2.5 Ciphertext-to-Plaintext Addition . . . 138

D-2.6 Ciphertext-to-Plaintext Multiplication . . . 139
D-2.7 Ciphertext-to-Ciphertext Multiplication . . . 139
D-2.7.1 ModRaise . . . 139
D-2.7.2 Polynomial Multiplication . . . 140
D-2.7.3 Relinearization . . . 140
D-2.7.4 Rescaling . . . 143
D-2.7.5 Summary . . . 143
D-2.7.6 Application to TFHE's Ciphertext-to-Ciphertext Multiplication . . . 145
D-2.8 Homomorphic Key Switching . . . 147
D-2.9 Homomorphic Rotation of Input Vector Slots . . . 147
D-2.9.1 Rotating Input Vector Slots by Updating the Plaintext Polynomial . . . 148
D-2.9.2 Updating the Plaintext Polynomial by Updating the Ciphertext Polynomials 153
D-2.9.3 Summary . . . 153
D-2.9.4 Encoding Example . . . 154
D-2.9.5 Rotation Example . . . 156
D-2.10 Application: Matrix Multiplication . . . 159
D-2.11 Noise Bootstrapping . . . 160
D-2.11.1 High-level Idea . . . 160
D-2.11.2 Modulus Switch . . . 161
D-2.11.3 Homomorphic Decryption . . . 161
D-2.11.4 CoeffToSlot and SlotToCoeff . . . 162
D-2.11.5 Digit Extraction . . . 164
D-2.11.6 Summary . . . 165

D-3 CKKS Scheme **168**
D-3.1 Encoding and Decoding . . . 170
D-3.1.1 Example . . . 173
D-3.2 Encryption and Decryption . . . 175
D-3.3 Ciphertext-to-Ciphertext Addition . . . 176
D-3.4 Ciphertext-to-Plaintext Addition . . . 176
D-3.5 Ciphertext-to-Ciphertext Multiplication . . . 176
D-3.5.1 Synthetic Ciphertext Derivation . . . 176
D-3.5.2 Relinearization Method 1 – Ciphertext Decomposition . . . 177
D-3.5.3 Relinearization Method 2 – Ciphertext Modulus Switch . . . 177
D-3.5.4 Rescaling . . . 178
D-3.5.5 Comparing BFV and CKKS Bootstrapping . . . 178
D-3.5.6 Summary . . . 179
D-3.6 Ciphertext-to-Plaintext Multiplication . . . 179
D-3.7 ModDrop . . . 180
D-3.7.1 Difference between Modulus Switch and ModDrop . . . 180
D-3.8 Homomorphic Key Switching . . . 180
D-3.9 Homomorphic Rotation of Input Vector Slots . . . 180
D-3.9.1 Example . . . 181
D-3.10 Contemplation on CKKS Encoding . . . 182
D-3.11 Homomorphic Conjugation . . . 182
D-3.12 Sparsely Packing Ciphertexts . . . 183

D-3.12.1 Forward Proof: Decoding of Sparsely Packed Ciphertext . . . 183
D-3.13 Modulus Bootstrapping . . . 184
D-3.13.1 High-level Idea . . . 184
D-3.13.2 Mathematical Expansion of the High-level Idea . . . 186
D-3.13.3 Details: CoeffToSlot . . . 186
D-3.13.4 Details: EvalExp . . . 187
D-3.13.5 Details: SlotToCoeff . . . 188
D-3.13.6 Reducing the Bootstrapping Overhead by Sparsely Packing Ciphertext . . . 189
D-3.13.7 Summary . . . 189
D-3.13.8 Reducing the Bootstrapping Noise . . . 190

D-4 BGV Scheme **194**
D-4.1 Encoding and Decoding . . . 195
D-4.2 Encryption and Decryption . . . 196
D-4.3 Ciphertext-to-Ciphertext Addition . . . 196
D-4.4 Ciphertext-to-Plaintext Addition . . . 197
D-4.5 Ciphertext-to-Plaintext Multiplication . . . 197
D-4.6 ModDrop . . . 197
D-4.7 Modulus Switch . . . 198
D-4.7.1 Difference between Modulus Switch and ModDrop . . . 198
D-4.8 Ciphertext-to-Ciphertext Multiplication . . . 199
D-4.9 Homomorphic Key Switching . . . 199
D-4.10 Homomorphic Rotation of Input Vector Slots . . . 199
D-4.11 Modulus Bootstrapping . . . 200
D-4.11.1 Discussion . . . 201
D-4.11.2 Comparing the Bootstrapping in BFV, BGV, and CKKS . . . 202

D-5 RNS-variant FHE Schemes **203**
D-5.1 Fast Base Conversion: FastBConv . . . 204
D-5.2 Small Montgomery Reduction Algorithm: SmallMont . . . 204
D-5.2.1 Improving FastBConv by Using SmallMont . . . 205
D-5.3 RNS-based ModRaise: $\mathsf{ModRaise}_{\mathsf{RNS}}$ . . . 205
D-5.4 RNS-based ModDrop: $\mathsf{ModDrop}_{\mathsf{RNS}}$ . . . 206
D-5.5 RNS-based Modulus Switch: $\mathsf{ModSwitch}_{\mathsf{RNS}}$ . . . 206
D-5.5.1 Comparing $\mathsf{ModSwitch}_{\mathsf{RNS}}$, $\mathsf{ModRaise}_{\mathsf{RNS}}$, and $\mathsf{ModDrop}_{\mathsf{RNS}}$ . . . 207
D-5.6 RNS-based Decryption . . . 207
D-5.6.1 BFV Decryption: $\mathsf{Dec}^{\mathsf{BFV}}_{\mathsf{RNS}}$ . . . 207
D-5.6.2 CKKS and BGV Decryption . . . 208
D-5.7 BGV's RNS-based Modulus Switch: $\mathsf{ModSwitch}^{\mathsf{BGV}}_{\mathsf{RNS}}$ . . . 208
D-5.8 Exact Fast Base Conversion: FastBConvEx . . . 208
D-5.9 Decomposing Multiplication: $\mathsf{DecompMult}_{\mathsf{RNS}}$ . . . 209
D-5.10 Applying RNS Techniques to FHE Operations . . . 210
D-5.10.1 Addition and Multiplication of Polynomials . . . 210
D-5.10.2 Key Switching . . . 210
D-5.10.3 Input Slot Rotation . . . 211
D-5.10.4 BFV's Ciphertext-to-Ciphertext Multiplication . . . 211

D-5.10.5 CKKS's Ciphertext-to-Ciphertext Multiplication . . . 212
D-5.10.6 BGV's Ciphertext-to-Ciphertext Multiplication . . . 212
D-5.10.7 BFV's Bootstrapping . . . 212
D-5.10.8 CKKS's Bootstrapping . . . 213
D-5.10.9 BGV's Bootstrapping . . . 213
D-5.10.10 Noise Impact of RNS Operations . . . 214
D-5.10.11 Python Source Code of RNS Primitives . . . 214

D-6 FHE Scheme Comparison and Summary **215**

D-7 Python Demo FHE Library **220**

Part I
Basic Math

This chapter explains the basic mathematical components of number theory: group, field, order, roots of unity, cyclotomic polynomial, polynomial ring, and decomposition. These are essential building blocks for post-quantum cryptography.

A-1 Modulo Arithmetic

- Reference

YouTube – Extended Euclidean Algorithm Tutorial.

A-1.1 Overview

All moduli in this chapter are integers $q \geq 2$, and all operands are integers unless stated otherwise.

⟨Definition A-1.1⟩ Integer Modulo

- **Modulo** is the operation of computing the remainder obtained when one number is divided by another. **modulo** is often abbreviated as **mod**.
- ***a* mod *q* (i.e., *a* modulo *q*)** is the remainder after dividing a by q, which is always an element of $\{0, 1, 2, 3, \cdots, q-1\}$. For example, $7 \bmod 5 = 2$, because the remainder of dividing 7 by 5 is 2.
- **Modulus:** Given $\boldsymbol{a}$ mod $\boldsymbol{q}$, we call the divisor q the modulus, whereas *modulo* refers to the operation.
- **Modulo Congruence (≡):** a is congruent to b modulo q (i.e., $a \equiv b \bmod q$) if they have the same remainder when divided by q. For example, $5 \equiv 12 \bmod 7$, because $5 \bmod 7 = 5$ and $12 \bmod 7 = 5$. In mathematics, the notation $a \equiv b \bmod q$ is identical to $a = b \pmod q$, meaning that the remainder of a divided by q is the same as the remainder of b divided by q. Note that this notation differs from $a = b \bmod q$, which states that a equals the remainder of b divided by q.
- **Congruence *vs.* Equality:**
  $a \equiv b \bmod q \iff a = b + k \cdot q \quad$ (for some integer k)

  This means that a and b are congruent modulo q if and only if a and b differ by some multiple of q. For example, $5 \equiv 12 \bmod 7 \iff 5 = 12 + (-1) \cdot 7$

A-1.2 Modulo Arithmetic

The supported modulo operations are addition, subtraction, and multiplication. The properties of these modulo operations are as follows:

⟨Theorem A-1.2.1⟩ Properties of Modulo Operations

For any integer x, the following is true:

1. **Addition:** $a \equiv b \bmod q \iff a + x \equiv b + x \bmod q$
2. **Subtraction:** $a \equiv b \bmod q \iff a - x \equiv b - x \bmod q$
3. **Multiplication:** $a \equiv b \bmod q \iff a \cdot x \equiv b \cdot x \bmod q$. This equivalence holds provided that

$\gcd(x, q) = 1$. Without this assumption, only the implication $a \equiv b \bmod q \Rightarrow a \cdot x \equiv b \cdot x \bmod q$ is guaranteed.

Proof.

For any integer x,

1. **Addition:** $a \equiv b \bmod q \iff a = b + kq$ (for some integer k) $\quad \#$ a and b differ by some multiple of q
   $\iff a + x = b + k \cdot q + x$
   $\iff a + x = b + x + k \cdot q \triangleright$ $a + x$ and $b + x$ differ by some multiple of q
   $\iff a + x \equiv b + x \bmod q$

2. **Subtraction:** $a \equiv b \bmod q \iff a = b + kq$ (for some integer k)
   $\iff a - x = b + k \cdot q - x$
   $\iff a - x = b - x + k \cdot q \triangleright$ $a - x$ and $b - x$ differ by some multiple of q
   $\iff a - x \equiv b - x \bmod q$

3. **Multiplication:** $a \equiv b \bmod q \iff a = b + kq$ (for some integer k)
   $\implies a \cdot x = b \cdot x + k \cdot q \cdot x$
   $\implies a \cdot x = b \cdot x + k_x \cdot q$ (where $k_x = k \cdot x$) $\triangleright$ $a \cdot x$ and $b \cdot x$ differ by some multiple of q
   $\implies a \cdot x \equiv b \cdot x \bmod q$
   Conversely, if x and q are coprime (i.e., $\gcd(x, q) = 1$), then x has a multiplicative inverse x^{-1} modulo q. From $a \cdot x \equiv b \cdot x \bmod q$
   $\implies a \cdot x \cdot x^{-1} \equiv b \cdot x \cdot x^{-1} \bmod q$
   $\implies a \equiv b \bmod q$

□

Based on the modulo operations in Theorem A-1.2.1, we can also derive the following properties of modulo arithmetic:

⟨Theorem A-1.2.2⟩ Properties of Modulo Arithmetic

1. **Associative:** $(a \cdot b) \cdot c \equiv a \cdot (b \cdot c) \bmod q$
2. **Commutative:** $(a \cdot b) \equiv (b \cdot a) \bmod q$
3. **Distributive:** $(a \cdot (b + c)) \equiv ((a \cdot b) + (a \cdot c)) \bmod q$
4. **Interchangeable:** Congruent values are interchangeable in modulo arithmetic.
   For example, suppose $(a \equiv b \bmod q)$ and $(c \equiv d \bmod q)$. Then, a and b are interchangeable, and c and d are interchangeable in modulo arithmetic as follows:
   $(a + c) \equiv (b + d) \equiv (a + d) \equiv (b + c) \bmod q$
   $(a - c) \equiv (b - d) \equiv (a - d) \equiv (b - c) \bmod q$
   $(a \cdot c) \equiv (b \cdot d) \equiv (a \cdot d) \equiv (b \cdot c) \bmod q$

The proof of Theorem A-1.2.2 is similar to that of Theorem A-1.2.1, which we leave as an exercise for the reader.

A-1.3 Inverse

⟨Definition A-1.3⟩ Inverse in Modulo Arithmetic

In modulo q (i.e., in the world of remainders where all numbers have been divided by q), for each $a \in \{0, 1, 2, \cdots, q-1\}$:

- **Additive Inverse** of a is denoted as a_+^{-1} which satisfies $a + a_+^{-1} \equiv 0 \bmod q$. For example, in modulo 11, $3_+^{-1} = 8$, because $3 + 8 \equiv 0 \bmod 11$.
- **Multiplicative Inverse** of a is denoted as a_*^{-1} which satisfies $a \cdot a_*^{-1} \equiv 1 \bmod q$. Such an inverse exists if and only if $\gcd(a, q) = 1$. For example, modulo 11, $3_*^{-1} = 4$, because $3 \cdot 4 \equiv 1 \bmod 11$.

A-1.4 Modulo Division

In modulo arithmetic, *modulo division* is different from regular numeric division. Strictly speaking, there is no separate operation called "modulo division", because the modulo operation already returns only the remainder of a division. In practice, one uses "modulo division" to mean multiplying by a modular inverse when it exists, i.e., when $\gcd(a, q) = 1$. *Modulo division* of b by a modulo q is equivalent to computing the *modular* multiplication $b \cdot a^{-1} \bmod q$. The result of *modulo division* is different from that of numeric division because *modulo division* always gives an integer (a residue modulo q) (as it multiplies two integers modulo q), whereas numeric division gives a real number. The inverse of an integer modulo q can be computed using the extended Euclidean algorithm (YouTube tutorial).

A-1.5 Centered Residue Representation

Throughout this section, we have assumed that the residues are positive integers. For example, the possible residues modulo q are assumed to be $\{0, 1, \cdots, q-1\}$. This system is called the canonical (i.e., unsigned) residue representation. On the other hand, there is also a counterpart system that assumes signed (i.e., centered) residues $\left\{-\frac{q}{2}, -\frac{q}{2}+1, \cdots, 0, \cdots, \frac{q}{2}-2, \frac{q}{2}-1\right\}$[1], with the residues centered around 0, and the total number of residues is the same, namely q. In both systems, a modulo operation changes a given value to another value within the system's residue range such that: (1) if the given value is greater than the upper bound of the residue range, one subtracts a suitable integer multiple of q; (2) if the value is less than the lower bound, one adds a suitable integer multiple of q. The only difference between these two (canonical and centered) systems is their upper bounds and lower bounds: 0 and $q-1$ in the canonical residue system, whereas $-\frac{q}{2}$ and $\frac{q}{2}-1$ in the centered residue system. The canonical residue representation assumes that $\mathbb{Z}_q = \{0, 1, \cdots, q-1\}$, whereas the centered residue system assumes that $\mathbb{Z}_q = \left\{-\frac{q}{2}, -\frac{q}{2}+1, \cdots, 0, \cdots, \frac{q}{2}-2, \frac{q}{2}-1\right\}$.

In both systems, the same properties hold for addition, subtraction, multiplication, and division (when the divisor is invertible). This can be proved using the reasoning from §A-1.2: any two congruent residues differ by an integer multiple of q in either representation.

Also, the same property holds for an inverse: an inverse of a modulo q is a^{-1} such that $a \cdot a^{-1} \equiv 1 \bmod q$.

[1] Here, we assume q is an even number. In the case where q is an odd number, the residues are $\left\{-\frac{q-1}{2}, -\frac{q-3}{2}, \cdots, 0, \cdots, \frac{q-3}{2}, \frac{q-1}{2}\right\}$

Using a signed residue representation is useful in certain cases. In an example of canonical (i.e., unsigned) residue representation, suppose we have the relation $a + b \bmod q$ and we know that in a given application, $a + b$ is guaranteed to be within the $[0, q-1]$ range (i.e., $0 \leq a + b \leq q - 1$). Then, $(a + b \bmod q) = a + b$, and thus we can remove the modulo operation, simplifying the relation. Now, suppose a different example of centered (i.e., signed) residue representation where we have the relation $a - b \bmod q$, and we know that in a given application, $a - b$ is guaranteed to be within the range $\left[-\frac{q}{2}, \frac{q}{2} - 1\right]$. Then, $(a - b \bmod q) = a - b$. However, notice that if the relation $a - b \bmod q$ were in a canonical residue representation, then we cannot remove the modulo operation because if $a - b$ is negative, then this becomes smaller than the lower bound of the canonical residue system (i.e., 0), and thus a modulo reduction (i.e., addition by one or more q) is needed.

In §D-5.8, we design the FastBConvEx operation based on this beneficial property of centered residue representation: in this algorithm design, we can simplify $(\mu + u \bmod b_\alpha)$ to $\mu + u$ because we know that $-\frac{b_\alpha}{2} \leq \mu + u < \frac{b_\alpha}{2}$.

A-2 Group

A-2.1 Definitions

⟨Definition A-2.1⟩ Group

Set Elements

- **Set ($\mathbb{S}$):** An unordered collection of elements: $\mathbb{S} = \{a, b, c, \ldots\}$
- **Set Operations ($+, \cdot$):** We consider two binary operations on $\mathbb{S}$: addition ($+$) and multiplication ($\cdot$).
- **Additive Identity ($0_{(+)}$ often written 0):** An element $i \in \mathbb{S}$ is an additive identity if for all $a \in \mathbb{S}$, $i + a = a = a + i$.
- **Multiplicative Identity ($1_{(\cdot)}$ often written 1):** An element $i \in \mathbb{S}$ is a multiplicative identity if for all $a \in \mathbb{S}$, $i \cdot a = a = a \cdot i$
- **Additive Inverse ($a^{-1}_{(+)}$):** For each $a \in \mathbb{S}$, its additive inverse $a^{-1}_{(+)}$, often written $-a$, is defined as an element such that $a + a^{-1}_{(+)} = 0_{(+)} = a^{-1}_{(+)} + a$ (i.e., additive identity)
- **Multiplicative Inverse ($a^{-1}_{(\cdot)}$):** For each $a \in \mathbb{S}$ that is invertible with respect to $(\cdot)$, its multiplicative inverse $a^{-1}_{(\cdot)}$, often written a^{-1}, is defined as an element such that $a \cdot a^{-1}_{(\cdot)} = 1_{(\cdot)} = a^{-1}_{(\cdot)} \cdot a$ (i.e., multiplicative identity)

Element Operation Features

- **Closed:** A set $\mathbb{S}$ is closed under the $(+)$ operation if for every $a, b \in \mathbb{S}$, it is the case that $a + b \in \mathbb{S}$. Likewise, a set $\mathbb{S}$ is closed under the $(\cdot)$ operation if for every $a, b \in \mathbb{S}$, it is the case that $a \cdot b \in \mathbb{S}$.
- **Associative:** For any $a, b, c \in \mathbb{S}$, $(a + b) + c = a + (b + c)$
- **Commutative:** For any $a, b \in \mathbb{S}$, $a + b = b + a$
- **Distributive:** If both $(+)$ and $(\cdot)$ are defined (e.g. in a ring), then $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$, and $(a + b) \cdot c = a \cdot c + b \cdot c$.

Group Types

- **Semigroup:** A semigroup is a set of elements which is closed and associative on a single operation ($+$ or $\cdot$)
- **Monoid:** A monoid is a semigroup with an identity element e (a neutral element that leaves any other element unchanged under the operation).
  (e.g., 0 is the identity element for $+$ operator, 1 is the identity element for the $\cdot$ operator)
- **Group:** A group is a monoid, and every element has an inverse with respect to the operation.
- **Abelian Group:** An abelian group is a group, plus its operation is commutative.

A-2.2 Examples

$\mathbb{Z}$ (i.e., the set of all integers) is an abelian group under addition $(+)$, because:

- **Closed:** For any integer $a, b \in \mathbb{Z}$, $a + b = c$ is also an integer (i.e. $a + b \in \mathbb{Z}$).
- **Associative:** For any integer $a, b, c \in \mathbb{Z}$, $(a + b) + c = a + (b + c)$.
- **Identity:** The additive identity is 0 because, for any $a \in \mathbb{Z}$, $a + 0 = a$.

- **Inverse:** For each $a \in \mathbb{Z}$, its additive inverse is $-a$, as $a + (-a) = 0$.
- **Commutative:** For any integer $a, b \in \mathbb{Z}$, $a + b = b + a$.

$\mathbb{Z}$ is a monoid under multiplication $(\cdot)$ because:
- **Closed:** For any integer $a, b \in \mathbb{Z}$, $a \cdot b = c$ is also an integer (i.e., $a \cdot b \in \mathbb{Z}$).
- **Associative:** For any integer $a, b, c \in \mathbb{Z}$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
- **Identity:** The multiplicative identity is 1, because for any $a \in \mathbb{Z}$, $a \cdot 1 = a$.
- **NO Inverse:** The integer 0 has no multiplicative inverse; for $a \neq 0$, its reciprocal $1/a$ is an integer only when $a = \pm 1$. Thus $(\mathbb{Z}, \cdot)$ is a monoid but not a group.

$\mathbb{R}^\times$ (i.e., the set of all nonzero real numbers) is an abelian group under multiplication $(\cdot)$, because:
- **Closed:** For any real number $a, b \in \mathbb{R}^\times$, $a \cdot b = c$ is also a real number (and remains in $\mathbb{R}^\times$).
- **Associative:** For any real number $a, b, c \in \mathbb{R}^\times$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
- **Identity:** The multiplicative identity is 1, as for any real number $a \in \mathbb{R}^\times$, $a \cdot 1 = a$.
- **Inverse:** For each real number $a \in \mathbb{R}^\times$, its multiplicative inverse is $\dfrac{1}{a}$, which is a non-zero real number $(\in \mathbb{R}^\times)$.

A-3 Field

- Reference: Fields and Cyclotomic Polynomials [1]

A-3.1 Definitions

⟨Definition A-3.1⟩ Field Definitions

- **Ring:** A set R that is an abelian group under addition $(+)$, equipped with a multiplication $(\cdot)$ that is closed and associative, and such that multiplication distributes over addition on both sides: $a \cdot (b + c) = a \cdot b + a \cdot c$ and $(a + b) \cdot c = a \cdot c + b \cdot c$ for all $a, b, c \in R$. (Multiplication is not necessarily commutative, e.g., a matrix multiplication, and an identity element for $(\cdot)$ is optional unless stated "ring with unity".)
- **Field:** A set F that is an abelian group under $(+)$, whose nonzero elements $F^\times = F \setminus \{0\}$ form an abelian group under $(\cdot)$, with multiplication distributing over addition.
- **Galois Field ($\mathrm{GF}(p^n)$):** A field with p^n elements for some prime p and positive integer n.
- **$\mathbb{Z}_p$ ($\mathbb{Z}/p\mathbb{Z}$):** For a prime p, the set $\{0, 1, \ldots, p-1\}$ with addition and multiplication modulo p forms a finite field. More generally, for any integer $m \geq 2$, $\mathbb{Z}_m$ is a commutative ring, and it is a field iff m is prime.

A-3.2 Examples

$\mathbb{Z}$ (the set of all integers) is a ring but not a field, because not all of its elements have a multiplicative inverse (as shown in §A-2.2).

$\mathbb{R}$ (the set of all real numbers) is a field. As shown in §A-2.2, it is an abelian group under $(+)$; its nonzero elements form an abelian group under $(\cdot)$, and multiplication distributes over addition.

$\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$ is a finite field because:

- **Closed:** For any $a, b \in \mathbb{Z}_7$, there exist $c_1, c_2 \in \mathbb{Z}_7$ such that $a + b \equiv c_1 \pmod 7$ and $a \cdot b \equiv c_2 \pmod 7$.
- **Associative:** For any $a, b, c \in \mathbb{Z}_7$, $(a + b) + c = a + (b + c)$, and $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
- **Commutative:** For any $a, b \in \mathbb{Z}_7$, $a + b = b + a$, and $a \cdot b = b \cdot a$.
- **Distributive:** For any $a, b, c \in \mathbb{Z}_7$, $(a + b) \cdot c = a \cdot c + b \cdot c$, and $a \cdot (b + c) = a \cdot b + a \cdot c$.
- **Identity:** The additive identity is 0, and the multiplicative identity is 1.
- **Inverse:** For any $a \in \mathbb{Z}_7$, there exists $a' \in \mathbb{Z}_7$ such that $a + a' \equiv 0 \pmod 7$ (e.g., the additive inverse of 3 is 4 since $3 + 4 \equiv 0 \pmod 7$). For any $a \in \mathbb{Z}_7^\times = \{1, \ldots, 6\}$, there exists $b \in \mathbb{Z}_7^\times$ such that $ab \equiv 1 \pmod 7$ (e.g., $3 \cdot 5 = 15 \equiv 1 \pmod 7$).

A-3.3 Theorems

⟨Theorem A-3.3⟩ Field Theorems

1. **Size of Finite Field:** Every finite field (also called a Galois Field) has p^n elements for some prime p and positive integer n, conversely, for each such prime power p^n, there exists a finite field of order p^n (unique up to isomorphism).
2. **Isomorphic Fields:** Any two finite fields $\mathbb{F}_1$ and $\mathbb{F}_2$ with the same number of elements are isomorphic, i.e., there exists a bijection $f : \mathbb{F}_1 \to \mathbb{F}_2$ such that for all $a, b \in \mathbb{F}_1$, $f(a + b) = f(a) + f(b)$ and $f(ab) = f(a)f(b)$.

A-4 Order

- Reference: Fields and Cyclotomic Polynomials [1]

A-4.1 Definitions

The same definition and the first three order theorems apply to any element of finite multiplicative order in a group, including roots of unity in $\mathbb{C}^\times$. An arbitrary nonzero complex number need not have finite order.

⟨Definition A-4.1⟩ Order Definition

$\mathsf{ord}_{\mathbb{F}}(a)$: For $a \in \mathbb{F}^\times$ (a finite field, §A-3.1), a's order is the smallest positive integer k such that $a^k = 1$.

A-4.2 Theorems

⟨Theorem A-4.2.1⟩ Order Property (I)

For $a \in \mathbb{F}^\times$, and $n \geq 1$, $a^n = 1$ if and only if $\mathbf{ord}_{\mathbb{F}}(a) \mid n$
(i.e., $\mathsf{ord}_{\mathbb{F}}(a)$ divides n).

Proof.

1. *Forward Proof:* If $\mathsf{ord}_{\mathbb{F}}(a) \mid n$, then for $\mathsf{ord}_{\mathbb{F}}(a) = k$ where k is a's order, and $n = lk$ for some integer l. Then, $a^n = a^{lk} = (a^k)^l = 1^l = 1$.
2. *Backward Proof:* If $a^n = 1$ and $\mathsf{ord}_{\mathbb{F}}(a) = k$, write $n = qk + r$ with $0 \leq r < k$. Then $1 = a^n = a^{qk+r} = (a^k)^q a^r = a^r$. By minimality of k, we must have $r = 0$, hence $k \mid n$.

□

⟨Theorem A-4.2.2⟩ Order Property (II)

If $\mathsf{ord}_{\mathbb{F}}(a) = k$, then for any $n \geq 1$, $\mathsf{ord}_{\mathbb{F}}(a^n) = \dfrac{k}{\gcd(k, n)}$.

Proof.

1. $a^k, a^{2k}, a^{3k}, \ldots = 1$.
2. Given $\mathsf{ord}_{\mathbb{F}}(a^n) = m$, $(a^n)^m, (a^n)^{2m}, (a^n)^{3m}, \ldots = 1$
3. Note that by definition of order, $x = k$ is the smallest value that satisfies $a^x = 1$. Thus, given $\mathsf{ord}_{\mathbb{F}}(a^n) = m$, then m is the smallest integer that makes $(a^n)^m = 1$. By Theorem A-4.2.1, $(a^n)^m = 1$ holds exactly when $k \mid mn$. The smallest possible integer m that makes mn a multiple of k is $m = \dfrac{k}{\gcd(k, n)}$.

□

⟨Theorem A-4.2.3⟩ Order Property (III)

Suppose k divides n. Then, $\mathsf{ord}_{\mathbb{F}}(a) = kn$ if and only if $\mathsf{ord}_{\mathbb{F}}(a^k) = n$.

Proof.

1. *Forward Proof:* Given $\mathsf{ord}_{\mathbb{F}}(a) = kn$, and given Theorem A-4.2.2, $\mathsf{ord}_{\mathbb{F}}(a^k) = \dfrac{nk}{\gcd(k, nk)} = \dfrac{nk}{k} = n$.
2. *Backward Proof:* Given $\mathsf{ord}_{\mathbb{F}}(a^k) = n$ and letting $\mathsf{ord}_{\mathbb{F}}(a) = m$, Theorem A-4.2.2 gives $\mathsf{ord}_{\mathbb{F}}(a^k) = \dfrac{m}{\gcd(m, k)} = n$, so $m = n \cdot \gcd(m, k)$ (i.e., m is some multiple of n). But since k divides n, k also divides m. This means that $\gcd(m, k) = k$. Hence, $\mathsf{ord}_{\mathbb{F}}(a) = m = n \cdot \gcd(m, k) = nk$.

□

⟨Theorem A-4.2.4⟩ Fermat's Little Theorem

Given $|\mathbb{F}| = p$ (a prime) and $a \in \mathbb{F}$, $a^p = a$.

Proof.

1. If $a = 0$, then $a^p = a = 0$.
2. If $a \neq 0$, then $a \in \mathbb{F}^\times$, the multiplicative group of the field, which has size $|\mathbb{F}^\times| = p - 1$. By Lagrange's theorem (in a finite group G, the order of any element divides $|G|$), the order of a divides $p - 1$, hence $a^{p-1} = 1$. Therefore $a^p = a$.

□

A-5 Polynomial Ring

- Reference 1: Polynomial Ring (Wikipedia) [2]
- Reference 2: Polynomial Rings (LibreTexts) [3]

A-5.1 Overview

A polynomial ring is a set of polynomials where polynomial computations over the $(+, \cdot)$ operators (e.g., $f_1 + f_2$, $f_1 \cdot (f_2 - f_3)$, $f_1 + f_2 + f_4$) are closed, associative, commutative, and distributive.

A polynomial ring $\mathbb{Z}_p[x]/(x^n + 1)$ is the set of all polynomials f_i that have the following properties:

⟨Summary A-5.1⟩ Ring

For a polynomial $f \in \mathbb{Z}_p[x]/(x^n + 1)$ where $f = c_0 + c_1x^1 + \cdots + c_{n-1}x^{n-1}$:

- **Coefficient Ring:** each coefficient $c_j \in \mathbb{Z}_p$.
- **Degree Bound:** Any $f' \in \mathbb{Z}_p[x]$ can be written as:

  $$f' = (x^n + 1)f_q + f_r, \qquad \deg f_r < n,$$

  so in the quotient ring $\mathbb{Z}_p[x]/(x^n + 1)$ we have $f' \equiv f_r \pmod{x^n + 1}$. f_q is called a quotient polynomial and f_r is called a remainder polynomial resulting from the polynomial division of f' divided by $x^n + 1$.
- **Polynomial Congruence:** If two polynomials are congruent, they belong to the same equivalence class, in which case they are interchangeable in the polynomial operations $(+, \cdot)$ in the polynomial ring. For example, if:

  $f' \equiv f_{r1} \in \mathbb{Z}_p[x]/(x^n + 1)$
  $f'' \equiv f_{r2} \in \mathbb{Z}_p[x]/(x^n + 1)$
  $f_{r1} + f_{r2} \equiv f_{r3} \in \mathbb{Z}_p[x]/(x^n + 1)$

  Then the polynomial operation result of $f' + f''$ is in the same equivalence class as:
  $f' + f'' \equiv f_{r1} + f_{r2} \equiv f_{r3} \in \mathbb{Z}_p[x]/(x^n + 1)$

To make the notation simple, we denote the polynomial ring $\mathbb{Z}_p[x]/(x^n + 1)$ as $\mathcal{R}_{\langle n,p \rangle}$

Recall that in $\mathbb{Z}_p$, any b writes $b = mp + r$ with $0 \leq r < p$, hence $b \equiv r \pmod{p}$ (the quotient m disappears). Similarly, in a polynomial ring $\mathcal{R}_{\langle n,p \rangle}$, a high-degree polynomial f_{big} can be divided by the polynomial modulo $x^n + 1$, which yields:

$f_{big} = (x^n + 1) \cdot (f_q) + f_r \equiv f_r \in \mathcal{R}_{\langle n,p \rangle}$

, whereas f_q is a quotient polynomial, and f_r is a remainder polynomial. In this case, f_{big} is congruent to (i.e., it is in the same equivalence class as) f_r. Thus, f_q can be eliminated, and f_r (i.e., the simplified version of f_{big}) can be used interchangeably for polynomial operations $(+, \cdot)$ in the polynomial ring. Polynomial simplification (i.e., reduction) in a polynomial ring is done by substituting $x^n \equiv -1$ into f_{big} because $x^n + 1 \equiv 0$ in the polynomial ring (this is the same as the case of a number ring modulo p, where

we reduce a number by substituting 0 for p). This way, a high-degree polynomial f_{big} can be recursively simplified to a polynomial of degree less than n by recursively substituting $x^n \equiv -1$ into f_{big}.

For a polynomial modulo, we normally choose a cyclotomic polynomial $x^n + 1$ (where n is 2^p for some integer p) as the divisor, as it provides computational efficiency.

A-5.1.1 Example

Given $f \in \mathbb{Z}_7[x]/(x^2+1)$, suppose $f = x^4 + 3x^3 + 11x^2 + 6x + 10$. Then,

$$\begin{aligned} f &= (x^2) \cdot (x^2) + 3x \cdot (x^2) + 11x^2 + 6x + 10 \\ &\equiv (-1)(-1) + 3x(-1) + (11 \bmod 7)(-1) + 6x + (10 \bmod 7) \\ &= 3x \in \mathbb{Z}_7[x]/(x^2+1) \end{aligned}$$

Thus, $f(x) = x^4 + 3x^3 + 11x^2 + 6x + 10$ is equivalent to ($\equiv$) $3x$ in the polynomial ring $\mathbb{Z}_7[x]/(x^2+1)$.

A-5.1.2 Discussion

Congruency: If two numbers are congruent, they belong to the same *congruence class*. The same is true for two congruent polynomials. If the computation results of two mathematical formulas belong to the same congruency class, then their computations wrap around within the modulus of their congruency. This is a useful property for cryptographic schemes where encryption & decryption computations wrap around their values within a limited set of binary bits. Congruency is useful for simplifying computations. For example, a large number or a complex polynomial can be *normalized* to a simpler number or polynomial by using the congruency rule, which reduces the computational overhead.

Polynomial Evaluation: Congruence classes need not have equal integer or polynomial representatives. In the example of §A-5.1.1, evaluating the displayed integer polynomials at $x = 0$ gives 10 and 0, respectively, although they represent the same element of $\mathbb{Z}_7[x]/(x^2+1)$. Evaluation of a quotient class is well-defined only in a target algebra that respects the coefficient modulus and sends the polynomial modulus to zero. Specifically, take an extension field of $\mathbb{Z}_p$ when p is prime and choose x_i with $x_i^n + 1 = 0$. Then $M_1 - M_2 = Q(x)(x^n+1)$ in $\mathbb{Z}_p[x]$ implies $M_1(x_i) = M_2(x_i)$ in that extension. Evaluating arbitrary integer lifts at a complex root does not in general preserve reduction modulo p. In characteristic different from 2, the roots of $x^n + 1$ are $2n$-th roots of unity; when n is a power of 2, they are primitive $2n$-th roots. See §A-7 and §A-9.

⟨Summary A-5.1.2⟩ Polynomial Evaluation over Polynomial Ring

Suppose $M_1 - M_2 = Q(x)(x^n + 1)$ in a coefficient field F. For a root x_i of $x^n + 1$ in an extension of F, evaluation gives $M_1(x_i) = M_2(x_i)$. If $F = \mathbb{Z}_p$, all coefficient arithmetic and evaluation take place in characteristic p.

Number Ring & Polynomial Ring: These two rings share many common characteristics, which are summarized in Table 1.

	Ring	Polynomial Ring
Set Elements	number	polynomial
Ring Notation & Definition	$\mathbb{Z}_p = \{0, 1, \ldots, p-1\}$ The set of all integers between 0 and p	$\mathbb{Z}_p[x]/(x^n+1)$ The set of all polynomials f such that $f = c_0 + c_1x^1 + c_2x^2 \cdots + c_{n-1}x^{n-1}$ where each coefficient $c_i \in \mathbb{Z}_p$ and f's degree is less than n
Supported Operations	$(+, \cdot)$ (Addition, Multiplication)	$(+, \cdot)$ (Addition, Multiplication)
$(+)$ Operation	We know how to add numbers	$f_a = a_0 + a_1x^1 + a_2x^2 \cdots + a_{d_a-1}x^{d_a-1}$ $f_b = b_0 + b_1x^1 + b_2x^2 \cdots + b_{d_b-1}x^{d_b-1}$ Then, $f_a + f_b$ is computed as: $f_c = \sum_{i=0}^{\max(d_a,d_b)-1} (a_i + b_i)x^i$
$(\cdot)$ Operation	We know how to multiply numbers	$f_a = a_0 + a_1x^1 + a_2x^2 \cdots + a_{d_a-1}x^{d_a-1}$ $f_b = b_0 + b_1x^1 + b_2x^2 \cdots + b_{d_b-1}x^{d_b-1}$ Then, $f_a \cdot f_b$ is computed as: $f_c = \sum_{i=0}^{d_a+d_b-2} \sum_{j=0}^{i} a_j b_{i-j} x^i$
Commutative **Associative** **Distributive** **Closed**	For $a, b \in \mathbb{Z}_p$ $a + b = b + a$ $(a + b) + c = a + (b + c)$ $a \cdot (b + c) = a \cdot b + a \cdot c$ $a + b \equiv c \in \mathbb{Z}_p$, $a \cdot b \equiv d \in \mathbb{Z}_p$	For $f_a, f_b \in \mathbb{Z}_p[x]/(x^n+1)$, $f_a + f_b = f_b + f_a$ $(f_a + f_b) + f_c = f_a + (f_b + f_c)$ $f_a \cdot (f_b + f_c) = f_a \cdot f_b + f_a \cdot f_c$ $f_a + f_b \equiv f_c \in \mathcal{R}_{\langle n,p \rangle}$, $f_a \cdot f_b \equiv f_d \in \mathcal{R}_{\langle n,p \rangle}$
Congruency ($\equiv$)	Two numbers $a \equiv b$ in modulo p if: $(a \bmod p) = (b \bmod p)$	Two polynomials $f_a \equiv f_b$ in $\mathbb{Z}_p[x]/(x^n+1)$ if: $f'_a = f_a \bmod (x^n+1) = \sum_{i=0}^{d_a} a_i x^i$ $f'_b = f_b \bmod (x^n+1) = \sum_{i=0}^{d_b} b_i x^i$, $d_a = d_b$ and $a_i \equiv b_i$ in modulo p for all $0 \le i \le d_a$

Table 1: Comparison between a number ring and a polynomial ring.
The table uses zero coefficients outside the displayed ranges; after multiplication, reduce modulo $x^n + 1$ and coefficient-wise modulo p.

A-5.2 Coefficient Rotation

The displayed formula assumes $0 \le h < n$. The element x is a unit in this quotient, since $x(-x^{n-1}) = 1$; hence negative powers are defined. General shifts reduce modulo $2n$, including the sign change after n positions.

Coefficient rotation is a process of shifting the entire coefficients of a polynomial (either to the left or right) in a polynomial ring. In order to rotate the entire coefficients of a polynomial by h positions to the left, we multiply x^{-h} with the polynomial.

For example, suppose we have a polynomial as follows:

$$f(x) = c_0 + c_1x^1 + c_2x^2 + \cdots + c_hx^h + \cdots + c_{n-1}x^{n-1} \in \mathcal{R}_{\langle n,p \rangle}$$

To shift the entire coefficients of f to the left by h positions (i.e., shift f's h-th coefficient to the constant term), we compute $f \cdot x^{-h}$, which is:

⟨Summary A-5.2⟩ Polynomial Rotation

Given the $(n-1)$-degree polynomial:

$f(x) = c_0 + c_1x^1 + c_2x^2 + \cdots + c_hx^h + \cdots + c_{n-1}x^{n-1} \in \mathcal{R}_{\langle n,p\rangle}$

The coefficients of $f(x)$ can be rotated to the left by h positions by multiplying to $f(x)$ by x^{-h} as follows:

$f(x) \cdot x^{-h} = c_0 \cdot x^{-h} + c_1x^1 \cdot x^{-h} + c_2x^2 \cdot x^{-h} + \cdots + c_hx^h \cdot x^{-h} + \cdots + c_{n-1}x^{n-1} \cdot x^{-h}$
$\equiv c_h + c_{h+1}x + c_{h+2}x^2 + \cdots + c_{n-1}x^{n-1-h} - c_0x^{n-h} - \cdots - c_{h-1}x^{n-1} \in \mathcal{R}_{\langle n,p\rangle}$

Note that multiplying the two polynomials f and x^{-h} will yield a congruent polynomial in $\mathcal{R}_{\langle n,p\rangle}$. Therefore, the rotated polynomial, which is the result of $f \cdot x^{-h}$, will also have a congruent polynomial in $\mathcal{R}_{\langle n,p\rangle}$.

Note that the coefficient signs change when they rotate around the boundary of $x^n(= -1)$, as the computation is conducted in the polynomial ring $\mathbb{Z}_p[x]/(x^n + 1)$.

A-5.2.1 Example

Suppose we have a polynomial $f \in \mathbb{Z}_8[x]/(x^4 + 1)$ as follows:

We use the centered residue system for $\mathbb{Z}_8$, i.e., $\{-4, -3, -2, -1, 0, 1, 2, 3\}$.

$f = 2 + 3x - 4x^2 - x^3$

The polynomial ring $\mathbb{Z}_8[x]/(x^4 + 1)$ has the following 4 congruence relationships:

$\boldsymbol{x^4 \equiv -1}$
$x^4 \cdot x^{-1} \equiv -1 \cdot x^{-1}$
$\boldsymbol{x^3 \equiv -x^{-1}}$

$x^4 \equiv -1$
$x^4 \cdot x^{-3} \equiv -1 \cdot x^{-3}$
$\boldsymbol{x \equiv -x^{-3}}$

$x^4 \equiv -1$
$x^4 \cdot x^{-2} \equiv -1 \cdot x^{-2}$
$\boldsymbol{x^2 \equiv -x^{-2}}$

Then, based on the coefficient rotation technique in Summary A-5.2, rotating 1 position to the left is equivalent to computing $f \cdot x^{-1}$ as follows:

$f \cdot x^{-1} = 2 \cdot (x^{-1}) + 3x \cdot (x^{-1}) - 4x^2 \cdot (x^{-1}) - x^3 \cdot (x^{-1})$
$\equiv -2x^3 + 3 - 4x^1 - x^2$
$= 3 - 4x^1 - x^2 - 2x^3$

As another example, rotating 3 positions to the left is equivalent to computing $f \cdot x^{-3}$ as follows:

$f \cdot x^{-3} = 2 \cdot (x^{-3}) + 3x \cdot (x^{-3}) - 4x^2 \cdot (x^{-3}) - x^3 \cdot (x^{-3})$

$$
\begin{aligned}
&\equiv -2x - 3x^2 + 4x^3 - 1 \\
&= -1 - 2x - 3x^2 + 4x^3 \\
&= -1 - 2x - 3x^2 + (4 \equiv -4 \bmod 8)x^3 \\
&\equiv -1 - 2x - 3x^2 - 4x^3
\end{aligned}
$$

A-6 Decomposition

Decomposition is a mathematical technique used to represent a number in a smaller base (radix) while preserving its value. This section will explain number decomposition and polynomial decomposition.

A-6.1 Number Decomposition

We fix a modulus $q \geq 2$ and write $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. Let $\gamma \in \mathbb{Z}_q$. Number decomposition expresses γ as a sum of multiple numbers in base β as follows:

$$\gamma = \gamma_1 \frac{q}{\beta^1} + \gamma_2 \frac{q}{\beta^2} + \cdots + \gamma_\ell \frac{q}{\beta^\ell}$$

where $\beta \geq 2$ and $\ell \geq 1$ are integers. For every canonical representative $0 \leq \gamma < q$ to have such a unique expansion with $\gamma_i \in \{0, 1, \ldots, \beta - 1\}$, assume $q = \beta^\ell$. More generally, $\beta^\ell \mid q$ only makes the weights integers: if $d = q/\beta^\ell > 1$, an exact expansion exists precisely when $d \mid \gamma$. Figure 1 illustrates full decomposition; see §A-6.3 for truncation. Each γ_i is a digit in the base-β representation of γ when $q = \beta^\ell$, with $i = 1$ the most significant digit. Binary bit shifts apply when both q and β are powers of 2.

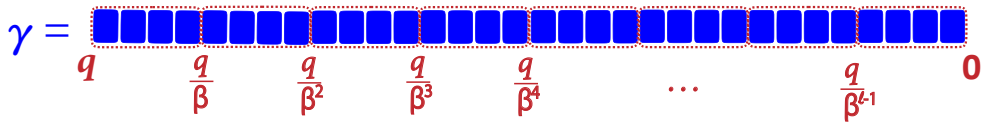


Figure 1: An illustration of number decomposition.

We define the decomposition of the number γ into base β with level ℓ as follows:

$$\mathsf{Decomp}^{\beta,\ell}(\gamma) = (\gamma_1, \gamma_2, \ \cdots, \ \gamma_\ell).$$

Number decomposition is also called radix decomposition, where the base β is referred to as a radix.

A-6.1.1 Example

Suppose we take $\gamma = 13$ in $\mathbb{Z}_{16}$. Suppose we want to decompose 13 with the base $\beta = 2$ and level $\ell = 4$. Then, 13 is decomposed as follows:

$$13 = 1 \cdot \frac{16}{2^1} + 1 \cdot \frac{16}{2^2} + 0 \cdot \frac{16}{2^3} + 1 \cdot \frac{16}{2^4}$$

$$\mathsf{Decomp}^{2,4}(13) = (1, 1, 0, 1)$$

A-6.2 Polynomial Decomposition

This time, suppose we have a polynomial f in the polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$. Therefore, each coefficient c_i of f is an integer modulo q. Polynomial decomposition expresses f as a sum of multiple polynomials in base β and level ℓ as follows:

⟨Summary A-6.2⟩ Polynomial Decomposition

Given $f \in \mathbb{Z}_q[x]/(x^n+1)$, fix integers $\beta \geq 2$, $\ell \geq 1$ with $q = \beta^\ell$. We write

$$f = \sum_{i=1}^{\ell} f_i \frac{q}{\beta^i}, \qquad f_i \in \mathbb{Z}_q[x]/(x^n+1)$$

where each f_i is obtained by decomposing every coefficient of f in base β. If $f = \sum_j c_j x^j$ with $c_j \in \mathbb{Z}_q$, then $c_j = \sum_{i=1}^{\ell} c_{j,i} \frac{q}{\beta^i}$ with $c_{j,i} \in \{0, \ldots, \beta-1\}$, and $f_i = \sum_j c_{j,i} x^j$. We denote the decomposition of the polynomial f into the base β with the level ℓ as follows:

$$\mathsf{Decomp}^{\beta,\ell}(f) = (f_1, f_2, \cdots, f_\ell)$$

A-6.2.1 Example

Suppose we have the following polynomial in the polynomial ring $\mathbb{Z}_{16}[x]/(x^4+1)$:

$$f = 6x^3 + 3x^2 + 14x + 7 \in \mathbb{Z}_{16}[x]/(x^4+1)$$

Suppose we want to decompose the above polynomial with base $\beta = 4$ and level $\ell = 2$. Then, each degree's coefficient of the polynomial f is decomposed as follows:

$$\boldsymbol{x^3}\text{: } 6 = 1 \cdot \frac{16}{4^1} + 2 \cdot \frac{16}{4^2}$$
$$\boldsymbol{x^2}\text{: } 3 = 0 \cdot \frac{16}{4^1} + 3 \cdot \frac{16}{4^2}$$
$$\boldsymbol{x^1}\text{: } 14 = 3 \cdot \frac{16}{4^1} + 2 \cdot \frac{16}{4^2}$$
$$\boldsymbol{x^0}\text{: } 7 = 1 \cdot \frac{16}{4^1} + 3 \cdot \frac{16}{4^2}$$

The decomposed polynomial is as follows:

$$f = 6x^3 + 3x^2 + 14x + 7 = (1x^3 + 0x^2 + 3x + 1) \cdot \frac{16}{4^1} + (2x^3 + 3x^2 + 2x + 3) \cdot \frac{16}{4^2}$$

$$\mathsf{Decomp}^{4,2}(6x^3 + 3x^2 + 14x + 7) = (1x^3 + 0x^2 + 3x + 1, 2x^3 + 3x^2 + 2x + 3)$$

A-6.2.2 Discussion

Note that after decomposition, the original coefficients of the polynomial have been reduced to smaller numbers. This characteristic is importantly used in the multiplication of polynomials in FHE ciphertexts to reduce the growth rate of the noise. Normally, the polynomial coefficients of ciphertexts are large because they are uniformly random numbers. Reducing such large constants is important for reducing the noise growth during homomorphic multiplication. We will discuss this in more detail in §D-1.6.

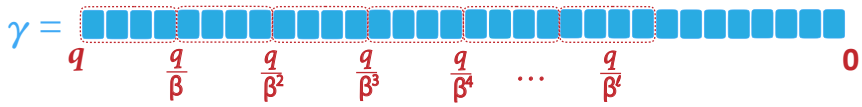


Figure 2: An illustration of approximate decomposition

A-6.3 Approximate Decomposition

Fix integers $\beta \geq 2$ and $\ell \geq 1$ with $\beta^\ell \mid q$, and let $d = q/\beta^\ell$. Truncation is needed when $d > 1$ and the input is not a multiple of d, even though the divisibility condition holds. Round the canonical representative of γ to the nearest multiple of d, reduce that multiple modulo q, and decompose it into the displayed weights. With a fixed nearest-integer tie rule, the result satisfies

$$\gamma \equiv \sum_{i=1}^{\ell} \gamma_i \frac{q}{\beta^i} + \varepsilon \pmod{q}, \qquad \gamma_i \in \{0, \ldots, \beta - 1\}, \qquad |\varepsilon| \leq \frac{q}{2\beta^\ell}.$$

The residual is an integer lift of the circular rounding error. The congruence is essential: for $q = 16$, $\beta = 2$, $\ell = 2$, the input 15 rounds to $16 \equiv 0 \pmod{16}$, with $\varepsilon = -1$. The polynomial case is coefficient-wise. If $\beta^\ell \nmid q$, the displayed rational weights do not define this integer-ring gadget; one may instead use the integer gadget in §A-6.4.

A-6.4 Gadget Decomposition

The gadget and the allowed digit set are part of the definition; arbitrary weights need not represent every residue. An exact integer choice valid for every $q \geq 2$ is $g_i = \beta^{i-1}$, with $\ell = \lceil \log_\beta q \rceil$ and the ordinary radix digits of the canonical representative. It satisfies $\gamma = \sum_{i=1}^{\ell} \gamma_i g_i$ over the integers. It differs from the descending weights below; their digit vectors must not be interchanged. For a truncated gadget, replace exact reconstruction by reconstruction modulo q with its residual and include that residual in any later noise analysis.

Gadget decomposition is a generalized form of number decomposition (§A-6.1). In number decomposition, a number γ is decomposed as follows:

$\gamma = \gamma_1 \frac{q}{\beta^1} + \gamma_2 \frac{q}{\beta^2} + \cdots + \gamma_\ell \frac{q}{\beta^\ell}$

In gadget decomposition, we decompose γ as follows:

$\gamma = \gamma_1 g_1 + \gamma_2 g_2 + \cdots + \gamma_\ell g_\ell$

We denote $\vec{g} = (g_1, g_2, \cdots, g_\ell)$ as a gadget vector, and $\mathsf{Decomp}^{\vec{g}}(\gamma) = (\gamma_1, \gamma_2, \ \cdots, \ \gamma_\ell)$

Then, $\gamma = \langle \mathsf{Decomp}^{\vec{g}}(\gamma), \vec{g} \rangle$

In the case of number decomposition (§A-6.1), its gadget vector is $\vec{g} = \left(\frac{q}{\beta}, \frac{q}{\beta^2}, \cdots, \frac{q}{\beta^\ell} \right)$.

A-7 Roots of Unity

- Reference: Fields and Cyclotomic Polynomials [1]

A-7.1 Definitions

⟨Definition A-7.1⟩ Definitions for Roots of Unity

- ***n*-th root of Unity:** A complex number ζ that satisfies the equation $\zeta^n = 1$
- **Primitive *n*-th Root of Unity:** Any n-th root of unity ζ such that $\mathsf{ord}_{\mathbb{C}}(\zeta) = n$. We denote $P(n)$ as a set of primitive n-th root of unity.

A primitive n-th root of unity is considered a *generator* of all n n-th roots of unity.

A-7.2 Theorems

⟨Theorem A-7.2.1⟩ Formula for *n*-th Root of Unity

Given $\zeta^n = 1$, there exist exactly n different n-th roots of unity:

$$\zeta = e^{2k\pi i/n} = \cos\left(\frac{2k\pi}{n}\right) + i \cdot \sin\left(\frac{2k\pi}{n}\right),$$

for n different k values, where $k = \{0, 1, \cdots, n-1\}$.

Proof.

1. Suppose $\zeta = e^{2k\pi i/n}$. Then, $\zeta^n = (e^{2k\pi i/n})^n = e^{2k\pi i}$, and since $\zeta^n = 1$, we need to find the k values such that $e^{2k\pi i} = 1$
2. Euler's formula states that $e^{i \cdot x} = \cos(x) + i \cdot \sin(x)$. Therefore, if $x = 2k\pi$, then $e^{2k\pi i} = \cos(2k\pi) + i \cdot \sin(2k\pi)$. This formula becomes 1 if $k = 0, 1, 2, ...$ Thus, $e^{2k\pi i} = 1$ for any integer $k \geq 0$.
3. If $\zeta = e^{2k\pi i/n} = \cos(\frac{2k\pi}{n}) + i \cdot \sin(\frac{2k\pi}{n})$, then the first n roots for $k = 0, 1, ... n-1$ are all distinct values, because they lie on the circle in the complex plane (where x-axis is a real value and y-axis is a complex value coefficient) at each angle $2k\pi/n$ for $k = \{0, 1, \cdots, n-1\}$.
4. Note $\zeta^n = 1$ is an n-th polynomial, so it can have at most n roots. Thus, we can consider the first n roots $e^{2k\pi i/n}$ for $k = \{0, 1, \cdots, n-1\}$ as the n distinct roots and ignore the rest of roots (i.e., $k \geq n$), considering them to be repetitions of the first n roots on a circle in the complex plane (see Figure 3).

□

⟨Theorem A-7.2.2⟩ Order of the Root of Unity

Given $\zeta \in \mathbb{C}$ (the complex number domain) and $\zeta^n = 1$ where $n \geq 1$, ζ is an n-th root of unity if and only if $\mathsf{ord}_{\mathbb{C}}(\zeta) \mid n$.

Proof. We use Theorem A-4.2.1:

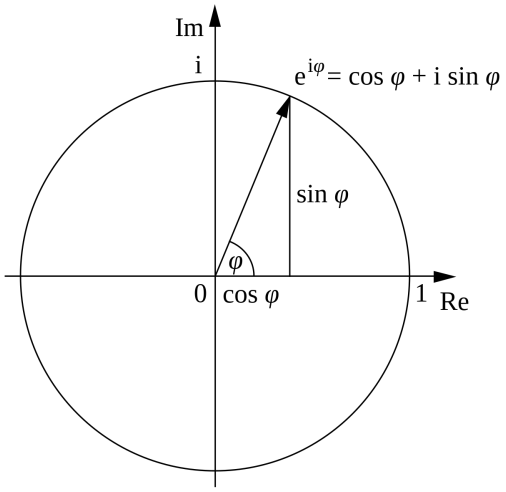


Figure 3: The figure illustrates a circle of Euler's formula in the complex plane (Source)

1. *Forward Proof:* Since $\mathsf{ord}_{\mathbb{C}}(\zeta) = k$ is the smallest integer such that $\zeta^k = 1$, for any n that satisfies $\zeta^n = 1$, n must be a multiple of k. This means that $k \mid n$.
2. *Backward Proof:* If $k \mid n$, then n is a multiple of k, which means that $\zeta^n = 1$.

□

⟨Theorem A-7.2.3⟩ Set of All n-th Roots of Unity

The set of all n-th roots of unity is the union $\bigcup_{d|n} P(d)$ (i.e., the union of all primitive d-th roots of unity where $d \mid n$).

Proof.

1. Let $\omega = e^{2\pi i/n}$. Given $\zeta^n = 1$, for each n-th root of unity ζ is, $\zeta = \omega^{k_i}$ for $k_i = \{0, 1, \cdots, n-1\}$. Note that according to Theorem A-4.2.1, $(\omega^{k_i})^n = 1$ if and only if $\mathsf{ord}_{\mathbb{C}}(\omega^{k_i}) \mid n$.
2. Let $\mathsf{ord}_{\mathbb{C}}(\omega^{k_i}) = d_i$. Then, $(\omega^{k_i})^{d_i} = 1$. Combining these two facts, each n-th root of unity ω^{k_i} is also the primitive d_i-th root of unity (i.e., a solution for $\zeta^{d_i} = 1$), that is, $\omega^{k_i} \in P(d_i)$.
3. Remember that for each $\mathsf{ord}_{\mathbb{C}}(\omega^{k_i}) = d_i$, $d_i \mid n$. For every d_i that divides n, all the (primitive) d_i-th roots of unity are also the n-th root of unity. This is because the (primitive) d_i-th root of unity that satisfies $\zeta^{d_i} = 1$ also satisfies $\zeta^n = 1$ (as n is a multiple of d_i).
4. Step 2 concludes that each n-th root of unity is a primitive d_i-th root of unity for some d_i that divides n. Step 3 concludes that each d_i-th root of unity, where d_i divides n, is also the n-th root of unity. Combining these two conclusions, the set of all n-th root of unity is equivalent to the union of all primitive d_i-th roots of unity where d_i divides n (i.e., $\bigcup_{d|n} P(d)$).

□

⟨Theorem A-7.2.4⟩ Condition for Primitive n-th Roots of Unity

Given an n-th root of unity $\zeta = \omega^k$ for $k = \{0, 1, \cdots, n-1\}$ where $\omega = e^{2\pi i/n}$, ζ is a primitive n-th root of unity if and only if $\gcd(n, k) = 1$ (i.e., k is co-prime to n).

Proof.

1. Note that $\zeta^n = 1$ and $\zeta = \omega$ for $k = 1$. Thus, $\mathsf{ord}_{\mathbb{C}}(\omega) = n$.
2. Theorem A-4.2.2 states that if $ord_{\mathbb{F}}(a) = k$, then for any $n \geq 1$, $ord_{\mathbb{F}}(a^n) = \dfrac{k}{\gcd(k,n)}$. Similarly, if $\mathsf{ord}_{\mathbb{C}}(\omega) = n$, then for any $k \geq 1$, $\mathsf{ord}_{\mathbb{C}}(\omega^k) = \dfrac{n}{\gcd(k,n)}$. For $k = 0$, the formula also holds directly: $\mathrm{ord}(\omega^0) = 1 = n/\gcd(n, 0)$; this includes $n = 1$.
3. Step 2 implies that $\mathsf{ord}_{\mathbb{C}}(\omega^k) = n$ (i.e., ω^k is a primitive n-th root of unity) if and only if $\gcd(k, n) = 1$.

□

⟨Theorem A-7.2.5⟩ The number of Primitive n-th Roots of Unity

The number of primitive n-th roots of unity is $\phi(n)$ (i.e., the number of elements in $\{1, \cdots, n\}$ that are coprime to n).

Proof.
1. Given $\zeta^n = 1$, the roots of unity are $\zeta = \omega^k$ where $\omega = e^{2\pi i/n}$ and $k = \{0, 1, \cdots, n-1\}$
2. By definition, ω^k is a primitive n-th root of unity if and only if $\mathsf{ord}_{\mathbb{C}}(\omega^k) = n$.
3. ω is a primitive n-th root of unity because $\mathsf{ord}_{\mathbb{C}}(\omega) = n$.
4. According to Theorem A-4.2.2, if $\mathsf{ord}_{\mathbb{C}}(\omega) = n$, then $\mathsf{ord}_{\mathbb{C}}(\omega^k) = \dfrac{n}{\gcd(k,n)}$. Therefore, in order for $\mathsf{ord}_{\mathbb{C}}(\omega^k) = n$, $\gcd(k, n)$ has to be 1. In other words, k and n have to be co-prime. For $n > 1$, the admissible exponents are the integers $1 \leq k < n$ coprime to n, of which there are $\phi(n)$. For $n = 1$, the sole root is 1, its order is 1, and $\phi(1) = 1$; the exponent $k = 0$ is admissible in this case.

□

A-8 Cyclotomic Polynomial

- Reference: Fields and Cyclotomic Polynomials [1]

A-8.1 Definitions

⟨Definition A-8.1⟩ Cyclotomic Polynomial

The n-th Cyclotomic Polynomial: is a polynomial whose roots are the primitive n-th roots of unity, that is:

$$\Phi_n(x) = \prod_{\zeta \in P(n)} (x - \zeta) = \prod_{\substack{0 \le k \le n-1, \\ \gcd(k,n)=1}} (x - \omega^k) \quad \text{, where } \omega = e^{2\pi i/n}$$

Remember the Euler's formula: $e^{2k\pi i/n} = \cos\left(\frac{2k\pi}{n}\right) + i \cdot \sin\left(\frac{2k\pi}{n}\right)$

A few pre-computed cyclotomic polynomials are as follows:

$\Phi_1(x) = x - 1$
$\Phi_2(x) = x + 1$
$\Phi_3(x) = x^2 + x + 1$
$\Phi_4(x) = x^2 + 1$
$\Phi_5(x) = x^4 + x^3 + x^2 + x + 1$
$\Phi_6(x) = x^2 - x + 1$
$\Phi_7(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$
$\Phi_8(x) = x^4 + 1$
$\Phi_9(x) = x^6 + x^3 + 1$
$\Phi_{10}(x) = x^4 - x^3 + x^2 - x + 1$

As one example,

$$\Phi_4(x) = \prod_{\substack{0 \le k \le 3, \\ \gcd(k,4)=1}} (x - \omega^k) = (x - \omega^1)(x - \omega^3) = (x - e^{2\pi i/4})(x - e^{2\cdot 3\pi i/4}) = (x - e^{\pi i/2})(x - e^{3\pi i/2})$$

$$= \left(x - \left(\cos\left(\frac{\pi}{2}\right) + i \cdot \sin\left(\frac{\pi}{2}\right)\right)\right) \cdot \left(x - \left(\cos\left(\frac{3\pi}{2}\right) + i \cdot \sin\left(\frac{3\pi}{2}\right)\right)\right)$$
$$= (x - i)(x + i) = x^2 + 1$$

A-8.2 Theorems

⟨Theorem A-8.2.1⟩ Roots of the M-th Cyclotomic Polynomial

Suppose that M is a power of 2 and the M-th cyclotomic polynomial $\Phi_M(x) = x^n + 1$ (where $M = 2n$). Then, the roots of the M-th cyclotomic polynomial are $\omega, \omega^3, \omega^5, \cdots, \omega^{2n-1}$, where $\omega = e^{i\pi/n}$

Proof. According to Definition A-8.1 in §A-8.1, the roots of $\Phi_M(x)$ are $e^{2k\pi i/M} = e^{2k\pi i/(2n)} = e^{k\pi i/n}$ where $0 \le k < M = 2n$ and $\gcd(k, M = 2n) = 1$, thus $k = \{1, 3, 5, \cdots, 2n-1\}$. If we let $\omega = e^{i\pi/n}$, then the roots of $\Phi_M(x)$ are $\omega, \omega^3, \omega^5, \cdots, \omega^{2n-1}$.

□

⟨Theorem A-8.2.2⟩ Polynomial Decomposition into Cyclotomic Polynomials

For any positive integer n,

$$x^n - 1 = \prod_{d|n} \Phi_d(x)$$

Proof.

1. The roots of $x^n - 1$ are all the n-th roots of unity. Thus, $x^n - 1 = (x - \omega^0)(x - \omega^1)...(x - \omega^{n-1})$, where $\zeta = \omega^k$.
2. Theorem A-7.2.3 states that each n-th root of unity (ω^k) is a primitive d-th root of unity for some d that divides n. In other words, each n-th root of unity belongs to some $P(d)$ where $d \mid n$. Meanwhile, by definition, $\Phi_d(x) = \Pi_{\zeta \in P(d)}(x - \zeta)$. Therefore, $x^n - 1$ is the product of all $\Phi_d(x)$ such that $d \mid n$.

□

⟨Theorem A-8.2.3⟩ Integer Coefficients of Cyclotomic Polynomials

A cyclotomic polynomial has only integer coefficients.

Proof.

We use strong induction on n. For $n = 1$, $\Phi_1(x) = x - 1$ is monic with integer coefficients. For $n > 1$, assume the result for every positive $d < n$ and set

$$F(x) = \prod_{\substack{d|n \\ d<n}} \Phi_d(x).$$

Then $F(x)$ is monic in $\mathbb{Z}[x]$. By Theorem A-8.2.2, $x^n - 1 = F(x)\Phi_n(x)$ in $\mathbb{C}[x]$. Long division by the monic polynomial F uses only integer arithmetic, so its quotient and remainder lie in $\mathbb{Z}[x]$. Uniqueness of polynomial division over $\mathbb{C}$ shows that this quotient is Φ_n and the remainder is zero. Thus $\Phi_n \in \mathbb{Z}[x]$, completing the induction. □

⟨Theorem A-8.2.4⟩ Formula for $\Phi_{nk}(x)$

If $k \mid n$, then $\Phi_{nk}(x) = \Phi_n(x^k)$.

Proof.

For positive integers $k \mid n$, Theorem A-4.2.3 gives $\text{ord}(\zeta) = nk$ if and only if $\text{ord}(\zeta^k) = n$. Consequently, $\Phi_n(x^k)$ and $\Phi_{nk}(x)$ have exactly the same roots. These roots are simple: every root η of Φ_n is nonzero and simple, and each polynomial $x^k - \eta$ has k distinct complex roots because its derivative kx^{k-1} is nonzero there. Both polynomials are monic and have the same simple roots, so they are equal. The map $\zeta \mapsto \zeta^k$ is k-to-one on these root sets, not one-to-one; multiplying once for every preimage would incorrectly repeat the roots of Φ_n. □

A-9 Roots of Unity and Cyclotomic Polynomial over Ring

In §A-7 and §A-8, we learned about the definition and properties of the μ-th roots of unity and the μ-th cyclotomic polynomial over complex numbers (i.e., $X \in \mathbb{C}$) as follows:

- **The μ-th roots of unity** are the solutions in $\mathbb{C}$ to the equation $X^\mu = 1$. In other words, all μ-th roots of unity can be written as $X = e^{2\pi ik/\mu}$ for integers k with $0 \le k < \mu$.
- **The primitive μ-th roots of unity (denoted as ω)** are those μ-th roots of unity whose order (§A-4.1) is μ (i.e., $\omega^\mu = 1$ and $\omega^d \neq 1$ for any $1 \le d < \mu$ with $d \mid \mu$).
- Given any primitive μ-th roots of unity ω, one can generate all primitive μ-th roots of unity by computing $\omega^{k'}$ such that k' is an integer $0 < k' < \mu$ and $\mathsf{gcd}(k', \mu) = 1$ (Theorem A-7.2.4 in §A-4.2).
- **The μ-th cyclotomic polynomial** is defined as a polynomial whose roots are the primitive μ-th roots of unity. That is,

$$\Phi_\mu(x) = \prod_{\omega \in P(\mu)} (x - \omega) = \prod_{\substack{0 \le k \le \mu - 1, \\ \mathsf{gcd}(k,\mu)=1}} (x - \omega^k)$$

Over $\mathbb{Z}_p$, the cyclotomic polynomial means the coefficient-wise reduction modulo the prime p of the integer polynomial Φ_μ. For the root factorizations in the following box and table, assume $\mu \ge 2$ and $\mu \mid p - 1$. Then $\mathbb{Z}_p^\times$ contains a primitive μ-th root, and all these roots lie in $\mathbb{Z}_p$. More generally, if $p \nmid \mu$, the same factorization is valid over a splitting field, which may be larger than $\mathbb{Z}_p$. If $p \mid \mu$, roots can be repeated and need not have order μ; the characterization below must not be applied in that case.

⟨Definition A-9⟩ Roots of Unity and Cyclotomic Polynomial over Ring $\mathbb{Z}_p$

- **The μ-th roots of unity (denoted as ω)** are the solutions for $X^\mu \equiv 1 \bmod p$. Note that in contrast to the complex case, these solutions cannot be expressed as $X = e^{2\pi ik/\mu}$.
- **The primitive μ-th roots of unity** are defined as those μ-th roots of unity whose order is μ (i.e., $\omega^\mu \equiv 1 \bmod p$, and $\omega^d \not\equiv 1 \bmod p$ for any $1 \le d < \mu$ with $d \mid \mu$).
- Given any primitive μ-th roots of unity ω, it can generate all primitive μ-th roots of unity by computing $\omega^{k'}$ such that k' is an integer $0 < k' < \mu$ and $\mathsf{gcd}(k', \mu) = 1$.
- **The μ-th cyclotomic polynomial** is defined as a polynomial whose roots are the primitive μ-th roots of unity. That is,

$$\Phi_\mu(x) = \prod_{\omega \in P(\mu)} (x - \omega) = \prod_{\substack{0 \le k \le \mu - 1, \\ \mathsf{gcd}(k,\mu)=1}} (x - \omega^k)$$

Note that in the μ-th cyclotomic polynomial, in both cases of over $X \in \mathbb{C}$ and over $X \in \mathbb{Z}_p$, each of their roots ω (i.e., the primitive μ-th root of unity) has the order μ (i.e., $\omega^\mu = 1$ over $X \in \mathbb{C}$, and $\omega^\mu \equiv 1 \bmod p$ over $X \in \mathbb{Z}_p$). Also note that each root ω can generate all roots of the μ-th cyclotomic polynomial by computing $\omega^{k'}$ such that $\mathsf{gcd}(k', \mu) = 1$.

Table 2 compares the properties of the roots of unity and the μ-th cyclotomic polynomial over $\mathbb{C}$ (the complex numbers) and over $\mathbb{Z}_p$ (the ring).

	Polynomial over $\mathbb{C}$ (Complex Number)	**Polynomial over $\mathbb{Z}_p$ (Ring)**
Definition of the μ-th Root of Unity	All $X \in \mathbb{C}$ such that $X^\mu = 1$, (which are computed as $X = e^{2\pi ik/\mu}$ for integer k where $0 \leq k \leq \mu - 1$)	All $X \in \mathbb{Z}_p$ such that $X^\mu \equiv 1 \bmod p$
Definition of the Primitive μ-th Root of Unity	Those μ-th roots of unity ω such that $\omega^\mu = 1$, and $\omega^d \neq 1$ for any $1 \leq d < \mu$ with $d \mid \mu$	Those μ-th roots of unity ω such that $\omega^\mu \equiv 1 \bmod p$, and $\omega^d \not\equiv 1 \bmod p$ for any $1 \leq d < \mu$ with $d \mid \mu$
Definition of the μ-th Cyclotomic Polynomial	The polynomial whose roots are the μ-th primitive roots of unity as follows: $\Phi_\mu(x) = \prod_{\omega \in P(\mu)}(x - \omega)$ (see Definition A-8.1 in §A-8.1)	
Finding Primitive μ-th Roots of Unity	For $\omega = e^{2\pi i/\mu}$, compute all ω^k such that $0 < k < \mu$ and $\mathsf{gcd}(k, \mu) = 1$ (Theorem A-7.2.4 in §A-7.2)	Find one primitive ω that is a root of the μ-th cyclotomic polynomial, and compute all $\omega^k \bmod p$ such that $0 < k < \mu$ and $\mathsf{gcd}(k, \mu) = 1$

Table 2: The roots of unity and cyclotomic polynomials over $X \in \mathbb{C}$ vs. over $X \in \mathbb{Z}_p$

A-10 Lagrange's Polynomial Interpolation

Suppose we are given $n+1$ two-dimensional coordinates $(x_0, y_0), (x_1, y_1), \cdots, (x_n, y_n)$, where all x values are distinct, but y values are not necessarily distinct. Lagrange's polynomial interpolation is a technique to find a unique polynomial of degree at most n that passes through such $n+1$ coordinates. The given points (x_i, y_i) may lie either in $\mathbb{C}^2$ (the complex plane, which includes the real numbers) or in $\mathbb{Z}_p^2$ for some prime p.

⟨Theorem A-10⟩ Lagrange's Polynomial Interpolation

Suppose we are given $n+1$ two-dimensional coordinates $(x_0, y_0), (x_1, y_1), \cdots, (x_n, y_n)$, whereas all X values are distinct but the Y values don't need to be distinct. The domain of (X, Y) can be either: $(x_i, y_i) \in \mathbb{C}^2$ (which includes the real domain) or $(x_i, y_i) \in \mathbb{Z}_p^2$ (where p is a prime). Then, there exists a unique polynomial $f(X)$ of degree at most n that passes through these $n+1$ coordinates. Such a polynomial $f(X)$ is computed as follows:

$$f(X) = \sum_{j=0}^{n} \frac{(X-x_0)\cdot(X-x_1)\cdots(X-x_{j-1})\cdot(X-x_{j+1})\cdots(X-x_n)}{(x_j-x_0)\cdot(x_j-x_1)\cdots(x_j-x_{j-1})\cdot(x_j-x_{j+1})\cdots(x_j-x_n)} \cdot y_j$$

$$f(X) = \sum_{j=0}^{n} \left(\prod_{\substack{0 \le k \le n \\ k \ne j}} \frac{X-x_k}{x_j-x_k} \cdot y_j \right)$$

Proof.

1. First, we will show that there exists an n-degree (or lesser degree) polynomial $f(X)$ that passes through the $n+1$ distinct coordinates: $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$. Such a polynomial $f(X)$ is designed as follows:

$$\begin{aligned} f(X) &= \sum_{j=0}^{n} \frac{(X-x_0)\cdot(X-x_1)\cdots(X-x_{j-1})\cdot(X-x_{j+1})\cdots(X-x_n)}{(x_j-x_0)\cdot(x_j-x_1)\cdots(x_j-x_{j-1})\cdot(x_j-x_{j+1})\cdots(x_j-x_n)} \cdot y_j \\ &= \sum_{j=0}^{n} \left(\prod_{\substack{0 \le k \le n \\ k \ne j}} \frac{X-x_k}{x_j-x_k} \cdot y_j \right) \\ &= \sum_{j=0}^{n} \ell_j(X) \cdot y_j \quad \triangleright \text{ where } \ell_j(X) = \prod_{\substack{0 \le k \le n \\ k \ne j}} \frac{X-x_k}{x_j-x_k} \end{aligned}$$

   We call $\{\ell_0(X), \ell_1(X), \ldots, \ell_n(X)\}$ the Lagrange basis for polynomials of degree $\le n$. Given this design of $f(X)$, notice that for each of $(x_i, y_i) \in \{(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)\}$, $\ell_i(x_{i'}) = 1$ for $i' = i$, and $\ell_i(x_{i'}) = 0$ for $i' \ne i$. Therefore, $f(x_i) = \sum_{j=0}^{n} \ell_j(x_i) \cdot y_j = 1 \cdot y_i = y_i$ for $0 \le i \le n$. In other words, $f(X)$ passes through the $n+1$ distinct coordinates: $\{(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)\}$. Such a satisfactory $f(X)$ can be computed in the case where the domain of (X, Y) is either: $(x_i, y_i) \in \mathbb{C}^2$ (i.e., real and complex numbers), or $(x_i, y_i) \in \mathbb{Z}_p^2$ (where p is a prime). Especially, a valid $f(X)$ can be computed also in the mod p domain, because as we learned from Fermat's Little Theorem in Theorem A-4.2.4 (§A-4.2), $a^{p-1} \equiv 1 \bmod p$ if and only if a and p are co-prime, and this means that if p is a prime, then $a^{p-1} \equiv 1 \bmod p$ for all $a \in \mathbb{Z}_p^\times$ (i.e., $\mathbb{Z}_p$ without $\{0\}$). Since every value in $\mathbb{Z}_p^\times$ has an inverse, we can perform each division in the formula for

$f(X) = \sum_{j=0}^{n} \frac{(X-x_0)\cdot(X-x_1)\cdots(X-x_{j-1})\cdot(X-x_{j+1})\cdots(X-x_n)}{(x_j-x_0)\cdot(x_j-x_1)\cdots(x_j-x_{j-1})\cdot(x_j-x_{j+1})\cdots(x_j-x_n)} \cdot y_j$ by multiplying with the corresponding inverses of those denominators.

2. Next, we will prove that no two distinct n-degree (or lesser degree) polynomials $f_1(X)$ and $f_2(X)$ can pass through the same $n+1$ distinct (X, Y) coordinates. Suppose there exist such two polynomials $f_1(X)$ and $f_2(X)$. Then $f_{1-2}(X) = f_1(X) - f_2(X)$ will be a new n-degree (or lesser degree) polynomial that passes through $(x_0, 0), (x_1, 0), \cdots, (x_n, 0)$. This means that $f_{1-2}(X)$ has $n+1$ distinct roots. In other words, $f_{1-2}(X)$ is an $n+1$-degree (or higher-degree) polynomial. However, this contradicts the assumption that $f_{1-2}(X)$ is an n-degree (or lesser degree) polynomial. Therefore, there exist no two polynomials $f_1(X)$ and $f_2(X)$ that pass through the same $n+1$ distinct (X, Y) coordinates.
3. We have shown that there exists some n-degree (or lesser degree) polynomial $f(X)$ that passes through $n+1$ distinct (X, Y) coordinates, and no such two or more distinct polynomials exists. Therefore, there exists only a unique polynomial that satisfies this requirement.

□

A-10.1 The Field Condition for Unique Interpolation

⟨Theorem A-10.1⟩ The Field Condition for Unique Interpolation

If a polynomial is defined over a field $\mathbb{F}$ (i.e., its x and y coordinates and its coefficients lie in $\mathbb{F}$), then for any $n+1$ distinct x values and any $n+1$ arbitrary (i.e., can be overlapped) y values in $\mathbb{F}$, there exists a unique polynomial of degree at most n that interpolates them.

Examples of such fields are $\mathbb{R}$ (real number domain), $\mathbb{C}$ (complex number domain), and any finite field $\mathbb{Z}_t$ (where t is a prime).

Proof.

1. Remember from Theorem A-10 that Lagrange's polynomial interpolation is defined as:

   $$f(X) = \sum_{j=0}^{n} \left(\prod_{\substack{0 \le k \le n \\ k \ne j}} \frac{X - x_k}{x_j - x_k} \cdot y_j \right)$$

   If every x, y values and coefficients of the polynomial $f(X)$ lie in a field, then $(x_j - x_k)^{-1}$ is guaranteed to exist within the same field, because every non-zero element of a field has an inverse, and $(x_j - x_k)$ is not zero (since $x_j \ne x_k$). This implies that $\prod_{\substack{0 \le k \le n \\ k \ne j}} \frac{X - x_k}{x_j - x_k} \cdot y_j$ also lies in the same field. Hence, a well-defined $f(X)$ is guaranteed to exist. And the degree of $f(X)$ is at most n, because its highest possible degree term is X^n.
2. For uniqueness of $f(X)$, suppose another polynomial $g(X)$ of degree at most n also passes through the same $n+1$ points. Let $h(X) = f(X) - g(X)$. Then the degree of $h(X)$ is at most n, and $h(x_i) = 0$ for all $(n+1)$ distinct nodes $x_0, \ldots, x_n$, so h has $(n+1)$ distinct roots. Over a field, a non-zero polynomial of degree d has at most d roots; a nonzero $h(X)$ of degree at most n could have at most n roots. Since $h(X)$ has $(n+1)$ roots, $h(X)$ must be the zero polynomial, i.e. $f(X) = g(X)$. Therefore $f(X)$ is unique.

□

A-11 Vector and Matrix

A-11.1 Vector Arithmetic

Geometric dot-product formulas in this chapter use real Euclidean vectors. For complex vectors, the Hermitian product defined below is the appropriate positive-definite inner product.

This section explains the basic arithmetic of vector and matrix computations, as well as advanced concepts such as vector/plane projection and the basis of planes (or spaces).

⟨Definition A-11.1⟩ Vector Arithmetic

- **Addition:** Given two $n \times 1$ vectors (i.e., n-dimensional vectors) composed of n numbers each:
  $\vec{a} = (a_0, a_1, \cdots, a_{n-1})$, $\vec{b} = (b_0, b_1, \cdots, b_{n-1})$

  Vector addition is defined as:
  $\vec{a} + \vec{b} = (a_0 + b_0,\ a_1 + b_1,\ \cdots,\ a_{n-1} + b_{n-1})$

- **Dot Product:** Given two n-dimensional vectors:
  $\vec{a} = (a_0, a_1, \ldots, a_{n-1})$, $\vec{b} = (b_0, b_1, \ldots, b_{n-1})$

  Dot product is defined as:
  $\langle \vec{a}, \vec{b} \rangle = \vec{a} \cdot \vec{b} = a_0 b_0 + a_1 b_1 + \cdots + a_{n-1} b_{n-1}$.
  $\vec{a} \cdot \vec{b}$ can also be expressed as $\|\vec{a}\|\,\|\vec{b}\| \cos\theta$, where θ is the angle between $\vec{a}$ and $\vec{b}$. If $\vec{a}$ and $\vec{b}$ point in the same direction, $\vec{a} \cdot \vec{b}$ attains its maximum value ($\|\vec{a}\|\,\|\vec{b}\|$). If $\vec{a}$ and $\vec{b}$ are orthogonal, then $\vec{a} \cdot \vec{b} = 0$.

- **Hadamard Product:** Given two n-dimensional vectors:
  $\vec{a} = (a_0, a_1, \ldots, a_{n-1})$, $\vec{b} = (b_0, b_1, \ldots, b_{n-1})$

  Hadamard product is defined as:
  $\vec{a} \odot \vec{b} = (a_0 b_0,\ a_1 b_1,\ \cdots,\ a_{n-1} b_{n-1})$

- **Hermitian Product:** Given two n-dimensional complex vectors:
  $\vec{a} = (a_0 + i \cdot a'_0,\ a_1 + i \cdot a'_1,\ \ldots,\ a_{n-1} + i \cdot a'_{n-1})$
  $\vec{b} = (b_0 + i \cdot b'_0,\ b_1 + i \cdot b'_1,\ \ldots,\ b_{n-1} + i \cdot b'_{n-1})$

  Hermitian product is a dot product with the 2nd operand as a conjugate (§A-12.2):
  $\langle \vec{a}, \vec{b} \rangle = \vec{a} \cdot \overline{\vec{b}}$
  $= (a_0 + i \cdot a'_0,\ a_1 + i \cdot a'_1,\ \cdots,\ a_{n-1} + i \cdot a'_{n-1}) \cdot (b_0 - i \cdot b'_0,\ b_1 - i \cdot b'_1,\ \cdots,\ b_{n-1} - i \cdot b'_{n-1})$

A-11.2 Various Types of Matrix

⟨Definition A-11.2⟩ Matrices

- An $n \times n$ **identity matrix** and a **reverse identity matrix** are defined as:

$$I_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}, I_n^R = \begin{bmatrix} 0 & \cdots & 0 & 0 & 1 \\ 0 & \cdots & 0 & 1 & 0 \\ 0 & \cdots & 1 & 0 & 0 \\ \vdots & \iddots & \vdots & \vdots & \vdots \\ 1 & 0 & 0 & \cdots & 0 \end{bmatrix}$$

- The **transpose of a matrix** X is defined as element-wise swapping along the diagonal line, denoted as X^T, which is:

$$X = \begin{bmatrix} a_1 & a_2 & a_3 & \cdots & a_n \\ b_1 & b_2 & b_3 & \cdots & b_n \\ c_1 & c_2 & c_3 & \cdots & c_n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ m_1 & m_2 & m_3 & \cdots & m_n \end{bmatrix}, X^T = \begin{bmatrix} a_1 & b_1 & c_1 & \cdots & m_1 \\ a_2 & b_2 & c_2 & \cdots & m_2 \\ a_3 & b_3 & c_3 & \cdots & m_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_n & b_n & c_n & \cdots & m_n \end{bmatrix}$$

- A **Vandermonde matrix** is an $(m+1) \times (n+1)$ matrix defined as:

$$Vander(x_0, x_1, \cdots, x_m) = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \cdots & x_m^n \end{bmatrix}$$

A-11.3 Matrix Arithmetic

Matrix-to-vector multiplication and matrix-to-matrix multiplication are defined as follows:

⟨Definition A-11.3⟩ Matrix Arithmetic

- **Matrix-to-Vector Multiplication:** Given a $m \times n$ matrix A and a n-dimensional vector x:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n} \\ a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n} \end{bmatrix} = \begin{bmatrix} \vec{a}_{1,*} \\ \vec{a}_{2,*} \\ \vec{a}_{3,*} \\ \vdots \\ \vec{a}_{m,*} \end{bmatrix}, \ \vec{x} = (x_1, x_2, \cdots, x_n)$$

  The result of $A \cdot \vec{x}$ is an m-dimensional vector computed as:

$$A \cdot \vec{x} = \left(\vec{a}_{1,*} \cdot \vec{x}, \ \vec{a}_{2,*} \cdot \vec{x}, \ \cdots, \ \vec{a}_{m,*} \cdot \vec{x} \right) = \left(\sum_{i=1}^{n} a_{1,i} \cdot x_i, \ \sum_{i=1}^{n} a_{2,i} \cdot x_i, \cdots, \ \sum_{i=1}^{n} a_{m,i} \cdot x_i \right)$$

- **Matrix-to-Matrix Multiplication:** Given a $m \times n$ matrix A and a $n \times k$ matrix B:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & a_{2,3} & \cdots & a_{2,n} \\ a_{3,1} & a_{3,2} & a_{3,3} & \cdots & a_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & a_{m,3} & \cdots & a_{m,n} \end{bmatrix}, B = \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & \cdots & b_{1,k} \\ b_{2,1} & b_{2,2} & b_{2,3} & \cdots & b_{2,k} \\ b_{3,1} & b_{3,2} & b_{3,3} & \cdots & b_{3,k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ b_{n,1} & b_{n,2} & b_{n,3} & \cdots & b_{n,k} \end{bmatrix}$$

The result of $A \cdot B$ is a $m \times k$ matrix computed as:

$$A \cdot B = \begin{bmatrix} \sum_{i=1}^{n} a_{1,i}b_{i,1} & \sum_{i=1}^{n} a_{1,i}b_{i,2} & \sum_{i=1}^{n} a_{1,i}b_{i,3} & \cdots & \sum_{i=1}^{n} a_{1,i}b_{i,k} \\ \sum_{i=1}^{n} a_{2,i}b_{i,1} & \sum_{i=1}^{n} a_{2,i}b_{i,2} & \sum_{i=1}^{n} a_{2,i}b_{i,3} & \cdots & \sum_{i=1}^{n} a_{2,i}b_{i,k} \\ \sum_{i=1}^{n} a_{3,i}b_{i,1} & \sum_{i=1}^{n} a_{3,i}b_{i,2} & \sum_{i=1}^{n} a_{3,i}b_{i,3} & \cdots & \sum_{i=1}^{n} a_{3,i}b_{i,k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^{n} a_{m,i}b_{i,1} & \sum_{i=1}^{n} a_{m,i}b_{i,2} & \sum_{i=1}^{n} a_{m,i}b_{i,3} & \cdots & \sum_{i=1}^{n} a_{m,i}b_{i,k} \end{bmatrix}$$

Given the above definitions of matrix and vector arithmetic, the following algebraic properties can be derived:

⟨Theorem A-11.3⟩ Matrix Arithmetic Properties

- **Associative:**
  $(AB)C = A(BC)$
  $A(Bx) = (AB)x$

- **Distributive:**
  $A(x + y) = Ax + Ay$

  These linearity identities do not assert preservation of dot, Hermitian, or Hadamard products. For example, with $A = 2I$, $x = y \neq 0$, one generally has $A(x \odot y) \neq (Ax) \odot (Ay)$. This is a failure of a universal identity, not an inequality valid for every input: $A = I$ does preserve the Hadamard product. Expressions such as $A\langle x, y\rangle$ are scalar multiples of a matrix, whereas $\langle Ax, Ay\rangle$ is a scalar; they must not be compared as two outputs of the same vector map.

- **NOT Commutative:**
  Matrix multiplication is not commutative in general. Both products must first be defined and have compatible shapes; some matrices commute, for example $AI = IA$. For a column vector x, the expression xA may be undefined even when Ax is defined.

Proof.

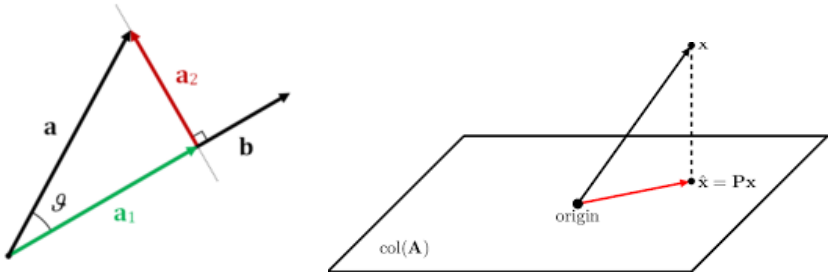
The properties described in A-11.3 can be demonstrated by expanding the formulas on both sides of each equation using a variable representation for each element in the vectors/matrices and comparing the resulting formulas. We leave this expansion as an exercise for the reader.

□

A-11.4 Projection

In this subsection the scalar field is $\mathbb{R}$. Every vector onto which we project, and every vector in an orthogonal basis, is assumed nonzero.

We consider orthogonal projection onto a line (vector projection) and onto a higher-dimensional subspace.



(a) Vector Projection (Source) (b) Orthogonal (Plane/Subspace) Projection (Source)

Figure 4

Vector Projection: For real vectors $\vec{a}$ and $\vec{b} \neq 0$, the signed scalar component along $\vec{b}$ is $c = (\vec{a} \cdot \vec{b})/\|\vec{b}\|$. Thus

$$\mathsf{Proj}_{\vec{b}}(\vec{a}) = c\frac{\vec{b}}{\|\vec{b}\|} = \frac{\vec{a} \cdot \vec{b}}{\|\vec{b}\|^2}\vec{b}.$$

For $\vec{a} \neq 0$, the length of this projection is $\|\vec{a}\|\,|\cos\theta|$; $\|\vec{a}\|\cos\theta$ is the signed component, which can be negative. Figure 4a illustrates the projection.

Orthogonal Projection: Given the vector $\vec{x}$ and a set of mutually orthogonal vectors $\vec{p}_0, \vec{p}_1, \cdots, \vec{p}_{n-1}$ that span the subspace P, the orthogonal projection $\mathsf{Proj}_P(\vec{x})$ measures how much of $\vec{x}$ lies in P (i.e., the component of $\vec{x}$ within P). In the example, $\vec{x}$'s projection onto P (red arrow) equals the sum of the projections of $\vec{x}$ onto each orthogonal basis vector $\vec{p}_i$ spanning P: $\mathsf{Proj}_P(\vec{x}) = \sum_{i=0}^{n-1} \mathsf{Proj}_{\vec{p}_i}(\vec{x})$. This computation can be viewed as expressing $\vec{x}$ in a coordinate system defined by the n orthogonal vectors.

⟨Definition A-11.4⟩ Vector and Orthogonal Projections

- **Vector Projection:** Given two vectors $\vec{a}$ and $\vec{b}$ in the same vector space, the vector projection of $\vec{a}$ on $\vec{b}$ is:
  $\mathsf{Proj}_{\vec{b}}(\vec{a}) = \vec{a}_p = \dfrac{\vec{a} \cdot \vec{b}}{||b||^2}\vec{b}$ ▷ where $||\vec{a}_p|| = ||\vec{a}||\,|\cos\theta|$
- **Orthogonal Basis:** If the n-dimensional plane (or subspace) P is spanned by the mutually orthogonal n-dimensional vectors $\vec{p}_0, \vec{p}_1, \cdots, \vec{p}_{n-1}$,
  then the matrix $P = \begin{bmatrix} \vec{p}_0 \\ \vec{p}_1 \\ \vdots \\ \vec{p}_{n-1} \end{bmatrix}$ is defined to be an orthogonal basis of plane P.

- **Orthogonal Projection:** Given the orthogonal basis matrix $P = \begin{bmatrix} \vec{p}_0 \\ \vec{p}_1 \\ \vdots \\ \vec{p}_{n-1} \end{bmatrix}$,

  vector $\vec{a}$'s orthogonal projection on P is:
  $\mathsf{Proj}_P(\vec{a}) = \sum_{i=0}^{n-1} \mathsf{Proj}_{\vec{p}_i}(\vec{a})$

Based on the definition of orthogonal projection, the following properties are derived:

Orthogonal Basis: In an n-dimensional vector space, any mutually orthogonal n vectors in the vector space span the subspace P that is identical to the entire vector space. Further, the orthogonal projection of any vector in the vector space on P is guaranteed to be a unique vector.

Non-orthogonal Basis: Let the rows of the invertible matrix P be a basis $\vec{p}_0, \ldots, \vec{p}_{n-1}$ of $\mathbb{R}^n$. If $\vec{v}$ is a column vector, its coordinates in this basis are $(P^T)^{-1}\vec{v}$, since $\vec{v} = P^T\vec{c}$; in general $P\vec{v}$ is a different invertible linear transformation, not these basis coordinates. Orthogonal projection onto the span remains well-defined even when the chosen basis is non-orthogonal. For a full-column-rank real matrix Q whose columns span a subspace, it is $Q(Q^TQ)^{-1}Q^T\vec{v}$. For a basis of the whole space the projection is simply $\vec{v}$. Only the sum of individual vector-projection formula requires an orthogonal basis.

⟨Theorem A-11.4⟩ Uniqueness of Transformed Vectors

- **Orthogonal Basis:** If some n vectors are a orthogonal basis of the plane P in the n-dimensional vector space, then P is the same as the entire vector space, and $\mathsf{Proj}_P(v)$ for every vector $\vec{v}$ in the vector space is guaranteed to be a unique vector.
- **Non-orthogonal Basis:** For a basis stored as the rows of P, every vector $\vec{v}$ has the unique coordinate vector $(P^T)^{-1}\vec{v}$. The orthogonal projection onto the entire space spanned by this basis is $\vec{v}$ itself.

A-11.5 Basis of a Polynomial Ring

The ring $\mathbb{Z}[X]/(X^n + 1)$ is a free $\mathbb{Z}$-module of rank n, not a vector space over $\mathbb{Z}$. A basis is a list $f_0, \ldots, f_{n-1}$ for which each element has a unique integer linear combination. Equivalently, require:

- **Linear Independence**: If $\sum_i z_i f_i = 0$ with $z_i \in \mathbb{Z}$, then every $z_i = 0$.
- **Spanning the Polynomial Ring:** A linear combination of the polynomials in the basis set can express any polynomial in the polynomial ring

Note that for a $(n-1)$-degree polynomial ring, the number of polynomials that form a basis of the polynomial ring is exactly n.

A-11.6 Isomorphism between Polynomials and Vectors over Integers (and Ring)

Let n be a power of 2, let t be prime with $2n \mid t-1$, and fix the n distinct roots $x_0, \ldots, x_{n-1}$ of $X^n + 1$ in $\mathbb{Z}_t$. Define σ by evaluation at these roots. Distinctness gives interpolation of degree less than n (§A-10.1);

being roots makes evaluation well-defined on the quotient and compatible with multiplication. Evaluation at arbitrary distinct nodes would give a linear bijection of coefficient representatives, but not generally the asserted ring isomorphism. Explicitly:

$\sigma : f(X) \in \mathbb{Z}_t[X]/(X^n+1) \longrightarrow (f(x_0), f(x_1), f(x_2), \cdots, f(x_{n-1})) \in \mathbb{Z}_t^n$

Now, we will explain why the mapping σ is isomorphic, which means that σ is a bijective one-to-one mapping from $\mathbb{Z}_t[X]/(X^n+1)$ to $\mathbb{Z}_t^n$, and it preserves the algebraic operations $(+, \cdot)$ (i.e., σ is a homomorphism for addition and multiplication).

Bijective: In the $(n-1)$-degree polynomial ring, a list of y values evaluated at some statically chosen n distinct x coordinates defines a unique polynomial because, algebraically, there exists only one $(n-1)$-degree (or a lesser degree) polynomial that passes through each given set of n distinct (x, y) coordinates. We proved this in Lagrange Polynomial Interpolation (Theorem A-10 in §A-10).

Homomorphic: The homomorphism of the mapping σ on the $(+, \cdot)$ operations means that the following two relationships hold:

$\sigma(f_a(X) + f_b(X)) = \sigma(f_a(X)) + \sigma(f_b(X))$

$\sigma(f_a(X) \cdot f_b(X)) = \sigma(f_a(X)) \odot \sigma(f_b(X))$ # $\odot$ is Hadamard vector multiplication (Summary A-11.1)

To prove our σ mapping's homomorphism, let's denote the input polynomials $f_a(X)$, $f_b(X)$, and their σ-mapped output vectors as follows:

$f_a(X) = a_0 + a_1X + a_2X^2 + \cdots + a_{n-1}X^{n-1}$

$\sigma(f_a(X)) = (f_a(x_0), f_a(x_1), f_a(x_2), \cdots, f_a(x_{n-1})) = \left(\sum_{i=0}^{n-1} a_i(x_0)^i, \sum_{i=0}^{n-1} a_i(x_1)^i, \sum_{i=0}^{n-1} a_i(x_2)^i, \cdots, \sum_{i=0}^{n-1} a_i(x_{n-1})^i\right)$

$f_b(X) = b_0 + b_1X + b_2X^2 + \cdots + b_{n-1}X^{n-1}$

$\sigma(f_b(X)) = (f_b(x_0), f_b(x_1), f_b(x_2), \cdots, f_b(x_{n-1})) = \left(\sum_{i=0}^{n-1} b_i(x_0)^i, \sum_{i=0}^{n-1} b_i(x_1)^i, \sum_{i=0}^{n-1} b_i(x_2)^i, \cdots, \sum_{i=0}^{n-1} b_i(x_{n-1})^i\right)$

Given the above setup, we can see that σ preserves homomorphism on the $(+)$ operation as follows:

$\boldsymbol{\sigma(f_a(X) + f_b(X))} = \boldsymbol{\sigma}\Big((a_0+b_0) + (a_1+b_1)X + (a_2+b_2)X^2 + \cdots + (a_{n-1}+b_{n-1})X^{n-1}\Big)$

$= \left(\sum_{i=0}^{n-1}(a_i+b_i)(x_0)^i, \sum_{i=0}^{n-1}(a_i+b_i)(x_1)^i, \sum_{i=0}^{n-1}(a_i+b_i)(x_2)^i, \cdots, \sum_{i=0}^{n-1}(a_i+b_i)(x_{n-1})^i\right)$

$= \left(\sum_{i=0}^{n-1} a_i(x_0)^i, \sum_{i=0}^{n-1} a_i(x_1)^i, \sum_{i=0}^{n-1} a_i(x_2)^i, \cdots, \sum_{i=0}^{n-1} a_i(x_{n-1})^i\right)$

$+ \left(\sum_{i=0}^{n-1} b_i(x_0)^i, \sum_{i=0}^{n-1} b_i(x_1)^i, \sum_{i=0}^{n-1} b_i(x_2)^i, \cdots, \sum_{i=0}^{n-1} b_i(x_{n-1})^i\right)$

$= \boldsymbol{\sigma(f_a(X)) + \sigma(f_b(X))}$

Also, we can see that σ preserves homomorphism on the $(\cdot)$ operation as follows:

$\boldsymbol{\sigma(f_a(X) \cdot f_b(X))} = \boldsymbol{\sigma}\left(\left(\sum_{i=0}^{n-1} a_iX^i\right) \cdot \left(\sum_{i=0}^{n-1} b_iX^i\right)\right)$

$= \left(\left(\sum_{i=0}^{n-1} a_ix_0^i\right)\left(\sum_{i=0}^{n-1} b_ix_0^i\right), \left(\sum_{i=0}^{n-1} a_ix_1^i\right)\left(\sum_{i=0}^{n-1} b_ix_1^i\right), \left(\sum_{i=0}^{n-1} a_ix_2^i\right)\left(\sum_{i=0}^{n-1} b_ix_2^i\right),\right.$

$$\cdots, \left(\sum_{i=0}^{n-1} a_i x_{n-1}^i\right)\left(\sum_{i=0}^{n-1} b_i x_{n-1}^i\right)\Bigg)$$
$$= \left(\sum_{i=0}^{n-1} a_i (x_0)^i,\ \sum_{i=0}^{n-1} a_i (x_1)^i,\ \cdots,\ \sum_{i=0}^{n-1} a_i (x_{n-1})^i\right)$$
$$\odot \left(\sum_{i=0}^{n-1} b_i (x_0)^i,\ \sum_{i=0}^{n-1} b_i (x_1)^i,\ \cdots,\ \sum_{i=0}^{n-1} b_i (x_{n-1})^i\right)$$
$$= \boldsymbol{\sigma}(f_a(X)) \odot \boldsymbol{\sigma}(f_b(X))$$

In summary, σ preserves the following homomorphism:
$\sigma(f_a(X) + f_b(X)) = \sigma(f_a(X)) + \sigma(f_b(X))$
$\sigma(f_a(X) \cdot f_b(X)) = \sigma(f_a(X)) \odot \sigma(f_b(X))$

However, for $\sigma(f_a(X) \cdot f_b(X)) = \sigma(f_a(X)) \odot \sigma(f_b(X))$, we need further reasoning to justify that this relation holds in polynomial rings, which is explained below.

Polynomial Ring Reduction: Suppose that we did not have the polynomial ring setup $X^n + 1$. Then, if we multiply $f_a(X)$ and $f_b(X)$, then $f_a(X) \cdot f_b(X)$ may become a new polynomial whose degree is higher than $n - 1$. This higher-degree polynomial would still decode into the expected correct vector. Suppose the following:
$\sigma(f_a(X)) = (f_a(x_0), f_a(x_1), \cdots, f_a(x_{n-1})) = (v_0, v_1, \cdots, v_{n-1})$
$\sigma(f_b(X)) = (f_b(x_0), f_b(x_1), \cdots, f_b(x_{n-1})) = (u_0, u_1, \cdots, u_{n-1})$

Then, the following is true:
$\sigma(f_a(X) \cdot f_b(X)) = (f_a(x_0) \cdot f_b(x_0), f_a(x_1) \cdot f_b(x_1), \cdots, f_a(x_{n-1}) \cdot f_b(x_{n-1})) = (v_0 u_0, v_1 u_1, \cdots, v_{n-1} u_{n-1})$
$= (v_0, v_1, \cdots, v_{n-1}) \odot (u_0, u_1, \cdots, u_{n-1})$

As shown above, even if $f_a(X) \cdot f_b(X)$ results in a polynomial with a degree higher than $n - 1$, it can be decoded into the expected correct vector. However, the σ mapping loses the property of isomorphism between a polynomial and a vector because if a polynomial's degree is higher than $n - 1$, then there can be more than 1 polynomial that passes through the given n distinct X coordinates: $\{x_0, x_1, \cdots, x_{n-1}\}$. This is a problem because, if the σ mapping supports only polynomial-to-vector mappings and not vector-to-polynomial mappings, then we cannot convert vectors into polynomials in the first place and do isomorphic computations. Another minor issue is that if the polynomial degree term is higher than $n - 1$, then the computational overhead of decoding (i.e., polynomial evaluation) becomes larger than before.

To resolve these two minor issues, we let the n distinct X coordinates of evaluation be the solutions of the polynomial ring modulo $X^n + 1$ (where n is some power of 2) and reduce $f_a(X) \cdot f_b(X)$ to a new polynomial modulo $X^n + 1$ whose degree is at most $n - 1$. Let $f_{ab}(X) = f_a(X) \cdot f_b(X)$, and $f'_{ab}(X)$ be the reduced polynomial such that $f_{ab}(X) = Q(X) \cdot (X^n + 1) + f'_{ab}(X)$ for some quotient polynomial $Q(X)$. Then, as illustrated in Summary A-5.1.2 (§A-5.1.2), $f_{ab}(X)$ and $f'_{ab}(X)$ evaluate to the same value if they are evaluated at the roots of $X^n + 1$ (by zeroing out the $Q(X)$ term). Therefore, if we let the n distinct evaluating points $\{x_0, x_1, \cdots, x_{n-1}\}$ be the roots of $X^n + 1$, then we can ensure that the decoded vector of $f'_{ab}(X)$ is identical to that of $f_{ab}(X)$, which we expect. Therefore, we can replace the higher-degree polynomial $f_{ab}(X)$ with the reduced polynomial $f'_{ab}(X)$ and continue with any further polynomial additions or multiplications using $f'_{ab}(X)$. Also, by applying polynomial ring reduction, we

can enhance the computational efficiency of polynomial addition and multiplication, as well as preserve the isomorphism of the σ mapping. Therefore, we can freely convert between vectors & polynomials and perform additions and multiplications.

For applying this polynomial ring reduction, the polynomial modulus can be a degree-n polynomial that splits into n distinct roots in the coefficient field. In practice, we often choose $X^n + 1$ as the polynomial ring modulus, which is the $(\mu = 2n)$-th cyclotomic polynomial (§A-8.1). The reason we let the polynomial ring modulus be a cyclotomic polynomial (especially the $(\mu = 2n)$-th cyclotomic polynomial, $X^n + 1$) is that its n distinct roots are well-defined (i.e., primitive $(\mu = 2n)$-th roots of unity) and thus can be quickly identified even when n is large.

Polynomial Coefficient Modulo Reduction: In addition, we often reduce the polynomial coefficients based on some modulus t to keep the size of the coefficients lower than a certain limit for the purpose of computational efficiency. Suppose two polynomials $f_c(X)$ and $f_d(X)$ have congruent coefficients modulo t as follows:

$f_c(X) = w_0 + w_1 \cdot X + w_2 \cdot X^2 + \cdots + w_{n-1} \cdot X^{n-1}$
$f_d(X) = w'_0 + w'_1 \cdot X + w'_2 \cdot X^2 + \cdots + w'_{n-1} \cdot X^{n-1}$
$w_i \equiv w'_i \bmod t$

Then, their evaluated value $f_c(x)$ and $f_d(x)$ for any $x \in \mathbb{Z}$ is guaranteed to be congruent modulo t, as shown below:

$f_c(x) = w_0 + w_1 \cdot x + w_2 \cdot x^2 + \cdots + w_{n-1} \cdot x^{n-1}$
$\equiv (w_0 + w_1 \cdot x + w_2 \cdot x^2 + \cdots + w_{n-1} \cdot x^{n-1}) \bmod t$
$\equiv (w_0 \bmod t) + (w_1 \bmod t) \cdot x + (w_2 \bmod t) \cdot x^2 + \cdots + (w_{n-1} \bmod t) \cdot x^{n-1}$
$\equiv w'_0 + w'_1 \cdot x + w'_2 \cdot x^2 + \cdots + w'_{n-1} \cdot x^{n-1}$
$= f_d(x)$

Summary: Since σ is bijective and homomorphic, σ is an isomorphic mapping between the $(n-1)$-degree polynomial ring $\mathbb{Z}_t[X]/(X^n + 1)$ and the n-dimensional vector space $\mathbb{Z}_t^n$.

A-11.6.1 Finding Appropriate Modulus t

To isomorphically evaluate a polynomial in $\mathbb{Z}_t[X]/(X^n + 1)$ into an n-dimensional vector, we need to evaluate the polynomial at n distinct roots of $X^n + 1 \bmod t$. However, $X^n + 1 \bmod t$ does not have n distinct roots for all combinations of (degree, modulus) $= (n, t)$. For example, if $n = 2$ and $t = 3$, then $X^2 + 1 \not\equiv 0 \bmod 3$ for any possible values of $X = \{0, 1, 2\}$. Therefore, our goal is to find a satisfactory t given a fixed n such that n distinct roots of $X^n + 1 \pmod t$ exist in order to use the isomorphic σ mapping.

We start with two constraints: (1) choose t to be a prime number; (2) ensure $t - 1$ is a multiple of $2n$.

We learned from Fermat's Little Theorem in Theorem A-4.2.4 (§A-4.2) the following: $a^{t-1} \equiv 1 \bmod t$ if and only if a and t are co-prime. This means that if t is a prime, then $a^{t-1} \equiv 1 \bmod t$ for all $a \in \mathbb{Z}_t^\times$ (i.e., $\mathbb{Z}_t$ without $\{0\}$). Suppose g is the generator of $\mathbb{Z}_t^\times$ whose powered values generate all elements of $\mathbb{Z}_t^\times$. Then, $\mathsf{Ord}_{\mathbb{Z}_t}(g) = t - 1$ and $g^{t-1} \equiv 1 \bmod t$. Since $t - 1 = k \cdot 2n$ for some k, $g^{k \cdot 2n} \equiv (g^k)^{2n} \equiv 1 \bmod t$. Then, $\mathsf{Ord}_{\mathbb{Z}_t}(g^k) \leq 2n$. However, since $\mathsf{Ord}_{\mathbb{Z}_t}(g) = t-1$, for all integers a such that $1 \leq a < t-1 = k \cdot 2n$, $g^a \not\equiv 1 \bmod t$. In other words, for all integers b such that $1 \leq b < 2n$, $(g^k)^b \not\equiv 1 \bmod t$. Thus, $\mathsf{Ord}_{\mathbb{Z}_t}(g^k) = 2n$.

Let $c = g^k$. Since $\mathsf{Ord}_{\mathbb{Z}_t}(c) = 2n$, $c^{2n} \equiv 1 \bmod t$. In other words, $(c^n)^2 \equiv 1 \bmod t$. Now, c^n can be only 1 or -1. The reason is that in the relation $X^2 \equiv 1 \bmod t$, X can be mathematically only 1 or $-1 \equiv t-1 \bmod t$. If we substitute $X = c^n$, then c^n can be only 1 or $-1 \equiv t-1 \bmod t$. But $\mathsf{Ord}_{\mathbb{Z}_t}(c) = 2n$, thus c^n cannot be 1 (because $c^n = 1$ would contradict $\mathrm{ord}(c) = 2n$). Thus, c^n can be only $-1 \equiv t-1 \bmod t$. If $c^n = -1 \equiv t-1 \bmod t$, then c is the root of $X^n + 1 \bmod t$, because $X^n + 1 = c^n + 1 \equiv (t-1) + 1 \equiv 0 \bmod t$.

In conclusion, given a cyclotomic polynomial $X^n + 1$, if we choose a prime t such that $t - 1 = k \cdot 2n$ for some integer k, then one root of $X^n + 1 \bmod t$ is: $X = c = g^k$.

Once we have found one root of $X^n + 1$, then we can derive all n distinct roots of $X^n + 1$. Suppose ω is one root of $X^n + 1 \bmod t$. Then we derive the following:

$\omega^n + 1 \equiv 0 \bmod t$

$\omega^n \equiv t - 1 \bmod t$

$\omega^{2n} \equiv (t-1) \cdot (t-1) \equiv t^2 - 2t + 1 \equiv 1 \bmod t$

Here $\omega^{2n} = 1$ and $\omega^n = -1 \neq 1$ in $\mathbb{Z}_t$, with odd prime t. Since n is a power of two, every proper divisor of $2n$ divides n. Hence the order of ω cannot be a proper divisor, so $\mathrm{ord}(\omega) = 2n$. This inference uses the power-of-two assumption; for general n, a root of $X^n + 1$ need not be primitive.

Now, we derive the remaining $n - 1$ distinct roots of $X^n + 1 \bmod t$ as follows:

$(\omega^3)^n + 1 \equiv (\omega^{2n}) \cdot \omega^n + 1 \equiv \omega^n + 1 \equiv t - 1 + 1 \equiv 0 \bmod t$

$(\omega^5)^n + 1 \equiv (\omega^{4n}) \cdot \omega^n + 1 \equiv \omega^n + 1 \equiv t - 1 + 1 \equiv 0 \bmod t$

$\vdots$

$(\omega^{2n-1})^n + 1 \equiv 0 \bmod t$

$\triangleright$ for any odd $k = 2j + 1$, $(\omega^{2j+1})^n = (\omega^{2n})^j \cdot \omega^n = 1^j \cdot (-1) = -1$, so $(\omega^k)^n + 1 = 0$

Note that $\omega, \omega^3, \omega^5, \cdots, \omega^{2n-1}$ are all distinct values in $\mathbb{Z}_t^\times$, because $\mathsf{Ord}_{\mathbb{Z}_t}(\omega) = 2n$. Thus, $\{\omega^{2i+1}\}_{i=0}^{n-1}$ are n distinct roots of $X^n + 1$. At the same time, since these are the roots of the cyclotomic polynomial $X^n + 1$, these are n distinct primitive $(\mu = 2n)$-th roots of unity.

We summarize our findings as follows:

⟨Theorem A-11.6.1⟩ Isomorphism between Polynomials and Vectors over Integers (Ring)

- Suppose we have an $(n-1)$-degree polynomial ring $\mathbb{Z}_t[X]/F(X)$ where $F(X)$ is an n-degree polynomial having n distinct roots $\{x_0, \cdots, x_{n-1}\}$, and an n-dimensional vector space $\mathbb{Z}_t^n$ (integers mod t) with vector $\vec{v}$.
  $\sigma : f(X) \in \mathbb{Z}_t[X]/F(X) \longrightarrow (f(x_0), f(x_1), f(x_2), \cdots, f(x_{n-1})) \in \mathbb{Z}_t^n$

  Then, σ preserves isomorphism over the $(+, \cdot)$ operations:
  $\sigma(f_a(X) + f_b(X)) = \sigma(f_a(X)) + \sigma(f_b(X))$
  $\sigma(f_a(X) \cdot f_b(X)) = \sigma(f_a(X)) \odot \sigma(f_b(X))$

- Suppose we have the $(\mu = 2n)$-th cyclotomic polynomial $X^n + 1 \bmod t$ such that t is a prime and $t - 1$ is some multiple of $2n$, and g is a generator of $\mathbb{Z}_t^\times$. Then, n distinct roots of $X^n + 1$ (i.e., primitive $(\mu = 2n)$-th roots of unity) can be efficiently computed as: $\{\omega^{2i+1}\}_{i=0}^{n-1}$ where

$\omega = g^{\frac{t-1}{2n}} \bmod t$.

A-11.7 Isomorphism between Polynomials and Vectors over Complex Numbers

Here $n = 2^r$ with $r \geq 1$, so n is even. All isomorphisms involving $\hat{\mathbb{C}}^n$ are real vector-space and real-algebra isomorphisms.

In Theorem A-11.5 (§A-11.5), we learned the isomorphic mapping

$$\sigma : f(X) \in \mathbb{Z}_t[X]/(X^n + 1) \longrightarrow (f(x_0), f(x_1), f(x_2), \cdots, f(x_{n-1})) \in \mathbb{Z}_t^n$$

, where $x_0, x_1, x_2, \cdots, x_{n-1} \in \mathbb{Z}_t$ are the ($(\mu = 2n)$-th primitive) roots of the cyclotomic polynomial $X^n + 1$, which are $\omega, \omega^3, \omega^5, \cdots, \omega^{2n-1}$, where ω can be any root of $X^n + 1$ (i.e., since each ω is a generator of all roots). In this subsection, we will demonstrate the isomorphism between a vector space and a polynomial ring over complex numbers as follows:

$\sigma_c : f(X) \in \mathbb{R}[X]/(X^n + 1) \longrightarrow (f(\omega), f(\omega^3), f(\omega^5), \cdots, f(\omega^{2n-1})) \in \hat{\mathbb{C}}^n$

, where $X \in \mathbb{C}$, and $\omega = e^{i\pi/n}$, the root (i.e., the primitive $(\mu = 2n)$-th root) of the cyclotomic polynomial $X^n + 1$ over complex numbers (Theorem A-8.2.1 in §A-8.2). We define $\hat{\mathbb{C}}^n$ to be an n-dimensional *special* vector space whose second-half elements of each vector are reverse-ordered conjugates of the first-half elements (e.g., $(v_0, v_1, \cdots, v_{\frac{n}{2}-1}, \overline{v}_{\frac{n}{2}-1}, \cdots, \overline{v}_1, \overline{v}_0)$).

A-11.7.1 Isomorphism between $\mathbb{C}^{\frac{n}{2}}$ and $\hat{\mathbb{C}}^n$

In this section, we treat both $\mathbb{C}^{\frac{n}{2}}$ and $\hat{\mathbb{C}}^n$ as vector spaces of complex vectors, where the mapping between them is defined as $\phi : \mathbb{C}^{\frac{n}{2}} \to \hat{\mathbb{C}}^n$ as follows:

$\phi(z_0, z_1, \ldots, z_{\frac{n}{2}-1}) = (z_0, z_1, \ldots, z_{\frac{n}{2}-1}, \overline{z_{\frac{n}{2}-1}}, \ldots, \overline{z_1}, \overline{z_0})$

In other words, $\hat{\mathbb{C}}^n$ is a conjugate extension of $\mathbb{C}^{\frac{n}{2}}$ such that the first-half elements of $\hat{\mathbb{C}}^n$ are identical to those of $\mathbb{C}^{\frac{n}{2}}$, and the second-half elements of $\hat{\mathbb{C}}^n$ are reverse-ordered conjugates of $\mathbb{C}^{\frac{n}{2}}$.

Now, we will demonstrate that the mapping ϕ between $\mathbb{C}^{\frac{n}{2}}$ and $\hat{\mathbb{C}}^n$ is an isomorphism.

Bijective: The mapping ϕ is trivially bijective. Every vector in $\mathbb{C}^{\frac{n}{2}}$ uniquely maps to a vector in $\hat{\mathbb{C}}^n$ by appending its reverse-ordered complex conjugates. Conversely, any vector in $\hat{\mathbb{C}}^n$ can uniquely map back to $\mathbb{C}^{\frac{n}{2}}$ by simply dropping the second half of its elements, establishing a perfect one-to-one correspondence.

Homomorphic: We will demonstrate that ϕ preserves homomorphism over addition ($+$) and element-wise vector multiplication ($\odot$). Given two vectors $\vec{u}, \vec{v} \in \mathbb{C}^{\frac{n}{2}}$:

- **Addition:** $\phi(\vec{u} + \vec{v}) = \phi(\vec{u}) + \phi(\vec{v})$ holds because complex conjugation distributes over addition (i.e., $\overline{u + v} = \overline{u} + \overline{v}$).
- **Element-wise Multiplication:** $\phi(\vec{u} \odot \vec{v}) = \phi(\vec{u}) \odot \phi(\vec{v})$ holds because the conjugate of a product is the product of the conjugates (i.e., $\overline{u \cdot v} = \overline{u} \cdot \overline{v}$).

Therefore, the mapping $\phi : \mathbb{C}^{\frac{n}{2}} \to \hat{\mathbb{C}}^n$ is an isomorphism.

Vector Space of Complex Vectors with either Real or Complex Scalars

By mathematical definition, a vector space must satisfy *closure under scalar multiplication*: for any vector $\vec{v}$ in the space and any scalar c from the scalar field, the product $c \cdot \vec{v}$ must also lie in the same vector space. Under this requirement, $\mathbb{C}^{\frac{n}{2}}$ is naturally a vector space of complex vectors with complex scalars (though the same is true with real scalars, since complex includes real). On the other hand, $\hat{\mathbb{C}}^n$ is a vector space of complex vectors *strictly with real scalars.*

To see why, recall that a vector $\vec{v} = (v_0, v_1, \ldots, v_{n-1}) \in \hat{\mathbb{C}}^n$ must satisfy the Hermitian-symmetry constraint: $v_{n-1-k} = \overline{v_k}$ for all k. Consider an example with $n = 4$:

$$\vec{v} = (1 + i,\ 2 + 2i,\ 2 - 2i,\ 1 - i) \in \hat{\mathbb{C}}^4$$

Now, let us multiply $\vec{v}$ by the complex scalar $c = i$:

$$i \cdot \vec{v} = (-1 + i,\ -2 + 2i,\ 2 + 2i,\ 1 + i)$$

For this result to still lie in $\hat{\mathbb{C}}^4$, the third entry must be the conjugate of the second entry. But $\overline{-2 + 2i} = -2 - 2i$, which is not equal to $2 + 2i$. So $i \cdot \vec{v} \notin \hat{\mathbb{C}}^4$, and closure fails.

More generally, for any nonzero $\vec{v} \in \hat{\mathbb{C}}^n$ and any scalar c, the product $c \cdot \vec{v}$ preserves Hermitian symmetry if and only if $\overline{c} = c$; that is, if and only if c is a real number. Therefore, $\hat{\mathbb{C}}^n$ is a valid vector space only when restricted to real scalars, and this is why $\hat{\mathbb{C}}^n$ is a vector space of complex vectors strictly with real scalars.

Throughout this book, we will regard both $\mathbb{C}^{\frac{n}{2}}$ and $\hat{\mathbb{C}}^n$ as vector spaces of complex vectors with real scalars.

Isomorphism of Vector Spaces: For a mapping between vector spaces to be an isomorphism, it has to be not only bijective and homomorphic over the $(+, \cdot)$ operations, but also homomorphic over scalar multiplication. Therefore, we need to prove that $\phi(c \cdot \vec{u}) = c \cdot \phi(\vec{u})$. This is always true in our case because the scalar c is a real number, meaning it is unaffected by complex conjugation (i.e., $\overline{c} = c$, and thus $\overline{c \cdot u_k} = c \cdot \overline{u_k}$).

Dimension of Vector Spaces: The dimension of a vector space is defined by how many scalars it takes to fully describe it. Therefore, the dimension of $\mathbb{C}^{\frac{n}{2}}$ is n because it has $\frac{n}{2}$ elements where each element requires 2 scalars to describe (one real and one imaginary part). The dimension of $\hat{\mathbb{C}}^n$ is also n because the first $\frac{n}{2}$ elements requires $\frac{n}{2} \cdot 2$ scalars to describe, and the latter $\frac{n}{2}$ elements are simply reverse-ordered conjugates of the former $\frac{n}{2}$ elements.

A-11.7.2 Isomorphism between $\hat{\mathbb{C}}^n$ and $\mathbb{R}[X]/(X^n+1)$

Now, we will demonstrate that σ_c is an isomorphism (i.e., bijective and homomorphic) between $\hat{\mathbb{C}}^n$ and $\mathbb{R}[X]/(X^n+1)$ by applying the same reasoning as described in the beginning of §A-11.6.

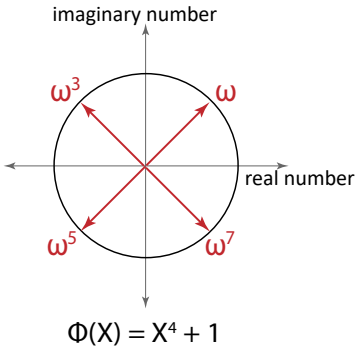


Figure 5: An illustration of the four roots of the 8th cyclotomic polynomial x^4+1

Bijective: Based on Euler's formula $e^{i\theta} = \cos\theta + i \cdot \sin\theta$ (§A-12.3), we can derive the following arithmetic relations: $\omega = \overline{\omega^{2n-1}}$, $\omega^3 = \overline{\omega^{2n-3}}, \cdots, \omega^{n-1} = \overline{\omega^{n+1}}$. In other words, the one-half roots are conjugates of the other-half roots. This can also be pictorially understood based on a complex plane in Figure 5, where red arrows represent the roots of the 8th cyclotomic polynomial X^4+1, comprising imaginary number and real number components. As shown in this figure, one half of the red arrows (i.e., roots) are a reflection of the other half on the x-axis (i.e., real number axis). This means that we can express these roots as an n-dimensional vector whose elements are the roots of X^n+1, such that its second-half elements are a reverse-ordered conjugate of the first-half elements. Based on this vector design, the σ_c mapping can be re-written as follows:

$$\sigma_c(f(X)) = (f(\omega), f(\omega^3), f(\omega^5), \cdots, f(\omega^{n-1}), f(\overline{\omega^{n-1}}), f(\overline{\omega^{n-3}}), \cdots, f(\overline{\omega^3}), f(\overline{\omega}))$$

Since $f(\overline{X}) = \overline{f(X)}$ (because $f(X)$ has strictly real coefficients), we can rewrite σ_c as:

$$\sigma_c(f(X)) = (f(\omega), f(\omega^3), f(\omega^5), \cdots, f(\omega^{n-3}), f(\omega^{n-1}), \overline{f(\omega^{n-1})}, \overline{f(\omega^{n-3})}, \cdots, \overline{f(\omega^3)}, \overline{f(\omega)})$$

This structure of vector exactly aligns with the definition of $\hat{\mathbb{C}}^n$: the second half of the elements of the n-dimensional vector $\vec{v}$ is a reverse-ordered conjugate of the first half.

To prove bijectivity, first note that distinct real polynomials of degree less than n cannot agree at the n distinct roots, by Lagrange interpolation (Theorem A-10 in §A-10). For surjectivity, take any $\vec{v} \in \hat{\mathbb{C}}^n$ and interpolate its values at the ordered roots to obtain a unique $f \in \mathbb{C}[X]$ of degree less than n. Conjugating the coefficients of f produces another polynomial with the same evaluations, because both nodes and values occur in reversed conjugate pairs. Uniqueness forces that conjugated polynomial to equal f, so every coefficient of f is real. Hence every $\vec{v} \in \hat{\mathbb{C}}^n$ has a real-polynomial preimage.

Homomorphic: σ_c is homomorphic, because based on the reasoning shown in §A-11.6, the relations $\sigma_c(f_a(X) + f_b(X)) = \sigma_c(f_a(X)) + \sigma_c(f_b(X))$ and $\sigma_c(f_a(X) \cdot f_b(X)) = \sigma_c(f_a(X)) \odot \sigma_c(f_b(X))$ mathematically hold regardless of whether the type of X is a modulo integer or a complex number. Additionally, for any real scalar $c \in \mathbb{R}$, $\sigma_c(c \cdot f(X)) = c \cdot \sigma_c(f(X))$ holds because scaling the polynomial and then evaluating it is the same as evaluating it and then scaling the result.

Since σ_c is both bijective and homomorphic over the $(+, \cdot)$ operations, it is an isomorphism.

⟨Theorem A-11.7⟩ Isomorphism between Polynomials and Vectors over Complex Numbers

The following mapping σ_c between polynomials and vectors over complex numbers is an isomorphism:
$\sigma_c : f(X) \in \mathbb{R}[X]/(X^n + 1) \longrightarrow (f(\omega), f(\omega^3), f(\omega^5), \cdots, f(\omega^{2n-1})) \in \hat{\mathbb{C}}^n \ (\cong \mathbb{C}^{\frac{n}{2}})$

, where $\omega = e^{i\pi/n}$, the root (i.e., the primitive ($\mu = 2n$)-th root) of the cyclotomic polynomial $X^n + 1$ over complex numbers, and $\hat{\mathbb{C}}^n$ is an n-dimensional vector space of complex vectors (with real scalars) whose second-half elements are reverse-ordered conjugates of the first-half elements.

A-11.8 Transforming Basis between Polynomial Ring and Vector Space of Complex Vectors with Real Scalars

A linear isomorphism over a fixed scalar field sends a basis to a basis: linearity preserves all linear combinations, injectivity preserves linear independence, and surjectivity preserves spanning. The evaluation map σ above is linear over $\mathbb{Z}_t$ with prime t, and σ_c is linear over $\mathbb{R}$. Those are the scalar fields used in the following basis-transfer argument.

The same principle holds between a polynomial ring and vector space of complex numbers (with real scalars). Given the polynomial ring $\mathbb{R}[X]/(x^n + 1)$, the most intuitive way to set up a basis of $\mathbb{R}[X]/(x^n + 1)$ is as follows:

$f_0(X) = 1$
$f_1(X) = X$
$f_2(X) = X^2$
$\vdots$
$f_{n-1}(X) = X^{n-1}$

These n polynomials are linearly independent, because each polynomial exclusively has its own unique exponent term, whereas one term cannot be expressed by a linear combination of the other terms. Also, these n polynomials span the polynomial ring $\mathbb{R}[X]/(x^n + 1)$, because each polynomial's scalar multiplication can express any coefficient value of its own exponent term, and summing all such polynomials can express any polynomial in the polynomial ring $\mathbb{R}[X]/(X^n + 1)$.

Now, we will apply the σ_c mapping to the above n polynomials that are a basis of the $(n-1)$-degree polynomial ring $\mathbb{R}[X]/(x^n + 1)$. Then, according to the principle of polynomial-to-vector basis transfer (explained in Theorem A-11.5 in §A-11.5), we can use these n polynomials (i.e., the basis of the $(n-1)$-degree polynomial ring) and the isomorphic polynomial-to-vector mapping σ_c to compute the basis of the n-dimensional special vector space $\hat{\mathbb{C}}^n$ as follows:

$$W = \begin{bmatrix} \sigma_c(f_0(X)) \\ \sigma_c(f_1(X)) \\ \sigma_c(f_2(X)) \\ \vdots \\ \sigma_c(f_{n-1}(X)) \end{bmatrix} = \begin{bmatrix} \sigma_c(1) \\ \sigma_c(X) \\ \sigma_c(X^2) \\ \vdots \\ \sigma_c(X^{n-1}) \end{bmatrix} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 \\ (\omega) & (\omega^3) & (\omega^5) & \cdots & (\omega^{2n-1}) \\ (\omega)^2 & (\omega^3)^2 & (\omega^5)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & (\omega^5)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ (\omega) & (\omega^3) & \cdots & (\overline{\omega})^3 & (\overline{\omega}) \\ (\omega)^2 & (\omega^3)^2 & \cdots & (\overline{\omega}^3)^2 & (\overline{\omega})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & \cdots & (\overline{\omega}^3)^{n-1} & (\overline{\omega})^{n-1} \end{bmatrix}$$

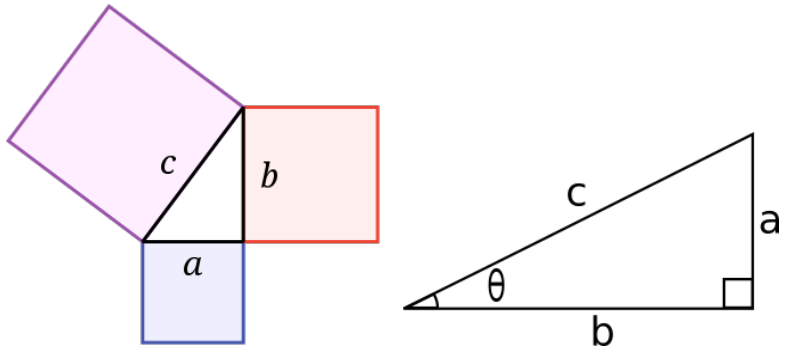
W is a valid basis of the n-dimensional special vector space $\hat{\mathbb{C}}^n$.

⟨Theorem A-11.7⟩ Transforming Basis between Polynomial Ring and Vector Space

If n polynomials form a basis of an $(n-1)$-degree polynomial ring and they are converted into n distinct vectors via an isomorphic mapping σ (or σ_c in the case of the complex number domain) from the $(n-1)$-degree polynomial ring to the n-dimensional vector space, then those converted n vectors form a basis of the n-dimensional vector space.

A-12 Euler's Formula

A-12.1 Pythagorean Identity



(a) Pythagorean Theorem (Source) (b) Pythagorean Identity (Source)
Figure 6

⟨Theorem A-12.1⟩ Pythagorean Identity

$\sin^2\theta + \cos^2\theta = 1$

Proof.
1. In Figure 6a, by Pythagorean theorem, $a^2 + b^2 = c^2$.
2. In Figure 6b, by definition of sin and cosine,
   $\sin\theta = \frac{a}{c}, \quad \cos\theta = \frac{b}{c}$
3. Combining step 1 and 2:
   $\sin^2\theta + \cos^2\theta = \frac{a^2}{c^2} + \frac{b^2}{c^2} = \frac{a^2 + b^2}{c^2} = 1$

□

A-12.2 Imaginary Number

⟨Definition A-12.2⟩ Imaginary Number

- $\boldsymbol{i}$ is defined to be an imaginary number that has the property: $\boldsymbol{i^2 = -1}$
- **Complex Number** is a number of the form $\boldsymbol{a + b\,i}$, where $\boldsymbol{a}$ and $\boldsymbol{b}$ are real numbers (e.g., $\boldsymbol{a = 5.6 + 4.3i}$)
- **Real Number** is a number that does not involve any imaginary number (e.g., $\boldsymbol{a = 13.4}$)
- $\overline{\boldsymbol{a}}$ is a **Conjugate** of $\boldsymbol{a}$ if $\boldsymbol{a}$ and $\overline{\boldsymbol{a}}$ have the same real number part and an opposite-signed imaginary number part
  (e.g., $\boldsymbol{a = 3 + i \cdot 3.4}$, $\overline{\boldsymbol{a}} \boldsymbol{= 3 - i \cdot 3.4}$)
- **Hermitian Vector** is a vector where the 2nd half of its elements is the complex conjugate of the 1st half in reverse order, as illustrated by the $\boldsymbol{n}$-dimensional vector below:

$$\vec{v} = (v_1, v_2, v_3, \cdots, v_{\frac{n}{2}-1}, v_{\frac{n}{2}}, \overline{v}_{\frac{n}{2}}, \overline{v}_{\frac{n}{2}-1}, \cdots, \overline{v}_3, \overline{v}_2, \overline{v}_1)$$

A-12.3 Euler's Formula

⟨Definition A-12.3⟩ Euler's Formula

$e^{i\theta} = \cos\theta + i \cdot \sin\theta$

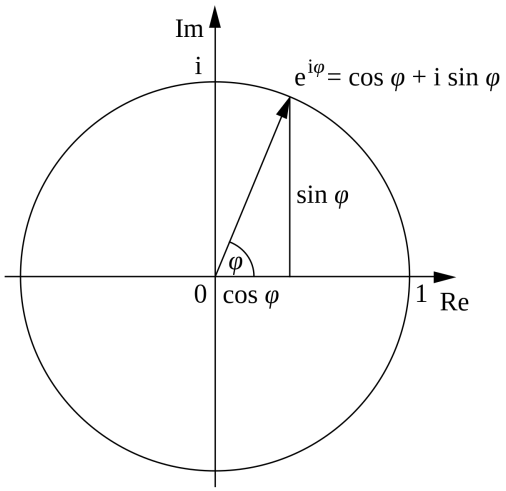


Figure 7: The figure illustrates Euler's formula on the unit circle in the complex plane (Source)

The value of $e^{i\theta}$ is represented as a coordinate on a circle in the complex plane in Figure 3, where the x-axis encodes the value's real number part and the y-axis encodes the value's imaginary number part. Note that as θ increases, the imaginary part ($\sin\theta$) oscillates between 1 and -1 (reaching i and $-i$ on the imaginary axis), and the real part ($\cos\theta$) oscillates between 1 and -1, with period 2π.

A-12.4 Vandermonde Matrix with Roots of Cyclotomic Polynomial over Complex Numbers

In this subsection, we will build a Vandermonde matrix (§A-11.2) with the n distinct roots of the μ-th cyclotomic polynomial over complex numbers (where μ is a power of 2) as follows:

⟨Theorem A-12.4⟩ Vandermonde Matrix with the Roots of (power-of-2)-th Cyclotomic Polynomial over Complex Numbers

Suppose we have an $n \times n$ (where n is a power of 2) Vandermonde matrix comprised of n distinct roots of the μ-th cyclotomic polynomial (explained in Theorem A-8.2.1 in §A-8.2), where μ is a power of 2 and $n = \frac{\mu}{2}$. In other words, $V = Vander(x_0, x_1, \cdots, x_{n-1})$, where each $x_j = (e^{i\pi/n})^{2j+1}$ for $0 \leq j < n$ (i.e., the primitive μ-th roots of unity). Then, the following holds:

$$V \cdot V^T = \begin{bmatrix} 0 & \cdots & 0 & 0 & n \\ 0 & \cdots & 0 & n & 0 \\ 0 & \cdots & n & 0 & 0 \\ \vdots & \iddots & \vdots & \vdots & \vdots \\ n & 0 & 0 & \cdots & 0 \end{bmatrix} = n \cdot I_n^R$$

And $V^{-1} = \dfrac{V^T \cdot I_n^R}{n}$

Proof.

1. Given $\omega = e^{i\pi/n}$, each $x_j = (\omega)^{2j+1}$. Thus, we can expand as follows:

$$V \cdot V^T = \begin{bmatrix} 1 & (\omega) & (\omega)^2 & \cdots & (\omega)^{n-1} \\ 1 & (\omega^3) & (\omega^3)^2 & \cdots & (\omega^3)^{n-1} \\ 1 & (\omega^5) & (\omega^5)^2 & \cdots & (\omega^5)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{2n-1}) & (\omega^{2n-1})^2 & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ (\omega) & (\omega^3) & (\omega^5) & \cdots & (\omega^{2n-1}) \\ (\omega)^2 & (\omega^3)^2 & (\omega^5)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & (\omega^5)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{k=0}^{n-1} \omega^{2k} & \sum_{k=0}^{n-1} \omega^{4k} & \sum_{k=0}^{n-1} \omega^{6k} & \cdots & \sum_{k=0}^{n-1} \omega^{2nk} \\ \sum_{k=0}^{n-1} \omega^{4k} & \sum_{k=0}^{n-1} \omega^{6k} & \sum_{k=0}^{n-1} \omega^{8k} & \cdots & \sum_{k=0}^{n-1} \omega^{2k(n+1)} \\ \sum_{k=0}^{n-1} \omega^{6k} & \sum_{k=0}^{n-1} \omega^{8k} & \sum_{k=0}^{n-1} \omega^{10k} & \cdots & \sum_{k=0}^{n-1} \omega^{2k(n+2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum_{k=0}^{n-1} \omega^{2nk} & \sum_{k=0}^{n-1} \omega^{2(n+1)k} & \sum_{k=0}^{n-1} \omega^{2(n+2)k} & \cdots & \sum_{k=0}^{n-1} \omega^{2(n+n-1)k} \end{bmatrix}$$

2. The $V \cdot V^T$ matrix's anti-diagonal elements are $\sum_{k=0}^{n-1} \omega^{2nk}$. We can derive the following:

$$\sum_{k=0}^{n-1} \omega^{2nk} = \sum_{k=0}^{n-1} (e^{i\pi/n})^{2nk} = \sum_{k=0}^{n-1} e^{2\pi ki} = \sum_{k=0}^{n-1} (\cos(2\pi k) + i\sin(2\pi k)) = \sum_{k=0}^{n-1} (1+0) = n$$

This means that The $V \cdot V^T$ matrix's anti-diagonal elements are n.

3. Next, we will prove that the $V \cdot V^T$ matrix has 0 for all positions except for the anti-diagonal ones. In other words, we will prove the following:

$$\sum_{k=0}^{n-1} \omega^{2k} = \sum_{k=0}^{n-1} \omega^{4k} = \sum_{k=0}^{n-1} \omega^{6k} = \cdots = \sum_{k=0}^{n-1} \omega^{2(n-1)k} = \sum_{k=0}^{n-1} \omega^{2(n+1)k} = \cdots = \sum_{k=0}^{n-1} \omega^{2(2n-1)k} = 0$$

For this proof, we will leverage the Geometric Sum formula $\sum_{i=0}^{n-1} x^i = \dfrac{x^n - 1}{x - 1}$:

⟨Theorem A-12.4.1⟩ Geometric Sum Formula

Let the geometric sum $S_n = 1 + x + x^2 + \cdots + x^{n-1}$
Then, $x \cdot S_n = x + x^2 + x^3 + \cdots + x^n$
$x \cdot S_n - S_n = (x + x^2 + x^3 + \cdots + x^n) - (1 + x + x^2 + \cdots + x^{n-1}) = x^n - 1$

$S_n \cdot (x-1) = x^n - 1$
$S_n = \dfrac{x^n - 1}{x - 1}$ ▷ with the constraint that $x \neq 1$

Leveraging the Geometric Sum formula $\sum_{i=0}^{n-1} x^i = \dfrac{x^n - 1}{x - 1}$,
$\sum_{k=0}^{n-1} \omega^{2mk} = \dfrac{(\omega^{2m})^n - 1}{\omega^{2m} - 1} = \dfrac{(\omega^{2n})^m - 1}{\omega^{2m} - 1} = \dfrac{1 - 1}{\omega^{2m} - 1} = 0$ for $1 \leq m \leq (2n-1)$ with $m \neq n$ ▷ since $\mathsf{Ord}(\omega) = 2n$

Therefore,
$\sum_{k=0}^{n-1} \omega^{2k} = \sum_{k=0}^{n-1} \omega^{4k} = \sum_{k=0}^{n-1} \omega^{6k} = \cdots = \sum_{k=0}^{n-1} \omega^{2(n-1)k} = \sum_{k=0}^{n-1} \omega^{2(n+1)k} = \cdots = \sum_{k=0}^{n-1} \omega^{2(2n-1)k} = 0$

4. Based on the proof of step 2 and 3, $V \cdot V^T = \begin{bmatrix} 0 & \cdots & 0 & 0 & n \\ 0 & \cdots & 0 & n & 0 \\ 0 & \cdots & n & 0 & 0 \\ \vdots & \iddots & \vdots & \vdots & \vdots \\ n & 0 & 0 & \cdots & 0 \end{bmatrix} = n \cdot I_n^R$

5. Given $V \cdot V^T = n \cdot I_n^R$,
   $V^{-1} \cdot V \cdot V^T = V^{-1} \cdot n \cdot I_n^R$
   $V^T = V^{-1} \cdot n \cdot I_n^R$
   $V^T \cdot I_n^R = V^{-1} \cdot n \cdot I_n^R \cdot I_n^R$
   $V^T \cdot I_n^R = V^{-1} \cdot n$ # since $I_n^R \cdot I_n^R = I_n$
   $V^{-1} = \dfrac{V^T \cdot I_n^R}{n}$

□

Later in the CKKS scheme (§D-3), we will use V^{-1} to encode a complex vector into a real number vector, and V to decode a real number vector into a complex vector (§D-3.2).

Condition for μ: The identity $V \cdot V^T = n \cdot I_n^R$ relies only on $\mathsf{Ord}(\omega) = 2n$ (step 3), so it holds for any $\mu = 2n$ (i.e., any even number). The nodes $\omega, \omega^3, \ldots, \omega^{2n-1}$ are likewise the n roots of $X^n + 1$ for any n, since $(\omega^{2j-1})^n = \omega^{2nj} \cdot \omega^{-n} = 1 \cdot (-1) = -1$.

Nevertheless, the reason we take μ to be a power of 2 does not come from this theorem, but from the FHE schemes themselves. In practice, CKKS, BFV, and BGV are most commonly instantiated over the power-of-2 cyclotomic ring $\mathbb{Z}[X]/(X^n + 1)$ with n a power of 2, because this choice makes $X^n + 1 = \Phi_{2n}(X)$ irreducible over rational numbers (which holds precisely when n is a power of 2), and also enables efficient negacyclic-NTT multiplication and a clean Ring-LWE security reduction. Once n (hence $\mu = 2n$) is fixed as a power of 2, the nodes are automatically the primitive μ-th roots of unity: by Theorem A-7.2.4 (§A-7.2), ω^k is a primitive μ-th root iff $\gcd(\mu, k) = 1$, and every odd exponent $1, 3, \ldots, 2n-1$ is coprime to $\mu = 2n$ iff $2n$ has no odd prime factor—i.e. iff $2n$ is a power of 2. So $\{\omega, \omega^3, \ldots, \omega^{2n-1}\}$ being exactly the primitive $(\mu = 2n)$-th roots is a consequence of the power-of-2 ring choice. Meanwhile, as far as μ is an even number, $V \cdot V^T = n \cdot I_n^R$ still holds.

A-12.5 Vandermonde Matrix with Roots of Cyclotomic Polynomial over Ring ($\mathbb{Z}_p$)

Assume p is prime and $2n \mid p-1$, and fix $\omega \in \mathbb{Z}_p^\times$ of order $2n$. Order the nodes as $x_j = \omega^{2j+1}$ for $0 \le j < n$. All matrix identities below hold in $\mathbb{Z}_p$; in particular n^{-1} exists. The displayed reverse identity depends on this node order; arbitrary row permutations change the pairing matrix.

Theorem A-12.4 (in §A-12.4) showed that $V \cdot V^T = n \cdot I_n^R$, where V is the Vandermonde matrix $V = \mathit{Vander}(x_0, x_1, \cdots, x_{n-1})$, where each x_i is the primitive μ-th root of unity over $X \in \mathbb{C}$ (i.e., complex number) and μ is a power of 2. In this subsection, we will show that the relation $V \cdot V^T = n \cdot I_n^R$ holds even if each x_i is the primitive μ-th root of unity over $X \in \mathbb{Z}_p$ (i.e., ring). In particular, we will prove Theorem A-12.4:

⟨Theorem A-12.5⟩ Vandermonde Matrix with Roots of (power-of-2)-th Cyclotomic Polynomial over Ring ($\mathbb{Z}_p$)

The proof takes the same format as that of Theorem A-12.4 (in §A-12.4). Suppose we have an $n \times n$ (where n is a power of 2) Vandermonde matrix comprising n distinct roots of the μ-th cyclotomic polynomial over $X \in \mathbb{Z}_p$ (ring), where μ is a power of 2 and $n = \frac{\mu}{2}$. In other words, $V = \mathit{Vander}(x_0, x_1, \cdots, x_{n-1})$, where each x_i is the root of $X^n + 1$ (i.e., the primitive ($\mu = 2n$)-th roots of unity). Then, the following holds:

$$V \cdot V^T = \begin{bmatrix} 0 & \cdots & 0 & 0 & n \\ 0 & \cdots & 0 & n & 0 \\ 0 & \cdots & n & 0 & 0 \\ \vdots & \iddots & \vdots & \vdots & \vdots \\ n & 0 & 0 & \cdots & 0 \end{bmatrix} = n \cdot I_n^R$$

And $V^{-1} = n^{-1} \cdot V^T \cdot I_n^R$

Proof.

1. $V \cdot V^T$ is expanded as follows:

$$V \cdot V^T = \begin{bmatrix} 1 & (\omega) & (\omega)^2 & \cdots & (\omega)^{n-1} \\ 1 & (\omega^3) & (\omega^3)^2 & \cdots & (\omega^3)^{n-1} \\ 1 & (\omega^5) & (\omega^5)^2 & \cdots & (\omega^5)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{2n-1}) & (\omega^{2n-1})^2 & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ (\omega) & (\omega^3) & (\omega^5) & \cdots & (\omega^{2n-1}) \\ (\omega)^2 & (\omega^3)^2 & (\omega^5)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & (\omega^5)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{k=0}^{n-1} \omega^{2k} & \sum_{k=0}^{n-1} \omega^{4k} & \sum_{k=0}^{n-1} \omega^{6k} & \cdots & \sum_{k=0}^{n-1} \omega^{2nk} \\ \sum_{k=0}^{n-1} \omega^{4k} & \sum_{k=0}^{n-1} \omega^{6k} & \sum_{k=0}^{n-1} \omega^{8k} & \cdots & \sum_{k=0}^{n-1} \omega^{2k(n+1)} \\ \sum_{k=0}^{n-1} \omega^{6k} & \sum_{k=0}^{n-1} \omega^{8k} & \sum_{k=0}^{n-1} \omega^{10k} & \cdots & \sum_{k=0}^{n-1} \omega^{2k(n+2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum_{k=0}^{n-1} \omega^{2nk} & \sum_{k=0}^{n-1} \omega^{2(n+1)k} & \sum_{k=0}^{n-1} \omega^{2(n+2)k} & \cdots & \sum_{k=0}^{n-1} \omega^{2(n+n-1)k} \end{bmatrix}$$

, where ω (i.e., the primitive ($\mu = 2n$)-th root of unity) has the order $2n$.

2. Note that the $V \cdot V^T$ matrix's anti-diagonal elements are $\sum_{k=0}^{n-1} \omega^{2nk}$. It can be seen that $\omega^{2n} \equiv 1 \bmod p$, because $\mathsf{Ord}_p(\omega) = 2n$. Thus, the $V \cdot V^T$ matrix's every anti-diagonal element is $\sum_{k=0}^{n-1} 1 = n$.

3. Next, we will prove that the $V \cdot V^T$ matrix has 0 for all other positions than the anti-diagonal ones. In other words, we will prove the following:
$\sum_{k=0}^{n-1} \omega^{2k} = \sum_{k=0}^{n-1} \omega^{4k} = \sum_{k=0}^{n-1} \omega^{6k} = \cdots = \sum_{k=0}^{n-1} \omega^{2(n-1)k} = \sum_{k=0}^{n-1} \omega^{2(n+1)k} = \cdots = \sum_{k=0}^{n-1} \omega^{2(2n-1)k} = 0$

   The above is true by the Geometric Sum formula (Theorem A-12.4.1). As shown in the proof step 3 of Theorem A-12.4, each element is of the form $\sum_{k=0}^{n-1}(\omega^{2m})^k$ for some integer m (where $2m$ is not a multiple of $2n$). Let $r = \omega^{2m}$. Then:
$\sum_{k=0}^{n-1} r^k = \frac{r^n - 1}{r - 1} = \frac{(\omega^{2m})^n - 1}{\omega^{2m} - 1} = \frac{(\omega^n)^{2m} - 1}{\omega^{2m} - 1}$

   Since ω is a root of $X^n + 1$, we know $\omega^n \equiv -1 \bmod p$. Thus:
$\frac{(-1)^{2m} - 1}{\omega^{2m} - 1} = \frac{1 - 1}{\omega^{2m} - 1} = 0$

   Therefore, the sum is 0 for all off-anti-diagonal positions.

4. According to step 1 and 2, the $V \cdot V^T$ matrix has n on its anti-diagonal positions and 0 for all other positions.
5. Now we will derive the formula for V^{-1}. Given $V \cdot V^T = n \cdot I_n^R$,
$V^{-1} \cdot V \cdot V^T = V^{-1} \cdot n \cdot I_n^R$
$V^T = V^{-1} \cdot n \cdot I_n^R$
$V^T \cdot I_n^R = V^{-1} \cdot n \cdot I_n^R \cdot I_n^R$
$V^T \cdot I_n^R = V^{-1} \cdot n$ # since $I_n^R \cdot I_n^R = I_n$

   Now, there is one caveat: modulo operation does not support direct number division (as explained in §A-1.4). This means that the formula $V^{-1} = \frac{V^T \cdot I_n^R}{n}$ in Theorem A-12.4 (in §A-12.4) is inapplicable in our case, because our modulo p arithmetic does not allow direct division of $V^T \cdot I_n^R$ by n. Therefore, we instead multiply $V^T \cdot I_n^R$ by the inverse of n (i.e., n^{-1}). We continue as follows:
$V^T \cdot I_n^R = V^{-1} \cdot n$
$V^T \cdot I_n^R \cdot n^{-1} = V^{-1} \cdot n \cdot n^{-1}$
$V^{-1} = n^{-1} \cdot V^T \cdot I_n^R$

□

We finally proved that $V \cdot V^T = n \cdot I_n^R$, and $V^{-1} = n^{-1} \cdot V^T \cdot I_n^R$. Later in the BFV scheme (§D-2), we will use V^{-1} to encode an integer vector into a vector of polynomial coefficients, and V to decode it back to the integer vector (§D-2.2).

Condition for μ: With p prime, $2n \mid p - 1$, and nodes ordered as ω^{2j+1}, the geometric-sum proof gives $VV^T = nI_n^R$ for every $n \geq 1$, not only powers of two. For example, $n = 3$, $p = 7$, and $\omega = 3$ satisfy these

assumptions. The power-of-two assumption is needed to identify $X^n + 1$ with the single cyclotomic polynomial Φ_{2n} and all its roots with primitive $2n$-th roots; it is not necessary for this matrix identity.

A-13 Modulo Rescaling

A-13.1 Rescaling Modulo of Congruence Relations

Let q, q' be positive integer moduli, $r = q'/q$, and fix the translation-equivariant nearest-integer rule $R(x) = \lfloor x + 1/2 \rfloor$. Thus $R(x + h) = R(x) + h$ for integer h. Write $\delta_x = R(rx) - rx$, so $|\delta_x| \leq 1/2$.

Original relation	Scaled relation modulo q'
$a \equiv b \pmod{q}$	$R(ra) \equiv R(rb) \pmod{q'}$
$a + c \equiv b + d \pmod{q}$	$R(ra) + R(rc) \equiv R(rb) + R(rd) + \eta \pmod{q'}$
$ac \equiv bd \pmod{q}$	$R(rac) \equiv R(rbd) \pmod{q'}$

Table 3: Scaling congruences, where $\eta = \delta_a + \delta_c - \delta_b - \delta_d$ and $|\eta| \leq 2$.

Proof. If $a = b + kq$, then $ra = rb + kq'$ and $R(ra) = R(rb) + kq'$. Thus rounding an entire congruence preserves it exactly, whether or not ra is an integer. The same argument applies to the entire products ac and bd.

For separately rounded summands, $a + c = b + d + kq$ gives

$$R(ra) + R(rc) - R(rb) - R(rd) = kq' + \delta_a + \delta_c - \delta_b - \delta_d.$$

The last four terms form the integer η, with $|\eta| \leq 2$. If they cancel (in particular, if all scaled summands are integers), the relation is exact. Otherwise this explicit error must be retained. □

These statements are forward implications, not general equivalences. For instance, with $q = 8$ and $q' = 2$, both $a = 0$ and $b = 1$ round to zero, although $a \not\equiv b \pmod{8}$. Rounding loses information. Also, separately scaling both factors of a product changes its scale to r^2; it cannot replace scaling the entire product once by r.

For a fixed integer multiplier s, the useful identity is $R(ra)s = ras + \delta_a s$, with error at most $|s|/2$. In an LWE phase $b - \sum_i a_i s_i$, separately rounding b and the a_i therefore introduces $\delta_b - \sum_i \delta_{a_i} s_i$. This error, rather than an undefined approximate-congruence symbol, is what later modulus-switching correctness must bound.

A-13.1.1 Example

Suppose we have the following congruence relation:

$b \equiv a \cdot s + \Delta \cdot m + e \bmod q$, where: $q = 30$, $s = 5$, $a = 10$, $\Delta = 10$, $m = 1$, $e = 10$, $b = 40$

First, we can test if the above congruence relation is true by plugging in the given example values as follows:

$b \equiv a \cdot s + \Delta \cdot m + e \bmod 30$

$40 \equiv 10 \cdot 5 + 10 \cdot 1 + 10 \bmod 30$

$40 \equiv 70 \bmod 30$

This congruence relation is true.

Now, suppose we want to rescale the modulo from $30 \to 3$. Then, based on the rescaling principles described in Table 3, we compute the rescaled values as follows:

$q' = 3, \quad s = 5, \quad m = 1$

$$\hat{a} = \left\lfloor a \cdot \frac{3}{30} \right\rfloor = \left\lfloor 10 \cdot \frac{3}{30} \right\rfloor = 1$$

$$\hat{\Delta} = \left\lfloor \Delta \cdot \frac{3}{30} \right\rfloor = \left\lfloor 10 \cdot \frac{3}{30} \right\rfloor = 1$$

$$\hat{e} = \left\lfloor e \cdot \frac{3}{30} \right\rfloor = \left\lfloor 10 \cdot \frac{3}{30} \right\rfloor = 1$$

$$\hat{b} = \left\lfloor b \cdot \frac{3}{30} \right\rfloor = \left\lfloor 40 \cdot \frac{3}{30} \right\rfloor = 4$$

The rescaled congruence relation from modulo $30 \to 3$ is derived as follows:

$$\left\lfloor b\frac{3}{30} \right\rfloor \equiv \left\lfloor s \cdot a\frac{3}{30} \right\rfloor + \left\lfloor m \cdot \Delta\frac{3}{30} \right\rfloor + \left\lfloor e\frac{3}{30} \right\rfloor \bmod 3$$

$\hat{b} \equiv \hat{a} \cdot s + \hat{\Delta} \cdot m + \hat{e} \bmod 3$ (an exact congruence relation, as all rescaled values have no decimals)

$4 \equiv 1 \cdot 5 + 1 \cdot 1 + 1 \bmod 3$

$4 \equiv 7 \bmod 3$

The example is exact because ra, $r\Delta$, and re are integers, so every rounding error is zero. If some scaled terms are not integers, exactness is still possible through cancellation; in general, retain the corresponding errors from Table 3.

A-14 Chinese Remainder Theorem

- **Reference 1:** Brilliant – Chinese Remainder Theorem [4]
- **Reference 2:** YouTube – Extended Euclidean Algorithm Tutorial

⟨Theorem A-14.1⟩ Chinese Remainder Theorem

Suppose we have pairwise coprime integers greater than one $n_0, n_1, n_2, \cdots, n_k$. Let $N = n_0 n_1 \cdots n_k$. We sample $k+1$ random integers $a_0, a_1, a_2, \cdots, a_k$ from each modulus $n_0, n_1, n_2, \ldots, n_k$ (i.e., $a_0 \in \mathbb{Z}_{n_0}$, $a_1 \in \mathbb{Z}_{n_1}$, $\cdots$, $a_k \in \mathbb{Z}_{n_k}$). Then, there exists one and only one solution x mod N such that $x \equiv a_i \pmod{n_i}$ for each $0 \le i \le k$. That is:

$$\begin{aligned} x &\equiv a_0 \bmod n_0 \\ x &\equiv a_1 \bmod n_1 \\ x &\equiv a_2 \bmod n_2 \\ &\vdots \\ x &\equiv a_k \bmod n_k \end{aligned}$$

To compute x, we first compute each y_i and z_i (for $0 \le i \le k$) as follows:

$$y_i = \frac{N}{n_i},\ z_i = y_i^{-1} \bmod n_i$$

Note that each y_i's inverse (i.e., y_i^{-1}) can be computed by using the Extended Euclidean algorithm (watch the YouTube tutorial). Then, the unique solution x can be computed as follows:

$$x = \sum_{i=0}^{k} a_i y_i z_i \ \triangleright \text{ Alternatively, we can compute } x = \sum_{i=0}^{k} |a_i z_i|_{n_i} y_i \text{ (where } |a_i z_i|_{n_i} = a_i z_i \bmod n_i)$$

Since such x is unique in mod N, there are isomorphic mappings between x mod N and $(a_0, a_1, a_2, \cdots, a_k)$.

Also, $y_i z_i \equiv (y_i z_i)^2 \bmod N \quad$ for all $0 \le i \le k$

Proof.

1. Given $x = \sum_{i=0}^{k} a_i y_i z_i$, let's compute x mod n_i for each i where $0 \le i \le k$:

$$\begin{aligned} x \bmod n_i &= \sum_{j=0}^{k} a_j y_j z_j \bmod n_i \\ &= a_0 y_0 z_0 + a_1 y_1 z_1 + a_2 y_2 z_2 + \cdots + a_k y_k z_k \bmod n_i \\ &= a_i y_i z_i \bmod n_i \ \triangleright \text{ because } y_j \equiv 0 \bmod n_i \text{ for all } j \neq i, \text{ as they are a multiple of } n_i \\ &= a_i \ \triangleright \text{ because } y_i z_i \equiv y_i y_i^{-1} \equiv 1 \bmod n_i \end{aligned}$$

Thus, the value of x in each modulo $n_0, n_1, n_2, \cdots, n_k$ is congruent with $a_0, a_1, a_2, \cdots, a_k$.

Alternatively, note that the following is also true:

$$\begin{aligned} x \bmod n_i &= \sum_{j=0}^{k} |a_j z_j|_{n_j} y_j \ (\bmod n_i) \\ &= |a_0 z_0|_{n_0} y_0 + |a_1 z_1|_{n_1} y_1 + |a_2 z_2|_{n_2} y_2 + \cdots + |a_k z_k|_{n_k} y_k \bmod n_i \end{aligned}$$

$= |a_i z_i|_{n_i} y_i \bmod n_i$
$= a_i$

2. Now, we prove that x is a unique solution modulo N. Suppose there were two solutions: x and x' such that:
$x \equiv x' \equiv a_0 \bmod n_0$
$x \equiv x' \equiv a_1 \bmod n_1$
$x \equiv x' \equiv a_2 \bmod n_2$
$\vdots$
$x \equiv x' \equiv a_k \bmod n_k$

   Then, by definition of modulo congruence, $n_0 \mid (x - x'), n_1 \mid (x - x'),\ n_2 \mid (x - x'), \cdots,\ n_k \mid (x - x')$. Also, since $n_0, n_1, n_2, \cdots, n_k$ are coprime, it must be the case that $n_0 n_1 n_2 n_3 \cdots n_k \mid (x - x')$, or $N \mid (x - x')$. This means that $x \equiv x' \bmod N$. Therefore, x is a unique solution in modulo N.

3. Now, we will prove that $y_i z_i \equiv (y_i z_i)^2 \bmod N \quad$ for all $0 \leq i \leq k$.

   In the case of modulo n_i, $y_i z_i \equiv 1 \bmod n_i$, since z_i is an inverse of y_i modulo n_i. In the case of all other modulo n_j where $i \neq j$, $y_i z_i \equiv 0 \bmod n_j$, because $y_i = \dfrac{N}{n_i}$ and thus n_j divides y_i.

   By squaring both sides of $(y_i z_i) \equiv 1 \bmod n_i$, we get $(y_i z_i)^2 \equiv 1 \bmod n_i$. Similarly, by squaring both sides of $(y_i z_i) \equiv 0 \bmod n_j$, we get $(y_i z_i)^2 \equiv 0 \bmod n_j$.

   Therefore, $y_i z_i - (y_i z_i)^2 \equiv 0 \bmod n_i$, and $y_i z_i - (y_i z_i)^2 \equiv 0 \bmod n_j$. In other words, $y_i z_i - (y_i z_i)^2 \equiv 0 \bmod n_j$ for all $0 \leq j \leq k$.

   Then, we do the similar reasoning as step 2: since every co-prime n_j divides $y_i z_i - (y_i z_i)^2$, $n_0 n_1 \cdots n_k = N$ divides $y_i z_i - (y_i z_i)^2$. Thus, $y_i z_i - (y_i z_i)^2 \equiv 0 \bmod N$, which is $y_i z_i \equiv (y_i z_i)^2 \bmod N$. This is true for all $0 \leq i \leq k$.

□

A-14.1 Application: Residue Number System (RNS)

In a modern processor, each data size is a maximum of 64 bits. If the data size exceeds 64 bits, its computations can be handled efficiently by using the Chinese remainder theorem, ensuring that each co-prime modulus n_i satisfies $\log_2 n_i \leq 64$ (where $N = n_0 \cdot n_1 \cdot \cdots \cdot n_k$), so that we can represent a large value a mod N as $\vec{a}_{crt} = (a_0, a_1, \cdots, a_k)$, where $a \equiv a_i \bmod n_i$. Then, for any pair of big numbers a and b mod N, we can compute $a + b \bmod N$ and $a \cdot b \bmod N$ as follows:

- $a + b \equiv \sum_{i=0}^{k} a_i y_i z_i + \sum_{i=0}^{k} b_i y_i z_i \equiv \sum_{i=0}^{k} (a_i y_i z_i + b_i y_i z_i) \equiv \sum_{i=0}^{k} (a_i + b_i) y_i z_i \bmod N$
- $a \cdot b \equiv \sum_{i=0}^{k} a_i y_i z_i \cdot \sum_{i=0}^{k} b_i y_i z_i \equiv \sum_{i=0}^{k} (a_i \cdot b_i)(y_i z_i)^2 + \sum_{i \neq j}^{k} (a_i \cdot b_j) y_i z_i y_j z_j \equiv \sum_{i=0}^{k} (a_i \cdot b_i)(y_i z_i)^2$
  ▷ Note that all terms $y_i z_i y_j z_j$ where $i \neq j$ are 0 modulo N, because $y_i y_j \bmod N \equiv 0$.
  This is because $y_i = n_0 n_1 \cdots n_{i-1} n_{i+1} \cdots$ and $y_j = n_0 n_1 \cdots n_{j-1} n_{j+1} \cdots$.

Thus $y_i y_j$ is a multiple of N.

$\equiv \sum_{i=0}^{k} (a_i \cdot b_i)(y_i z_i) \bmod N$

$\triangleright$ This is because $(y_i z_i) \equiv (y_i z_i)^2$ as shown in step 3 in the proof of Theorem A-14.1

Thus, the Chinese remainder theorem gives us the following useful formula:

⟨Theorem A-14.2⟩ Application of the Chinese Remainder Theorem

Suppose there are two big numbers $a = \sum_{i=0}^{k} a_i y_i z_i \bmod N$ and $b = \sum_{i=0}^{k} b_i y_i z_i \bmod N$ where N is a the product of co-prime moduli $n_0 \cdot n_1 \cdots n_k$, we have an isomorphism as follows:
$a \xrightarrow{\sigma} \vec{a}_{crt} = (a_0, a_1, \cdots, a_k)$
$b \xrightarrow{\sigma} \vec{b}_{crt} = (b_0, b_1, \cdots, b_k)$

Based on the above isomorphism, the following is true:

- $a + b \equiv \sum_{i=0}^{k} (a_i + b_i) y_i z_i \bmod N \iff \vec{a}_{crt} + \vec{b}_{crt} = ((a_i + b_i) \bmod n_i)_{i=0}^{k}$
- $a \cdot b \equiv \sum_{i=0}^{k} (a_i \cdot b_i) y_i z_i \bmod N \iff \vec{a}_{crt} \odot \vec{b}_{crt} = ((a_i b_i) \bmod n_i)_{i=0}^{k}$

, where each element-wise addition/multiplication can be independently done modulo n_i

A-15 Taylor Series

The Taylor series is a way to represent an analytic function near a point by an infinite power series (assuming the series converges to the function in that neighborhood). Truncating this series gives polynomial approximations of the function. Formally speaking, the Taylor series of a function is an infinite sum of the evaluations of the function's derivatives at a single point. Given function $f(X)$, its Taylor series centered at a is expressed as follows:

$$f(a) + \frac{f'(a)}{1!}(X-a) + \frac{f''(a)}{2!}(X-a)^2 + \frac{f'''(a)}{3!}(X-a)^3 + \cdots = \sum_{d=0}^{\infty} \frac{f^{(d)}(a)}{d!}(X-a)^d$$

Truncating after degree D gives the D-th Taylor polynomial $T_D(X)$. Inside a neighborhood where the Taylor series represents f, the error tends to zero as $D \to \infty$, but it need not decrease at every successive degree or monotonically with the distance from a. For a real function with $D+1$ continuous derivatives on the interval joining a and X, Taylor's theorem gives

$$|f(X) - T_D(X)| \leq \frac{\sup_u |f^{(D+1)}(u)|}{(D+1)!}|X-a|^{D+1},$$

where the supremum is over that interval. Increasing the degree outside the series' convergence region gives no general accuracy guarantee.

A-16 Efficient Polynomial Multiplication by FFT and NTT

- Reference: Polynomials and the Fast Fourier Transform (FFT) [5]

A-16.1 Background and Motivation

This chapter computes the negacyclic product modulo $X^n + 1$, not the unreduced degree-$2n-2$ product. Whenever quotient reduction is used, the evaluation nodes must be roots of $X^n + 1$. Coefficients outside $0, \ldots, n-1$ are zero in the convolution formula. Complexity counts field arithmetic operations, not bit operations or floating-point error.

Given two $(n-1)$-degree polynomials:

$A(X) = \sum_{i=0}^{n-1} a_i X^i, \quad B(X) = \sum_{i=0}^{n-1} b_i X^i$

, the polynomial multiplication $C(X) = A(X) \cdot B(X)$ is computed as follows:

$C(X) = \sum_{i=0}^{2n-2} c_i X^i$, where $c_i = \sum_{k=0}^{i} a_k b_{i-k}$

This operation of computing $\vec{c} = (c_0, c_1, \cdots, c_{2n-2})$ is also called the convolution of $\vec{a}$ and $\vec{b}$, denoted as $\vec{c} = \vec{a} \otimes \vec{b}$. The time complexity of this operation (i.e., the total number of multiplications between two numbers) is $O(n^2)$.

Another way of multiplying two polynomials is based on **point-value representation**. The point-value representation of an $(n-1)$-degree (or lesser degree) polynomial $A(X)$ is a set of n coordinates $\{(x_0, y_0), (x_1, y_1), \cdots (x_{n-1}, y_{n-1})\}$, where each x_i is a distinct X coordinate (whereas each y_i is not necessarily a distinct Y coordinate). Given a point-value representation of an $(n-1)$-degree (or lesser degree) polynomial, we can use polynomial interpolation (§A-10) to derive the polynomial. Let's denote the point-value representation of $(n-1)$-degree (or lesser degree) polynomial $A(X)$ and $B(X)$ as follows:

$A(X) : ((x_0, y_0^{\langle a \rangle}), (x_1, y_1^{\langle a \rangle}), \cdots (x_{n-1}, y_{n-1}^{\langle a \rangle}))$

$B(X) : ((x_0, y_0^{\langle b \rangle}), (x_1, y_1^{\langle b \rangle}), \cdots (x_{n-1}, y_{n-1}^{\langle b \rangle}))$

Then, the point-value representation of the polynomial $C(X) = A(X) \cdot B(X)$ can be computed as a Hadamard product (Definition A-11.1 in §A-11.1) of the y values of the point-value representation of $A(X)$ and $B(X)$ as follows:

$C(X) : ((x_0, y_0^{\langle c \rangle}), (x_1, y_1^{\langle c \rangle}), \cdots (x_{n-1}, y_{n-1}^{\langle c \rangle}))$, where $y_i^{\langle c \rangle} = y_i^{\langle a \rangle} \cdot y_i^{\langle b \rangle}$

However, we cannot derive polynomial $C(X)$ based on these n coordinates because the degree of $C(X)$ is $2n-2$ (or less than $2n-2$). But if we regard all polynomials (including $A(X)$, $B(X)$ and $C(X)$) to be in the polynomial ring $\mathbb{R}[X]/(X^n + 1)$ (or $\mathbb{Z}_p[X]/(X^n + 1)$), then we can reduce the $(2n-2)$-degree polynomial $C(X)$ to a congruent $(n-1)$-degree (or lesser degree) polynomial in the ring. Then, the n coordinates of $C(X)$ are sufficient to derive $C(X)$.

However, the time complexity of this new method is still $O(n^2)$. The Hadamard product between two polynomials' point-value representations takes $O(n)$, but evaluating a polynomial at n distinct x values takes $O(n^2)$ (because each polynomial has n terms, and we have to compute each term for n distinct x values). The polynomial interpolation for deriving $C(X)$ also takes $O(n^2)$.

To solve this efficiency problem, this section will explain an efficient technique for polynomial evaluation, which can evaluate a polynomial at n distinct roots of unity in $O(n \log n)$. This technique is classified into 2 types: Fast Fourier Transform (FFT) and Number-theoretic Transform (NTT). These two types

are technically almost the same, with the only difference that the FFT assumes a polynomial ring over complex numbers (§A-7), whereas the NTT assumes a polynomial ring over a finite field (e.g., integers modulo a prime) (§A-9). Polynomial multiplication based on FFT (or NTT) comprises 3 steps: (1) forward FFT (or NTT); (2) point-value multiplication; and (3) inverse FFT (or NTT).

A-16.2 Forward FFT (or NTT)

For the radix-two recursion, take $n = 2^L$. For the finite-field NTT, also require a prime p with $2n \mid p-1$ and choose ω of order $2n$ in $\mathbb{Z}_p^\times$. These conditions ensure distinct nodes and invertibility of n and all nonzero geometric-sum denominators. For the complex transform, use $\omega = e^{\pi i/n}$.

We assume a polynomial ring of $\mathbb{R}[X]/(X^n+1)$ for FFT, and $\mathbb{Z}_p[X]/(X^n+1)$ for NTT (where X^n+1 is a cyclotomic polynomial). The x coordinates to evaluate the target polynomial are the n distinct roots of X^n+1, which are $\omega^1, \omega^3, \ldots, \omega^{2n-1}$, where ω is the primitive $2n$-th root of unity. Then, the point-value representation of the polynomial $A(X)$ is $\big((x_0, y_0^{\langle a \rangle}), (x_1, y_1^{\langle a \rangle}), \cdots (x_{n-1}, y_{n-1}^{\langle a \rangle})\big)$, where:

$$y_i^{\langle a \rangle} = A(\omega^{2i+1}) = \sum_{j=0}^{n-1} a_j \cdot (\omega^{2i+1})^j = \sum_{j=0}^{n-1} a_j \cdot \omega^{(2i+1)\cdot j}$$

We call the vector $\vec{y}^{\langle a \rangle} = (y_0^{\langle a \rangle}, y_1^{\langle a \rangle}, \cdots, y_{n-1}^{\langle a \rangle})$ the Discrete Fourier Transform (DFT) of the coefficient vector $\vec{a} = (a_0, a_1, \cdots, a_{n-1})$. We write this as $\vec{y}^{\langle a \rangle} = \mathsf{DFT}(\vec{a})$. As explained in §A-16.1, the computation of the DFT takes $O(n^2)$, because we have to evaluate n distinct X values for a polynomial that has n terms.

A-16.2.1 High-level Idea

FFT (or NTT) is an improved method for computing the DFT, which reduces the time complexity from $O(n^2)$ to $O(n \log n)$. The high-level idea of FFT is to split the $(n-1)$-degree (or lesser degree) target polynomial $A(X)$ to evaluate into 2 half-degree polynomials $A_0(X)$ and $A_1(X)$ as follows:

$$\begin{aligned} A(X) &= a_0 + a_1 X + a_2 X^2 + \cdots + a_{n-1} X^{n-1} \\ &= A_0(X^2) + X \cdot A_1(X^2) \end{aligned}$$

$A_0(X) = a_0 + a_2 X + a_4 X^2 + \cdots + a_{n-2} X^{\frac{n}{2}-1}$

$A_1(X) = a_1 + a_3 X + a_5 X^2 + \cdots + a_{n-1} X^{\frac{n}{2}-1}$

The above method of splitting a polynomial into two half-degree polynomials is called the Cooley-Tukey step. As we split $A(X)$ into two smaller-degree polynomials $A_0(X)$ and $A_1(X)$, evaluating $A(X)$ at the odd-powered primitive $2n$-th roots of unity $\{\omega^1, \omega^3, \omega^5, \cdots, \omega^{2n-1}\}$ is equivalent to evaluating $A_0(X)$ and $A_1(X)$ at the n listed squared nodes (only $n/2$ of which are distinct) $\{(\omega^2)^1, (\omega^2)^3, (\omega^2)^5, \ldots, (\omega^2)^{2n-1}\}$ and computing $A_0(X^2) + X \cdot A_1(X^2)$. However, remember that the primitive $2n$-th root of unity ω has order $2n$ (i.e., $\omega^{2n} = 1$ and $\omega^m \neq 1$ for all integers $1 \leq m < 2n$). Therefore, the second half of $\{(\omega^2)^1, (\omega^2)^3, (\omega^2)^5, \ldots, (\omega^2)^{2n-1}\}$ is a repetition of the first half. This implies that we only need to evaluate $A_0(X)$ and $A_1(X)$ at $\dfrac{n}{2}$ distinct x coordinates each, instead of n distinct coordinates, because the polynomial evaluation results for the other half are the same as those of the first half (as their input x to the polynomial is the same).

We recursively split $A_0(X)$ and $A_1(X)$ into half-degree polynomials and evaluate them at half-counted (i.e., $n/2$) n-th roots of unity. Then, the total number of rounds of splitting is $\log n$, and the maximum number of root-to-coefficient multiplications in each round is n, which aggregates to $O(n \log n)$.

A-16.2.2 Details

Suppose we have a polynomial ring that is either $\mathbb{Z}_p[X]/(X^8+1)$ (i.e., over a finite field with prime p) or $\mathbb{R}[X]/(X^8+1)$ (over complex numbers). We denote by ω a primitive $(2n=16)$-th root of unity, and the 8 distinct roots of X^8+1 are: $\{\omega^1, \omega^3, \omega^5, \omega^7, \omega^9, \omega^{11}, \omega^{13}, \omega^{15}\}$.

Now, we define our target polynomial to evaluate as follows:
$A(X) = a_0 + a_1X + a_2X^2 + a_3X^3 + a_4X^4 + a_5X^5 + a_6X^6 + a_7X^7$

We split this 7-degree polynomial into the following two 3-degree polynomials (using the Cooley-Tukey step):
$A_0(X) = a_0 + a_2X + a_4X^2 + a_6X^3$
$A_1(X) = a_1 + a_3X + a_5X^2 + a_7X^3$
$A(X) = A_0(X^2) + X \cdot A_1(X^2)$

We recursively split the two 3-degree polynomials above into 1-degree polynomials as follows:
$A_{0,0}(X) = a_0 + a_4X, \quad A_{0,1}(X) = a_2 + a_6X$
$A_0(X) = A_{0,0}(X^2) + X \cdot A_{0,1}(X^2)$

$A_{1,0}(X) = a_1 + a_5X, \quad A_{1,1}(X) = a_3 + a_7X$
$A_1(X) = A_{1,0}(X^2) + X \cdot A_{1,1}(X^2)$

$$A(X) = A_0(X^2) + X \cdot A_1(X^2)$$
$$= \underbrace{(\underbrace{A_{0,0}(X^4)}_{\text{FFT Level 1}} + X^2 \cdot \underbrace{A_{0,1}(X^4)}_{\text{FFT Level 1}})}_{\text{FFT Level 2}} + X \cdot \underbrace{(\underbrace{A_{1,0}(X^4)}_{\text{FFT Level 1}} + X^2 \cdot \underbrace{A_{1,1}(X^4)}_{\text{FFT Level 1}})}_{\text{FFT Level 2}}$$
$$\text{FFT Level 3}$$

To evaluate $A(X)$ at the n distinct roots $X = \{\omega^1, \omega^3, \ldots, \omega^{15}\}$, we evaluate each FFT level of the above formula at $X = \{\omega^1, \omega^3, \cdots, \omega^{15}\}$, starting from level $1 \leq l \leq 3$.

FFT Level $l = 1$: We evaluate $A_{0,0}(X^4)$, $A_{0,1}(X^4)$, $A_{1,0}(X^4)$, and $A_{1,1}(X^4)$ at $X = \{\omega^1, \omega^3, \cdots, \omega^{15}\}$. However, notice that plugging in $X = \{\omega^1, \omega^3, \cdots, \omega^{15}\}$ to X^4 results in only 2 distinct values: ω^4 and ω^{12} (which correspond to roots of X^2+1). This is because the order of ω is $2n$ (i.e., $\omega^{2n}=1$), and thus $(\omega^1)^4 = (\omega^5)^4 = (\omega^9)^4 = (\omega^{13})^4 = \omega^4$, and $(\omega^3)^4 = (\omega^7)^4 = (\omega^{11})^4 = (\omega^{15})^4 = \omega^{12}$. Therefore, we only need to evaluate $A_{0,0}(X^4)$, $A_{0,1}(X^4)$, $A_{1,0}(X^4)$, and $A_{1,1}(X^4)$ at 2 distinct x values instead of 8, where each evaluation requires a constant number of arithmetic operations: computing 1 multiplication and 1 addition. As there are a total of 4 polynomials to evaluate (i.e., $A_{0,0}(X^4)$, $A_{0,1}(X^4)$, $A_{1,0}(X^4)$, $A_{1,1}(X^4)$), we compute the FFT a total of $4 \cdot 2 = 8$ times.

FFT Level $l = 2$: Based on the evaluation results from FFT Level 1 as building blocks, we evaluate $A_0(X^2)$ and $A_1(X^2)$ at $X = \{\omega^1, \omega^3, \cdots, \omega^{15}\}$. However, notice that plugging in $X = \{\omega^1, \omega^3, \cdots, \omega^{15}\}$ to X^2 results in only 4 distinct values: ω^2, ω^6, ω^{10}, and ω^{14} (which correspond to roots of X^4+1). This is because the order of ω is $2n$ (i.e., $\omega^{2n}=1$), and thus $(\omega^1)^2 = (\omega^9)^2$, $(\omega^3)^2 = (\omega^{11})^2$, $(\omega^5)^2 = (\omega^{13})^2$, and $(\omega^7)^2 = (\omega^{15})^2$. Therefore, we only need to evaluate $A_0(X^2)$ and $A_1(X^2)$ at 4 distinct x values instead of 8, where each evaluation requires a constant number of arithmetic operations: computing 1

multiplication and 1 addition (where we use the results from FFT Level 1 as building blocks, and the computational structure of FFT Level 2 is the same as that of FFT Level 1). There are a total of 2 polynomials to evaluate (i.e., $A_0(X^2), A_1(X^2)$); thus, we compute the FFT a total of $2 \cdot 4 = 8$ times.

FFT Level $l = 3$: Based on the evaluation results from FFT Level 2 as building blocks, we evaluate $A(X)$ at $X = \{\omega^1, \omega^3, \cdots, \omega^{15}\}$. For this last level of computation, we need to evaluate all 8 distinct X values, since they are all unique values, and each evaluation requires a constant number of arithmetic operations: computing 1 multiplication and 1 addition. There is a total of 1 polynomial to evaluate (i.e., $A(X)$); thus, we compute the FFT a total of $1 \cdot 8 = 8$ times.

Generalization: Suppose that the degree of the target polynomial to evaluate is at most $n-1$ (with n terms), and we define $L = \log_2 n$ (i.e., the total number of FFT levels). Then, the forward FFT operation requires a total of L FFT levels, where each l-th level requires the evaluation of 2^{L-l} polynomials at 2^l distinct X values. Therefore, the total number of FFT computations for the forward FFT is: $\sum_{l=1}^{L}(2^{L-l} \cdot 2^l) = L \cdot 2^L = n \log n$. Therefore, the time complexity of the forward FFT is $O(n \log n)$.

Using the FFT technique, we reduce the number of x points to evaluate to half as the level decreases (while the number of polynomials to evaluate doubles), and their growth and reduction cancel each other, resulting in $O(n)$ for each level. Since there are $\log n$ such levels, the total time complexity is $O(n \log n)$. The core enabler of this optimization is the special property of the x evaluation coordinates: its power (i.e., ω^i) is cyclic. To enforce this cyclic property, this negacyclic FFT uses the odd powers of a primitive $2n$-th root; the ordinary cyclic DFT instead uses all powers of a primitive n-th root.

A-16.3 Point-wise Multiplication

Once we have applied the forward FFT operation (§A-16.2) to polynomial $A(X)$ and $B(X)$ as $\vec{y}^{\langle a \rangle}$ and $\vec{y}^{\langle b \rangle}$, computing the point-value representation of $C(X) = A(X) \cdot B(X)$ can be done in $O(n)$ using the Hadamard product $\vec{y}^{\langle c \rangle} = \vec{y}^{\langle a \rangle} \odot \vec{y}^{\langle b \rangle}$ (as explained in §A-16.1).

A-16.4 Inverse FFT (or NTT)

So far, we have computed:

$C(X) : \big((x_0, y_0^{\langle c \rangle}), (x_1, y_1^{\langle c \rangle}), \cdots (x_{n-1}, y_{n-1}^{\langle c \rangle})\big)$, where $y_i^{\langle c \rangle} = y_i^{\langle a \rangle} \cdot y_i^{\langle b \rangle}$

Our final step is to convert $y_i^{\langle c \rangle}$ back to $\vec{c}$, the polynomial coefficients of $C(X)$. We call this reversing operation the inverse FFT.

Given an $(n-1)$-degree polynomial $C(X) = \sum_{i=0}^{n-1} c_i X^i$, the forward FFT process is computationally equivalent to evaluating the polynomial at n distinct roots of $X^n + 1$, which are $2n$-th roots of unity as follows:

$y_i^{\langle c \rangle} = C(\omega^{2i+1}) = \sum_{j=0}^{n-1} c_j \cdot (\omega^{2i+1})^j = \sum_{j=0}^{n-1} c_j \cdot \omega^{(2i+1)j}$

The above evaluation is equivalent to computing the following matrix-vector multiplication:

$$\vec{y}^{\langle c \rangle} = W \cdot \vec{c}, \quad \text{where } W = \begin{bmatrix} (\omega^1)^0 & (\omega^1)^1 & \cdots & (\omega^1)^{n-1} \\ (\omega^3)^0 & (\omega^3)^1 & \cdots & (\omega^3)^{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ (\omega^{2n-1})^0 & (\omega^{2n-1})^1 & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}, \quad \vec{c} = (c_0, c_1, \cdots, c_{n-1})$$

We denote each element of W as: $(W)_{i,j} = \omega^{(2i+1)j}$. The inverse FFT is a process of reversing the above computation. For this inversion, our goal is to find an inverse matrix W^{-1} such that $W^{-1} \cdot \vec{y}^{\langle c \rangle} = W^{-1} \cdot (W \cdot \vec{c}) = (W^{-1} \cdot W) \cdot \vec{c} = I_n \cdot \vec{c} = \vec{c}$. As a solution, we propose the inverse matrix W^{-1} as follows:

$(W^{-1})_{j,k} = n^{-1} \cdot \omega^{-(2k+1)j}$

Now, we will show why $W^{-1} \cdot W = I_n$. Each element of $W^{-1} \cdot W$ is computed as:

$(W^{-1} \cdot W)_{j,i} = \sum_{k=0}^{n-1} (n^{-1} \cdot \omega^{-(2k+1)j} \cdot \omega^{(2k+1)i}) = n^{-1} \cdot \sum_{k=0}^{n-1} \omega^{(2k+1)(i-j)}$

In order for $W^{-1} \cdot W$ to be I_n, the following should hold:

$$(W^{-1} \cdot W)_{j,i} = \begin{cases} 1 \text{ if } j = i \\ 0 \text{ if } j \neq i \end{cases}$$

If $j = i$ holds, the above condition is satisfied because:

$(W^{-1} \cdot W)_{j,i} = n^{-1} \cdot \sum_{k=0}^{n-1} \omega^{(2k+1)(0)} = n^{-1} \cdot \sum_{k=0}^{n-1} 1 = n^{-1} \cdot n = 1$

In the case of $j \neq i$, we will leverage the Geometric Sum formula $\sum_{i=0}^{n-1} x^i = \frac{x^n - 1}{x - 1}$ (the proof is provided below):

⟨Theorem A-16.4⟩ Geometric Sum Formula

Let the geometric sum $S_n = 1 + x + x^2 + \cdots + x^{n-1}$
Then, $x \cdot S_n = x + x^2 + x^3 + \cdots + x^n$
$x \cdot S_n - S_n = (x + x^2 + x^3 + \cdots + x^n) - (1 + x + x^2 + \cdots + x^{n-1}) = x^n - 1$
$S_n \cdot (x - 1) = x^n - 1$
$S_n = \frac{x^n - 1}{x - 1}$ ▷ with the constraint that $x \neq 1$

Our goal is to compute $\sum_{k=0}^{n-1} \omega^{(2k+1)(i-j)} = \omega^{i-j} \sum_{k=0}^{n-1} (\omega^{2(i-j)})^k$. Leveraging the Geometric Sum formula with $x = \omega^{2(i-j)}$:

$\omega^{i-j} \sum_{k=0}^{n-1} (\omega^{2(i-j)})^k = \omega^{i-j} \frac{(\omega^{2(i-j)})^n - 1}{\omega^{2(i-j)} - 1}$

$= \omega^{i-j} \frac{(\omega^{2n})^{i-j} - 1}{\omega^{2(i-j)} - 1}$

$= \omega^{i-j} \frac{(1)^{i-j} - 1}{\omega^{2(i-j)} - 1}$ ▷ since the order of ω is $2n$, $\omega^{2n} = 1$

▷ Here, the denominator can't be 0. Since $i \neq j$ and $|i - j| < n$, the exponent $2(i - j)$ is not a multiple of $2n$.

$= 0$

Thus, $(W^{-1} \cdot W)_{j,i}$ is 1 if $j = i$, and 0 if $j \neq i$. Therefore, the inverse FFT can be computed as $\vec{c} = W^{-1} \cdot y^{\langle c \rangle}$, where:

$c_i = \sum_{j=0}^{n-1} (W^{-1})_{i,j} \cdot y_j^{\langle c \rangle}$

$= \sum_{j=0}^{n-1} \left(n^{-1} \cdot \omega^{-(2j+1)i} \right) \cdot y_j^{\langle c \rangle}$ ▷ since $(W^{-1})_{j,k} = n^{-1} \cdot \omega^{-(2k+1)j}$, and $(W^{-1})_{i,j} = n^{-1} \cdot \omega^{-(2j+1)i}$

$= n^{-1} \cdot \sum_{j=0}^{n-1} y_j^{\langle c \rangle} \cdot \omega^{-(2j+1)i}$

$= n^{-1} \sum_{j=0}^{n-1} y_j^{\langle c \rangle} \cdot (\omega^{-2ji} \cdot \omega^{-i})$ ▷ since $\omega^{-(2j+1)i} = \omega^{-2ji-i} = \omega^{-2ji} \cdot \omega^{-i}$

$= n^{-1} \omega^{-i} \sum_{j=0}^{n-1} y_j^{\langle c \rangle} (\omega^{-2})^{ji}$

By reusing the recursive splitting technique explained in the forward process (§A-16.2.2), this inverse operation also achieves a time complexity of $O(n \log n)$.

Part II

Post-quantum Cryptography

This chapter explains lattice-based cryptographic schemes: LWE cryptosystem, RLWE cryptosystem, GLWE cryptosystem, GLev cryptosystem, and GGSW cryptosystem. These are the essential building blocks for FHE schemes.

Required Background

- §A-2: Group
- §A-3: Field
- §A-5: Polynomial Ring

B-1 Lattice-based Cryptography

- Reference: Lattice-Based Cryptography and the Learning with Errors Problem [6]

Lattice-based cryptography provides candidates for post-quantum security, under precisely specified computational assumptions and parameter choices. Resistance to every quantum attack is not proved unconditionally. This section introduces the LWE (Learning with Errors), RLWE (Ring Learning with Errors), GLWE, GLev, and GGSW constructions used later.

B-1.1 Overview

All equations in this overview are over $\mathbb{Z}_q$. Masks are sampled uniformly and independently; errors are small integers from a specified distribution, reduced modulo q only for arithmetic.

Suppose we have a single unknown k-dimensional vector $\vec{s}$ as a secret key, many publicly known k-dimensional vectors $\vec{a}^{\langle i \rangle}$.

And suppose we have a large set of the following dot products $\vec{s} \cdot \vec{a}^{\langle i \rangle}$:

$$\vec{s} \cdot \vec{a}^{\langle 0 \rangle} = s_0 \cdot a_0^{\langle 0 \rangle} + s_1 \cdot a_1^{\langle 0 \rangle} + \cdots + s_{k-1} \cdot a_{k-1}^{\langle 0 \rangle} = b^{\langle 0 \rangle}$$
$$\vec{s} \cdot \vec{a}^{\langle 1 \rangle} = s_0 \cdot a_0^{\langle 1 \rangle} + s_1 \cdot a_1^{\langle 1 \rangle} + \cdots + s_{k-1} \cdot a_{k-1}^{\langle 1 \rangle} = b^{\langle 1 \rangle}$$
$$\vec{s} \cdot \vec{a}^{\langle 2 \rangle} = s_0 \cdot a_0^{\langle 2 \rangle} + s_1 \cdot a_1^{\langle 2 \rangle} + \cdots + s_{k-1} \cdot a_{k-1}^{\langle 2 \rangle} = b^{\langle 2 \rangle}$$
$$\vdots$$

In the noiseless case, k samples determine $\vec{s}$ if their coefficient matrix is invertible over $\mathbb{Z}_q$. For prime q, this means full rank; over a composite modulus the determinant must be a unit. Merely having k equations does not ensure independence or unique recovery.

However, suppose that in each equation above, we randomly add an unknown small noise $e^{\langle i \rangle}$ (i.e., error) as follows:

$$b^{\langle 0 \rangle} = s_0 \cdot a_0^{\langle 0 \rangle} + s_1 \cdot a_1^{\langle 0 \rangle} + \cdots + s_{k-1} \cdot a_{k-1}^{\langle 0 \rangle} + e^{\langle 0 \rangle} \pmod{q}$$
$$b^{\langle 1 \rangle} = s_0 \cdot a_0^{\langle 1 \rangle} + s_1 \cdot a_1^{\langle 1 \rangle} + \cdots + s_{k-1} \cdot a_{k-1}^{\langle 1 \rangle} + e^{\langle 1 \rangle} \pmod{q}$$
$$b^{\langle 2 \rangle} = s_0 \cdot a_0^{\langle 2 \rangle} + s_1 \cdot a_1^{\langle 2 \rangle} + \cdots + s_{k-1} \cdot a_{k-1}^{\langle 2 \rangle} + e^{\langle 2 \rangle} \pmod{q}$$
$$\vdots$$

Noise prevents simply treating these as exact linear equations for $\vec{s}$. If each of k errors has r possible values, enumerating all r^k error vectors is one possible attack. This is an upper bound for that particular enumeration, not a lower bound for all algorithms and not a security proof. Additional samples may help an attacker and can always be ignored. LWE hardness is a computational assumption for specified dimension, modulus, secret and error distributions, and sample count; sufficiently small or structured errors can make some instances easy.

Based on this intuition, the mathematical hard problem that constitutes lattice-based cryptography is as follows:

⟨Summary B-1.1⟩ The LWE (Learning with Errors) and RLWE Problems

LWE Problem

Consider samples of the form: $b = \vec{s} \cdot \vec{a} + e$ (where e is a small noise to be explained later). For each sample, choose $\vec{a}$ uniformly from $\mathbb{Z}_q^k$ and independently draw a small integer error e from an explicitly specified distribution χ, then reduce e modulo q. The secret $\vec{s}$ is sampled once from its specified distribution and reused. The error is not uniform over $\mathbb{Z}_q$. A security experiment permits a polynomial number of such samples in the security parameter.

$(\vec{a}^{\langle 1 \rangle}, b^{\langle 1 \rangle})$, where $b^{\langle 1 \rangle} = \vec{s} \cdot \vec{a}^{\langle 1 \rangle} + e^{\langle 1 \rangle}$
$(\vec{a}^{\langle 2 \rangle}, b^{\langle 2 \rangle})$, where $b^{\langle 2 \rangle} = \vec{s} \cdot \vec{a}^{\langle 2 \rangle} + e^{\langle 2 \rangle}$
$(\vec{a}^{\langle 3 \rangle}, b^{\langle 3 \rangle})$, where $b^{\langle 3 \rangle} = \vec{s} \cdot \vec{a}^{\langle 3 \rangle} + e^{\langle 3 \rangle}$
$\vdots$

Suppose that the attacker has a sufficiently large number of $(\vec{a}^{\langle i \rangle}, b^{\langle i \rangle})$ tuples. Given this setup, the following hard problems constitute the defense mechanism of the LWE (Learning with Errors) cryptosystem:

- **Search problem:** Given the samples, recover the secret $\vec{s}$. The search-LWE assumption states that no efficient algorithm succeeds with non-negligible probability for the selected parameter family.
- **Decision problem:** Distinguish LWE samples $(\vec{a}, \langle \vec{a}, \vec{s} \rangle + e)$ from independent uniform pairs in $\mathbb{Z}_q^k \times \mathbb{Z}_q$. The decisional-LWE assumption states that every efficient distinguisher has negligible advantage for the selected parameter family.

Search-to-decision reductions require hypotheses; the two formulations are not unconditionally interchangeable for every modulus and distribution. For example, Regev's reduction in Lemma 4.2 assumes a prime modulus bounded by a polynomial in the dimension, with its stated error and oracle conditions. His worst-case lattice reduction also has explicit noise and modulus requirements and is a quantum reduction. See Regev, On Lattices, Learning with Errors, Random Linear Codes, and Cryptography, Theorem 1.1 and Section 4.

RLWE Problem

For the algebraic RLWE form, use $B = AS + E$ in $\mathbb{Z}_q[X]/(X^n + 1)$, with polynomial representatives of degree less than n. Search asks for S; decision distinguishes these pairs from uniform ring pairs. This notation alone does not transfer every LWE security theorem: the ring, modulus, and error/secret distributions must satisfy the hypotheses of a ring-LWE result.

Small-secret scope: A normal-form reduction can replace a uniform LWE secret by a secret drawn from the *error distribution*. For a prime-power modulus, choose an invertible mask matrix A_0 using only its masks, with $\vec{b}_0 = A_0\vec{s} + \vec{e}_0$. For a fresh row sample $(\vec{a}, b)$ set

$$\vec{a}' = -\vec{a}A_0^{-1}, \qquad b' = b - \vec{a}A_0^{-1}\vec{b}_0 = \vec{a}'\vec{e}_0 + e.$$

The new mask is uniform and the new secret has distribution χ^k; uniform input pairs also map to uniform pairs. The preprocessing data are internal to this reduction. In the ring version the same identity requires an invertible initial ring mask. See Applebaum–Cash–Peikert–Sahai, Section 3.1, Lemma 2. This argument does not turn Gaussian errors into a uniformly ternary secret. The later ternary-secret

constructions therefore use a decisional assumption for their actual distribution, unless a separate distribution-matching reduction is supplied. Neither a dimension count nor the adjective "small" supplies that reduction.

B-1.2 LWE Cryptosystem

For the following correctness formulas, assume $t \mid q$, set $\Delta = q/t$, and let $[x]_q$ denote the centered integer representative of x mod q. Rounding is performed on this representative, and the decoded result is reduced modulo t. All bit-shift descriptions apply only when Δ is a power of two; in general scaling means multiplication or division by Δ. The complete ciphertext is $(\vec{a}, b)$, not b alone.

The LWE cryptosystem uses the following encryption formula: $b = \vec{s} \cdot \vec{a} + \Delta \cdot m + e$ (where $\vec{s}$ is a secret key, $\vec{a}$ is a publicly known random vector picked per encryption, m is a plaintext, e is small noise randomly picked per encryption from a normal distribution, and b is a ciphertext). Δ is a scaling factor of the plaintext M (shifting m by $\log_2\Delta$ bits to the left). Before encrypting the plaintext, we left-shift the plaintext several bits (i.e., $\log_2\Delta$ bits) to secure sufficient space to store the error in the lower bits.

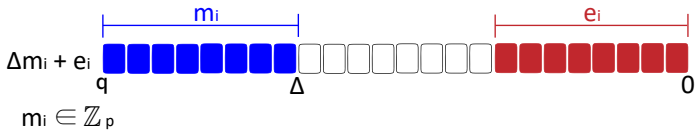


Figure 8: An illustration of LWE's plaintext scaling and adding a noise: $\Delta \cdot m + e \in \mathbb{Z}_q$

Figure 8 visually illustrates the term $\Delta \cdot m + e$, where the plaintext m left-shifted by $\log_2\Delta$ bits and noised by the noise e. The actual encryption and decryption formulas are as follows:

⟨Summary B-1.2⟩ Lattice-based LWE Cryptosystem

- **Encryption:** $b^{\langle i \rangle} = \vec{s} \cdot \vec{a}^{\langle i \rangle} + \Delta \cdot m^{\langle i \rangle} + e^{\langle i \rangle}$, where $b^{\langle i \rangle}$ and $\vec{a}^{\langle i \rangle}$ are publicly known also to the attacker, while $\vec{s}, m^{\langle i \rangle}, e^{\langle i \rangle}$ are unknown (only known by the secret key owner).
- **Decryption:** $\left\lceil [b^{\langle i \rangle} - \vec{s} \cdot \vec{a}^{\langle i \rangle}]_q / \Delta \right\rfloor \bmod t = m^{\langle i \rangle} \left(\text{provided } |e^{\langle i \rangle}| < \frac{\Delta}{2} \right)$

$\lfloor \rceil_\Delta$ means rounding the number to the nearest multiple of Δ. For example, $\lfloor 16 \rceil_{10} = 20$, which is rounding 16 to the nearest multiple of 10. As another example, $\lfloor 17 \rceil_8 = 16$, which rounds 17 to the nearest multiple of 8 (note that 17 is closer to 16 than to 24; thus, it is rounded to 16).

Correctness: In the decryption scheme, computing $b^{\langle i \rangle} - \vec{s} \cdot \vec{a}^{\langle i \rangle}$ gives $\Delta \cdot m^{\langle i \rangle} + e^{\langle i \rangle}$, which is Figure 8. Then, $\lceil \Delta \cdot m^{\langle i \rangle} + e^{\langle i \rangle} \rfloor_\Delta$ (i.e., rounding the value to the nearest multiple of Δ) gives $\Delta \cdot m^{\langle i \rangle}$, provided the added noise $|e^{\langle i \rangle}| < \frac{\Delta}{2}$. That is, if the noise is less than $\frac{\Delta}{2}$, it will disappear during the rounding. Finally, right-shifting $\Delta \cdot m^{\langle i \rangle}$ by $\log_2\Delta$ bits gives $m^{\langle i \rangle}$. To summarize, if we ensure $|e^{\langle i \rangle}| < \frac{\Delta}{2}$ (which is why the noise $e^{\langle i \rangle}$ should be smaller than this threshold), then we can eliminate $e^{\langle i \rangle}$ during the decryption's rounding process and retrieve the original $\Delta \cdot m^{\langle i \rangle}$. The reason we scaled $m^{\langle i \rangle}$ by Δ is to: (i) create space for storing $e^{\langle i \rangle}$ in the lower bits during encryption such that the noise bits do not interfere with the plaintext bits (to avoid corrupting the plaintext bits); and (ii) blow away the noise $e^{\langle i \rangle}$ stored in the lower bits during decryption without corrupting the plaintext $m^{\langle i \rangle}$.

Security: Fix the actual secret distribution, error distribution, modulus, dimension, and sample bound Q. Let $\delta(T, Q)$ be the maximum difference in a time-T distinguisher's acceptance probabilities between a real LWE sample oracle and an independent uniform-pair oracle. The same secret is reused, while every query has a fresh independent uniform mask and fresh error. This is a decisional assumption, not merely a claim that key recovery is difficult.

In the adaptive left/right encryption game, a hidden bit ϑ selects one of each submitted pair (m_0, m_1). Define the guessing bias $\epsilon = |\Pr[\widehat{\vartheta} = \vartheta] - 1/2|$. A reduction chooses ϑ itself, takes $(\vec{a}, z)$ from its sample oracle for each query, and returns $(\vec{a}, z + \Delta m_\vartheta)$. With real samples this is exactly encryption. With uniform samples, the fresh z remains uniform after any message offset, even conditional on the entire preceding transcript. Induction therefore makes the complete view independent of ϑ. Outputting whether the adversary guessed correctly gives

$$\epsilon \leq \delta(T', Q),$$

where T' includes the simulation overhead. No extra factor Q is needed because the assumed advantage already permits Q samples. If IND advantage is instead defined as the output-probability difference between the two message experiments, the corresponding bound is $2\delta(T', Q)$. This covers repeated and adaptively chosen plaintexts through a classical encryption interface, without additional secret-dependent public material. For quantum attackers the underlying decisional assumption must cover quantum distinguishers; this statement does not grant superposition encryption queries.

Decryption subtracts the secret-dependent term, rounds to remove a sufficiently small error, divides by Δ, and reduces modulo t. The noise bound and the security assumption are distinct requirements: small enough for correct decoding does not by itself mean large enough for security.

B-1.3 RLWE Cryptosystem

In the RLWE cryptosystem, the formula in ⟨Summary B-1.2⟩ is the same, but $\vec{s}, \vec{a}^{\langle i \rangle}, b^{\langle i \rangle}, m^{\langle i \rangle}, e^{\langle i \rangle}$ are replaced by polynomials $S, A^{\langle i \rangle}, B^{\langle i \rangle}, M^{\langle i \rangle}, E^{\langle i \rangle}$ as follows:

⟨Summary B-1.3⟩ Lattice-based RLWE Cryptosystem

- **Encryption:** $B^{\langle i \rangle} = S \cdot A^{\langle i \rangle} + \Delta \cdot M^{\langle i \rangle} + E^{\langle i \rangle}$, where $B^{\langle i \rangle}$ and $A^{\langle i \rangle}$ are publicly known also to the attacker, while $S, M^{\langle i \rangle}, E^{\langle i \rangle}$ are unknown (only known by the secret key owner).
- **Decryption:** $\left\lceil [B^{\langle i \rangle} - S \cdot A^{\langle i \rangle}]_q / \Delta \right\rfloor \bmod t = M^{\langle i \rangle}$
  $\left(\text{provided } ||E^{\langle i \rangle}||_\infty < \dfrac{\Delta}{2}, \text{ meaning each coefficient of } E^{\langle i \rangle} \text{ has a magnitude less than } \dfrac{\Delta}{2}\right)$
  $\lceil \rfloor_\Delta$ is equivalent to rounding each term's coefficient in the polynomial.

The security reduction just given applies to ring pairs under the corresponding decisional RLWE assumption, and to k ring masks under the actual GLWE assumption. Expanding a ring product into coefficient equations gives a structured negacyclic matrix, not independent uniform LWE rows. Thus the proof template transfers, but a scalar LWE hardness theorem does not automatically certify a ring or module parameter family.

B-2 LWE Cryptosystem

- Reference: TFHE Deep Dive: Part I - Ciphertext types [7]

B-2.1 Setup

Initially assume $2 \leq t < q$ and $t \mid q$. The notation $e \xleftarrow{\chi_\sigma} \mathbb{Z}_q$ means sampling an integer from a specified discrete Gaussian (or a specified rounded Gaussian) and reducing it modulo q. Noise bounds concern its integer lift, not the nonnegative residue. A Gaussian has unbounded tails, so correctness is conditional on the bound and parameters must make its failure probability suitably small. Let $[0, t-1]$ be the plaintext range, and $[0, q-1]$ the ciphertext range, where $t \ll q$ (t is much smaller than q). Randomly pick a vector $\vec{s}$ of length k comprising k ternary numbers sampled from $\{-1, 0, 1\}$ as a secret key (denoted as $\vec{s} \xleftarrow{\$} \{-1, 0, 1\}^k$). Let $\Delta = \frac{q}{t}$, the scaling factor of plaintext.

B-2.2 Encryption

1. Suppose we have a plaintext $m \in \mathbb{Z}_t$ to encrypt.
2. Randomly pick a vector $\vec{a} \in \mathbb{Z}_q^k$ (of length k) as a one-time random public mask (denoted as $\vec{a} \xleftarrow{\$} \mathbb{Z}_q^k$).
3. Randomly pick a small one-time noise $e \in \mathbb{Z}_q$ sampled from the Gaussian distribution χ_σ (denoted as $e \xleftarrow{\chi_\sigma} \mathbb{Z}_q$).
4. Scale m by Δ, which is to compute $\Delta \cdot m$. This converts $m \in \mathbb{Z}_t$ into $\Delta \cdot m \in \mathbb{Z}_q$.
5. Compute $b = \vec{a} \cdot \vec{s} + \Delta \cdot m + e \in \mathbb{Z}_q$.

The LWE encryption formula is summarized as follows:

⟨Summary B-2.2⟩ LWE Encryption

Initial Setup: $\Delta = \frac{q}{t}$, $\vec{s} \xleftarrow{\$} \{-1, 0, 1\}^k$, where t divides q

Encryption Input: $m \in \mathbb{Z}_t$, $\vec{a} \xleftarrow{\$} \mathbb{Z}_q^k$, $e \xleftarrow{\chi_\sigma} \mathbb{Z}_q$

1. Scale up $m \longrightarrow \Delta \cdot m \ \in \mathbb{Z}_q$
2. Compute $b = \vec{a} \cdot \vec{s} + \Delta m + e \pmod q$
3. $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) = (\vec{a}, b) \ \in \mathbb{Z}_q^{k+1}$

B-2.3 Decryption

In the formulas below, the phase is first lifted to a centered representative $[b - \vec{a} \cdot \vec{s}]_q$. The notation $\mathsf{LWE}_{S,\sigma}^{-1}$ denotes this phase extraction; the subsequent rounding and reduction modulo t complete message decryption. The same convention applies to RLWE and GLWE. Use a fixed nearest-integer rule, such as $\lceil x \rfloor = \lfloor x + 1/2 \rfloor$.

1. Given the ciphertext $(\vec{a}, b)$ where $b = \vec{a} \cdot \vec{s} + \Delta \cdot m + e \in \mathbb{Z}_q$, compute $b - \vec{a} \cdot \vec{s}$, which gives the same value as $\Delta \cdot m + e \in \mathbb{Z}_q$.
2. Round $\Delta \cdot m + e \in \mathbb{Z}_q$ to the nearest multiple of Δ (i.e., round it as a base Δ number), which is denoted as $\lceil \Delta \cdot m + e \rfloor_\Delta$. This rounding operation successfully eliminates e and gives Δm, provided e is small enough to be $|e| < \frac{\Delta}{2}$. If $|e| \geq \frac{\Delta}{2}$, then some of the higher bits of the noise e will overlap with the plaintext m, won't be blown away, and will corrupt some lower bits of the plaintext m.
3. Compute $\frac{\Delta m}{\Delta}$, which is equivalent to right-shifting $\lceil \Delta \cdot m + e \rfloor_\Delta$ by $\log_2 \Delta$ bits. (Here we assume Δ is a power of 2; if Δ is not a power of 2, scaling up or down m by Δ is equivalent to multiplying or dividing the value by Δ.)

The LWE decryption formula is summarized as follows:

⟨Summary B-2.3⟩ LWE Decryption

Decryption Input: $\mathsf{ct} = (\vec{a}, b) \in \mathbb{Z}_q^{k+1}$

1. $\mathsf{LWE}_{S,\sigma}^{-1}(\mathsf{ct}) = b - \vec{a} \cdot \vec{s} = \Delta m + e \pmod q$
2. Scale down $\left\lceil \frac{\Delta m + e}{\Delta} \right\rfloor \bmod t = m \in \mathbb{Z}_t$

For correct decryption, the noise e should be $|e| < \frac{\Delta}{2}$.

During decryption, the secret key owner can subtract $\vec{a} \cdot \vec{s}$ from b because he can directly compute $\vec{a} \cdot \vec{s}$ by using his secret key $\vec{s}$.

Scaling places neighboring message representatives Δ apart, so nearest-integer decoding tolerates errors with $|e| < \Delta/2$. If Δ is a power of two, encryption scaling is a left shift, as in Figure 8. Decryption must round to the nearest multiple first and only then divide or right-shift. A bare right shift is incorrect for signed errors: $\Delta = 8$, $m = 1$, and $e = -1$ give $7/8$, whose nearest integer is 1 but whose truncation is 0.

B-2.3.1 In the Case of t not Dividing q

In Summary B-2.2 (§B-2.2), we assumed that t divides q. In this case, there is no upper or lower limit on the size of plaintext m: its value is allowed to wrap around modulo t indefinitely, yet the decryption works correctly. This is because any wrap-around of m modulo t is carried by the mod-q reduction of the scaled plaintext during decryption.

If $t \nmid q$, set $\Delta = \lfloor q/t \rfloor$ and $r = q - \Delta t$. A plaintext overflow $m = m' + vt$ changes the effective error from e to $e - rv$. Therefore $q \gg t$ alone does not guarantee decryption for arbitrarily many overflows. The derivation below gives an explicit sufficient condition.

For this derivation let $t \geq 3$ be odd; primality is unnecessary here, though batching may impose additional conditions (§A-11.6.1). Use centered integer representatives: $m' \in [-(t-1)/2, (t-1)/2]$ and ciphertext representatives in $[-q/2, q/2) \cap \mathbb{Z}$. Write $m = m' + vt$ with integer v. Then:

$$\left\lfloor \frac{q}{t} \right\rfloor \cdot m + e$$

$$
\begin{aligned}
&= \left\lfloor \frac{q}{t} \right\rfloor \cdot m' + \left\lfloor \frac{q}{t} \right\rfloor \cdot vt + e \triangleright \text{ applying } m = m' + vt \\
&= \left\lfloor \frac{q}{t} \right\rfloor \cdot m' + \frac{q}{t} \cdot vt - \left(\frac{q}{t} - \left\lfloor \frac{q}{t} \right\rfloor \right) \cdot vt + e \\
&= \left\lfloor \frac{q}{t} \right\rfloor \cdot m' + qv - \left(\frac{q}{t} - \left\lfloor \frac{q}{t} \right\rfloor \right) \cdot vt + e \\
&= \left\lfloor \frac{q}{t} \right\rfloor \cdot m' + qv - \epsilon \cdot vt + e \triangleright \text{ where } \epsilon = \frac{q}{t} - \left\lfloor \frac{q}{t} \right\rfloor, \text{ a fractional value between } [0, 1)
\end{aligned}
$$

The actual decoder includes rounding: $\lceil [b - \vec{a} \cdot \vec{s}]_q / \Delta \rfloor \bmod t$. Substituting the preceding identity gives:

$$
\begin{aligned}
&\left\lceil \frac{1}{\Delta} \cdot (b - \vec{a} \cdot \vec{s} \bmod q) \right\rfloor \bmod t \\
&= \left\lceil \frac{1}{\Delta} \cdot (\Delta m + e \bmod q) \right\rfloor \bmod t \\
&= \left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot (m' + vt) + e \bmod q \right) \right\rfloor \bmod t \\
&= \left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot m' + \left\lfloor \frac{q}{t} \right\rfloor \cdot vt + e \bmod q \right) \right\rfloor \bmod t \\
&= \left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot m' + \left(\frac{q}{t} - \epsilon \right) \cdot vt + e \bmod q \right) \right\rfloor \bmod t \\
&= \left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot m' + vq - \epsilon vt + e \bmod q \right) \right\rfloor \bmod t \\
&= \left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e \bmod q \right) \right\rfloor \bmod t
\end{aligned}
$$

In order for the decryption to work, we ideally want to eliminate the inner modulo q reduction. That is, assuming the centered residue system $\left[-\frac{q}{2}, \frac{q}{2} \right) \cap \mathbb{Z}$, we want $-\frac{q}{2} \leq \left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e < \frac{q}{2}$, or more strictly, $\left| \left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e \right| < \frac{q}{2}$.

To ensure these conditions hold, we will make a special assumption: $|-\epsilon vt + e| < \frac{\Delta}{2}$. Applying this assumption to the expression above, we can derive the following relation:

$$
\begin{aligned}
&\left| \left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e \right| \\
&\leq \left| \left\lfloor \frac{q}{t} \right\rfloor \cdot m' \right| + |-\epsilon vt + e| \\
&\leq \left| \left\lfloor \frac{q}{t} \right\rfloor \cdot \frac{t-1}{2} \right| + |-\epsilon vt + e| \triangleright \text{ only odd } t \text{ is needed for this centered range} \\
&\leq \left| \frac{q}{t} \cdot \frac{t-1}{2} \right| + |-\epsilon vt + e| \\
&= \frac{q}{2} - \frac{q}{2t} + |-\epsilon vt + e| \\
&< \frac{q}{2} - \frac{q}{2t} + \frac{\Delta}{2} \triangleright \text{ applying our special assumption } |-\epsilon vt + e| < \frac{\Delta}{2} \\
&= \frac{q}{2} + \frac{1}{2} \cdot \left(\Delta - \frac{q}{t} \right)
\end{aligned}
$$

$< \frac{q}{2}$ ▷ since $\Delta < \frac{q}{t}$

Therefore, if we assume $|-\epsilon vt + e| < \frac{\Delta}{2}$, then $\left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e \bmod q$ can be simplified to $\left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e$. We continue with the following derivation:

$$\left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e \bmod q \right) \right\rfloor \bmod t$$

$$= \left\lceil \frac{1}{\lfloor \frac{q}{t} \rfloor} \cdot \left(\left\lfloor \frac{q}{t} \right\rfloor \cdot m' - \epsilon vt + e \right) \right\rfloor \bmod t$$

$$= \left\lceil m' + \frac{-\epsilon vt + e}{\lfloor \frac{q}{t} \rfloor} \right\rfloor \bmod t$$

$$= m' + \left\lceil \frac{-\epsilon vt + e}{\lfloor \frac{q}{t} \rfloor} \right\rfloor \bmod t \text{ ▷ since } \lceil m' \rfloor = m'$$

$$= m' \bmod t \text{ ▷ applying special assumption } |-\epsilon vt + e| < \frac{\Delta}{2} = \frac{\lfloor \frac{q}{t} \rfloor}{2}$$

Thus $|e - rv| < \Delta/2$, where $r = \epsilon t = q \bmod t$, is a sufficient correctness condition. A simpler conservative bound is $|e| + r|v| < \Delta/2$. It requires a bound on v as well as on the noise. For example, $q = 1024$, $t = 3$, $\Delta = 341$, $m = 513 = 171t$, and $e = 0$ give centered phase -171 and decoded value $\lceil -171/341 \rfloor \bmod 3 = 2$, although $m \bmod 3 = 0$. Increasing q relative only to t and e cannot justify unbounded plaintext overflow.

We can generalize the formula for the plaintext's scaling factor in Summary B-2.2 (in §B-2.2) as $\left\lfloor \frac{q}{t} \right\rfloor$, where t is an odd integer.

⟨Summary B-2.3.1⟩ Noise Budget for an Odd Plaintext Modulus t

Given the plaintext's scaling factor $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$ and t is an odd integer, the LWE decryption works correctly as long as the error-bounding condition holds:
$|-\epsilon vt + e| < \frac{\Delta}{2}$
, where $\epsilon = \frac{q}{t} - \left\lfloor \frac{q}{t} \right\rfloor$ is a fractional value between $[0, 1)$, and v accounts for the t-overflows of the plaintext m.

B-3 RLWE Cryptosystem

The RLWE cryptosystem's ciphertext is a tuple (A, B), where $B = S \cdot A + \Delta \cdot M + E$. The random public mask A and the secret key S are $(n-1)$-degree polynomials. The message M and the noise E are $(n-1)$-degree polynomials. Like in LWE, a new random public mask A is created for each ciphertext, whereas the same secret key S is used for all ciphertexts. In this section, we denote each ciphertext instance as (A, B) instead of $(A^{\langle i \rangle}, b^{\langle i \rangle})$ for simplicity.

In RLWE, all polynomials are computed in the polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$, where $x^n + 1$ is a cyclotomic polynomial with $n = 2^f$ for some integer f and the polynomial coefficients are in $\mathbb{Z}_q$. Thus, all polynomials in RLWE have the coefficient range $\mathbb{Z}_q$ and the maximum polynomial degree of $n - 1$. For simplicity, we denote $\mathcal{R}_{\langle n,q \rangle} = \mathbb{Z}_q[x]/(x^n + 1)$.

B-3.1 Setup

Let t be the size of plaintext, and q the size of ciphertext, where $t < q$ (t is much smaller than q) and $t|q$ (i.e., t divides q). Randomly pick a $(n-1)$-degree polynomial $S \in \mathcal{R}_{\langle n,q \rangle}$ whose coefficients are either $\{-1, 0, 1\}$ as a secret key. Let $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$ be the scaling factor of plaintext.

Notice that RLWE's setup parameters are similar to that of LWE. One difference is that S is not a vector of length k sampled from $\{-1, 0, 1\}$, but an $(n-1)$-degree polynomial encoding n secret coefficients, where each coefficient is a randomly picked ternary number from $\{-1, 0, 1\}$ (denoted as $S \xleftarrow{\$} \mathcal{R}_{\langle n,tern \rangle}$).

B-3.2 Encryption

Polynomial representatives have degree at most $n - 1$, including when leading coefficients vanish. Noise coefficients are integer samples reduced modulo q, using the convention of §B-2.3. A correctness inequality is a condition on these lifted coefficients, not a deterministic guarantee for an unbounded Gaussian distribution.

1. Suppose we have an $(n-1)$-degree polynomial $M \in \mathcal{R}_{\langle n,t \rangle}$ whose coefficients represent the plaintext numbers to encrypt.
2. Randomly pick an $(n-1)$-degree polynomial $A \in \mathcal{R}_{\langle n,q \rangle}$ as a one-time random public mask (denoted as $A \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}$).
3. Randomly pick a small polynomial $E \in \mathcal{R}_{\langle n,q \rangle}$ as a one-time noise, whose n coefficients are small numbers in $\mathbb{Z}_q$ randomly sampled from the Gaussian distribution χ_σ (denoted as $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$).
4. Scale M by Δ, which is to compute $\Delta \cdot M$. This converts $M \in \mathcal{R}_{\langle n,t \rangle}$ into $\Delta \cdot M \in \mathcal{R}_{\langle n,q \rangle}$.
5. Compute $B = A \cdot S + \Delta \cdot M + E \bmod \mathcal{R}_{\langle n,q \rangle}$ (i.e., reduce the degree by n and the coefficient by modulo q).
6. The final ciphertext is (A, B).

The RLWE encryption formula is summarized as follows:

⟨Summary B-3.2⟩ RLWE Encryption

Initial Setup: $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$, $S \xleftarrow{\$} \mathcal{R}_{\langle n,tern \rangle}$

Encryption Input: $M \in \mathcal{R}_{\langle n,t \rangle}$, $A \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}$, $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$

1. Scale up $M \longrightarrow \Delta M \ \in \mathcal{R}_{\langle n,q \rangle}$
2. Compute $B = A \cdot S + \Delta M + E \mod \mathcal{R}_{\langle n,q \rangle}$
3. $\mathsf{RLWE}_{S,\sigma}(\Delta M + E) = (A, B) \ \in \mathcal{R}^2_{\langle n,q \rangle}$

B-3.3 Decryption

1. Given the ciphertext (A, B) where $B = A \cdot S + \Delta \cdot M + E \in \mathcal{R}_{\langle n,q \rangle}$, compute $B - A \cdot S = \Delta \cdot M + E$.
2. Round each coefficient of the polynomial $\Delta \cdot M + E \in \mathcal{R}_{\langle n,q \rangle}$ to the nearest multiple of Δ (i.e., round it as a base Δ number), which is denoted as $\lceil \Delta \cdot M + E \rfloor_\Delta$. This rounding operation successfully eliminates E and gives $\Delta \cdot M$. One caveat is that the noise E's each coefficient e_i should be small enough to be $|e_i| < \frac{\Delta}{2}$ in order to be eliminated during the rounding. Otherwise, some of e_i's higher bits will overlap and corrupt the plaintext m_i coefficient's lower bits and won't be blown away.
3. Compute $\frac{\Delta \cdot M}{\Delta}$, which is equivalent to scaling down each polynomial coefficient in $\Delta \cdot M$ by Δ (or right-shifting each coefficient by $\log_2 \Delta$ bits if Δ is a power of 2).

In summary, the RLWE decryption formula is summarized as follows:

⟨Summary B-3.3⟩ RLWE Decryption

Decryption Input: $\mathsf{ct} = (A, B) \ \in \mathcal{R}^2_{\langle n,q \rangle}$

1. $\mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}) = B - A \cdot S = \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$
2. Scale down $\left\lceil \frac{\Delta M + E}{\Delta} \right\rfloor \mod t = M \ \in \mathcal{R}_{\langle n,t \rangle}$

For $t \mid q$, require $\|E\|_\infty < \Delta/2$. If $t \nmid q$, use the coefficientwise effective-error bound from §B-2.3.1: for unreduced coefficient $m_i = m'_i + v_i t$ with centered m'_i, require $|e_i - (q \bmod t)v_i| < \Delta/2$ under that subsection's odd-t assumptions. The condition $q \gg t$ alone is insufficient.

B-4 GLWE Cryptosystem

The GLWE cryptosystem is a generalized form to encompass both the LWE and RLWE cryptosystems. The GLWE cryptosystem's ciphertext is a tuple $(\{A_i\}_{i=0}^{k-1}, B)$, where $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E$. The public mask $\{A_i\}_{i=0}^{k-1}$ and the secret key $\{S_i\}_{i=0}^{k-1}$ are a list of k $(n-1)$-degree polynomials, each. The message M and the noise E are $(n-1)$-degree polynomials, each. Like in LWE and RLWE, a random public mask A is created for each ciphertext, whereas the same secret key S is used for all ciphertexts. In this section, we denote each ciphertext instance as $(\{A_i\}_{i=0}^{k-1}, B)$ instead of $(\{A_i\}_{i=0}^{k-1,\langle j \rangle}, B^{\langle j \rangle})$ for simplicity.

B-4.1 Setup

Let t be the size of plaintext, and q the size of ciphertext, where $t < q$ (t is much smaller than q) and $t|q$ (i.e., t divides q). Randomly pick a list of k $(n-1)$-degree polynomials as a secret key, where each polynomial coefficient is a randomly picked ternary number in $\{-1, 0, 1\}$ (i.e., $\{S_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}_{\langle n,tern \rangle}^k$). Let $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$ be the scaling factor of plaintext.

Notice that GLWE's setup parameters are similar to that of RLWE. One difference is that S is not an $(n-1)$-degree polynomial encoding n secret coefficients, but a list of k such $(n-1)$-degree polynomials encoding total $n \cdot k$ secret coefficients.

B-4.2 Encryption

Polynomial representatives have degree at most $n-1$, including when leading coefficients vanish. Noise coefficients are integer samples reduced modulo q, using the convention of §B-2.3. A correctness inequality is a condition on these lifted coefficients, not a deterministic guarantee for an unbounded Gaussian distribution.

Suppose we have an $(n-1)$-degree polynomial $M \in \mathcal{R}_{\langle n,t \rangle}$ whose coefficients represent the plaintext numbers to encrypt.

1. Randomly pick a list of k $(n-1)$-degree polynomials $\{A_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}^k$ as a one-time public mask.
2. Randomly pick a small polynomial $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$ as a one-time noise, whose n coefficients are small numbers in $\mathbb{Z}_q$ randomly sampled from the Gaussian distribution χ_σ.
3. Scale M by Δ, which is to compute $\Delta \cdot M$. This converts $M \in \mathcal{R}_{\langle n,t \rangle}$ into $\Delta \cdot M \in \mathcal{R}_{\langle n,q \rangle}$.
4. Compute $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E \in \mathcal{R}_{\langle n,q \rangle}$.

The GLWE encryption formula is summarized as follows:

⟨Summary B-4.2⟩ GLWE Encryption

Initial Setup: $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$, $\{S_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}_{\langle n,tern \rangle}^k$

Encryption Input: $M \in \mathcal{R}_{\langle n,t \rangle}$, $\{A_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}^{k}$, $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$

1. Scale up $M \longrightarrow \Delta M \ \in \mathcal{R}_{\langle n,q \rangle}$
2. Compute $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$
3. $\mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (\{A_i\}_{i=0}^{k-1}, B) \ \in \mathcal{R}_{\langle n,q \rangle}^{k+1}$

B-4.3 Decryption

1. Given the ciphertext $(\{A_i\}_{i=0}^{k-1}, B)$ where $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E \in \mathcal{R}_{\langle n,q \rangle}$, compute $B - \sum_{i=0}^{k-1} (A_i \cdot S_i) = \Delta \cdot M + E$.
2. Round each coefficient of the polynomial $\Delta \cdot M + E \in \mathcal{R}_{\langle n,q \rangle}$ to the nearest multiple of Δ (i.e., round it as a base Δ number), which is denoted as $\lceil \Delta \cdot M + E \rfloor_\Delta$. This operation successfully eliminates E and gives $\Delta \cdot M$. One caveat is that E's each coefficient e_i has to be $|e_i| < \dfrac{\Delta}{2}$ to be eliminated during the rounding. Otherwise, some of e_i's higher bits will overlap the plaintext m_i coefficient's lower bit and won't be eliminated during decryption, corrupting the plaintext m_i.
3. Compute $\dfrac{\Delta \cdot M}{\Delta}$, which is equivalent to scaling down each polynomial coefficient in $\Delta \cdot M$ by Δ.

In summary, the GLWE decryption formula is summarized as follows:

⟨Summary B-4.3⟩ GLWE Decryption

Decryption Input: $\mathsf{ct} = (\{A_i\}_{i=0}^{k-1}, B) \ \in \mathcal{R}_{\langle n,q \rangle}^{k+1}$

1. $\mathsf{GLWE}_{S,\sigma}^{-1}(\mathsf{ct}) = B - \sum_{i=0}^{k-1} (A_i \cdot S_i) = \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$
2. Scale down $\left\lceil \dfrac{\Delta M + E}{\Delta} \right\rfloor \bmod t = M \ \in \mathcal{R}_{\langle n,t \rangle}$

For correct decryption, every noise coefficient e_i of polynomial E should be: $|e_i| < \dfrac{\Delta}{2}$.

B-4.3.1 Discussion

The following size comparisons describe representation and brute-force search space only. Concrete security depends jointly on n, k, q, the error and secret distributions, and sample count; increasing one dimension is not by itself a security reduction.

1. **LWE** is a special case of GLWE where the polynomial ring's degree $n = 1$. That is, all polynomials in $\{A_i\}_{i=0}^{k-1}, \{S_i\}_{i=0}^{k-1}, E$, and M are zero-degree polynomial constants. Instead, there are k such constants for A_i and S_i, so each of them forms a vector.
2. **RLWE** is a special case of GLWE where $k = 1$. That is, the secret key S is a single polynomial S_0, and each encryption is processed by only a single polynomial A_0 as a public mask.

3. **Size of n:** The polynomial $S_i = s_{i,0} + s_{i,1}X + \cdots + s_{i,n-1}X^{n-1}$ has n coefficients. With independent uniform ternary coefficients, the complete secret has 3^{nk} possible values. This is a count of exhaustive-search candidates, not a lower bound on the best attack or a proof of the search assumption (§B-1.1). Higher-degree plaintext polynomials can represent more coefficients; the resulting throughput also depends on the operation costs.
4. **Size of k:** Increasing k adds secret polynomials $(S_0, S_1, \ldots, S_{k-1})$ and public mask polynomials $(A_0, A_1, \ldots, A_{k-1})$. Each ciphertext still has one message polynomial M and one error polynomial E. Comparing two choices of n, k requires their actual ring/module assumptions and distributions, not only the number of secret coefficients.
5. **Reducing the Ciphertext Size:** A seeded ciphertext can store a fresh independent public seed d and B. A specified public expansion G produces all kn mask coefficients, with parameter/index domain separation and an unbiased coefficient mapping. Its input is never the secret. Because the seed is public, ordinary hidden-seed PRG security does not justify replacing $(d, G(d))$ by (d, A) for independent uniform A: testing $A = G(d)$ distinguishes those distributions. For a fixed expansion, the necessary joint assumption concerns $(d, \langle G(d), S\rangle + E)$ versus (d, V) for uniform V, with the actual secret/error distributions and sample count.

A precise seeded model: If expansion is an ideal random oracle $H : \{0,1\}^h \to \mathcal{R}^k_{\langle n,q\rangle}$, a classical adversary making Q encryption queries and Q_H oracle queries has guessing bias

$$\epsilon \le \delta(T', Q) + \min\left\{1, \frac{QQ_H + Q(Q-1)/2}{2^h}\right\}.$$

Here δ is the actual-distribution decisional GLWE advantage, with the convention of §B-1.2. To simulate a ciphertext, choose its seed independently and program the previously unassigned $H(d)$ with the uniform mask from a challenge sample; add the selected message to its body. The simulation fails only if the seed was queried earlier or repeats an earlier encryption seed. At the jth encryption at most $Q_H + j - 1$ points are occupied, giving the displayed union bound. Outside that event real samples give real encryption; uniform sample bodies hide the message. Abort with an independent fair guess on failure. This proves the stated classical random-oracle result. It neither replaces a concrete XOF by an ideal oracle without an assumption nor proves the same bound for quantum oracle queries. See the Kyber v3.02 specification, Sections 4.3.1 and 5.4, for the corresponding distinction between public XOF expansion and secret pseudorandom generation. Reusing a mask also exposes the difference of two message phases plus their error difference, so seed freshness is essential.

B-4.4 An Alternative Version of GLWE

The following is an alternative version of ⟨Summary B-4.2⟩, where the sign of each A_iS_i is flipped in the encryption and decryption formula as follows:

⟨Summary B-4.4⟩ An Alternative GLWE Cryptosystem

Initial Setup: $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$, $\{S_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}^k_{\langle n,tern\rangle}$

Encryption Input: $M \in \mathcal{R}_{\langle n,t\rangle}$, $\{A_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}^k_{\langle n,q\rangle}$, $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q\rangle}$

1. Scale up $M \longrightarrow \Delta M \ \in \mathcal{R}_{\langle n,q \rangle}$
2. Compute $B = -\sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$
3. $\mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (\{A_i\}_{i=0}^{k-1}, B) \ \in \mathcal{R}^{k+1}_{\langle n,q \rangle}$

Decryption Input: $\mathsf{ct} = (\{A_i\}_{i=0}^{k-1}, B) \ \in \mathcal{R}^{k+1}_{\langle n,q \rangle}$

1. $\mathsf{GLWE}^{-1}_{S,\sigma}(\mathsf{ct}) = B + \sum_{i=0}^{k-1} (A_i \cdot S_i) = \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$
2. Scale down $\left\lceil \frac{\Delta M + E}{\Delta} \right\rfloor \bmod t = M \ \in \mathcal{R}_{\langle n,t \rangle}$

For correct decryption, every noise coefficient e_i of polynomial E should be: $|e_i| < \frac{\Delta}{2}$.

Even if the A_iS_i terms flip their signs, the decryption stage cancels out those terms by adding their equivalent double-sign-flipped terms; thus, the same correctness of decryption is preserved as in the original version.

B-4.5 Public Key Encryption

Scope: This subsection is restricted to $k = 1$, the standard RLWE public-key structure, with sufficiently large power-of-two ring dimension n and a parameter/distribution family for which the required decisional RLWE assumptions are justified. The vector notation below is retained with a single component. It is not a generic public-key construction for arbitrary GLWE dimensions or the $n = 1$ LWE special case. Such generalizations require a suitable public matrix construction and their own security argument. As in the setup above, assume $t \mid q$ for the displayed decryption bound.

The encryption scheme in §B-4.2 assumes that it is the secret key owner who encrypts each plaintext. In this section, we explain a public key encryption scheme in which we create a public key counterpart of the secret key. Anyone who knows the public key can encrypt the plaintext in such a way that only the secret key owner can decrypt it. The high-level idea is that a portion of the components to be used in the encryption stage is pre-computed at the setup stage and published as a public key. At the actual encryption stage, the public key is multiplied by an additional randomness (U) and added to additional noise ($E_1, \vec{E}_2$) to create unpredictable randomness for each encrypted ciphertext. The actual scheme is as follows:

⟨Summary B-4.5⟩ RLWE Public Key Encryption ($k = 1$)

Initial Setup:

- The scaling factor $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$
- The secret key $\vec{S} = \{S_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}^k_{\langle n,tern \rangle}$
- The public key pair $(PK_1, \overrightarrow{PK}_2) \in \mathcal{R}^{k+1}_{\langle n,q \rangle}$ is generated as follows:
  $\vec{A} = \{A_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}^k_{\langle n,q \rangle}, \quad E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$
  $PK_1 = \vec{A} \cdot \vec{S} + E \in \mathcal{R}_{\langle n,q \rangle}$

$\overrightarrow{PK}_2 = \vec{A} \in \mathcal{R}^k_{\langle n,q\rangle}$

Encryption Input: $M \in \mathcal{R}_{\langle n,t\rangle}$, $U \xleftarrow{\$} \mathcal{R}_{\langle n,tern\rangle}$, $E_1 \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q\rangle}$, $\vec{E}_2 \xleftarrow{\chi_\sigma} \mathcal{R}^k_{\langle n,q\rangle}$

1. Scale up $M \longrightarrow \Delta M \ \in \mathcal{R}_{\langle n,q\rangle}$
2. Compute the following:
   $B = PK_1 \cdot U + \Delta M + E_1 \ \in \mathcal{R}_{\langle n,q\rangle}$
   $\vec{D} = \overrightarrow{PK}_2 \cdot U + \vec{E}_2 \in \mathcal{R}^k_{\langle n,q\rangle}$ ▷ $\overrightarrow{PK}_2 \cdot U$ multiplies each element of $\overrightarrow{PK}_2$ by U
3. $\mathsf{GLWE}_{S,\sigma}(\Delta M + E_{all}) = (\vec{D}, B) \ \in \mathcal{R}^{k+1}_{\langle n,q\rangle}$ ▷ where $E_{all} = E \cdot U + E_1 - \vec{E}_2 \cdot \vec{S}$

Decryption Input: $\mathsf{ct} = (\vec{D}, B) \ \in \mathcal{R}^{k+1}_{\langle n,q\rangle}$

1. $\mathsf{GLWE}^{-1}_{S,\sigma}(\mathsf{ct}) = B - \vec{D} \cdot \vec{S} = \Delta M + E_{all} \ \in \mathcal{R}_{\langle n,q\rangle}$
2. Scale down $\left\lceil \dfrac{\Delta M + E_{all}}{\Delta} \right\rfloor \bmod t = M \ \in \mathcal{R}_{\langle n,t\rangle}$

For correct decryption, every noise coefficient e_i of polynomial E_{all} should be: $|e_i| < \dfrac{\Delta}{2}$.

The equation in the 1st step of the decryption process is derived as follows:
$\mathsf{GLWE}^{-1}_{S,\sigma}(\ \mathsf{ct} = (\vec{D}, B)\ ) = B - \vec{D} \cdot \vec{S}$
$= (PK_1 \cdot U + \Delta M + E_1) - (\overrightarrow{PK}_2 \cdot U + \vec{E}_2) \cdot \vec{S}$
$= (\vec{A} \cdot \vec{S} + E) \cdot U + \Delta M + E_1 - (\vec{A} \cdot U) \cdot \vec{S} - \vec{E}_2 \cdot \vec{S}$
$= (U \cdot \vec{A}) \cdot \vec{S} + E \cdot U + \Delta M + E_1 - (U \cdot \vec{A}) \cdot \vec{S} - \vec{E}_2 \cdot \vec{S}$
$= \Delta M + E \cdot U + E_1 - \vec{E}_2 \cdot \vec{S}$
$= \Delta M + E_{all}$ ▷ where $E_{all} = E \cdot U + E_1 - \vec{E}_2 \cdot \vec{S}$

Security: For $k = 1$, write the public key as (A, P) with $P = AS + E$, and the ciphertext as $(D, B) = (AU + E_2, PU + E_1 + \Delta M)$. Assume decisional RLWE for the actual distribution of S and error E, and for the actual distribution of U with two samples whose errors have the distributions of E_2, E_1. In this subsection both secrets are ternary and all three error laws are the same, so one corresponding two-sample RLWE assumption covers both uses. It does not follow merely from a theorem stated for Gaussian secrets.

The single-challenge IND-CPA proof has two hybrids. First replace $P = AS + E$ by an independent uniform ring element. A reduction embeds one RLWE sample as the public key and can perform every encryption itself by choosing fresh U, E_1, E_2; it does not need S. Next, with uniform (A, P), replace the challenge's $(D, B - \Delta M)$ by independent uniform ring elements. A reduction takes two samples $(A, Z_2), (P, Z_1)$ sharing the same unknown secret U, uses their masks as the public key, and returns $(Z_2, Z_1 + \Delta M)$ once the challenge messages are chosen. Real samples give $Z_2 = AU + E_2$ and $Z_1 = PU + E_1$; uniform samples hide the chosen message completely. Other encryptions are publicly computable with fresh known randomness. Thus, with the guessing-bias convention of §B-1.2,

$$\epsilon_{\mathrm{PKE}} \le \delta_S(T'_1, 1) + \delta_U(T'_2, 2).$$

The two terms use the stated actual distributions and include reduction overhead. This is a complete conditional CPA reduction for the scoped construction; it does not certify a numerical parameter

family or secret-dependent public-key encryption queries. See Lyubashevsky–Peikert–Regev, Section 1.1, Applications. For an unrestricted vector version, the second hybrid would require rank-one RLWE with secret U and $k+1$ samples, not merely the rank-k GLWE assumption used in the first hybrid.

The cancellation calculation proves a separate correctness identity with $E_{all} = EU + E_1 - E_2 S$.

For lifted coefficients in the negacyclic ring,

$$\|E_{all}\|_\infty \le \|E\|_\infty \|U\|_1 + \|E_1\|_\infty + \|E_2\|_\infty \|S\|_1.$$

The right-hand side being less than $\Delta/2$ suffices for decryption when $t \mid q$.

The unrestricted construction would be insecure for $n = 1$ and ternary U, even if the LWE vector dimension k were large: an attacker need only try $U \in \{-1, 0, 1\}$ and test whether all $D_i - A_i U$ are small, then decode $B - PK_1U$. In particular, $U = 0$ occurs with probability $1/3$ and leaves $B = \Delta m + E_1$. This is why the general GLWE security claim cannot be inferred from the displayed algebra.

B-5 GLev

A GLev ciphertext is a list of GLWE ciphertexts that encrypt the list of plaintexts $\frac{q}{\beta^1}M, \frac{q}{\beta^2}M, \ldots, \frac{q}{\beta^l}M$, where M is a plaintext encoded in a polynomial. Note that each i-th GLWE ciphertext of a GLev ciphertext uses a different plaintext scaling factor, which is: $\Delta_i = \frac{q}{\beta^i}$. The structure of GLev ciphertext is visually depicted in Figure 9.

For these integer-ring weights require an integer $\beta \geq 2$ and $\beta^l \mid q$. The inequality $t \leq \beta$ is needed only if the first row is intended to recover coefficients represented in $\{0, \ldots, t-1\}$. GLev is also used as a gadget ciphertext for a general ring message; in that use, each row need not separately decode the original message. Exact reconstruction with the descending gadget for every residue additionally requires $q = \beta^l$, as explained in §A-6.

B-5.1 Encryption

⟨Summary B-5.1⟩ GLev Encryption

$$\mathsf{GLev}^{\beta,l}_{S,\sigma}(M) = \left\{\mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta^i}M + E_i\right)\right\}_{i=1}^{l} \in \mathcal{R}^{(k+1)\cdot l}_{\langle n,q\rangle}$$

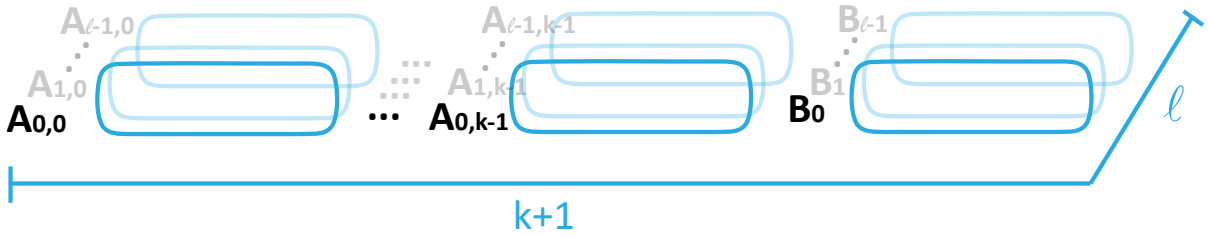


Figure 9: An illustration of a GLev ciphertext

B-5.2 Decryption

The first row has phase $(q/\beta)M + E_1 \pmod q$. If $\|E_1\|_\infty < q/(2\beta)$, nearest-integer decoding recovers $M \bmod \beta$ coefficientwise. It recovers a chosen original representative only when that representative is uniquely determined by its class modulo β, for example coefficients in $\{0, \ldots, t-1\}$ with $t \leq \beta$. Row i similarly decodes modulo β^i if $\|E_i\|_\infty < q/(2\beta^i)$. Later rows have a smaller error tolerance; they are not necessarily undecodable, nor is separate decoding required for gadget operations.

B-5.3 Lev and RLev

Lev is GLev with $n = 1$. RLev is GLev with $k = 1$.

B-6 GGSW

The GGSW cryptosystem is a list of GLev ciphertexts. In the GGSW cryptosystem, the secret key S is a list of k polynomials (i.e., $S_0, S_1, \ldots S_{k-1}$), and each i-th GLev ciphertext in the GGSW ciphertext encrypts the plaintext $-S_0 \cdot M, -S_1 \cdot M, \ldots, -S_{k-1} \cdot M$, and M. This is visually depicted in Figure 10.

B-6.1 Encryption

⟨Summary B-6.1⟩ GGSW Encryption

$$\mathsf{GGSW}^{\beta,l}_{S,\sigma}(M) = \left\{ \{\mathsf{GLev}^{\beta,l}_{S,\sigma}(-S_i \cdot M)\}_{i=0}^{k-1}, \mathsf{GLev}^{\beta,l}_{S,\sigma}(M) \right\} \in \mathcal{R}^{(k+1)\cdot l\cdot(k+1)}_{\langle n,q\rangle}$$

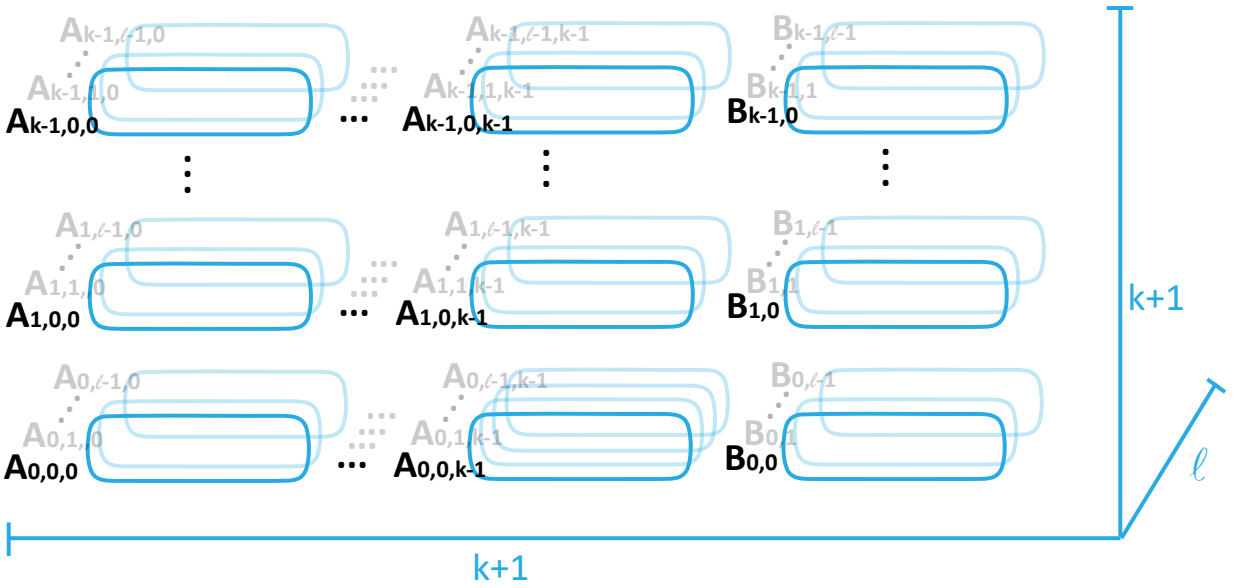


Figure 10: An illustration of a GGSW ciphertext

B-6.2 Decryption

Decrypting the final GLev block recovers M under the message-range and noise conditions stated for GLev. Another block's first row decodes $-S_i M \bmod \beta$ under its corresponding error bound. Recovering $M \bmod \beta$ from this product requires S_i to be a unit in $\mathbb{Z}_\beta[X]/(X^n+1)$. Nonzero is insufficient: in $\mathbb{Z}_5[X]/(X^2+1)$, both $X-2$ and $X+2$ are nonzero, but $(X-2)(X+2)=0$. No inversion of S_i is needed when using the final block.

B-6.3 GSW and RGSW

GSW is GGSW with $n=1$. RGSW is GGSW with $k=1$.

Part III

Generic Fully Homomorphic Encryption

This chapter explains the generic techniques of homomorphic computation adopted by various FHE schemes such as TFHE, CKKS, BGV, and BFV,

As we learned from §B-4, $\mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (A_0, A_1, \cdots, A_{k-1}, B) \in \mathcal{R}^{k+1}_{\langle n,q\rangle}$, where $\mathcal{R}_{\langle n,q\rangle} = \mathbb{Z}_q[x]/(x^n+1)$, and B is computed as $B = \sum_{i=0}^{k-1}(A_i \cdot S_i) + \Delta \cdot M + E$. Each A_i is an $(n-1)$-degree polynomial as a public key, whose each coefficient is uniformly randomly sampled from $\mathcal{R}_{\langle n,q\rangle}$. E is an $(n-1)$-degree polynomial as a noise, whose each coefficient is sampled from $\mathcal{R}_{\langle n,q\rangle}$ based on the Gaussian distribution χ_σ. S is a list of k $(n-1)$-degree polynomials as a secret key, such that $S = (S_0, S_1, \cdots S_{k-1}) \in \mathcal{R}^k_{\langle n,q\rangle}$, and each polynomial S_i's each coefficient is a randomly sampled binary number in $\mathbb{Z}_2$ (i.e., $\{0,1\}$).

Based on this GLWE setup, this section will explain the following 5 homomorphic operations: ciphertext-to-ciphertext addition, ciphertext-to-plaintext addition, ciphertext-to-plaintext multiplication, ciphertext-to-ciphertext multiplication, and key switching.

C-1 GLWE Ciphertext-to-Ciphertext Addition

Both inputs must have the same secret key, ring, modulus q, and scale $\Delta = q/t$, with $t \mid q$ for the stated plaintext arithmetic modulo t. Throughout these operation chapters, $\mathsf{GLWE}_{S,\sigma}(\text{phase})$ is shorthand for a ciphertext with that phase; an evaluated ciphertext need not have the same fresh Gaussian distribution or standard deviation σ. All noise bounds use integer lifts before coefficient reduction.

- Reference: TFHE Deep Dive - Part II - Encodings and linear leveled operations [8]

Suppose we have two GLWE ciphertexts encrypting two different plaintexts $M^{\langle 1\rangle}, M^{\langle 2\rangle}$:

$\mathsf{GLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle}) = \mathsf{ct}^{\langle 1\rangle} = (A_0^{\langle 1\rangle}, A_1^{\langle 1\rangle}, \ldots, A_{k-1}^{\langle 1\rangle}, B^{\langle 1\rangle}) \in \mathcal{R}_{\langle n,q\rangle}^{k+1}$

$\mathsf{GLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle}) = \mathsf{ct}^{\langle 2\rangle} = (A_0^{\langle 2\rangle}, A_1^{\langle 2\rangle}, \ldots, A_{k-1}^{\langle 2\rangle}, B^{\langle 2\rangle}) \in \mathcal{R}_{\langle n,q\rangle}^{k+1}$

Let's define the following ciphertext addition operation:

$\mathsf{ct}^{\langle 1\rangle} + \mathsf{ct}^{\langle 2\rangle} = (A_0^{\langle 1\rangle} + A_0^{\langle 2\rangle},\ A_1^{\langle 1\rangle} + A_1^{\langle 2\rangle}, \ldots, A_{k-1}^{\langle 1\rangle} + A_{k-1}^{\langle 2\rangle},\ B^{\langle 1\rangle} + B^{\langle 2\rangle})$

Then, the following is true:

⟨Summary C-1⟩ GLWE Homomorphic Addition

$$
\begin{aligned}
&\mathsf{GLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle}) + \mathsf{GLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle}) \\
&= (\{A_i^{\langle 1\rangle}\}_{i=0}^{k-1},\ B^{\langle 1\rangle}) + (\{A_i^{\langle 2\rangle}\}_{i=0}^{k-1},\ B^{\langle 2\rangle}) \\
&= (\{A_i^{\langle 1\rangle} + A_i^{\langle 2\rangle}\}_{i=0}^{k-1},\ B^{\langle 1\rangle} + B^{\langle 2\rangle}) \\
&= \mathsf{GLWE}_{S,\sigma}(\Delta(M^{\langle 1\rangle} + M^{\langle 2\rangle}) + E^{\langle 3\rangle}) \quad \triangleright \text{ where } E^{\langle 3\rangle} = E^{\langle 1\rangle} + E^{\langle 2\rangle}
\end{aligned}
$$

The phase identity is unconditional algebraically. Decoding returns $M^{\langle 1\rangle} + M^{\langle 2\rangle} \bmod t$ provided $\|E^{\langle 1\rangle} + E^{\langle 2\rangle}\|_\infty < \Delta/2$; for example, the sum of the two input noise bounds being less than $\Delta/2$ suffices.

Proof.

1. Define the following notations:

$$
\begin{aligned}
A_0^{\langle 3\rangle} &= A_0^{\langle 1\rangle} + A_0^{\langle 2\rangle} \\
A_1^{\langle 3\rangle} &= A_1^{\langle 1\rangle} + A_1^{\langle 2\rangle} \\
&\vdots \\
A_{k-1}^{\langle 3\rangle} &= A_{k-1}^{\langle 1\rangle} + A_{k-1}^{\langle 2\rangle} \\
E^{\langle 3\rangle} &= E^{\langle 1\rangle} + E^{\langle 2\rangle} \\
B^{\langle 3\rangle} &= B^{\langle 1\rangle} + B^{\langle 2\rangle}
\end{aligned}
$$

2. Derive the following:

$$
\begin{aligned}
B^{\langle 3\rangle} &= B^{\langle 1\rangle} + B^{\langle 2\rangle} \\
&= \sum_{i=0}^{k-1} (A_i^{\langle 1\rangle} \cdot S_i) + \Delta \cdot M^{\langle 1\rangle} + E^{\langle 1\rangle} + \sum_{i=0}^{k-1} (A_i^{\langle 2\rangle} \cdot S_i) + \Delta \cdot M^{\langle 2\rangle} + E^{\langle 2\rangle} \\
&= \sum_{i=0}^{k-1} ((A_i^{\langle 1\rangle} + A_i^{\langle 2\rangle}) \cdot S_i) + \Delta \cdot (M^{\langle 1\rangle} + M^{\langle 2\rangle}) + (E^{\langle 1\rangle} + E^{\langle 2\rangle}) \quad \triangleright \text{ commutative and distributive rules} \\
&= \sum_{i=0}^{k-1} (A_i^{\langle 3\rangle} \cdot S_i) + \Delta \cdot (M^{\langle 1\rangle} + M^{\langle 2\rangle}) + E^{\langle 3\rangle}
\end{aligned}
$$

3. Since $B^{\langle 3\rangle} = \sum_{i=0}^{k-1} (A_i^{\langle 3\rangle} \cdot S_i) + \Delta \cdot (M^{\langle 1\rangle} + M^{\langle 2\rangle}) + E^{\langle 3\rangle}$,
   this means that $(A_0^{\langle 3\rangle}, A_1^{\langle 3\rangle}, A_2^{\langle 3\rangle}, \ldots, A_{k-1}^{\langle 3\rangle}, B^{\langle 3\rangle})$ form the ciphertext: $\mathsf{GLWE}_{S,\sigma}(\Delta \cdot (M^{\langle 1\rangle} + M^{\langle 2\rangle}) + E^{\langle 3\rangle})$.

4. Thus,
   $\mathsf{GLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle}) + \mathsf{GLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle})$
   $= (A_0^{\langle 1\rangle} + A_0^{\langle 2\rangle},\ A_1^{\langle 1\rangle} + A_1^{\langle 2\rangle}, \ldots, A_{k-1}^{\langle 1\rangle} + A_{k-1}^{\langle 2\rangle},\ B^{\langle 1\rangle} + B^{\langle 2\rangle})$
   $= (A_0^{\langle 3\rangle}, A_1^{\langle 3\rangle}, A_2^{\langle 3\rangle}, \ldots, A_{k-1}^{\langle 3\rangle}, B^{\langle 3\rangle})$
   $= (\{A_i^{\langle 3\rangle}\}_{i=0}^{k-1}, B^{\langle 3\rangle})$
   $= \mathsf{GLWE}_{S,\sigma}(\Delta(M^{\langle 1\rangle} + M^{\langle 2\rangle}) + E^{\langle 3\rangle})$

□

C-1.1 Discussion

Noise Elimination: Secret-key subtraction removes the mask products exactly. Rounding removes the resulting error only while its coefficient magnitudes remain below $\Delta/2$. Valid inputs can add to an invalid output if the combined error exceeds that bound.

Noise Growth: The new error is $E^{\langle 3\rangle} = E^{\langle 1\rangle} + E^{\langle 2\rangle}$. For independent zero-mean errors, variances add; with equal variance σ^2, summing h independent samples gives variance $h\sigma^2$. Zero mean alone is insufficient. In general, $\mathrm{Var}(e_1 + e_2) = \mathrm{Var}(e_1) + \mathrm{Var}(e_2) + 2\,\mathrm{Cov}(e_1, e_2)$. Adding a ciphertext to itself produces $2e$ and variance $4\sigma^2$, not $2\sigma^2$. A deterministic coefficient bound uses the triangle inequality and requires no independence assumption.

Plaintext Overflow when $t \nmid q$: Mask products and ciphertext components are always interpreted modulo q. When $t \mid q$, plaintext wrap-around modulo t is compatible with multiplication by $\Delta = q/t$. When $t \nmid q$ and $\Delta = \lfloor q/t \rfloor$, write each unreduced plaintext coefficient as $m = m' + vt$ with centered m'. The effective error is $e - (q \bmod t)v$. Under the assumptions of §B-2.3.1, require $|e - (q \bmod t)v| < \Delta/2$. This condition depends on the overflow v; $q \gg t$ alone does not ensure it. The subsequent reduction modulo t is part of message decryption.

C-2 GLWE Ciphertext-to-Plaintext Addition

Use the same ring and scale for the ciphertext and the integer lift of Λ. Assume $t \mid q$, $\Delta = q/t$, and $\|E\|_\infty < \Delta/2$ for the stated decoded plaintext modulo t.

Suppose we have a GLWE ciphertext ct and a new plaintext polynomial Λ as follows:
$\mathsf{ct} = \mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (A_0, A_1, \ldots, A_{k-1}, B) \in \mathcal{R}^{k+1}_{\langle n,q \rangle}$
Λ: a new plaintext polynomial
$\Delta\Lambda$: a Δ-scaled new plaintext polynomial

Let's define the following ciphertext-to-plaintext addition operation:
$\mathsf{ct} + \Delta\Lambda = (A_0,\ A_1, \ldots, A_{k-1},\ B + \Delta\Lambda)$

Then, the following is true:

⟨Summary C-2⟩ GLWE Homomorphic Addition with a Plaintext

$$\begin{aligned}
&\mathsf{GLWE}_{S,\sigma}(\Delta M + E) + \Delta\Lambda \\
&= (\{A_i\}_{i=0}^{k-1},\ B) + \Delta\Lambda \\
&= (\{A_i\}_{i=0}^{k-1},\ B + \Delta\Lambda) \\
&= \mathsf{GLWE}_{S,\sigma}(\Delta(M + \Lambda) + E)
\end{aligned}$$

This means that adding a (Δ-scaled) plaintext polynomial Λ to a GLWE ciphertext that encrypts M and decrypting it yields $M + \Lambda$.

Proof.

1. Since $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E$,
$$B + \Delta \cdot \Lambda = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E + \Delta \cdot \Lambda = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot (M + \Lambda) + E$$
This means that $(A_0, A_1, \ldots A_{k-1}, B + \Delta\Lambda)$ form the ciphertext $\mathsf{GLWE}_{S,\sigma}(\Delta(\Lambda + M) + E)$
2. Thus,
$\mathsf{GLWE}_{S,\sigma}(\Delta M + E) + \Delta\Lambda$
$= (A_0,\ A_1, \ldots A_{k-1},\ B + \Delta\Lambda)$
$= (\{A_i\}_{i=0}^{k-1},\ B + \Delta\Lambda)$
$= \mathsf{GLWE}_{S,\sigma}(\Delta(M + \Lambda) + E)$

□

Noise Growth: The phase error relative to the unreduced message $M + \Lambda$ remains E. With $t \mid q$, reducing that message modulo t does not alter the effective error. If $t \nmid q$, a message wrap-around can add the overflow-dependent error described in §B-2.3.1; the statement that noise stays unchanged must then be understood relative to the unreduced message.

C-3 GLWE Ciphertext-to-Plaintext Multiplication

For message decoding, assume $t \mid q$ and $\Delta = q/t$. Work with fixed integer coefficient lifts of the public multiplier Λ when bounding noise. Equality of ciphertext phases below does not assert equality of fresh-encryption distributions.

- Reference: TFHE Deep Dive - Part II - Encodings and linear leveled operations [8]

Suppose we have a GLWE ciphertext ct:
$\mathsf{ct} = \mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (A_0, A_1, \ldots, A_{k-1}, B) \in \mathcal{R}^{k+1}_{\langle n,q \rangle}$

and a new plaintext polynomial Λ as follows:
$\Lambda = \sum_{i=0}^{n-1} (\Lambda_i \cdot X^i) \in \mathcal{R}_{\langle n,q \rangle}$

Let's define the following ciphertext-to-plaintext multiplication operation:
$\Lambda \cdot \mathsf{ct} = (\Lambda \cdot A_0, \Lambda \cdot A_1, \ldots, \Lambda \cdot A_{k-1}, \Lambda \cdot B)$

NTT multiplication uses $O(n \log n)$ field operations when the transform's root and invertibility conditions hold (§A-16); for instance, prime q with $2n \mid q-1$ supports the negacyclic transform. An arbitrary modulus does not automatically provide such an NTT. The following algebraic identity holds independently of the multiplication algorithm:

⟨Summary C-3⟩ GLWE Ciphertext-to-Plaintext Multiplication

$$\begin{aligned}
&\Lambda \cdot \mathsf{GLWE}_{S,\sigma}(\Delta M + E) \\
&= \Lambda \cdot (\{A_i^{\langle 1 \rangle}\}_{i=0}^{k-1},\ B^{\langle 1 \rangle}) \\
&= (\{\Lambda \cdot A_i^{\langle 1 \rangle}\}_{i=0}^{k-1},\ \Lambda \cdot B^{\langle 1 \rangle}) \\
&= \mathsf{GLWE}_{S,\sigma}(\Delta(M \cdot \Lambda) + \Lambda \cdot E)
\end{aligned}$$

The resulting phase is $\Delta M\Lambda + \Lambda E$. It decodes to $M\Lambda \bmod t$ only when $\|\Lambda E\|_\infty < \Delta/2$.

Proof.

1. Define the following notations:
   $A'_0 = \Lambda \cdot A_0$
   $A'_1 = \Lambda \cdot A_1$
   $\vdots$
   $A'_{k-1} = \Lambda \cdot A_{k-1}$
   $E' = \Lambda \cdot E$
   $B' = \Lambda \cdot B$

2. Derive the following:
   $B' = \Lambda \cdot B$
   $= \Lambda \cdot (\sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E) = \sum_{i=0}^{k-1} (\Lambda \cdot A_i \cdot S_i) + \Delta \cdot \Lambda \cdot M + \Lambda \cdot E$
   ▷ by the distributive property of a polynomial ring

$$= \sum_{i=0}^{k-1} ((\Lambda \cdot A_i) \cdot S_i) + \Delta \cdot (\Lambda \cdot M) + (\Lambda \cdot E)$$
$$= \sum_{i=0}^{k-1} (A'_i \cdot S_i) + \Delta \cdot (\Lambda \cdot M) + (E')$$

3. Since $B' = \sum_{i=0}^{k-1} (A'_i \cdot S_i) + \Delta \cdot (\Lambda \cdot M) + (E')$,
   $(A'_0, A'_1, \ldots, A'_{k-1}, B')$ form the ciphertext $\mathsf{GLWE}_{S,\sigma}(\Delta \cdot \Lambda \cdot M + E')$.
4. Thus,
   $\Lambda \cdot \mathsf{GLWE}_{S,\sigma}(\Delta M + E)$
   $= (\Lambda \cdot A_0,\ \Lambda \cdot A_1, \ldots, \Lambda \cdot A_{k-1},\ \Lambda \cdot B)$
   $= (\{A'_i\}_{i=0}^{k-1},\ \Lambda \cdot B)$
   $= \mathsf{GLWE}_{S,\sigma}(\Delta(M \cdot \Lambda) + \Lambda \cdot E)$

□

Secret-key subtraction removes the mask products exactly, but rounding removes $E' = \Lambda E$ only within the decoding bound. In the negacyclic ring, each output coefficient is a signed sum of at most n products, so

$$\|\Lambda E\|_\infty \leq \|\Lambda\|_1 \|E\|_\infty \leq n \|\Lambda\|_\infty \|E\|_\infty.$$

Here $\|\Lambda\|_1$ is the sum of the absolute values of its lifted coefficients.

Repeated multiplications can increase the noise bound until decoding fails. Decryption reads the result but does not refresh an existing ciphertext. For large multipliers, a precomputed GLev representation can give a better bound through gadget decomposition (§A-6.4), as below.

C-3.1 Gadget Decomposition for Noise Suppression

Here use the exact descending gadget $g_i = q/\beta^i$ with $q = \beta^l$ and unsigned digits in $\{0, \ldots, \beta - 1\}$. The precomputed input is $\mathsf{GLev}^{\beta,l}_{S,\sigma}(\Delta M)$, not an arbitrary single GLWE ciphertext. Such a low-noise GLev representation is additional encrypted input and cannot be assumed to appear for free from a ciphertext after previous operations.

In the ciphertext-to-plaintext multiplication $\Lambda \cdot \mathsf{GLWE}_{S,\sigma}(\Delta M)$, the noise E grows to $E' = \Lambda \cdot E$. To limit this noise growth, we introduce a technique based on decomposing Λ (§A-6.1) and a GLev encryption (§B-5.1) of M as follows:

$$\Lambda = \Lambda_1 \frac{q}{\beta^1} + \Lambda_2 \frac{q}{\beta^2} + \cdots + \Lambda_l \frac{q}{\beta^l} \longrightarrow \mathsf{Decomp}^{\beta,l}(\Lambda) = (\Lambda_1, \Lambda_2, \cdots, \Lambda_l)$$

$$\mathsf{GLev}^{\beta,l}_{S,\sigma}(\Delta M) = \left\{ \mathsf{GLWE}_{S,\sigma}\left(\Delta M \frac{q}{\beta^1} + E_1\right), \mathsf{GLWE}_{S,\sigma}\left(\Delta M \frac{q}{\beta^2} + E_2\right), \cdots \mathsf{GLWE}_{S,\sigma}\left(\Delta M \frac{q}{\beta^l} + E_l\right) \right\}$$

We will encrypt the plaintext M as $\mathsf{GLev}^{\beta,l}_{S,\sigma}(\Delta M)$ instead of $\mathsf{GLWE}_{S,\sigma}(\Delta M)$, and compute $\mathsf{Decomp}^{\beta,l}(\Lambda) \cdot \mathsf{GLev}^{\beta,l}_{S,\sigma}(\Delta M)$ instead of $\Lambda \cdot \mathsf{GLWE}_{S,\sigma}(\Delta M)$. Notice that the results of both computations are the same as follows:

$$\mathsf{Decomp}^{\beta,l}(\Lambda) \cdot \mathsf{GLev}^{\beta,l}_{S,\sigma}(\Delta M)$$
$$= (\Lambda_1, \Lambda_2, \cdots, \Lambda_l) \cdot \left(\mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta}\Delta M + E_1\right),\ \mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta^2}\Delta M + E_2\right),\ \cdots,\ \mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta^l}\Delta M + E_l\right) \right)$$
$$= \Lambda_1 \cdot \mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta}\Delta M + E_1\right) + \Lambda_2 \cdot \mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta^2}\Delta M + E_2\right) + \cdots + \Lambda_l \cdot \mathsf{GLWE}_{S,\sigma}\left(\frac{q}{\beta^l}\Delta M + E_l\right)$$

$$
\begin{aligned}
&= \mathsf{GLWE}_{S,\sigma}\left(\Lambda_1 \cdot \frac{q}{\beta}\Delta M + \Lambda_1 E_1\right) + \mathsf{GLWE}_{S,\sigma}\left(\Lambda_2 \cdot \frac{q}{\beta^2}\Delta M + \Lambda_2 E_2\right) + \cdots + \mathsf{GLWE}_{S,\sigma}\left(\Lambda_l \cdot \frac{q}{\beta^l}\Delta M + \Lambda_l E_l\right) \\
&= \mathsf{GLWE}_{S,\sigma}\left(\Lambda_1 \cdot \frac{q}{\beta}\Delta M + \Lambda_2 \cdot \frac{q}{\beta^2}\Delta M + \cdots + \Lambda_l \cdot \frac{q}{\beta^l}\Delta M + E_{all}\right) \\
&= \mathsf{GLWE}_{S,\sigma}\left(\left(\Lambda_1 \cdot \frac{q}{\beta} + \Lambda_2 \cdot \frac{q}{\beta^2} + \cdots + \Lambda_l \cdot \frac{q}{\beta^l}\right) \cdot \Delta M + E_{all}\right) \triangleright \text{ where } E_{all} = \sum_{i=1}^{l} \Lambda_i E_i \\
&= \mathsf{GLWE}_{S,\sigma}\left(\Lambda \cdot \Delta M + E_{all}\right) \triangleright \text{ whose decryption is } \Lambda \cdot M
\end{aligned}
$$

For exact decomposition, the final error is $E_{all} = \sum_{i=1}^{l} \Lambda_i E_i$. If $\|E_i\|_\infty \leq B_{\mathrm{g}}$, the unsigned digits give

$$\|E_{all}\|_\infty \leq \sum_{i=1}^{l} \|\Lambda_i\|_1 \|E_i\|_\infty \leq nl(\beta - 1)B_{\mathrm{g}}.$$

A $\beta/2$ digit bound would require an explicitly balanced decomposition instead. This can improve the worst-case bound compared with a large multiplier, but does not guarantee a smaller realized error for every input. Both computations decode to the same message only if their respective error bounds hold. For truncated decomposition $\Lambda \equiv \sum_i \Lambda_i g_i + \rho \pmod q$, the final phase instead has error $\sum_i \Lambda_i E_i - \rho\Delta M$; this residual must also be bounded. Figure 11 illustrates the intended bound reduction.

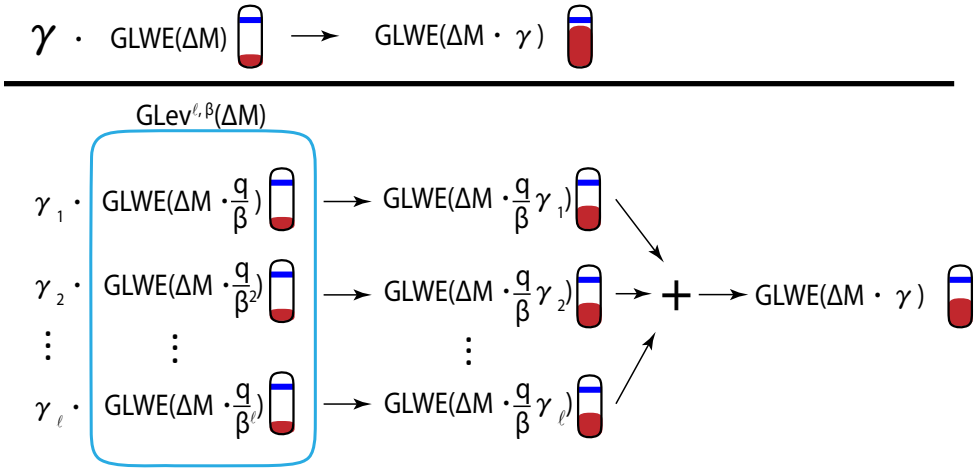


Figure 11: Noise reduction in ciphertext-to-plaintext multiplication by gadget decomposition.

C-3.1.1 Discussion

Nevertheless, the decomposition technique is still very useful: for GLWE key-switching (§C-5), we will show how to key-switch by combining decomposed mask polynomials $\mathsf{Decomp}^{\beta,l}(A_i)$ with a precomputed key-switching key $\mathsf{KSK}_i = \mathsf{GLev}^{\beta,l}_{S',\sigma}(S_i)$, so gadget decomposition can be repeatedly leveraged across key-switching calls even though each individual application outputs a standard GLWE ciphertext.

Meanwhile, for the technique to repeatedly re-initialize the noise E of regular ciphertexts, we will describe TFHE's noise bootstrapping technique in §D-1.8.

C-4 GLWE Modulus Switching

Modulus switching scales and rounds a ciphertext's public coefficients from modulus q to modulus $\hat{q}$. Preservation of the decoded message is conditional on the resulting noise bound. The hidden noise is not directly computed by the evaluator: its transformation is inferred from the new phase. We use the exact scale-and-round identities of §A-13 and retain all rounding-error terms.

C-4.1 LWE Modulus Switching

Throughout this chapter assume $t \mid q$ and $t \mid \hat{q}$, so $\Delta = q/t$ and $\hat{\Delta} = \hat{q}/t$ are integers. Put $r = \hat{q}/q$ and use the fixed tie rule $R(x) = \lfloor x + 1/2 \rfloor$. All scaled expressions are first evaluated on integer lifts; final ciphertext coefficients are reduced modulo $\hat{q}$. If scales are instead chosen by floors without these divisibilities, the new error contains the additional message-dependent term $(r\Delta - \hat{\Delta})m$ (or $(r\Delta - \hat{\Delta})M$), and the overflow analysis of §B-2.3.1 is also required. The displayed correctness claim cannot be extended by simply omitting those terms.

- Reference: Modulus Switching in LWE

Recall that the LWE cryptosystem (§B-2.2) comprises the following components:

- **Setup:** $\Delta = \dfrac{q}{t}, \quad S = (s_0, s_1, \ldots, s_{k-1}) \xleftarrow{\$} \mathbb{Z}_2^k$
- **Encryption Input:** $m \in \mathbb{Z}_t, \quad \vec{a} = (a_0, a_1, \ldots, a_{k-1}) \xleftarrow{\$} \mathbb{Z}_q^k, \quad e \xleftarrow{\chi_\sigma} \mathbb{Z}_q$
- **Encryption:** $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) = (\vec{a}, b) \in \mathbb{Z}_q^{k+1} \quad (\text{where } b = \vec{a} \cdot \vec{s} + \Delta m + e \in \mathbb{Z}_q)$
- **Decryption:** $\mathsf{LWE}_{S,\sigma}^{-1}(\mathsf{ct}) = b - \vec{a} \cdot \vec{s} = \Delta m + e \quad \in \mathbb{Z}_q$

For modulus reduction, $0 < r < 1$. The message and small secret stay fixed while the public ciphertext coefficients and scale shrink. The old error is multiplied by r, but coefficientwise rounding adds a separate error depending on the secret. Hence the total error need not decrease. Applications such as §D-1.8, §D-2.7, and §D-3.5 must use their own scale and correctness conditions.

The high-level idea of LWE modulus switch is to rescale the congruence relationship of the LWE scheme. LWE's homomorphic computation algorithms include the following: ciphertext-to-ciphertext addition, ciphertext-to-plaintext addition, ciphertext-to-plaintext multiplication, ciphertext-to-ciphertext multiplication. However, all congruence relationships used in these algorithms are essentially rewritten versions of the following single fundamental congruence relationship: $b = \vec{a} \cdot \vec{s} + \Delta m + e \bmod q$. Thus, modulus switch of an LWE ciphertext from $q \to q'$ is equivalent to rescaling the modulo of the above congruence relationship from $q \to q'$.

Based on this insight, the LWE cryptosystem's modulus switching from $q \to \hat{q}$ (where $q > \hat{q}$) is a process of converting the original LWE ciphertext $\mathsf{LWE}_{S,\sigma}(\Delta m + e)$ as follows:

⟨Summary C-4.1⟩ LWE Modulus Switching

Given an LWE ciphertext $(\vec{a}, b)$ where $b = \vec{a} \cdot \vec{s} + \Delta m + e \bmod q$ and $m \in \mathbb{Z}_t$, modulus switch of the ciphertext from q to $\hat{q}$ is equivalent to updating (A, b) to $(\vec{\hat{a}}, \hat{b})$ as follows:

$\vec{\hat{a}} = (\hat{a}_0, \hat{a}_1, \ldots, \hat{a}_{k-1})$, where each $\hat{a}_i = \left\lceil a_i \frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$ ▷ $\lceil \rfloor$ means rounding to the nearest integer

$\hat{b} = \left\lceil b \frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

$\mathsf{LWE}_{S,\sigma}(\hat{\Delta}m + \hat{e} + \epsilon_{all}) = (\hat{a}_0, \hat{a}_1, \ldots, \hat{b}) \in \mathbb{Z}_{\hat{q}}^{k+1}$ ▷ where ϵ_{all} is a small rounding error

The above update effectively changes $\hat{e}$ and $\hat{\Delta}$ as follows:

$\hat{e} = \left\lceil e \frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$,

$\hat{\Delta} = \Delta \frac{\hat{q}}{q}$ ▷ which should be an integer

Meanwhile, S and m stay the same as before.

Note that in order for $(\hat{a}_0, \hat{a}_1, \ldots, \hat{b}) \in \mathbb{Z}_{\hat{q}}^{k+1}$ to be a valid LWE ciphertext of $\hat{\Delta}m$, we need to prove that the following relationship holds:

$$\hat{b} = \sum_{i=0}^{k-1} \hat{a}_i \cdot s_i + \hat{\Delta}m + \hat{e} + \epsilon_{all} \in \mathbb{Z}_{\hat{q}}$$

Proof.

1. Note the following:
   $\hat{b} = \left\lceil b \frac{\hat{q}}{q} \right\rfloor = b \frac{\hat{q}}{q} + \epsilon_b$ (where $-0.5 \leq \epsilon_b \leq 0.5$, a rounding drift error)
   $\hat{a_i} = \left\lceil a_i \frac{\hat{q}}{q} \right\rfloor = a_i \frac{\hat{q}}{q} + \epsilon_{a_i}$ (where $-0.5 \leq \epsilon_{a_i} \leq 0.5$)
   $\hat{e} = \left\lceil e \frac{\hat{q}}{q} \right\rfloor = e \frac{\hat{q}}{q} + \epsilon_e$ (where $-0.5 \leq \epsilon_e \leq 0.5$)
2. Note the following:
   $b = \vec{a} \cdot \vec{s} + \Delta m + e = \sum_{i=0}^{k-1} (a_i s_i) + \Delta m + e \in \mathbb{Z}_q$
   $b = \sum_{i=0}^{k-1} (a_i s_i) + \Delta m + e + H \cdot q$ (where modulo q is replaced by adding $H \cdot q$, some multiple of q)
3. According to step 1 and 2:
   $$\begin{aligned}
   &\hat{b} = b \frac{\hat{q}}{q} + \epsilon_b \in \mathbb{Z}_{\hat{q}} \\
   &= \left(\sum_{i=0}^{k-1} (a_i s_i) + \Delta m + e + H \cdot q \right) \cdot \frac{\hat{q}}{q} + \epsilon_b \\
   &= \frac{\hat{q}}{q} \cdot \sum_{i=0}^{k-1} (a_i s_i) + \frac{\hat{q}}{q} \cdot \Delta m + \frac{\hat{q}}{q} \cdot e + \frac{\hat{q}}{q} \cdot H \cdot q + \epsilon_b \\
   &= \sum_{i=0}^{k-1} \left(\frac{\hat{q}}{q} \cdot a_i s_i \right) + \hat{\Delta} m + (\hat{e} - \epsilon_e) + \hat{q} \cdot H + \epsilon_b \\
   &= \sum_{i=0}^{k-1} ((\hat{a}_i - \epsilon_{a_i}) \cdot s_i) + \hat{\Delta} m + \hat{e} - \epsilon_e + \hat{q} \cdot H + \epsilon_b \\
   &= \sum_{i=0}^{k-1} (\hat{a}_i s_i - \epsilon_{a_i} s_i) + \hat{\Delta} m + \hat{e} - \epsilon_e + \epsilon_b \in \mathbb{Z}_{\hat{q}}
   \end{aligned}$$

$$= \sum_{i=0}^{k-1} \hat{a}_i s_i + \hat{\Delta} m + \left(\hat{e} - \epsilon_e + \epsilon_b - \sum_{i=0}^{k-1} \epsilon_{a_i} s_i \right) \in \mathbb{Z}_{\hat{q}}$$

$$= \sum_{i=0}^{k-1} \hat{a}_i s_i + \hat{\Delta} m + \hat{e} + \epsilon_{all} \in \mathbb{Z}_{\hat{q}} \triangleright \text{ where } \epsilon_{all} = \left(-\epsilon_e + \epsilon_b - \sum_{i=0}^{k-1} \epsilon_{a_i} s_i \right)$$

A deterministic bound on this auxiliary error is

$$|\epsilon_{all}| \leq 1 + \tfrac{1}{2}\|\vec{s}\|_1 \leq 1 + \tfrac{k}{2}$$

for binary or ternary secrets. It is an upper bound, not the actual error value. The error that decoding must remove is the total

$$e_{\text{new}} = \hat{e} + \epsilon_{all} = re + \epsilon_b - \sum_i \epsilon_{a_i} s_i, \qquad |e_{\text{new}}| \leq r|e| + \tfrac{1}{2}(1 + \|\vec{s}\|_1).$$

Requiring this bound to be less than $\hat{\Delta}/2$ suffices for correct decoding. The terms need not be independent or zero-mean: b depends on the masks. Increasing k does not itself make the error smaller.

Caution: Bounding ϵ_{all} alone is insufficient; it must be combined with the scaled input error. Pure exact scaling preserves the ratio $|e|/\Delta$. The extra rounding error is what can exhaust the smaller output budget.

4. To summarize, $\hat{b}$ is approximately as follows:
   $\hat{b} = \sum_{i=0}^{k-1} \hat{a}_i s_i + \hat{\Delta} m + \hat{e} + \epsilon_{all} \in \mathbb{Z}_{\hat{q}}$
   Thus, $(\hat{a}_0, \hat{a}_1, \ldots, \hat{b}) = \mathsf{LWE}_{S,\sigma}(\hat{\Delta} m + \hat{e} + \epsilon_{all})$, which decodes to m when $|e_{\text{new}}| < \hat{\Delta}/2$.

□

C-4.2 Example

Suppose we have the following LWE setup:

$t = 4$
$q = 64$
$k = 4$
$\Delta = \frac{q}{t} = 16$
$m = 1 \in \mathbb{Z}_t$
$S = (s_0, s_1, s_2, s_3) = (0, 1, 1, 0) \in \{-1, 0, 1\}^4$
$A = (a_0, a_1, a_2, a_3) = (-25, 12, -3, 7) \in \mathbb{Z}_q^4$
$e = 1 \in \mathbb{Z}_q$
$b = a_0 s_0 + a_1 s_1 + a_2 s_2 + a_3 s_3 + \Delta m + e = 26 \in \mathbb{Z}_q$
$\mathsf{LWE}_{S,\sigma}(\Delta m + e) = \mathsf{ct} = (a_0, a_1, a_2, a_3, b) = (-25, 12, -3, 7, 26) \in \mathbb{Z}_q^{k+1}$

Now, suppose we want modulus switching from $q = 64$ to $\hat{q} = 32$, which gives:
$\hat{\Delta} = \Delta \cdot \frac{32}{64} = 8$
$\hat{e} = \left\lceil 1 \cdot \frac{32}{64} \right\rfloor = 1$

$\mathsf{LWE}_{S,\sigma}(\hat{\Delta}m + \hat{e} + \epsilon_{all}) = \hat{\mathsf{ct}} = (\hat{a_0}, \hat{a_1}, \hat{a_2}, \hat{a_3}, \hat{b})$
$= \left(\left\lceil -25 \cdot \frac{32}{64} \right\rfloor, \left\lceil 12 \cdot \frac{32}{64} \right\rfloor, \left\lceil -3 \cdot \frac{32}{64} \right\rfloor, \left\lceil 7 \cdot \frac{32}{64} \right\rfloor, \left\lceil 26 \cdot \frac{32}{64} \right\rfloor \right)$
$= (-12, 6, -1, 4, 13) \in \mathbb{Z}_{\hat{q}}^{k+1}$

Now, verify if the following LWE constraint holds:
$\hat{b} = \hat{a}_0 s_0 + \hat{a}_1 s_1 + \hat{a}_2 s_2 + \hat{a}_3 s_3 + \hat{\Delta} m + \hat{e} + \epsilon_{all} \in \mathbb{Z}_{32}$
$13 = 0 + 6 - 1 + 0 + 8 \cdot 1 + 1 - 1 \in \mathbb{Z}_{32}$
$\epsilon_{all} = -1, \qquad e_{\text{new}} = \hat{e} + \epsilon_{all} = 1 - 1 = 0$
The auxiliary rounding correction is -1, making the congruence exact. The individual rounded values are:
$\hat{a_0} = \lceil -12.5 \rfloor = -12$, $\hat{a_2} = \lceil -1.5 \rfloor = -1$, $\hat{a_3} = \lceil 3.5 \rfloor = 4$, and $\hat{e} = \lceil 0.5 \rfloor = 1$

If we solve the LWE decryption formula:
$\hat{b} - (\hat{a}_0 s_0 + \hat{a}_1 s_1 + \hat{a}_2 s_2 + \hat{a}_3 s_3) = 13 - (0(-12) + 1(6) + 1(-1) + 0(4)) = 13 - (6 - 1) = 8 = \hat{\Delta} m + \hat{e} + \epsilon_{all} \in \mathbb{Z}_{32}$

$m = \left\lceil \frac{8}{\hat{\Delta}} \right\rfloor = \left\lceil \frac{8}{8} \right\rfloor = 1$, which is correct.

C-4.3 Discussion

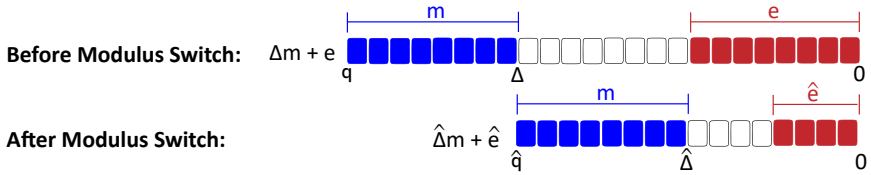


Figure 12: An illustration of scaled plaintext with a noise: $\Delta \cdot m + e \in \mathbb{Z}_q$

Relative Noise and Rounding: Figure 12 illustrates the shrinking numerical scale. Without rounding, $re/(r\Delta) = e/\Delta$, so the relative noise does not worsen merely because the modulus is smaller. Actual coefficientwise rounding contributes $\epsilon_b - \sum_i \epsilon_{a_i} s_i$. Relative to $\hat{\Delta} = r\Delta$, this extra error can become significant. Correctness must use the total-error bound, rather than a comparison of binary digit positions.

C-4.4 RLWE Modulus Switching

RLWE modulus switching is similar to LWE modulus switching. Recall that the RLWE cryptosystem (§B-3.2) comprises the following components:

- **Setup:** $\Delta = \frac{q}{t}, \quad S = s_0 + s_1 X + s_2 X^2 + \cdots + s_{n-1} X^{n-1} \xleftarrow{\$} \mathcal{R}_{\langle n, tern \rangle}$

- **Encryption Input:**
  $M \in \mathcal{R}_{\langle n,t \rangle}$
  $A = a_0 + a_1 X + a_2 X^2 + \cdots + a_{n-1} X^{n-1} \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}$
  $E = e_0 + e_1 X + e_2 X^2 + \cdots + e_{n-1} X^{n-1} \xleftarrow{\chi\sigma} \mathcal{R}_{\langle n,q \rangle}$

- **Encryption:** $\mathsf{RLWE}_{S,\sigma}(\Delta \cdot M + E) = (A, B) \in \mathcal{R}^2_{\langle n,q \rangle}$
  , where $B = A \cdot S + \Delta \cdot M + E = b_0 + b_1 X + b_2 X^2 + \cdots + b_{n-1} X^{n-1}$

- **Decryption:** $\mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}) = B - A \cdot S = \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$

RLWE modulus switching is done as follows:

⟨Summary C-4.4⟩ RLWE Modulus Switching

For an RLWE ciphertext (A, B) where $B = AS + \Delta M + E$ and $M \in \mathcal{R}_{\langle n,t \rangle}$, modulus switch of the ciphertext from q to $\hat{q}$ is equivalent to updating (A, B) to $(\hat{A}, \hat{B})$ as follows:

$\hat{A} = \hat{a}_0 + \hat{a}_1 X + \hat{a}_2 X^2 + \cdots + \hat{a}_{n-1} X^{n-1}$, where each $\hat{a}_i = \left\lceil a_i \frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

$\hat{B} = \hat{b}_0 + \hat{b}_1 X + \hat{b}_2 X^2 + \cdots + \hat{b}_{n-1} X^{n-1}$, where each $\hat{b}_i = \left\lceil b_i \frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

$\mathsf{RLWE}_{S,\sigma}(\hat{\Delta} M + \hat{E} + E^{\langle \epsilon_{all} \rangle}) = (\hat{A}, \hat{B}) \in \mathcal{R}^2_{\langle n,\hat{q} \rangle}$

The above update effectively changes Δ and E as follows:

$\hat{\Delta} = \Delta \frac{\hat{q}}{q}$ ▷ which should be an integer

$\hat{E} = \hat{e}_0 + \hat{e}_1 X + \hat{e}_2 X^2 + \cdots + \hat{e}_{n-1} X^{n-1}$, where each $\hat{e}_i = \left\lceil e_i \frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

Meanwhile, S and M stay the same as before.

The proof is similar to that of LWE modulus switching.

Proof

1. Note the following:
   $\hat{b}_i = \left\lceil b_i \frac{\hat{q}}{q} \right\rfloor = b_i \frac{\hat{q}}{q} + \epsilon_{b_i}$ (where $-0.5 \leq \epsilon_{b_i} \leq 0.5$, a rounding drift error)
   $\hat{a}_i = \left\lceil a_i \frac{\hat{q}}{q} \right\rfloor = a_i \frac{\hat{q}}{q} + \epsilon_{a_i}$ (where $-0.5 \leq \epsilon_{a_i} \leq 0.5$)
   $\hat{e}_i = \left\lceil e_i \frac{\hat{q}}{q} \right\rfloor = e_i \frac{\hat{q}}{q} + \epsilon_{e_i}$ (where $-0.5 \leq \epsilon_{e_i} \leq 0.5$)
2. Note the following:
   $$\begin{aligned}
   & B - A \cdot S \\
   &= (b_0 + b_1 X + \cdots + b_{n-1} X^{n-1}) - (a_0 + a_1 X + \cdots + a_{n-1} X^{n-1})(s_0 + s_1 X + \cdots + s_{n-1} X^{n-1}) \\
   &= \left(b_0 - \left(\sum_{i=0}^{0} (a_{0-i} s_i) - \sum_{i=1}^{n-1} (a_{n-i} s_i) \right) \right) \\
   &+ \left(b_1 - \left(\sum_{i=0}^{1} (a_{1-i} s_i) - \sum_{i=2}^{n-1} (a_{n+1-i} s_i) \right) \right) \cdot X \\
   &+ \left(b_2 - \left(\sum_{i=0}^{2} (a_{2-i} s_i) - \sum_{i=3}^{n-1} (a_{n+2-i} s_i) \right) \right) \cdot X^2 \\
   &\ \vdots \\
   &+ \left(b_{n-1} - \left(\sum_{i=0}^{n-1} (a_{n-1-i} s_i) - \sum_{i=n}^{n-1} (a_{n+n-1-i} s_i) \right) \right) \cdot X^{n-1} \quad \triangleright \text{ Grouping the terms by same exponents}
   \end{aligned}$$

$$= \sum_{h=0}^{n-1} \left(b_h - \left(\sum_{i=0}^{h} (a_{h-i} s_i) - \sum_{i=h+1}^{n-1} (a_{n+h-i} s_i) \right) \right) \cdot X^h$$

Thus,

$$B = \sum_{h=0}^{n-1} b_h X^h$$

$$A \cdot S = \sum_{h=0}^{n-1} \left(\sum_{i=0}^{h} (a_{h-i} s_i) - \sum_{i=h+1}^{n-1} (a_{n+h-i} s_i) \right) \cdot X^h$$

3. Based on step 2,
$B = A \cdot S + \Delta M + E$
$$\sum_{h=0}^{n-1} b_h X^h = \sum_{h=0}^{n-1} \left(\sum_{i=0}^{h} (a_{h-i} s_i) - \sum_{i=h+1}^{n-1} (a_{n+h-i} s_i) \right) \cdot X^h + \Delta \sum_{h=0}^{n-1} m_h X^h + \sum_{h=0}^{n-1} e_h X^h \in \mathcal{R}_{\langle n,q \rangle}$$
$$\sum_{h=0}^{n-1} b_h X^h = \sum_{h=0}^{n-1} \left(\sum_{i=0}^{h} (a_{h-i} s_i) - \sum_{i=h+1}^{n-1} (a_{n+h-i} s_i) \right) \cdot X^h + \Delta \sum_{h=0}^{n-1} m_h X^h + \sum_{h=0}^{n-1} e_h X^h + H \cdot q$$
Here $H = \sum_i c_i X^i$ has integer coefficients c_i, so Hq has coefficients $c_i q$. This converts the congruence into an equality of integer polynomial lifts.

4. According to step 1 and 3, for each j in $0 \le j \le n-1$:
$$\hat{b}_j = b_j \frac{\hat{q}}{q} + \epsilon_{b_j} \in \mathbb{Z}_{\hat{q}}$$
$$= \left(\sum_{i=0}^{j} (a_{j-i} s_i) - \sum_{i=j+1}^{n-1} (a_{n+j-i} s_i) + \Delta m_j + e_j + c_j \cdot q \right) \cdot \frac{\hat{q}}{q} + \epsilon_{b_j}$$
$$= \frac{\hat{q}}{q} \sum_{i=0}^{j} (a_{j-i} s_i) - \frac{\hat{q}}{q} \sum_{i=j+1}^{n-1} (a_{n+j-i} s_i) + \frac{\hat{q}}{q} \cdot \Delta m_j + \frac{\hat{q}}{q} e_j + \frac{\hat{q}}{q} \cdot c_j \cdot q + \epsilon_{b_j}$$
$$= \sum_{i=0}^{j} \left(\frac{\hat{q}}{q} \cdot a_{j-i} s_i \right) - \sum_{i=j+1}^{n-1} \left(\frac{\hat{q}}{q} \cdot a_{n+j-i} s_i \right) + \hat{\Delta} m_j + (\hat{e}_j - \epsilon_{e_j}) + \hat{q} \cdot c_j + \epsilon_{b_j}$$
$$= \sum_{i=0}^{j} ((\hat{a}_{j-i} - \epsilon_{a_{j-i}}) \cdot s_i) - \sum_{i=j+1}^{n-1} ((\hat{a}_{n+j-i} - \epsilon_{a_{n+j-i}}) \cdot s_i) + \hat{\Delta} m_j + (\hat{e}_j - \epsilon_{e_j}) + \hat{q} \cdot c_j + \epsilon_{b_j}$$
$$= \sum_{i=0}^{j} (\hat{a}_{j-i} s_i) - \sum_{i=j+1}^{n-1} (\hat{a}_{n+j-i} s_i) - \sum_{i=0}^{j} (\epsilon_{a_{j-i}} s_i) + \sum_{i=j+1}^{n-1} (\epsilon_{a_{n+j-i}} s_i) + \hat{\Delta} m_j + (\hat{e}_j - \epsilon_{e_j}) + \hat{q} \cdot c_j + \epsilon_{b_j}$$
$$= \left(\sum_{i=0}^{j} (\hat{a}_{j-i} s_i) - \sum_{i=j+1}^{n-1} (\hat{a}_{n+j-i} s_i) \right) + \hat{\Delta} m_j + \hat{e}_j + \left(\epsilon_{b_j} - \epsilon_{e_j} - \sum_{i=0}^{j} (\epsilon_{a_{j-i}} s_i) + \sum_{i=j+1}^{n-1} (\epsilon_{a_{n+j-i}} s_i) \right) + \hat{q} \cdot c_j$$
$$= \left(\sum_{i=0}^{j} (\hat{a}_{j-i} s_i) - \sum_{i=j+1}^{n-1} (\hat{a}_{n+j-i} s_i) \right) + \hat{\Delta} m_j + \hat{e}_j + \epsilon_{all} \in \mathbb{Z}_{\hat{q}}$$
▷ where $\epsilon_{all} = \epsilon_{b_j} - \epsilon_{e_j} - \sum_{i=0}^{j} (\epsilon_{a_{j-i}} s_i) + \sum_{i=j+1}^{n-1} (\epsilon_{a_{n+j-i}} s_i)$

5. To summarize, for each $0 \le j \le n-1$, each polynomial degree coefficient $\hat{b_j}$ is approximately as follows:
$$\hat{b}_j = \left(\sum_{i=0}^{j} (\hat{a}_{j-i} s_i) - \sum_{i=j+1}^{n-1} (\hat{a}_{n+j-i} s_i) \right) + \hat{\Delta} m_j + \hat{e}_j + \epsilon_{all}$$
Thus, $(\hat{A}, \hat{B}) = \mathsf{RLWE}_{S,\sigma}(\hat{\Delta} M + \hat{E} + E^{\langle \epsilon_{all} \rangle})$, which decodes to M only under the total coefficient-error

bound stated below.

□

C-4.5 GLWE Modulus Switching

GLWE modulus switching is an extension of RLWE modulus switching. The only difference is that while RLWE's A and S are a single polynomial each, GLWE's A and S are a list of k polynomials each. Thus, the same modulus switching technique as RLWE can be applied to GLWE for its k polynomials.

Recall that the GLWE cryptosystem (§B-4.2) is comprised of the following components:

- **Initial Setup:** $\Delta = \frac{q}{t}, \quad \{S_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}^k_{\langle n,tern\rangle}$
- **Encryption Input:** $M \in \mathcal{R}_{\langle n,t\rangle}, \quad \{A_i\}_{i=0}^{k-1} \xleftarrow{\$} \mathcal{R}^k_{\langle n,q\rangle}, \quad E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q\rangle}$
- **Encryption:** $\mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (\{A_i\}_{i=0}^{k-1}, B) \in \mathcal{R}^{k+1}_{\langle n,q\rangle}$
  , where $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta M + E \in \mathcal{R}_{\langle n,q\rangle}$
- **Decryption:** $\mathsf{GLWE}^{-1}_{S,\sigma}(\mathsf{ct}) = B - \sum_{i=0}^{k-1} (A_i \cdot S_i) = \Delta M + E \in \mathcal{R}_{\langle n,q\rangle}$

GLWE modulus switching is done as follows:

⟨Summary C-4.5⟩ GLWE Modulus Switching

Given a GLWE ciphertext $(\{A_i\}_{i=0}^{k-1}, B)$ where $B = \vec{A} \cdot \vec{S} + \Delta M + E \bmod q$ and $M \in \mathcal{R}_{\langle n,t\rangle}$, the modulus switch of the ciphertext from q to $\hat{q}$ is equivalent to updating $(\{A\}_{i=0}^{k-1}, B)$ to $(\{\hat{A}_i\}_{i=0}^{k-1}, \hat{B})$ as follows:

$\hat{A}_i = \hat{a}_{i,0} + \hat{a}_{i,1}X + \hat{a}_{i,2}X^2 + \cdots + \hat{a}_{i,n-1}X^{n-1}$, where each $\hat{a}_{i,j} = \left\lceil a_{i,j}\frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

$\hat{B} = \hat{b}_0 + \hat{b}_1X + \hat{b}_2X^2 + \cdots + \hat{b}_{n-1}X^{n-1}$, where each $\hat{b}_j = \left\lceil b_j\frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

$\mathsf{GLWE}_{S,\sigma}(\hat{\Delta}M + \hat{E} + E^{\langle\epsilon_{all}\rangle}) = (\{\hat{A}_i\}_{i=0}^{k-1}, \hat{B}) \in \mathcal{R}^{k+1}_{\langle n,\hat{q}\rangle}$

The above update effectively changes E and Δ as follows:

$\hat{E} = \hat{e}_0 + \hat{e}_1X + \hat{e}_2X^2 + \cdots + \hat{e}_{n-1}X^{n-1}$, where each $\hat{e}_j = \left\lceil e_j\frac{\hat{q}}{q} \right\rfloor \in \mathbb{Z}_{\hat{q}}$

$\hat{\Delta} = \Delta\frac{\hat{q}}{q}$ ▷ which should be an integer

Meanwhile, $\vec{S}$ and M stay the same as before.

The proof is similar to that of RLWE modulus switching. The modulus-switched GLWE ciphertext's culminating rounding drift error for each j-th polynomial coefficient in its congruence relationship (i.e., $B = \sum_{i=0}^{k-1} A_i \cdot S_i + \Delta M + E$) is as follows:

$$\epsilon_{j,all} = \epsilon_{b_j} - \epsilon_{e_j} - \sum_{l=0}^{k-1}\sum_{i=0}^{j}(\epsilon_{a_{l,j-i}} \cdot s_{l,i}) + \sum_{l=0}^{k-1}\sum_{i=j+1}^{n-1}(\epsilon_{a_{l,n+j-i}} \cdot s_{l,i})$$

▷ derived from the proof step 4 of Summary C-4.4: $\epsilon_{all} = \epsilon_{b_j} - \epsilon_{e_j} - \sum_{i=0}^{j} (\epsilon_{a_{j-i}} s_i) + \sum_{i=j+1}^{n-1} (\epsilon_{a_{n+j-i}} s_i)$

For a GLWE ciphertext, let ϵ_B and ϵ_{A_l} be the coefficientwise rounding-error polynomials. The exact new error is

$$E_{\text{new}} = rE + \epsilon_B - \sum_{l=0}^{k-1} \epsilon_{A_l} S_l, \qquad \|E_{\text{new}}\|_\infty \le r\|E\|_\infty + \tfrac{1}{2}\left(1 + \sum_{l=0}^{k-1} \|S_l\|_1\right).$$

For ternary secrets this is at most $r\|E\|_\infty + (1+kn)/2$; RLWE is the case $k = 1$. Requiring the bound to be less than $\hat{\Delta}/2$ suffices for correct coefficientwise decoding. The auxiliary term used in the proof satisfies $|\epsilon_{j,\text{all}}| \le 1 + \frac{1}{2}\sum_l \|S_l\|_1$. A sharper probabilistic estimate needs explicit distributional or independence assumptions; neither independence nor a small error follows from the central limit theorem alone. The hidden $\hat{E}$ is an analysis device, not a value the evaluator separately computes.

C-5 GLWE Key Switching

For the exact descending gadget below assume $q = \beta^l$, integer $\beta \geq 2$, and use the unsigned coefficient digits of §A-6.2. Both keys use the same ring and modulus, while their numbers of components k and k' may differ. The key owner precomputes the encryptions of the old secret components under the new key. Assume $t \mid q$ for the decoded-message bound. The arithmetic proof and the security of the published evaluation material are separate. The conditional security arguments at the end of this section distinguish independent-key acyclic graphs, same-key raw affine messages, and other key-dependent cases.

- Reference: TFHE Deep Dive - Part III - Key switching and leveled multiplications [9]

Under the security conditions stated below, key switching changes a GLWE ciphertext from secret key S to S' by public evaluation with a precomputed key-switching key, without revealing the plaintext. It preserves the phase's message term but replaces E by a generally different error E'. Correct decoding is conditional on the new error bound; key switching is not a noise refresh.

Remember that $\mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (A_0, A_1, \ldots, A_{k-1}, B) \in \mathcal{R}^{k+1}_{\langle n,q \rangle}$

, where $B = \sum_{i=0}^{k-1} (A_i \cdot S_i) + \Delta \cdot M + E$

, and the secret key S is a list of k polynomials: $S = (S_0, S_1, \ldots, S_{k-1})$

Also, remember that GLev (§B-5) is defined as follows:

$\mathsf{GLev}^{\beta,l}_{S,\sigma}(M) = \left\{ \mathrm{GLWE}_{S,\sigma}\left(\frac{q}{\beta^i} \cdot M + E_i\right) \right\}_{i=1}^{l} \in \mathcal{R}^{(k+1)\cdot l}_{\langle n,q \rangle}$

Now, let us denote each of the k key-switching keys as follows:

$KSK_i = \mathsf{GLev}^{\beta,l}_{S',\sigma}(S_i)$

$= \left(\mathsf{GLWE}_{S',\sigma}\left(\frac{q}{\beta^1} S_i + E_{i,1}\right), \mathsf{GLWE}_{S',\sigma}\left(\frac{q}{\beta^2} S_i + E_{i,2}\right), \ldots, \mathsf{GLWE}_{S',\sigma}\left(\frac{q}{\beta^l} S_i + E_{i,l}\right) \right) \in \mathcal{R}^{(k'+1)\cdot l}_{\langle n,q \rangle}$

, which is a list of GLWE encryptions of the secret key S by S', and the new secret key S' is a list of k' polynomials, $(S'_0, S'_1, \ldots, S'_{k'-1})$. Then, the GLWE ciphertext's key can be switched from $S \to S'$ as follows:

⟨Summary C-5⟩ GLWE Key Switching

Given $\mathsf{GLWE}_{S,\sigma}(\Delta M + E) = (A, B)$,

$$\mathsf{GLWE}_{S',\sigma}(\Delta M + E') = (\overbrace{0, 0, \ldots, 0}^{k'}, B) - \sum_{i=0}^{k-1} \langle \mathsf{Decomp}^{\beta,l}(A_i), KSK_i \rangle$$

Proof.

1. Given the principle of polynomial decomposition (§A-6.2) and the polynomial $A_i \in \mathbb{Z}_q[x]/(x^n + 1)$, its decomposition is as follows:

   $\mathsf{Decomp}^{\beta,l}(A_i) = (A_{i,1}, A_{i,2}, \ldots, A_{i,l})$, where

$A_i = A_{i,1}\frac{q}{\beta^1} + A_{i,2}\frac{q}{\beta^2} + \cdots + A_{i,l}\frac{q}{\beta^l}$

2. $\langle \mathsf{Decomp}^{\beta,l}(A_i), KSK_i \rangle$
$= A_{i,1} \cdot \mathsf{GLWE}_{S',\sigma}\left(\frac{q}{\beta^1}S_i + E_{i,1}\right) + A_{i,2} \cdot \mathsf{GLWE}_{S',\sigma}\left(\frac{q}{\beta^2}S_i + E_{i,2}\right) + \cdots + A_{i,l} \cdot \mathsf{GLWE}_{S',\sigma}\left(\frac{q}{\beta^l}S_i + E_{i,l}\right)$
$\triangleright$ where each GLWE ciphertext is an encryption of $S_i\frac{q}{\beta}, S_i\frac{q}{\beta^2}, \cdots, S_i\frac{q}{\beta^l}$ as plaintext with the plaintext scaling factor 1

$= \mathsf{GLWE}_{S',\sigma}\left(\left(A_{i,1}\frac{q}{\beta^1} + A_{i,2}\frac{q}{\beta^2} + \cdots + A_{i,l}\frac{q}{\beta^l}\right) \cdot S_i + E_i\right)$ $\triangleright$ where $E_i = \sum_{j=1}^{l} A_{i,j}E_{i,j}$
$= \mathsf{GLWE}_{S',\sigma}\left((A_i) \cdot S_i + E_i\right) = \mathsf{GLWE}_{S',\sigma}(A_iS_i + E_i)$

3. $\sum_{i=0}^{k-1} \langle \mathsf{Decomp}^{\beta,l}(A_i), KSK_i \rangle = \sum_{i=0}^{k-1} (\mathsf{GLWE}_{S',\sigma}(A_iS_i + E_i))$
$= \mathsf{GLWE}_{S',\sigma}\left(\sum_{i=0}^{k-1}(A_iS_i + E_i)\right)$

4. B is equivalent to a trivial GLWE encryption with S', where its every $A_0, A_1, \ldots, A_{k'-1}$ is 0. That is, $\mathsf{GLWE}_{S',\sigma}(B) = (\overbrace{0, 0, \ldots}^{k'}, B)$. Note that $\mathsf{GLWE}_{S',\sigma}(B)$ can be created without the knowledge of S', because its all A_iS_i terms are 0.

5. $\mathsf{GLWE}_{S',\sigma}(B) - \mathsf{GLWE}_{S',\sigma}\left(\sum_{i=0}^{k-1}(A_iS_i + E_i)\right)$
$= \mathsf{GLWE}_{S',\sigma}(B - \sum_{i=0}^{k-1}(A_iS_i + E_i))$
$= \mathsf{GLWE}_{S',\sigma}(\Delta M + E')$ $\triangleright$ where $E' = E - \sum_{i=0}^{k-1} E_i$

6. If we explicitly expand the above relation,
$\mathsf{GLWE}_{S',\sigma}(B) - \mathsf{GLWE}_{S',\sigma}\left(\sum_{i=0}^{k-1}(A_iS_i + E_i)\right)$
$= (\overbrace{0, 0, \ldots}^{k'}, B) - (\overbrace{A'_0, A'_1, \ldots, A'_{k'-1}}^{k'}, B')$ $\triangleright$ where $B' = \sum_{i=0}^{k'-1} A'_iS'_i + \sum_{i=0}^{k-1} A_iS_i + \sum_{i=0}^{k-1} E_i$
$(\overbrace{-A'_0, -A'_1, \ldots, -A'_{k'-1}}^{k'}, B - B')$

The decryption of the above ciphertext gives us:
$\mathsf{GLWE}^{-1}_{S',\sigma}\left((\overbrace{-A'_0, -A'_1, \ldots, -A'_{k'-1}}^{k'}, B - B')\right)$
$= B - B' - \sum_{i=0}^{k'-1} -A'_iS'_i$
$= B - \sum_{i=0}^{k'-1} A'_iS'_i - \sum_{i=0}^{k-1} A_iS_i - \sum_{i=0}^{k-1} E_i + \sum_{i=0}^{k'-1} A'_iS'_i$

$$= B - \sum_{i=0}^{k-1} A_i S_i - \sum_{i=0}^{k-1} E_i$$
$$= \Delta M + E - \sum_{i=0}^{k-1} E_i$$
$$= \Delta M + E' \triangleright \text{ where } E' = E - \sum_{i=0}^{k-1} E_i$$

Strictly speaking, $B - \sum_{i=0}^{k-1} A_i S_i = \Delta M + E + Kq$ where K is a polynomial representing the wrap-around coefficient values as multiples of q. However, since all the above computations are done in modulo q, the Kq term gets eliminated.

7. Thus, $(0, 0, \ldots, B) - \sum_{i=0}^{k-1} \langle \mathsf{Decomp}^{\beta,l}(A_i), KSK_i \rangle = \mathsf{GLWE}_{S',\sigma}(\Delta M + E')$

   $\mathsf{GLWE}_{S',\sigma}(\Delta M + E')$ is an encryption of plaintext ΔM with the plaintext scaling factor 1. However, we can re-interpret this ciphertext as an encryption of plaintext M with the plaintext scaling factor Δ. This way, we can recover the ciphertext's original scaling factor Δ without any additional computation. □

Noise and Truncation: For exact decomposition,

$$E' = E - \sum_{i=0}^{k-1} \sum_{j=1}^{l} A_{i,j} E_{i,j}, \qquad \|E'\|_\infty \le \|E\|_\infty + \sum_{i,j} \|A_{i,j}\|_1 \|E_{i,j}\|_\infty.$$

If all key-switching errors have norm at most B_{ks}, the unsigned-digit bound is $\|E'\|_\infty \le \|E\|_\infty + knl(\beta - 1)B_{\text{ks}}$. It must remain below $\Delta/2$ for guaranteed decoding. An algebraic phase identity does not imply that the new ciphertext has the original error distribution.

For a truncated descending gadget with integer weights, write $A_i \equiv \sum_j A_{i,j}(q/\beta^j) + \rho_i \pmod q$. The same public algorithm gives

$$E' = E - \sum_{i,j} A_{i,j} E_{i,j} + \sum_i \rho_i S_i,$$

so the bound must also include $\sum_i \|\rho_i\|_\infty \|S_i\|_1$. Omitting this residual would invalidate the proof. An exact integer gadget for general q is possible, but its weights and decomposition must be defined consistently in both the ciphertext digits and the key-switching key.

Failure probability: Assume $t \mid q$ and $\Delta = q/t$. Write $p_\chi(B) = \Pr_{e \leftarrow \chi}[|e| > B]$ and let $\mathcal{B}$ be the event that some input or key error exceeds its bound. For Q fresh input ciphertexts and one set of kl key-switching ciphertexts, the coefficientwise union bound gives

$$\Pr[\mathcal{B}] \le nQp_{\chi_{\text{in}}}(B_{\text{in}}) + nklp_{\chi_{\text{ks}}}(B_{\text{ks}}).$$

If the preceding deterministic error bound, including every truncation residual, is below $\Delta/2$ on this event's complement, this is also a bound on failure of any of the Q switched ciphertexts. It does not require independent output errors. The sampling law, its tail function, the number of samples and the target failure rate must still be specified; a discrete Gaussian and a rounded continuous Gaussian are different distributions.

Security of the published switching material: Use the actual-distribution decisional GLWE advantage $\delta(T, Q)$ and guessing-bias convention of §B-1.2. Here each auxiliary ciphertext has a raw ring

phase; its message is not first reduced modulo a separate plaintext modulus. There are two directly provable cases.

For the same key $S' = S$, let L be a public vector of ring elements and c a public ring constant. Given a challenge sample (A', Z), a reduction simulates encryption of the raw affine function $\langle L, S\rangle + c$ by returning $(A' - L, Z + c)$, since

$$Z + c = \langle A' - L, S\rangle + E + \langle L, S\rangle + c.$$

For the KDM real-function-versus-zero game, choose a hidden bit ϑ internally and return $(A' - \vartheta L, Z + \vartheta c)$. Real samples give precisely the selected raw encryption; uniform samples remain an independent uniform pair after these translations. This holds conditional on every earlier transcript, so it covers adaptively selected public L, c and polynomially many samples. Outputting whether the bit was guessed gives $\epsilon_{\text{aff}} \le \delta(T', Q)$, or $2\delta(T', Q)$ for the output-probability-difference convention. The row message $(q/\beta^j)S_i$ is obtained by placing q/β^j in component i of L and zeros elsewhere. Thus this same-key raw switching material does not require an additional circular-security assumption, provided its masks are independent uniform and its error law is covered by the sample assumption. See Brakerski–Vaikuntanathan, Section 1.1, KDM Security.

For independently generated keys in a fixed directed acyclic graph, define an edge $u \to v$ to mean raw encryption of a public efficient function $f_{uv}(S_u)$ under S_v. Process destination keys in reverse topological order, starting with sinks. When replacing all ciphertexts under S_v by uniform pairs, the reduction knows every other key and can compute all incoming messages. Every outgoing ciphertext whose message depends on the unknown S_v has already been replaced by a uniform pair. This gives an exact adjacent-hybrid simulation using the decisional oracle for S_v. If Q_v counts all samples under that key, the resulting joint-view bound is $\epsilon_{\text{DAG}} \le \sum_v \delta_v(T'_v, Q_v)$. This statement assumes no secret corruptions or decryption queries and fresh independent randomness for every auxiliary encryption. Same-key raw affine loops can be handled by the preceding mask shift; a general cycle does not admit the acyclic simulation order.

The affine proof requires the *actual phase message* to be affine over the ciphertext ring. Composing a function with reduction modulo t and scaling by $\lfloor q/t \rfloor$ can add a secret-dependent wrap term; it vanishes automatically when $t \mid q$ and $\Delta = q/t$. Moreover, a coefficient-linear map need not be ring multiplication: a general secret automorphism $S(X^j)$, coefficient or bit extraction, and S^2 are not automatically of the form $LS + c$. Non-affine self-dependent material, general multi-key cycles, related-key distributions, and fixed seeded-mask formats need their own matching proof or explicitly stated KDM/seeded assumption. These restrictions do not negate the two conditional reductions above, and those reductions do not certify the hardness of an unspecified parameter family.

Once the input ciphertexts and all published auxiliary material jointly hide their messages, public key switching and the other public homomorphic operations preserve this indistinguishability by efficient postprocessing. This says neither that the output has the fresh-encryption distribution nor that circuit privacy, chosen-ciphertext security, or correctness follows without their separate conditions.

Part IV
Fully Homomorphic Encryption Schemes

This chapter explains the four most well-known FHE schemes: TFHE, CKKS, BGV, and BFV, as well as their RNS-variant versions.

D-1 TFHE Scheme

TFHE supports homomorphic computation on encoded finite-domain integers, especially Boolean circuits. Its fast bootstrapping is useful for deep circuits when the encoding and noise bounds hold. Programmable bootstrapping combines ciphertext refresh with evaluation of a selected lookup function; this capability alone is not the formal cryptographic notion of functional encryption. The construction and error analysis are based on the original TFHE paper [10].

In TFHE, each plaintext is encrypted as an LWE ciphertext. Therefore, TFHE's ciphertext-to-ciphertext addition, ciphertext-to-plaintext addition, and ciphertext-to-plaintext multiplication are implemented based on GLWE's homomorphic addition and multiplication described in Part III, with $n = 1$ to make GLWE an LWE.

This section will explain TFHE's novel components: key switching, ciphertext-to-ciphertext multiplication, coefficient extraction, and noise bootstrapping.

Required Background

- §A-1: Modulo Arithmetic
- §A-2: Group
- §A-3: Field
- §A-4: Order
- §A-5: Polynomial Ring
- §A-6: Decomposition
- §A-13: Modulo Rescaling
- §B-1: Lattice-based Cryptography
- §B-2: LWE Cryptosystem
- §B-3: RLWE Cryptosystem
- §B-4: GLWE Cryptosystem
- §B-5: GLev
- §B-6: GGSW
- §C-1: GLWE Ciphertext-to-Ciphertext Addition
- §C-2: GLWE Ciphertext-to-Plaintext Addition
- §C-3: GLWE Ciphertext-to-Plaintext Multiplication
- §C-4.1: LWE Modulus Switching
- §C-5: GLWE Key Switching

D-1.1 Encryption and Decryption

TFHE encrypts and decrypts ciphertexts using LWE (§B-2), equivalently GLWE (§B-4) with $n = 1$. The TFHE construction considered here samples the secret from $\{0,1\}^k$. Binary and ternary secrets are both choices within the broader LWE framework; a ternary secret is not part of the definition of LWE.

⟨Summary D-1.1⟩ TFHE Encryption and Decryption

Initial Setup: $\Delta = \frac{q}{t}$, $\vec{s} \xleftarrow{\$} \mathbb{Z}_2^k$ ▷ where t divides q, and each element of $\vec{s}$ is a 0-degree polynomial

---

Encryption Input: $m \in \mathbb{Z}_t$, $\vec{a} \xleftarrow{\$} \mathbb{Z}_q^k$, $e \xleftarrow{\chi_\sigma} \mathbb{Z}_q$ ▷ each element of $\vec{a}$ is a 0-degree polynomial

1. Scale up $m \longrightarrow \Delta \cdot m \ \in \mathbb{Z}_q$
2. Compute $b = \vec{a} \cdot \vec{s} + \Delta m + e \mod q$
3. $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) = (\vec{a}, b) \ \in \mathbb{Z}_q^{k+1}$

Decryption Input: $\mathsf{ct} = (\vec{a}, b) \ \in \mathbb{Z}_q^{k+1}$

1. $\mathsf{LWE}_{\vec{s},\sigma}^{-1}(\mathsf{ct}) = b - \vec{a} \cdot \vec{s} = \Delta m + e \pmod q$
2. Scale down $\left\lceil \frac{\Delta m + e}{\Delta} \right\rfloor = m \ \in \mathbb{Z}_t$ ▷ i.e., modulus switch from $q \to t$

Condition for Correct Decryption:

- The noise e grown over homomorphic operations should be: $|e| < \frac{\Delta}{2}$.

In this section, we will often write $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e)$ as $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m)$ for simplicity, because $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) \approx \mathsf{LWE}_{\vec{s},\sigma}(\Delta m)$ (i.e., they decrypt to the same message). Even in the case that we write $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m)$ instead of $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e)$, you should assume this as an encryption of $\Delta m + e$ (i.e., the noise is included inside the scaled message).

D-1.2 Homomorphic Ciphertext-to-Ciphertext Addition

TFHE's ciphertext-to-ciphertext addition uses LWE's ciphertext-to-ciphertext addition scheme, which is equivalent to GLWE's ciphertext-to-ciphertext addition scheme (§C-1) with $n = 1$.

⟨Summary D-1.2⟩ TFHE Ciphertext-to-Ciphertext Addition

$$
\begin{aligned}
&\mathsf{LWE}_{\vec{s},\sigma}(\Delta m^{\langle 1 \rangle} + e^{\langle 1 \rangle}) + \mathsf{LWE}_{\vec{s},\sigma}(\Delta m^{\langle 2 \rangle} + e^{\langle 2 \rangle}) \\
&= (\vec{a}^{\langle 1 \rangle},\ b^{\langle 1 \rangle}) + (\vec{a}^{\langle 2 \rangle},\ b^{\langle 2 \rangle}) \\
&= (\vec{a}^{\langle 1 \rangle} + \vec{a}^{\langle 2 \rangle},\ b^{\langle 1 \rangle} + b^{\langle 2 \rangle}) \\
&= \mathsf{LWE}_{\vec{s},\sigma}(\Delta(m^{\langle 1 \rangle} + m^{\langle 2 \rangle}) + e^{\langle 1+2 \rangle})
\end{aligned}
$$

D-1.3 Homomorphic Ciphertext-to-Plaintext Addition

TFHE's ciphertext-to-plaintext addition (where λ is a constant to add) uses LWE's ciphertext-to-plaintext addition scheme, which is equivalent to GLWE's ciphertext-to-plaintext addition scheme (§C-2) with

$n = 1$.

⟨Summary D-1.3⟩ TFHE Ciphertext-to-Plaintext Addition

$$\begin{aligned}&\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) + \Delta\lambda \\ &= (\vec{a},\ b) + \Delta\lambda \\ &= (\vec{a},\ b + \Delta\lambda) \\ &= \mathsf{LWE}_{\vec{s},\sigma}(\Delta(m+\lambda) + e)\end{aligned}$$

D-1.4 Homomorphic Ciphertext-to-Plaintext Multiplication

TFHE's ciphertext-to-plaintext multiplication uses LWE's ciphertext-to-plaintext multiplication scheme, which is equivalent to GLWE's ciphertext-to-plaintext multiplication scheme (§C-3) with $n = 1$.

⟨Summary D-1.4⟩ TFHE Ciphertext-to-Plaintext Multiplication

$$\begin{aligned}&\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) \cdot \lambda \\ &= (\vec{a},\ b) \cdot \lambda \\ &= (\lambda \cdot \vec{a},\ \lambda \cdot b) \\ &= \mathsf{LWE}_{\vec{s},\sigma}(\Delta(m \cdot \lambda) + \lambda \cdot e)\end{aligned}$$

D-1.5 Homomorphic Key Switching

- Reference: TFHE Deep Dive - Part III - Key switching and leveled multiplications [9]

TFHE key switching changes the LWE secret from $\vec{s}$ to $\vec{s}'$, possibly changing its dimension. With this chapter's phase $b - \langle \vec{a}, \vec{s} \rangle$, it is the $n = 1$ instance of §C-5. The minus sign below corresponds to this convention; the alternative convention in §B-4.4 uses the opposite sign.

⟨Summary D-1.5⟩ TFHE Key Switching

Given $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) = (\vec{a}, b)$,

$$\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m + e') = (0, b) - \langle \mathsf{Decomp}^{\beta,l}(\vec{a}),\ \mathsf{Lev}^{\beta,l}_{\vec{s}',\sigma}(\vec{s}) \rangle$$

D-1.6 Homomorphic Ciphertext-to-Ciphertext Multiplication

A separate leveled multiplication of two LWE ciphertexts is constructed and conditionally proved in §D-2.7.6. It uses tensor-product evaluation keys, an exact $\Delta = q/t$ scale and explicit rounding/noise bounds. It is not this section's GSW external product, and its evaluation keys do not replace the GGSW keys used by blind rotation.

The exact decomposition identities in the following proof assume $q = \beta^l$. For a truncated decomposition write $c = \langle \mathsf{Decomp}(c), \vec{g} \rangle + \rho_c \pmod q$. Its residual must be included in the output error, together with the errors in the GSW rows. The notation σ in a homomorphic output does not assert that its noise distribution remains the fresh encryption distribution.

- Reference: TFHE Deep Dive - Part III - Key switching and leveled multiplications [9]

TFHE supports multiplication of two ciphertexts in the form: $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1) \cdot \mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2)$.

The 1st term $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 + e_1)$ comes from one of the following:
- A fresh LWE encryption (§B-4.2) of plaintext m_1.
- A homomorphically added result of two LWE ciphertexts (§C-1).
- A homomorphically multiplied result of a LWE ciphertext with a plaintext (§C-3).

The 2nd term $\mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2)$ comes from one of the following:
- A fresh GSW encryption (§B-6.1) of plaintext m_2.
- Converted from $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m_2 + e_2)$ into $\mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2)$ by *circuit bootstrapping* (this will be covered in the future).

Remember the following:

$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 + e_1) = (\vec{a}, b) \in \mathbb{Z}_q^{k+1}$, where $b = \vec{a} \cdot \vec{s} + \Delta m_1 + e_1$

$\mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2) = \left\{ \{\mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(-s_i \cdot m_2)\}_{i=0}^{k-1}, \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(m_2) \right\} \in \mathbb{Z}_q^{(k+1)\cdot l \cdot (k+1)}$ ▷ from §B-6.1

Let's use the following notations:

$\mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2) = \bar{\mathsf{ct}} = (\bar{\mathsf{ct}}_0, .\bar{\mathsf{ct}}_1, \cdots \bar{\mathsf{ct}}_k)$

$\bar{\mathsf{ct}}_i = \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(-s_i \cdot m_2)$ for $0 \leq i \leq (k-1)$

$\bar{\mathsf{ct}}_k = \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(m_2)$

$\mathsf{ct} = \mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 + e_1) = (\vec{a}, b) = (a_0, a_1, \cdots, a_{k-1}, b) = (\mathsf{ct}_0, \mathsf{ct}_1, \cdots, \mathsf{ct}_k)$

Let's define the following TFHE ciphertext multiplication operation:

$$\mathsf{ct} \cdot \bar{\mathsf{ct}} = \sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(\mathsf{ct}_i), \bar{\mathsf{ct}}_i \rangle$$

Then, the following is true:

⟨Summary D-1.6⟩ TFHE Ciphertext-to-Ciphertext Multiplication

$\mathsf{ct} = \mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 + e_1) = (a_0, a_1, \cdots, a_{k-1}, b)$

$\bar{\mathsf{ct}} = \mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2) = (\mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(-s_0 \cdot m_2), \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(-s_1 \cdot m_2), \cdots, \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(-s_{k-1} \cdot m_2), \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(m_2))$

$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 + e_1) \cdot \mathsf{GSW}^{\beta,l}_{\vec{s},\sigma}(m_2)$

$= \sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(\mathsf{ct}_i), \bar{\mathsf{ct}}_i \rangle$

$\approx \mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 m_2)$

This means that multiplying two TFHE ciphertexts (one is in LWE and another in GSW) and decrypting the resulting LWE ciphertext gives the same result as multiplying their two original plaintexts.

Proof.

1. $\sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(\mathsf{ct}_i), \bar{\mathsf{ct}}_i \rangle$
   $= \langle \mathsf{Decomp}^{\beta,l}(a_0), \bar{\mathsf{ct}}_0 \rangle + \langle \mathsf{Decomp}^{\beta,l}(a_1), \bar{\mathsf{ct}}_1 \rangle + \cdots + \langle \mathsf{Decomp}^{\beta,l}(a_{k-1}), \bar{\mathsf{ct}}_{k-1} \rangle + \langle \mathsf{Decomp}^{\beta,l}(b), \bar{\mathsf{ct}}_k \rangle$
   ▷ expanding the dot product of two vectors

2. For $i = k$:
   $\mathsf{Decomp}^{\beta,l}(b) = (b_1, b_2, \ldots, b_l)$, where $b = b_1\frac{q}{\beta^1} + b_2\frac{q}{\beta^2} + \cdots + b_l\frac{q}{\beta^l}$ ▷ from §A-6.2

   $\bar{\mathsf{ct}}_k = \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(m_2) = \left(\mathsf{LWE}_{\vec{s},\sigma}\left(m_2\frac{q}{\beta^1} + e_{2,1}\right), \mathsf{LWE}_{\vec{s},\sigma}\left(m_2\frac{q}{\beta^2} + e_{2,2}\right), \ldots, \mathsf{LWE}_{\vec{s},\sigma}\left(m_2\frac{q}{\beta^l} + e_{2,l}\right)\right)$

   Therefore:
   $\langle \mathsf{Decomp}^{\beta,l}(b), \bar{\mathsf{ct}}_k\rangle$
   $= b_1 \cdot \mathsf{LWE}_{\vec{s},\sigma}\left(m_2\frac{q}{\beta^1} + e_{2,1}\right) + b_2 \cdot \mathsf{LWE}_{\vec{s},\sigma}\left(m_2\frac{q}{\beta^2} + e_{2,2}\right) + \cdots + b_l \cdot \mathsf{LWE}_{\vec{s},\sigma}\left(m_2\frac{q}{\beta^l} + e_{2,l}\right)$
   $= \mathsf{LWE}_{\vec{s},\sigma}\left(b_1 m_2\frac{q}{\beta^1} + b_1 e_{2,1}\right) + \mathsf{LWE}_{\vec{s},\sigma}\left(b_2 m_2\frac{q}{\beta^2} + b_2 e_{2,2}\right) + \cdots + \mathsf{LWE}_{\vec{s},\sigma}\left(b_l m_2\frac{q}{\beta^l} + b_l e_{2,l}\right)$ ▷ from §C-3
   $= \mathsf{LWE}_{\vec{s},\sigma}\left(b_1 m_2\frac{q}{\beta^1} + b_2 m_2\frac{q}{\beta^2} + \cdots + b_l m_2\frac{q}{\beta^l} + b_1 e_{2,1} + \cdots + b_l e_{2,l}\right)$ ▷ from §C-1
   $= \mathsf{LWE}_{\vec{s},\sigma}\left(m_2 \cdot \left(b_1\frac{q}{\beta^1} + b_2\frac{q}{\beta^2} + \cdots + b_l\frac{q}{\beta^l}\right) + e^{\langle k\rangle}\right)$ ▷ where $e^{\langle k\rangle} = \sum_{i=1}^{l} b_i e_{2,i}$
   $= \mathsf{LWE}_{\vec{s},\sigma}(m_2 b + e^{\langle k\rangle})$ ▷ from §A-6.2
3. For $0 \le i \le (k-1)$:
   $\mathsf{Decomp}^{\beta,l}(a_i) = (a_{\langle i,1\rangle}, a_{\langle i,2\rangle}, \cdots, a_{\langle i,l\rangle})$, where $a_i = a_{\langle i,1\rangle}\frac{q}{\beta^1} + a_{\langle i,2\rangle}\frac{q}{\beta^2} + \cdots + a_{\langle i,l\rangle}\frac{q}{\beta^l}$

   $\bar{\mathsf{ct}}_i = \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(-s_i m_2)$
   $= \left(\mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2\frac{q}{\beta^1} + e_{i,1}\right), \mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2\frac{q}{\beta^2} + e_{i,2}\right), \cdots, \mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2\frac{q}{\beta^l} + e_{i,l}\right)\right)$

   Therefore:
   $\langle \mathsf{Decomp}^{\beta,l}(a_0), \bar{\mathsf{ct}}_0\rangle + \langle \mathsf{Decomp}^{\beta,l}(a_1), \bar{\mathsf{ct}}_1\rangle + \cdots + \langle \mathsf{Decomp}^{\beta,l}(a_{k-1}), \bar{\mathsf{ct}}_{k-1}\rangle$
   $= \sum_{i=0}^{k-1} \langle \mathsf{Decomp}^{\beta,l}(a_i), \bar{\mathsf{ct}}_i\rangle$
   $= \sum_{i=0}^{k-1} \left(a_{\langle i,1\rangle} \cdot \mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2\frac{q}{\beta^1} + e_{i,1}\right) + a_{\langle i,2\rangle} \cdot \mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2\frac{q}{\beta^2} + e_{i,2}\right) + \right.$
   $\left. \cdots + a_{\langle i,l\rangle} \cdot \mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2\frac{q}{\beta^l} + e_{i,l}\right)\right)$
   $= \sum_{i=0}^{k-1} \left(\mathsf{LWE}_{\vec{s},\sigma}\left(-a_{\langle i,1\rangle} s_i m_2\frac{q}{\beta^1} + a_{\langle i,1\rangle} e_{i,1}\right) + \mathsf{LWE}_{\vec{s},\sigma}\left(-a_{\langle i,2\rangle} s_i m_2\frac{q}{\beta^2} + a_{\langle i,2\rangle} e_{i,2}\right) + \right.$
   $\left. \cdots + \mathsf{LWE}_{\vec{s},\sigma}\left(-a_{\langle i,l\rangle} s_i m_2\frac{q}{\beta^l} + a_{\langle i,l\rangle} e_{i,l}\right)\right)$
   $= \sum_{i=0}^{k-1} \mathsf{LWE}_{\vec{s},\sigma}\left(-a_{\langle i,1\rangle} s_i m_2\frac{q}{\beta^1} - a_{\langle i,2\rangle} s_i m_2\frac{q}{\beta^2} + \cdots - a_{\langle i,l\rangle} s_i m_2\frac{q}{\beta^l} + a_{\langle i,1\rangle} e_{i,1} + \cdots + a_{\langle i,l\rangle} e_{i,l}\right)$
   $= \sum_{i=0}^{k-1} \mathsf{LWE}_{\vec{s},\sigma}\left(-s_i m_2 \cdot \left(a_{\langle i,1\rangle}\frac{q}{\beta^1} + a_{\langle i,2\rangle}\frac{q}{\beta^2} + \cdots + a_{\langle i,l\rangle}\frac{q}{\beta^l}\right) + a_{\langle i,1\rangle} e_{i,1} + \cdots + a_{\langle i,l\rangle} e_{i,l}\right)$
   $= \sum_{i=0}^{k-1} \mathsf{LWE}_{\vec{s},\sigma}(-s_i m_2 a_i + e^{\langle i\rangle})$ ▷ where $e^{\langle i\rangle} = a_{\langle i,1\rangle} e_{i,1} + \cdots + a_{\langle i,l\rangle} e_{i,l}$
4. According to step 2 and 3,
   $\sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(\mathsf{ct}_i), \bar{\mathsf{ct}}_i\rangle$

$$
\begin{aligned}
&= \sum_{i=0}^{k-1} \mathsf{LWE}_{\vec{s},\sigma}(-s_i m_2 a_i + e^{\langle i \rangle}) + \mathsf{LWE}_{\vec{s},\sigma}(m_2 b + e^{\langle k \rangle}) \\
&= \mathsf{LWE}_{\vec{s},\sigma}\left(\sum_{i=0}^{k-1}(-s_i m_2 a_i) + m_2 b + \sum_{i=0}^{k} e^{\langle i \rangle}\right) \triangleright \text{ addition of two LWE ciphertexts} \\
&= \mathsf{LWE}_{\vec{s},\sigma}\left(m_2 b - \sum_{i=0}^{k-1} m_2 a_i s_i + \sum_{i=0}^{k} e^{\langle i \rangle}\right) \\
&= \mathsf{LWE}_{\vec{s},\sigma}\left(m_2 (b - \sum_{i=0}^{k-1} a_i s_i) + \sum_{i=0}^{k} e^{\langle i \rangle}\right) \\
&= \mathsf{LWE}_{\vec{s},\sigma}\left(m_2(\Delta m_1 + e_1) + \sum_{i=0}^{k} e^{\langle i \rangle}\right) \\
&= \mathsf{LWE}_{\vec{s},\sigma}\left(\Delta m_1 m_2 + m_2 e_1 + \sum_{i=0}^{k} e^{\langle i \rangle}\right) \\
&\approx \mathsf{LWE}_{\vec{s},\sigma}(\Delta m_1 m_2) \triangleright \text{ provided the complete error } m_2 e_1 + \sum_{i=0}^{k} e^{\langle i \rangle} \text{ lies within the output decoding interval}
\end{aligned}
$$

□

To reduce the noise growth, noise bootstrapping is needed (will be discussed in §D-1.8).

D-1.6.1 Generalization to GLWE-to-GGSW Multiplication

As in the scalar proof, exact descending gadget weights require $q = \beta^l$. The displayed calculation suppresses the GGSW row errors only to explain the message algebra. In full, if ρ_{A_i}, ρ_B are decomposition residuals and $E^{\text{key}}_{i,j}$ are evaluation-key errors, the output error is

$$
E_{\text{out}} = M_2 E - M_2\Big(\rho_B - \sum_{i=0}^{k-1} \rho_{A_i} S_i\Big) + \sum_{i=0}^{k}\sum_{j=1}^{l} D_{i,j} E^{\text{key}}_{i,j} \pmod q.
$$

All these terms require bounds. In particular, $\|M_2 E\|_\infty \le \|M_2\|_1 \|E\|_\infty$; small input noise alone does not imply small output noise.

We can further generalize TFHE's LWE-to-GSW multiplication to GLWE-to-GGSW multiplication between the following two ciphertexts: $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1) \cdot \mathsf{GGSW}^{\beta,l}_{\vec{S},\sigma}(M_2)$, where M_1, M_2, and S are $(n-1)$-degree polynomials.

The 1st term $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1)$ comes from one of the following:

- A fresh GLWE encryption (§B-4.2) of plaintext M_1.
- A homomorphically added result of two GLWE ciphertexts (§C-1).
- A homomorphically multiplied result of a GLWE ciphertext with a plaintext (§C-3).

The 2nd term $\mathsf{GGSW}^{\beta,l}_{\vec{S},\sigma}(M_2)$ comes from one of the following:

- A fresh GGSW encryption (§B-6.1) of plaintext M_2.
- Converted from $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_2)$ into $\mathsf{GGSW}^{\beta,l}_{\vec{S},\sigma}(M_2)$ by *circuit bootstrapping* (this will be covered in the future).

Remember the following:

$\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1) = (A_0, A_1, \cdots, A_{k-1}, B) \in \mathcal{R}_{n,q}^{k+1}$, where $B = \sum_{i=0}^{k-1}(A_i \cdot S_i) + \Delta M_1 + E$

▷ from §B-4.2

$\mathsf{GGSW}_{\vec{S},\sigma}^{\beta,l}(M_2) = \left\{\{\mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(-S_i \cdot M_2)\}_{i=0}^{k-1}, \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(M_2)\right\} \in \mathcal{R}_{\langle n,q\rangle}^{(k+1)\cdot l\cdot(k+1)}$ ▷ from §B-6.1

Let's use the following notations:

$\mathsf{GGSW}_{\vec{S},\sigma}^{\beta,l}(M_2) = \bar{C} = (\bar{C}_0, .\bar{C}_1, \cdots \bar{C}_k)$

$\bar{C}_i = \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(-S_i \cdot M_2)$ for $0 \leq i \leq (k-1)$

$\bar{C}_k = \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(M_2)$

$\mathsf{ct} = \mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1) = (C_0, C_1, \cdots, C_k) = (A_0, A_1, \cdots, A_{k-1}, B)$

Let's define the following TFHE ciphertext multiplication operation:

$\mathsf{ct} \cdot \bar{C} = \sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(C_i), \bar{C}_i \rangle$

Then, the following is true:

⟨Summary D-1.6.1⟩ Generalization to GLWE-to-GGSW Multiplication

$\mathsf{ct} = \mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1) = (A_0, A_1, \cdots, A_{k-1}, B)$

$\bar{C} = \mathsf{GGSW}_{\vec{S},\sigma}^{\beta,l}(M_2)$

$= (\mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(-S_0 \cdot M_2), \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(-S_1 \cdot M_2), \cdots, \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(-S_{k-1} \cdot M_2), \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(M_2))$

$\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1) \cdot \mathsf{GGSW}_{\vec{S},\sigma}^{\beta,l}(M_2) = \sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(C_i), \bar{C}_i \rangle \approx \mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1 M_2)$

This means that multiplying two TFHE ciphertexts (one is in GLWE and another in GGSW) and decrypting the resulting GLWE ciphertext gives the same result as multiplying their two original plaintexts.

Proof.

1. $\sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(C_i), \bar{C}_i \rangle$
   $= \langle \mathsf{Decomp}^{\beta,l}(A_0), \bar{C}_0 \rangle + \langle \mathsf{Decomp}^{\beta,l}(A_1), \bar{C}_1 \rangle + \cdots + \langle \mathsf{Decomp}^{\beta,l}(A_{k-1}), \bar{C}_{k-1} \rangle + \langle \mathsf{Decomp}^{\beta,l}(B), \bar{C}_k \rangle$
   ▷ expanding the dot product of two vectors
2. For $i = k$:
   $\mathsf{Decomp}^{\beta,l}(B) = (B_1, B_2, \cdots, B_l)$, where $B = B_1\frac{q}{\beta^1} + B_2\frac{q}{\beta^2} + \cdots + B_l\frac{q}{\beta^l}$ ▷ from §A-6.2
   $\bar{C}_k = \mathsf{GLev}_{\vec{S},\sigma}^{\beta,l}(M_2) = \left(\mathsf{GLWE}_{\vec{S},\sigma}\left(M_2\frac{q}{\beta^1}\right), \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2\frac{q}{\beta^2}\right), \cdots, \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2\frac{q}{\beta^l}\right)\right)$
   ▷ we omit the noise terms $E_{2,1}, \ldots, E_{2,l}$ in each GLWE ciphertext for simplicity

Therefore:

$\langle \mathsf{Decomp}^{\beta,l}(B), \bar{C}_k \rangle$

$= B_1 \cdot \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2 \frac{q}{\beta^1}\right) + B_2 \cdot \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2 \frac{q}{\beta^2}\right) + \cdots + B_l \cdot \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2 \frac{q}{\beta^l}\right)$

$= \mathsf{GLWE}_{\vec{S},\sigma}\left(B_1 M_2 \frac{q}{\beta^1}\right) + \mathsf{GLWE}_{\vec{S},\sigma}\left(B_2 M_2 \frac{q}{\beta^2}\right) + \cdots + \mathsf{GLWE}_{\vec{S},\sigma}\left(B_l M_2 \frac{q}{\beta^l}\right)$ ▷ from §C-3

$= \mathsf{GLWE}_{\vec{S},\sigma}\left(B_1 M_2 \frac{q}{\beta^1} + B_2 M_2 \frac{q}{\beta^2} + \cdots + B_l M_2 \frac{q}{\beta^l}\right)$ ▷ from §C-1

$= \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2 \cdot \left(B_1 \frac{q}{\beta^1} + B_2 \frac{q}{\beta^2} + \cdots + B_l \frac{q}{\beta^l}\right)\right)$

$= \mathsf{GLWE}_{\vec{S},\sigma}(M_2 B)$ ▷ from §A-6.2

3. For $0 \leq i \leq (k-1)$:

$\mathsf{Decomp}^{\beta,l}(A_i) = (A_{\langle i,1 \rangle}, A_{\langle i,2 \rangle}, \cdots, A_{\langle i,l \rangle})$, where $A_i = A_{\langle i,1 \rangle} \frac{q}{\beta^1} + A_{\langle i,2 \rangle} \frac{q}{\beta^2} + \cdots + A_{\langle i,l \rangle} \frac{q}{\beta^l}$

$\bar{C}_i = \mathsf{GLev}^{\beta,l}_{\vec{S},\sigma}(-S_i M_2) = \left(\mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \frac{q}{\beta^1}\right), \mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \frac{q}{\beta^2}\right), \cdots, \mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \frac{q}{\beta^l}\right)\right)$

Therefore:

$\langle \mathsf{Decomp}^{\beta,l}(A_0), \bar{C}_0 \rangle + \langle \mathsf{Decomp}^{\beta,l}(A_1), \bar{C}_1 \rangle + \cdots + \langle \mathsf{Decomp}^{\beta,l}(A_{k-1}), \bar{C}_{k-1} \rangle$

$= \sum_{i=0}^{k-1} \langle \mathsf{Decomp}^{\beta,l}(A_i), \bar{C}_i \rangle$

$= \sum_{i=0}^{k-1} \left(A_{\langle i,1 \rangle} \cdot \mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \frac{q}{\beta^1}\right) + A_{\langle i,2 \rangle} \cdot \mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \frac{q}{\beta^2}\right) + \cdots + A_{\langle i,l \rangle} \cdot \mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \frac{q}{\beta^l}\right)\right)$

$= \sum_{i=0}^{k-1} \left(\mathsf{GLWE}_{\vec{S},\sigma}\left(-A_{\langle i,1 \rangle} S_i M_2 \frac{q}{\beta^1}\right) + \mathsf{GLWE}_{\vec{S},\sigma}\left(-A_{\langle i,2 \rangle} S_i M_2 \frac{q}{\beta^2}\right) + \cdots + \mathsf{GLWE}_{\vec{S},\sigma}\left(-A_{\langle i,l \rangle} S_i M_2 \frac{q}{\beta^l}\right)\right)$

$= \sum_{i=0}^{k-1} \mathsf{GLWE}_{\vec{S},\sigma}\left(-A_{\langle i,1 \rangle} S_i M_2 \frac{q}{\beta^1} + -A_{\langle i,2 \rangle} S_i M_2 \frac{q}{\beta^2} + \cdots + -A_{\langle i,l \rangle} S_i M_2 \frac{q}{\beta^l}\right)$

$= \sum_{i=0}^{k-1} \mathsf{GLWE}_{\vec{S},\sigma}\left(-S_i M_2 \cdot \left(A_{\langle i,1 \rangle} \frac{q}{\beta^1} + A_{\langle i,2 \rangle} \frac{q}{\beta^2} + \cdots + A_{\langle i,l \rangle} \frac{q}{\beta^l}\right)\right)$

$= \sum_{i=0}^{k-1} \mathsf{GLWE}_{\vec{S},\sigma}(-S_i M_2 A_i)$

4. According to step 2 and 3,

$\sum_{i=0}^{k} \langle \mathsf{Decomp}^{\beta,l}(C_i), \bar{C}_i \rangle$

$= \sum_{i=0}^{k-1} \mathsf{GLWE}_{\vec{S},\sigma}(-S_i M_2 A_i) + \mathsf{GLWE}_{\vec{S},\sigma}(M_2 B)$

$= \mathsf{GLWE}_{\vec{S},\sigma}\left(\sum_{i=0}^{k-1}(-S_i M_2 A_i) + M_2 B\right)$ ▷ addition of two GLWE ciphertexts

$= \mathsf{GLWE}_{\vec{S},\sigma}\left(B M_2 - \sum_{i=0}^{k-1} M_2 A_i S_i\right)$

$= \mathsf{GLWE}_{\vec{S},\sigma}\left(M_2 (B - \sum_{i=0}^{k-1} A_i S_i)\right)$

$= \mathsf{GLWE}_{\vec{S},\sigma}(M_2(\Delta M_1 + E))$

$= \mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1 M_2 + M_2 E)$

$\approx \mathsf{GLWE}_{\vec{S},\sigma}(\Delta M_1 M_2)$ ▷ provided the full output error satisfies the decoding bound

□

D-1.7 Coefficient Extraction

- Reference: TFHE Deep Dive - Part IV - Programmable Bootstrapping [11]

In TFHE, coefficient extraction is a process of extracting a coefficient of a polynomial that is encrypted as GLWE ciphertext. The extracted coefficient is in the form of LWE ciphertext (§B-2).

Note that in the GLWE cryptosystem, plaintext M is encoded as a polynomial, where each coefficient encodes the plaintext value $m_0, m_1, \cdots, m_{n-1}$.

Suppose we have a GLWE ciphertext setup as the following:

$$M = \sum_{j=0}^{n-1} m_j X^j \in \mathcal{R}_{\langle n,t \rangle}$$

$$S = \left(S_0 = \sum_{j=0}^{n-1} s_{0,j} X^j, S_1 = \sum_{j=0}^{n-1} s_{1,j} X^j, \ \cdots, S_{k-1} = \sum_{j=0}^{n-1} s_{k-1,j} X^j \right)$$

$$\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M) = \left(A_0 = \sum_{j=0}^{n-1} a_{0,j} X^j, A_1 = \sum_{j=0}^{n-1} a_{1,j} X^j, \ \cdots, A_{k-1} = \sum_{j=0}^{n-1} a_{k-1,j} X^j, B = \sum_{j=0}^{n-1} b_j X^j \right)$$

$$B = \sum_{i=0}^{k-1} A_i S_i + \Delta M + E$$

$$E = \sum_{i=0}^{n-1} e_i X^i$$

Note that:

$$\begin{aligned}
&\Delta M + E = B - \sum_{i=0}^{k-1} A_i S_i \\
&= (\Delta m_0 + \Delta m_1 X + \ \cdots \ + \Delta m_{n-1} X^{n-1}) + (e_0 + e_1 X + \ \cdots \ + e_{n-1} X^{n-1}) \\
&= (\Delta m_0 + e_0) + (\Delta m_1 + e_1) X + \ \cdots \ + (\Delta m_{n-1} + e_{n-1}) X^{n-1}
\end{aligned}$$

Another way to write the formula is:

$$\begin{aligned}
&B - \sum_{i=0}^{k-1} A_i S_i \\
&= (b_0 + b_1 X + \ \cdots \ + b_{n-1} X^{n-1}) \\
&-(a_{0,0} + a_{0,1} X + \ \cdots \ + a_{0,n-1} X^{n-1})(s_{0,0} + s_{0,1} X + \ \cdots \ + s_{0,n-1} X^{n-1}) \\
&-(a_{1,0} + a_{1,1} X + \ \cdots \ + a_{1,n-1} X^{n-1})(s_{1,0} + s_{1,1} X + \ \cdots \ + s_{1,n-1} X^{n-1}) \\
&- \cdots \\
&-(a_{k-1,0} + a_{k-1,1} X + \ \cdots \ + a_{k-1,n-1} X^{n-1})(s_{k-1,0} + s_{k-1,1} X + \ \cdots \ + s_{k-1,n-1} X^{n-1}) \\
&= \left(b_0 - \left(\sum_{i=0}^{k-1} \sum_{j=0}^{0} (a_{i,0-j} s_{i,j}) - \sum_{i=0}^{k-1} \sum_{j=1}^{n-1} (a_{i,n-j} s_{i,j}) \right) \right) \\
&+ \left(b_1 - \left(\sum_{i=0}^{k-1} \sum_{j=0}^{1} (a_{i,1-j} s_{i,j}) - \sum_{i=0}^{k-1} \sum_{j=2}^{n-1} (a_{i,n+1-j} s_{i,j}) \right) \right) \cdot X \\
&+ \left(b_2 - \left(\sum_{i=0}^{k-1} \sum_{j=0}^{2} (a_{i,2-j} s_{i,j}) - \sum_{i=0}^{k-1} \sum_{j=3}^{n-1} (a_{i,n+2-j} s_{i,j}) \right) \right) \cdot X^2 \\
&\ \cdots \\
&+ \left(b_{n-1} - \left(\sum_{i=0}^{k-1} \sum_{j=0}^{n-1} (a_{i,n-1-j} s_{i,j}) - \sum_{i=0}^{k-1} \sum_{j=n}^{n-1} (a_{i,n+(n-1)-j} s_{i,j}) \right) \right) \cdot X^{n-1}
\end{aligned}$$

▷ Grouping the terms by same exponents

$$= \sum_{h=0}^{n-1} \left(b_h - \left(\sum_{i=0}^{k-1} \sum_{j=0}^{h} (a_{i,h-j} s_{i,j}) - \sum_{i=0}^{k-1} \sum_{j=h+1}^{n-1} (a_{i,n+h-j} s_{i,j}) \right) \right) \cdot X^h$$

$$= \sum_{h=0}^{n-1} C_h \cdot X^h, \text{ where } C_h = b_h - \left(\sum_{i=0}^{k-1} \sum_{j=0}^{h} (a_{i,h-j} s_{i,j}) - \sum_{i=0}^{k-1} \sum_{j=h+1}^{n-1} (a_{i,n+h-j} s_{i,j}) \right)$$

In the above $(n-1)$-degree polynomial, notice that each X^h term's coefficient, C_h, can be expressed as an LWE ciphertext ct_h as follows:

$S' = (s_{0,0}, s_{0,1}, \cdots, s_{0,n-1}, s_{1,0}, s_{1,1}, \cdots, s_{1,n-1}, \cdots, s_{k-1,n-1}) = (s'_0, s'_1, \cdots, s'_{nk-1}) \in \mathbb{Z}_q^{nk}$

$\mathsf{ct}_h = (a'_0, a'_1, \cdots, a'_{nk-1}, b_h) \in \mathbb{Z}_q^{nk+1}$

, where $a'_{n \cdot i+j} = \begin{cases} a_{i,h-j} \text{ (if } 0 \le j \le h) \\ -a_{i,n+h-j} \text{ (if } h+1 \le j \le n-1) \end{cases}$, b_h is directly obtained from the polynomial B

Note that $b_h - \sum_{i=0}^{nk-1} s'_i a'_i = \Delta m_h + e_h$. This means that C_h can be replaced by its encrypted version, $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h)$, an LWE ciphertext ct_h encrypting the h-th coefficient of M. Therefore, we just extracted $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h)$ from $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M)$. This operation is called coefficient extraction, which does not add any noise because it simply extracts an LWE ciphertext by reordering the polynomial of the GLWE ciphertext.

Once we have $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h)$, we can key-switch it from $\vec{s}' \to \vec{s}$ (§D-1.5).

⟨Summary D-1.7⟩ GLWE Ciphertext's Coefficient Extraction

Given the following GLWE ciphertext:

$M = \sum_{j=0}^{n-1} m_j X^j \in \mathcal{R}_{\langle n,t \rangle}$

$\vec{S} = \left(S_0 = \sum_{j=0}^{n-1} s_{0,j} X^j, S_1 = \sum_{j=0}^{n-1} s_{1,j} X^j, \cdots, S_{k-1} = \sum_{j=0}^{n-1} s_{k-1,j} X^j \right)$

$\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M) = \left(A_0 = \sum_{j=0}^{n-1} a_{0,j} X^j, A_1 = \sum_{j=0}^{n-1} a_{1,j} X^j, \cdots, A_{k-1} = \sum_{j=0}^{n-1} a_{k-1,j} X^j, B = \sum_{j=0}^{n-1} b_j X^j \right)$

$B = \sum_{i=0}^{k-1} A_i S_i + \Delta M + E \bmod q, \quad E = \sum_{i=0}^{n-1} e_i X^i$

$\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h)$ is an LWE ciphertext that encrypts ΔM's h-th coefficient (i.e., Δm_h). $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h)$ can be extracted from $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta M)$ as follows:

$\vec{s}' = (s_{0,0}, s_{0,1}, \cdots, s_{0,n-1}, s_{1,0}, s_{1,1}, \cdots, s_{1,n-1}, \cdots, s_{k-1,n-1}) = (s'_0, s'_1, \cdots, s'_{nk-1}) \in \mathbb{Z}_q^{nk}$

$\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h) = (a'_0, a'_1, \cdots, a'_{nk-1}, b_h) \in \mathbb{Z}_q^{nk+1}$

, where $a'_{n \cdot i+j} = \begin{cases} a_{i,h-j} \text{ (if } 0 \le j \le h) \\ -a_{i,n+h-j} \text{ (if } h+1 \le j \le n-1) \end{cases}$, b_h is obtained from the polynomial B

Once we have $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m_h)$, key-switch it from $\vec{s}' \to \vec{s}$ (§D-1.5).

D-1.8 Noise Bootstrapping

- Reference: TFHE Deep Dive - Part IV - Programmable Bootstrapping [11]

Homomorphic addition generally increases the noise budget even when each fresh error has mean zero. For independent errors of variance σ^2, a sum of r errors has variance $r\sigma^2$; adding the same ciphertext r times gives error re and variance $r^2\sigma^2$. Multiplication and external products also introduce message-dependent, evaluation-key and decomposition errors. Bootstrapping refreshes a ciphertext before its accumulated error crosses a decoding boundary (illustrated in Figure 8); it replaces the old error by a new bounded evaluation error, rather than by zero.

As explained in the beginning of this section, TFHE uses LWE (which is GLWE with polynomial degree 0) to encrypt & decrypt a plaintext. That is, each plaintext is m (a single number), encoded as a zero-degree polynomial. Further, the secret key S that encrypts each m is a vector $\{s_0, s_1, \cdots, s_{k-1}\}$ instead of a polynomial. On the other hand, TFHE's noise bootstrapping uses homomorphic addition between GLWE ciphertexts and homomorphic multiplication between GLWE and GGSW ciphertexts.

Suppose we have a TFHE ciphertext as follows:

$$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m) = (a_0, a_1, \cdots a_{k-1}, b)$$
$$b = \sum_{i=0}^{k-1} a_i s_i + \Delta m + e_b$$
$$\vec{s} = (s_0, s_1, \cdots s_{k-1})$$

, where e_b is a big noise accumulated over a series of many ciphertext (or plaintext) multiplications. The goal of noise bootstrapping is to convert $(a_0, a_1, \cdots a_{k-1}, b)$ into $(a'_0, a'_1, \cdots a'_{k-1}, b')$ such that:

$$b' = \sum_{i=0}^{k-1} a'_i s_i + \Delta m + e_s$$

, where e_s is a re-initialized noise.

D-1.8.1 Overview

Bootstrapping evaluates a rounding lookup table at the encrypted phase. Conceptually, let $j = [\Delta m + e]_q$ and define a length-q cyclic table by

$$F_q(j) = \left[\left\lfloor j/\Delta \right\rceil\right]_t, \qquad V_q(X) = \sum_{j=0}^{q-1} F_q(j) X^j.$$

Here the centered error satisfies $|e| < \Delta/2$ and ties require a fixed convention. Thus each message occupies an interval centered at Δm, rather than the one-sided interval $[\Delta m, \Delta(m+1))$. This conceptual cyclic table explains lookup; it is not a GLWE ciphertext in the smaller ring $\mathcal{R}_{\langle n,q \rangle}$.

The actual construction uses a degree-at-most-$n-1$ polynomial $V(X)$, modulus switching to $2n$, and the negacyclic lookup restriction explained next. We use V for the unscaled table and initialize a GLWE accumulator encrypting ΔV. For a public table the trivial ciphertext $(0, \ldots, 0, \Delta V)$ suffices. The output GLWE secret $\vec{S}_{bk}$ consists of k' binary polynomials; it is secret and is not itself the public bootstrapping

key. That key is the collection

$$\mathsf{BK} = \{\mathsf{GGSW}^{\beta,l}_{\vec{S}_{bk},\sigma}(s_i)\}_{i=0}^{k-1}.$$

Blind rotation uses these encryptions, not the secret $\vec{S}_{bk}$, to rotate the encrypted accumulator. Coefficient extraction (§D-1.7) then gives an LWE encryption of Δm under the flattened GLWE key, followed by key switching back to $\vec{s}$.

1. **Input:** an LWE encryption of Δm with error e.
2. Modulus-switch its public components to $2n$ using §C-4.1.
3. Blindly rotate the encryption of ΔV by the resulting encrypted phase, using the coefficient-rotation operation of Summary A-5.2.1 in §A-5.2.
4. Extract the constant coefficient and switch its secret key back to $\vec{s}$.
5. **Output:** an encryption of the same m, provided the input phase remains inside its lookup interval and the output evaluation error is below the decoding threshold.

The input error affects which table entry is selected. A correct lookup makes the selected message independent of that error, while the homomorphic evaluation produces a new error. This distinction is essential: bootstrapping cannot recover a message after its input phase has crossed into another lookup interval.

D-1.8.2 Modulus Switch for Noise Bootstrapping

To avoid using the giant $(q-1)$-degree polynomial V_q, we will compress q possible ciphertext elements $\Delta m + e \in \mathbb{Z}_q$ into n distinct exponents of the $(n-1)$-degree polynomial V, where each $v_j X^j$ term in V represents a mapping from $j \to v_j$ (i.e., noisy plaintext to noise-free plaintext). However, notice that when we rotate the coefficients of the $(n-1)$-degree polynomial V to the left, as $v_j X^j$ rotates across the boundary between X^0 and X^{n-1} degree terms, v_j's sign flips to $-v_j$ (as shown in the example of §A-5.2.1). Due to this coefficient sign flip, the $(n-1)$-degree polynomial V can theoretically encode total $2n$ distinct coefficient states as follows: $(v_0, v_1, v_2, \cdots, v_{n-1}, -v_0, -v_1, \cdots, -v_{n-1})$. To move each of these $2n$ distinct coefficients to the constant term's coefficient position in V (i.e., shifting the coefficient v_j to the leftmost term in V), the rotating computation of $V \cdot X^{-j}$ can use $2n$ distinct j values, which are $\{0, 1, 2, \cdots, n-1, n, \cdots, 2n-1\}$, to move each of $(v_0, v_1, v_2, \cdots, v_{n-1}, -v_0, -v_1, \cdots, -v_{n-1})$ coefficients to the constant term's position. This implies that the exponent j in $V \cdot X^{-j}$ can use any of the $2n$ distinct values to cover all possible $2n$ (sign-flipped) coefficient states of V. Also, remember that $j = \Delta m + e$. Therefore, we will switch the modulo of $\Delta m + e$ from $q \to 2n$. Using the LWE modulus switching technique (§C-4.1), our original LWE ciphertext $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e) = (a_0, a_1, \cdots a_{k-1}, b) \in \mathbb{Z}_q^{k+1}$ (i.e., the initial input to the noise bootstrapping procedure) will be converted into the following:

$\mathsf{LWE}_{\vec{s},\sigma}(\hat{\Delta} m) = (\hat{a}_0, \hat{a}_1, \cdots \hat{a}_{k-1}, \hat{b}) \in \mathbb{Z}_{2n}^{k+1}$
$\vec{s} = (s_0, s_1, \cdots s_{k-1}) \in \mathbb{Z}_2^k$ ▷ the secret key stays the same, as each s_i is binary
$\hat{\Delta} = \Delta \dfrac{2n}{q} = \dfrac{2n}{t} \in \mathbb{Z}_{2n}$
$\hat{a}_i = \left\lceil a_i \dfrac{2n}{q} \right\rfloor \in \mathbb{Z}_{2n}$
$\hat{e} = \dfrac{2n}{q} e + \delta_b - \sum_{i=0}^{k-1} s_i \delta_{a_i}$, where $\delta_b = \hat{b} - \dfrac{2n}{q} b$ and $\delta_{a_i} = \hat{a}_i - \dfrac{2n}{q} a_i$ are component rounding errors.

In particular,

$$|\hat{e}| \leq \frac{2n}{q}|e| + \frac{1 + \|\vec{s}\|_1}{2}.$$

The simple integer-grid description assumes $t \mid 2n$ in addition to $t \mid q$.

$$\hat{b} = \left\lceil b\frac{2n}{q} \right\rfloor \equiv \sum_{i=0}^{k-1} \hat{a}_i s_i + \hat{\Delta} m + \hat{e} \in \mathbb{Z}_{2n}$$

The degree of $V(X)$ and Security: The ring dimension is n, the polynomial degree is at most $n-1$, and the rotation period is $2n$. A lookup table needs room for noise-tolerant intervals, not just one coefficient per message. Ring dimension, modulus, secret distribution and error distribution must be selected jointly for security and correctness; increasing the degree alone is not a complete security argument.

D-1.8.3 Halving the Plaintext Space To be Used

In $\mathbb{Z}_q[X]/(X^n+1)$, the identity $X^n = -1$ forces a lookup table to be antiperiodic. Write $V(X) = \sum_{j=0}^{n-1} v_j X^j$, and let $r = j \bmod 2n$. Then

$$[X^0](V(X)X^{-j}) = F(j) = \begin{cases} v_r, & 0 \le r < n, \\ -v_{r-n}, & n \le r < 2n. \end{cases}$$

Here $[X^0]$ denotes the constant coefficient. Consequently $F(j+n) = -F(j)$; the second half of the table cannot be chosen independently of the first half. This is the actual restriction, rather than a prohibition on intermediate rotations or on negative rotation indices.

Solution: For an arbitrary lookup function, reserve one padding bit and restrict the admissible message set to a specified contiguous half of $\mathbb{Z}_t$ (with t even). Design a constant lookup interval centered at each admissible $\hat{\Delta} m$, and leave a strict guard against rounding boundaries. A length-n interval of phase indices can be assigned freely, and the other half is fixed by antiperiodicity. Signed messages may use negative phase indices; their coefficients must then include the sign dictated by $X^n = -1$.

⟨Summary D-1.8.3⟩ Structure of Lookup Table Polynomial $V(X)$

Let $\hat{\Delta} = 2n/t$, let $\mathcal{D}$ be the admissible half-domain, and choose an antiperiodic function $F : \mathbb{Z}_{2n} \to \mathbb{Z}_q$ such that

$$F(\hat{\Delta} m + u) = m \quad (m \in \mathcal{D},\ |u| < \hat{\Delta}/2), \qquad F(j+n) = -F(j).$$

Boundary values use a fixed tie convention. Set

$$V(X) = \sum_{j=0}^{n-1} F(j) X^j.$$

The accumulator encrypts $\Delta V(X)$ at modulus q, so a successful lookup produces a constant term Δm, not an unscaled m. Correctness requires the total switched error (including component rounding) to stay in its assigned interval. For programmable bootstrapping replace m on the right by the desired output $f(m)$ and choose the output scale explicitly.

The encrypted lookup polynomial is also called an accumulator. A function already satisfying the required antiperiodicity may use a larger input domain; the restriction is on the function and its encoding.

D-1.8.4 Blind Rotation

Blind rotation refers to rotating the coefficients of an *encrypted* polynomial so that it is not possible to know how many positions the polynomial's coefficients have been rotated. After the rotation, it is not possible to see which coefficient has moved to which degree term. Blind rotation uses the basic polynomial rotation method 1 technique (Summary A-5.2 in §A-5.2), with the difference that the $V(X) \cdot X^{-i}$ computation is done homomorphically.

The phase convention in this chapter gives $X^{\hat{\Delta}m+\hat{e}_b} = X^{\hat{b}-\sum_{i=0}^{k-1}\hat{a}_i s_i}$. Therefore blind rotation starts with $X^{-\hat{b}}$ and applies conditional factors $X^{\hat{a}_i s_i}$. The evaluator is given $\mathsf{BK} = \{\mathsf{GGSW}^{\beta,l}_{\vec{S}_{bk},\sigma}(s_i)\}_{i=0}^{k-1}$, encrypting the input LWE secret bits under the output GLWE secret $\vec{S}_{bk}$. The evaluator does not receive that secret. Below V_i denotes the unscaled lookup polynomial; its accumulator ciphertext encrypts ΔV_i.

1. Initialize the GLWE accumulator encrypting ΔV under the output secret $\vec{S}_{bk}$, or use a trivial encryption for a public table.
2. Compute $V_0 = V \cdot X^{-\hat{b}}$, which is basically rotating V's polynomials by $\hat{b}$ positions to the left. Since $\hat{b}$ is a known value visible in the LWE ciphertext, we can directly compute the rotation of $V_0 = V \cdot X^{-\hat{b}}$.
3. Compute $V_1 = V_0 \cdot X^{\hat{a}_0 s_0} = s_0 \cdot (V_0 \cdot X^{\hat{a}_0} - V_0) + V_0$. This formula works for the special case where $s_0 \in \{0, 1\}$: if $s_0 = 0$, then $V_1 = V_0$; else if $s_0 = 1$, then $V_1 = V_0 \cdot X^{\hat{a}_0}$. Computing $s_0 \cdot (V_0 \cdot X^{\hat{a}_0} - V_0) + V_0$ is done as a TFHE homomorphic addition and multiplication. We call this blind rotation of V_0, where the selection bit s_0 (i.e., the 1st element of the secret key vector S) is encrypted as a GGSW ciphertext by using $\vec{S}_{bk}$ (the output GLWE secret key) and V_0 is an encrypted polynomial as a GLWE ciphertext. Multiplying GLWE-encrypted V_0 with $x^{\hat{a}_0}$ is done by GLWE ciphertext-to-plaintext multiplication (§C-3), and subtracting the result by GLWE-encrypted V_0 is done by GLWE ciphertext-to-ciphertext addition/subtraction (§C-1), and multiplying the result by GGSW-encrypted s_0 is done by GLWE-to-GGSW multiplication (§D-1.6.1), and adding the result with GLWE-encrypted V_0 is done by GLWE ciphertext-to-ciphertext addition. If $s_0 = 1$, then the formula's $X^{\hat{a}_0}$ term gets multiplied to V_0, which effectively rotates V_0's coefficients by $\hat{a}_0$ positions to the right. Else if $s_0 = 0$, then V_0 does not get rotated and stays the same. In both cases, the resulting V_1 is encrypted as a new GLWE ciphertext. The evaluator knows the public candidate rotation $\hat{a}_0$, but not the encrypted selection bit s_0. Hiding the selected branch relies on the security assumptions of the input and evaluation-key encryptions; possession of the appropriate secret key permits its recovery.
4. By using the same blind rotation method as in the previous step, compute the GLWE encryption of $V_2 = V_1 \cdot X^{\hat{a}_1 s_1} = s_1 \cdot (V_1 \cdot X^{\hat{a}_1} - V_1) + V_1$. Note that we have the following publicly known components: $\hat{a}_1$, a GLWE encryption of V_1, and a GGSW ciphertext of s_1 encrypted by using $\vec{S}_{bk}$.
5. Continue to compute the GLWE encryption of $V_3, V_4, \cdots, V_k$ in the same manner, and we finally get a GLWE encryption of V_k, whose computed value is equivalent to:
$$\begin{aligned} V' &= V_k \\ &= V_{k-1} \cdot X^{\hat{a}_{k-1} s_{k-1}} \\ &= V_0 \cdot X^{\hat{a}_0 s_0} X^{\hat{a}_1 s_1} \cdots X^{\hat{a}_{k-1} s_{k-1}} \\ &= V \cdot X^{-\hat{b}+\sum_{i=0}^{k-1} \hat{a}_i s_i} \\ &= V \cdot X^{-(\hat{\Delta}m+\hat{e}_b)} \end{aligned}$$

This means that the GLWE encryption of the final polynomial V_k will have the coefficient m in its constant term, as $V(X)$ is designed to have the mapping $(\hat{\Delta}m + \hat{e}_*, m)$.

Intermediate rotations may cross any boundary. Only the final phase $\hat{\Delta}m + \hat{e}$ must belong to the admissible centered interval described in §D-1.8.3. This is a subset of $\mathbb{Z}_{2n}$, not $\mathbb{Z}_t$; it need not be $[0, n)$ when signed encodings are used.

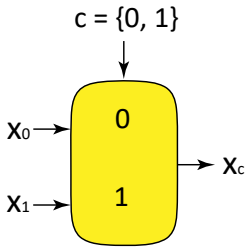


Figure 13: An illustration of the MUX logic gate

Homomorphic MUX Logic Gate: In step 3, the formula $s_0 \cdot (V_0 \cdot X^{\hat{a}_0} - V_0) + V_0$ implements the MUX logic gate as shown in Figure 13, where in our case s_0 is the selection bit that chooses between the two inputs: V_0 and $V_0 \cdot X^{\hat{a}_0}$. If s_0 is 1, then the output is $V_0 \cdot X^{\hat{a}_0}$; otherwise, the output is V_0. In our design, the homomorphic computation of $s_0 \cdot (V_0 \cdot X^{\hat{a}_0} - V_0) + V_0$ effectively implements a homomorphic MUX gate, where the two data inputs are GLWE-encrypted and the selection bit is GGSW-encrypted.

Dimensions of the GLWE and GGSW Ciphertexts: A GLWE ciphertext contains $k'+1$ polynomials of degree at most $n-1$, hence $(k'+1)n$ coefficients. Each GGSW bootstrapping-key entry encrypts one input bit s_i and contains $l(k'+1)^2$ polynomials, hence $l(k'+1)^2 n$ coefficients. Flattening the output GLWE secret gives an LWE secret of dimension $k'n$. Choosing $k' = 1$ yields RLWE/RGSW; this common efficiency choice still requires an appropriate security parameter set.

D-1.8.5 Coefficient Extraction

Next, use coefficient extraction (§D-1.7) on the rotated accumulator to obtain $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m)$ under a vector $\vec{s}'$ of length $k'n$. Extraction itself adds no noise. If the input phase selects the correct lookup interval, the decoded message no longer depends on the old error $\hat{e}_b$; the output instead contains the accumulator and blind-rotation evaluation error e_s, which must satisfy its own decoding bound.

In fact, the homomorphic MUX logic in the blind rotation procedure (§D-1.8.4) involves numerous ciphertext multiplications and additions, which can accumulate additional noise until we reach the point of coefficient extraction. To limit the accumulating noise during the series of MUX logic operations, we can carefully adjust the security parameters. For example, we can design a narrower Gaussian distribution for sampling the noise e, while designing a sufficiently large n to compensate for the reduced noise. Meanwhile, increasing q can make the system less secure because it becomes more vulnerable to lattice reduction attacks.

D-1.8.6 Key Switching

The output of the coefficient extraction is $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m)$, which is encrypted by a secret key $\vec{s}'$ whose length is $k' \cdot n$. For consistency, we need to switch its key from $\vec{s}' \rightarrow \vec{s}$, where $\vec{s}$ is our original k-length key. This key switching can be done by using the technique explained in Summary D-1.5 (in §D-1.5).

D-1.8.7 Noise Bootstrapping Summary

TFHE's noise bootstrapping procedure is summarized as follows:

⟨Summary D-1.8.7⟩ TFHE Noise Bootstrapping Procedure

Lookup Table Encryption: Initialize an encryption of $\Delta V(X)$ under the output GLWE secret $\vec{S}_{bk}$ of length k', or use the trivial encryption for a public table. Blind rotation uses the public GGSW collection BK defined above.

1. **Modulus Switch:** Change the modulus of the TFHE ciphertext $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m + e_b)$ from $q \to 2n$ to get $\mathsf{LWE}_{\vec{s},\sigma}(\hat{\Delta} m + \hat{e}_b)$, where $\hat{\Delta} = \Delta \cdot \dfrac{2n}{q} = \dfrac{2n}{t}$.

2. **Blind Rotation:** Rotate the GLWE-encrypted polynomial V by $(\hat{b} - \sum_{i=0}^{k-1} \hat{a}_i s_i) = (\hat{\Delta} m + \hat{e}_b)$ positions to the left, by recursively computing:
   $V_0 = V \cdot X^{-\hat{b}}$
   $V_1 = V_0 \cdot X^{\hat{a}_0 s_0} = s_0 \cdot (V_0 \cdot X^{\hat{a}_0} - V_0) + V_0$
   $\vdots$
   $V_k = V_{k-1} \cdot X^{\hat{a}_{k-1} s_{k-1}} = s_{k-1} \cdot (V_{k-1} \cdot X^{\hat{a}_{k-1}} - V_{k-1}) + V_{k-1}$
   $= V_0 \cdot X^{\hat{a}_0 s_0} X^{\hat{a}_1 s_1} \cdots X^{\hat{a}_{k-1} s_{k-1}}$
   $= V \cdot X^{-\hat{b} + \sum_{i=0}^{k-1} \hat{a}_i s_i}$
   $= V \cdot X^{-(\hat{\Delta} m + \hat{e}_b)}$

   Each step of the actual blind rotation above is computed as the following TFHE ciphertext-to-ciphertext multiplication and addition:
   $\mathsf{GLWE}_{\vec{S}_{bk},\sigma}(\Delta V_{i+1}) = \mathsf{GGSW}^{\beta,l}_{\vec{S}_{bk},\sigma}(s_i) \cdot (\mathsf{GLWE}_{\vec{S}_{bk},\sigma}(\Delta V_i) \cdot X^{\hat{a}_i} - \mathsf{GLWE}_{\vec{S}_{bk},\sigma}(\Delta V_i)) + \mathsf{GLWE}_{\vec{S}_{bk},\sigma}(\Delta V_i)$

3. **Coefficient Extraction:** Homomorphically extract the constant term's coefficient m from the rotated polynomial V_k, which is $\mathsf{LWE}_{\vec{s}',\sigma}(\Delta m)$, where $\vec{s}'$ is a vector of length $k' \cdot n$.

4. **Key Switching:** Homomorphically switch the key of the LWE ciphertext from $\vec{s}' \to \vec{s}$.

Halving the Usable Plaintext Range: Reserve a specified half-domain for an arbitrary LUT and center each lookup interval at its encoded message. The table must satisfy $F(j+n) = -F(j)$, and the full switched error must stay inside the assigned interval. Signed phase indices and intermediate rotations beyond $n-1$ are permitted.

Choice of k': For computational efficiency, k' is set to be 1, which simplifies the GLWE and GGSW ciphertexts as RLWE and RGSW ciphertexts.

D-1.8.8 Example: Noise Bootstrapping

Suppose the GLWE security setup: $n = 16$, $t = 8$, $q = 64$, $k = 8$
$\mathbb{Z}_{t=8} = \{-4, -3, -2, -1, 0, 1, 2, 3\}$
$\mathbb{Z}_{q=64} = \{-32, -31, -30, \cdots, 29, 30, 31\}$
$\Delta = \dfrac{q}{t} = \dfrac{64}{8} = 8$

And suppose we have the following LWE ciphertext:
$\vec{s} = (1,0,0,1,1,1,0,1) \in \mathbb{Z}_2^{k=8}$
$m = 1 \in \mathbb{Z}_{t=8}$
$\Delta m = 1 \cdot 8 = 8 \in \mathbb{Z}_{q=64}$
$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m) = (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7, b) = (8, -28, 4, -32, 0, 31, -6, 7, 24) \in \mathbb{Z}_{q=64}^{k+1=9}$
$e = 2 \in \mathbb{Z}_{q=64}$ (should be the case that $|e| < \dfrac{\Delta}{2} = 4$ for correct decryption)
$b = \sum_{i=0}^{7} a_i s_i + \Delta m + e = (8 - 32 + 31 + 7) + 8 + 2 = 24 \in \mathbb{Z}_{q=64}$

Now then, the TFHE noise bootstrapping procedure is as follows:

1. **Modulus Switch:** Switch the modulus of $\mathsf{LWE}_{\vec{s},\sigma}(\Delta m)$ From $q \to 2n$, which is from $64 \to 32$. After the modulus switch, the original LWE ciphertext is converted as follows:
$\mathbb{Z}_{2n=32} = \{-16, -15, -14, \cdots, 13, 14, 15\}$
$\vec{s} = (1,0,0,1,1,1,0,1) \in \mathbb{Z}_2^{k=8}$
$\hat{\Delta} = \Delta \dfrac{2n}{64} = 8\dfrac{32}{64} = 4$
$\hat{\Delta} m = 4 \cdot 1 = 4 \in \mathbb{Z}_{2n=32}$
$\hat{e} = \hat{b} - \sum_{i=0}^{7} \hat{a}_i s_i - \hat{\Delta} m = 12 - (4 - 16 + 16 + 4) - 4 = 0 \pmod{32}$. The component $\hat{a}_5 = 16$ is a noncentered representative of -16; either represents the same residue.

$\mathsf{LWE}_{\vec{s},\sigma}(\hat{\Delta} m) = (\hat{a}_0, \hat{a}_1, \hat{a}_2, \hat{a}_3, \hat{a}_4, \hat{a}_5, \hat{a}_6, \hat{a}_7, \hat{b}) \in \mathbb{Z}_{2n=32}^{k+1=9}$
$= (\left\lceil 8\dfrac{32}{64} \right\rfloor, \left\lceil -28\dfrac{32}{64} \right\rfloor, \left\lceil 4\dfrac{32}{64} \right\rfloor, \left\lceil -32\dfrac{32}{64} \right\rfloor, \left\lceil 0\dfrac{32}{64} \right\rfloor, \left\lceil 31\dfrac{32}{64} \right\rfloor, \left\lceil -6\dfrac{32}{64} \right\rfloor, \left\lceil 7\dfrac{32}{64} \right\rfloor, \left\lceil 24\dfrac{32}{64} \right\rfloor)$
$= (4, -14, 2, -16, 0, 16, -3, 4, 12)$

The exact switched phase is $12 - (4 - 16 + 16 + 4) = 4 \pmod{32}$. The scaled input error $2 \cdot 32/64 = 1$ is canceled here by the component-rounding contribution -1, giving $\hat{e} = 0$. In general it is incorrect to replace the total switched error by the rounded value of $2ne/q$.
2. **Blind Rotation:** Use the admissible messages $\{-2, -1, 0, 1\}$ and $|\hat{e}| < 2$. The table is centered at the phase values $-8, -4, 0, 4$. At the unused tie boundaries choose rounding toward the higher integer. The unscaled lookup coefficients are

$$(v_0, \ldots, v_{15}) = (0,0,1,1,1,1,2,2,2,2,1,1,1,1,0,0).$$

Negative indices select the negated coefficients dictated by $X^{16} = -1$. The accumulator encrypts

Phase j in VX^{-j}	$-10, -9, -8, -7$	$-6, -5, -4, -3$	$-2, -1, 0, 1$	$2, 3, 4, 5$
Constant coefficient	-2	-1	0	1

Table 4: Centered lookup for $n = 16, q = 64, t = 8$. Boundary points are assigned a tie convention; correctness uses the strict bound $|\hat{e}| < 2$.

$\Delta V = 8V$; the lookup indices use the different scale $\hat{\Delta} = 4$.
During the actual bootstrapping, we will do a blind rotation of Table 4's $V(X)$ (which is GLWE-encrypted) by $\hat{b} - \sum_{i=0}^{7} \hat{a}_i s_i = 4$ positions to the left, which is computed as follows:

$\hat{\Delta}m + \hat{e} = \hat{b} - \sum_{i=0}^{7} \hat{a}_i s_i = 12 - (4 - 16 + 16 + 4) = 4 \bmod 32 \in \mathbb{Z}_{2n=32}$

In Table 4, phase 4 selects $v_4 = 1 = m$. The accumulator contains $\Delta v_4 = 8$, so extraction produces a ciphertext decrypting to $m = 1$, not the scalar ciphertext "1". The toy example uses $k' = 8$ output GLWE polynomials, hence extraction gives a 128-dimensional LWE secret before key switching back to dimension 8; these small parameters are illustrative, not secure.

The actual blind rotation is computed as follows:

$\vec{s} = (1, 0, 0, 1, 1, 1, 0, 1)$

$\mathsf{LWE}_{\vec{s},\sigma}(\hat{\Delta}m) = (\hat{a}_0, \hat{a}_1, \hat{a}_2, \hat{a}_3, \hat{a}_4, \hat{a}_5, \hat{a}_6, \hat{a}_7, \hat{b}) = (4, -14, 2, -16, 0, 16, -3, 4, 12)$

$V_0 = V \cdot X^{-\hat{b}} = V \cdot X^{-12} = v_{12} + v_{13}X + v_{14}X^2 + \cdots$

$V_1 = V_0 \cdot X^{\hat{a}_0 s_0} = s_0 \cdot (V_0 \cdot X^{\hat{a}_0} - V_0) + V_0 = V_0 \cdot X^4 = v_8 + v_9X + v_{10}X^2 + \cdots$

$V_2 = V_1 \cdot X^{\hat{a}_1 s_1} = s_1 \cdot (V_1 \cdot X^{\hat{a}_1} - V_1) + V_1 = V_1 = v_8 + v_9X + v_{10}X^2 + \cdots$

$V_3 = V_2 \cdot X^{\hat{a}_2 s_2} = s_2 \cdot (V_2 \cdot X^{\hat{a}_2} - V_2) + V_2 = V_2 = v_8 + v_9X + v_{10}X^2 + \cdots$

$V_4 = V_3 \cdot X^{\hat{a}_3 s_3} = s_3 \cdot (V_3 \cdot X^{\hat{a}_3} - V_3) + V_3 = V_3 \cdot X^{-16} = -v_8 - v_9X - v_{10}X^2 - \cdots$

$V_5 = V_4 \cdot X^{\hat{a}_4 s_4} = s_4 \cdot (V_4 \cdot X^{\hat{a}_4} - V_4) + V_4 = V_4 \cdot X^0 = -v_8 - v_9X - v_{10}X^2 - \cdots$

$V_6 = V_5 \cdot X^{\hat{a}_5 s_5} = s_5 \cdot (V_5 \cdot X^{\hat{a}_5} - V_5) + V_5 = V_5 \cdot X^{16} = v_8 + v_9X + v_{10}X^2 + \cdots$

$V_7 = V_6 \cdot X^{\hat{a}_6 s_6} = s_6 \cdot (V_6 \cdot X^{\hat{a}_6} - V_6) + V_6 = V_6 = v_8 + v_9X + v_{10}X^2 + \cdots$

$V_8 = V_7 \cdot X^{\hat{a}_7 s_7} = s_7 \cdot (V_7 \cdot X^{\hat{a}_7} - V_7) + V_7 = V_7 \cdot X^4 = v_4 + v_5X + v_6X^2 + \cdots$

The final output of blind rotation is the GLWE ciphertext of V_8, $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta V_8)$, whose constant term's coefficient is $v_4 = m = 1$.

Each step of the actual blind rotation above is computed as the following TFHE ciphertext-to-ciphertext multiplication:

$\mathsf{GLWE}_{\vec{S},\sigma}(\Delta V_{i+1}) = \mathsf{GGSW}^{\beta,l}_{\vec{S},\sigma}(s_i) \cdot (\mathsf{GLWE}_{\vec{S},\sigma}(\Delta V_i) \cdot X^{\hat{a}_i} - \mathsf{GLWE}_{\vec{S},\sigma}(\Delta V_i)) + \mathsf{GLWE}_{\vec{S},\sigma}(\Delta V_i)$

We will leave this computation for the reader's exercise.

3. **Coefficient Extraction:** At the end of blind rotation, we finally get the following GLWE ciphertext:

$\mathsf{GLWE}_{\vec{S},\sigma}(\Delta V_8)$

$= \mathsf{GLWE}_{\vec{S},\sigma}(\Delta \cdot (v_4 + v_5X + v_6X^2 + v_7X^3 + v_8X^4 + v_9X^5 + v_{10}X^6 + v_{11}X^7 + v_{12}X^8 + v_{13}X^9 + v_{14}X^{10} + v_{15}X^{11} - v_0X^{12} - v_1X^{13} - v_2X^{14} - v_3X^{15}))$

$= \mathsf{GLWE}_{\vec{S},\sigma}(\Delta \cdot (1 + X + 2X^2 + 2X^3 + 2X^4 + 2X^5 + X^6 + X^7 + X^8 + X^9 - X^{14} - X^{15}))$

$= \left(A_0 = \sum_{j=0}^{15} a_{0,j}X^j, A_1 = \cdots, A_{k-1} = \cdots, B = \sum_{j=0}^{15} b_jX^j \right)$

Now, we extract the constant term's coefficient of the encrypted polynomial $\mathsf{GLWE}_{\vec{S},\sigma}(\Delta \cdot (1 + X + 2X^2 + \cdots))$ by using the coefficient extraction formula (Summary D-1.7). Specifically, we will extract the constant term's coefficient, which corresponds to $\mathsf{LWE}_{\vec{s'},\sigma}(\Delta m_0)$. We extract $\mathsf{LWE}_{\vec{s'},\sigma}(\Delta m_0)$ by computing the following:

$\mathsf{LWE}_{\vec{s'},\sigma}(\Delta m_0) = (a'_0, a'_1, \cdots, a'_{nk-1}, b_h)$ ▷ where $h = 0$

, where $a'_{n \cdot i+j} = \begin{cases} a_{i,0-j} \text{ (if } 0 \le j \le 0) \\ -a_{i,n+0-j} \text{ (if } 0+1 \le j \le n-1) \end{cases}$, b_0 is obtained from the polynomial B

Finally use §D-1.5 to switch the extracted ciphertext from the flattened key $\vec{s}\,'$ back to the original $\vec{s}$.

D-1.8.9 Discussion

- **Programmable Bootstrapping**: Replace the identity lookup in §D-1.8 by a table for $f(m)$, with centered input intervals, an explicit output scale and the required negacyclic layout. This yields gates such as those in §D-1.8.10. An encrypted table can hide table entries under its encryption assumptions, but this alone is not a construction of functional encryption with function-restricted decryption keys. Identity and programmable bootstrapping have comparable noise bounds for the same accumulator and evaluation parameters; an encrypted table's initial noise and the chosen encoding must also be counted.
- **Bootstrapping Noise**: The CMux operations in §D-1.8.4 introduce evaluation-key and decomposition errors. These form part of the new output noise; coefficient extraction does not erase them or introduce additional noise. Correctness needs both a successful table selection and a bound on the accumulated output error.

D-1.8.10 Application: Gate Bootstrapping

In this example $n = 8$. The error bound below concerns the total phase error after forming the linear combination and modulus switching. The accumulator is an encryption of ΔV at modulus q, so the extracted output uses the same message scale as the input.

Besides the CMux used during blind rotation (§D-1.8.4), a LUT can implement Boolean gates such as AND, NAND, OR and XOR. In this example $q = 32$, $t = 8$, $n = 8$, $\Delta = q/t = 4$ and $\hat{\Delta} = 2n/t = 2$. Encode Boolean inputs by $0 \mapsto -1$ and $1 \mapsto 1$. Assume the total error after the affine combination and modulus switching satisfies $\hat{e} \in [-1, 1]$. The three possible affine phases are then centered at $-6, -2, 2$.

Lookup Table Polynomial $V(X) = 1 + 1X + 1X^2 + 1X^3 + 1X^4 + 1X^5 + 1X^6 + 1X^7$							
Decoded		**Encoded**		**Linear combination of encodings**	**Scaled Encoded Combination**	**Bootstrapping**	**Decoded result**
d_1	d_2	m_1	m_2	$m_1 + m_2 - 1$	$\hat{\Delta} \cdot (m_1 + m_2 - 1)$	$V(X) \cdot X^{-(\hat{\Delta} \cdot (m_1 + m_2 - 1) + \hat{e})}$	$d_1 \wedge d_2$
0	0	-1	-1	-3	-6	constant term's coeff. is -1	0
0	1	-1	1	-1	-2	constant term's coeff. is -1	0
1	0	1	-1	-1	-2	constant term's coeff. is -1	0
1	1	1	1	1	2	constant term's coeff. is 1	1

Table 5: An example truth table for an AND operation with an additional encoding.

In Table 5, $V(X) = \sum_{i=0}^{7} X^i$ returns -1 at phases $-6, -2$ and $+1$ at phase 2, including the allowed error intervals. Form the affine ciphertext as $\mathsf{ct}_1 + \mathsf{ct}_2 - (0, \Delta)$; subtracting an unscaled 1 would encode a different offset. Initialize the accumulator with ΔV. Extraction and the usual key switch then produce an LWE encryption at scale Δ of the encoded AND result. The error bound is on the aggregate switched phase, not on each input error separately; larger supported ranges require a new correctness and security parameter analysis.

This gate table is antiperiodic, so it may use both halves of the $2n = 16$ rotation indices consistently. For a general lookup function, reserve the admissible half-domain and centered intervals specified in §D-1.8.3. The identity lookup of §D-1.8.1 likewise needs a compatible encoding; the restriction is not a ban on rotations exceeding $n - 1$.

Besides the AND gate, other logic gates can be built in a similar manner, each of which is based on a different linear combination formula and LUT layout.

Division: Modular inversion, integer division and rounding are different operations. A nonunit has no multiplicative inverse in $\mathbb{Z}_q$, but this does not prevent a lookup table from evaluating division by a public constant or another bounded unary division function on its specified domain. General integer division can also be implemented with Boolean circuits and gate bootstrapping; division by zero needs an application-defined convention.

D-1.8.11 Application: Neural Networks Bootstrapping

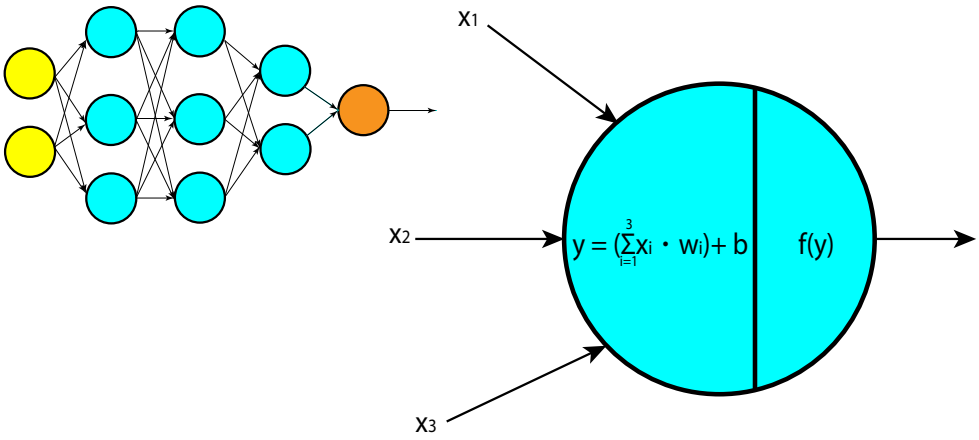


Figure 14: An illustration of neural networks

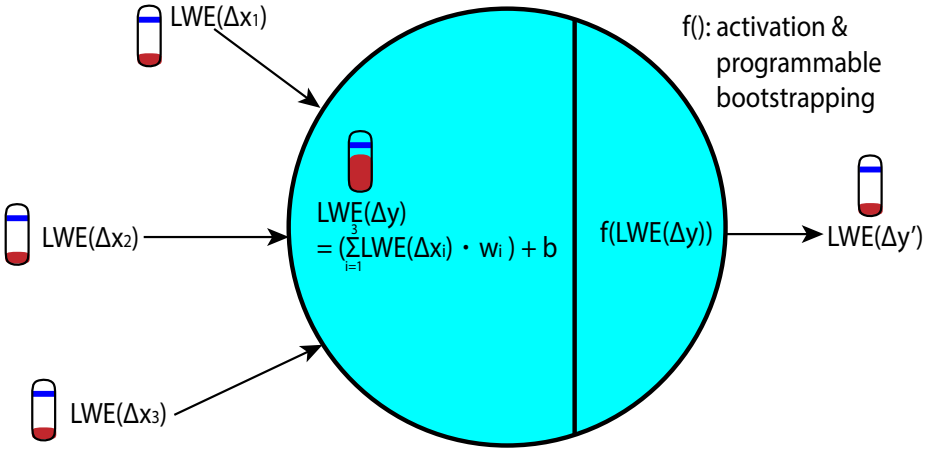


Figure 15: Programmable bootstrapping in a quantized neural network. The diagram's added bias denotes the encoded constant $(0, \Delta b)$; the text specifies the quantization, scales and admissible lookup ranges.

Homomorphic encryption can be applied to the neurons of deep neural networks, in which each neuron is generally comprised of two steps of computation:

1. **Linear Combination of Input Values:** An input feature value (or intermediate value) set $(x_1, x_2, \cdots, x_n)$, a weight set $(w_1, w_2, \cdots, w_n)$, and a bias b are computed as: $y = \sum_{i=1}^{n} x_i w_i + b$.

2. **Activation Function:** $f(y)$ is computed, where f is a non-linear activation function such as the sin function, ReLU, sigmoid, hyperbolic tangent, etc.

With quantized integer features and public integer weights, the first step is implemented as

$$\sum_{i=1}^{n} \mathsf{LWE}_{\vec{s},\sigma}(\Delta x_i) w_i + (0, \Delta b)$$

. It uses ciphertext addition (§C-1) and ciphertext-to-plaintext multiplication (§C-3). The input and bias scales must agree, and the weighted sum's range and noise must fit the next lookup.

The activation can be evaluated through a finite lookup table after choosing input and output quantization, clipping ranges and scales. Nonpolynomial real functions cannot be represented exactly by a finite real arithmetic polynomial, although polynomial approximation and interpolation on finite domains are possible. Programmable bootstrapping evaluates the selected discrete function and refreshes its ciphertext. Repeating this supports deep networks only when each linear stage stays within the lookup domain and noise budget, and the accumulated failure probability is acceptable.

D-1.9 TFHE on a Discrete Torus

- Reference: Guide to Fully Homomorphic Encryption over the [Discretized] Torus [12]

Torus $\mathbb{T}$ is a continuous real number domain between 0 and 1 that wraps around, that is $[0, 1)$.

A discrete torus $\mathbb{T}_t$ is a finite real number set: $\left(0, \frac{1}{t}, \frac{2}{t}, \cdots, \frac{t-1}{t}\right)$

In the previous subsections, we learned the TFHE scheme based on the following setup:

$m \in \mathbb{Z}_t \ \vec{s} \in \{0,1\}^k \ e \in \mathbb{Z}_q$

$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m) = (a_0, a_1, \cdots, a_{k-1}, b) \in \mathbb{Z}_q^{k+1}$

Dividing the integer ciphertext components by q gives the corresponding discrete-torus representation (the original theory can also use the continuous torus):

$m \in \mathbb{T}_t, \ \vec{s} \in \{0,1\}^k, \ e \in \mathbb{T}_q$

$\mathsf{LWE}_{\vec{s},\sigma}(m) = \mathsf{ct} = (a_0, a_1, \cdots, a_{k-1}, b) \in \mathbb{T}_q^{k+1}$

$b = \sum_{i=0}^{k-1} (a_i s_i) + m + e \in \mathbb{T}_q$

$\mathsf{LWE}^{-1}_{\vec{s},\sigma}(\mathsf{ct}) = \left\lceil b - \sum_{i=0}^{k-1} (a_i s_i) \right\rfloor_{\frac{1}{t}} = \left\lceil m + e \right\rfloor_{\frac{1}{t}} = m$, given $|e| < \frac{1}{2t}$

▷ where $\lceil x \rfloor_{\frac{1}{t}}$ means rounding x to the nearest multiple of $\frac{1}{t}$

The torus is the additive group $\mathbb{T} = \mathbb{R}/\mathbb{Z}$, with multiplication by integers; arbitrary real multiplication is not a well-defined ring operation on it. A discrete implementation represents $a/q \bmod 1$ by the integer $a \bmod q$ (for example with fixed-width integers), so floating-point storage is not required. Normalizing the integer encoding by q sends Δm_{int} to m_{int}/t when $t \mid q$. Thus the embedding scale and message spacing $1/t$ remain present even when the symbol Δ is omitted. Correctness requires the centered torus

error to satisfy $|e| < 1/(2t)$. FFT arithmetic may use floating point internally; that is separate from the ciphertext representation.

Both the torus-based and integer-ring-based TFHE schemes are built based on the same fundamental principles.

D-2 BFV Scheme

BFV supports exact arithmetic on plaintext polynomials modulo an integer plaintext modulus. Unlike approximate CKKS arithmetic, its decoded result is exact when every operation remains within the BFV correctness bound; encoding integers alone does not guarantee successful decryption for arbitrary noise or circuit depth. Applications must also choose a plaintext modulus and encoding that preserve their intended integer semantics. The reference scheme is that of Fan and Vercauteren [13].

In BFV, each plaintext is encrypted as an RLWE ciphertext. Therefore, BFV's ciphertext-to-ciphertext addition, ciphertext-to-plaintext addition, and ciphertext-to-plaintext multiplication are implemented based on GLWE's homomorphic addition and multiplication (as we learned in Part III), with $k = 1$ to make GLWE an RLWE.

Required Background

- §A-1: Modulo Arithmetic
- §A-2: Group
- §A-3: Field
- §A-4: Order
- §A-5: Polynomial Ring
- §A-6: Decomposition
- §A-7: Roots of Unity
- §A-8: Cyclotomic Polynomial
- §A-9: Roots of Unity and Cyclotomic Polynomial over Ring
- §A-11: Vector and Matrix
- §A-12: Euler's Formula
- §A-13: Modulo Rescaling
- §A-14: Chinese Remainder Theorem
- §A-10: Lagrange's Polynomial Interpolation
- §A-16: Efficient Polynomial Multiplication by FFT and NTT
- §B-1: Lattice-based Cryptography
- §B-3: RLWE Cryptosystem
- §B-4: GLWE Cryptosystem
- §C-1: GLWE Ciphertext-to-Ciphertext Addition
- §C-2: GLWE Ciphertext-to-Plaintext Addition
- §C-3: GLWE Ciphertext-to-Plaintext Multiplication
- §C-4.4: RLWE Modulus Switching
- §C-5: GLWE Key Switching

D-2.1 Single Value Encoding

This binary-polynomial encoding requires $0 \leq m < 2^n$. Evaluation at $X = 2$ preserves integer addition and multiplication only while the chosen integer coefficient lifts do not wrap modulo t and the product degree is strictly less than n. It is not a ring homomorphism from $\mathbb{Z}_t[X]/(X^n + 1)$ to the unbounded integers. For example, with $n = 2$, the encoding X of 2 squares to -1 in the quotient ring, not the integer 4.

BFV supports two encoding schemes: single value encoding and batch encoding. In this subsection, we will explain the single value encoding scheme.

⟨Summary D-2.1⟩ BFV Encoding

Input Integer: Decompose an integer m satisfying $0 \leq m < 2^n$ as follows:
$m = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \cdots + b_1 \cdot 2^1 + b_0 \cdot 2^0$, where each $b_i \in \{0, 1\}$

Encoded Polynomial: $M(X) = b_0 + b_1 X + b_2 X^2 + \cdots + b_{n-1} X^{n-1} \in \mathcal{R}_{\langle n,t \rangle}$

Decoding: $M(X = 2) = m$

Let's analyze whether the encoding scheme in Summary D-2.1 ensures correct decoding after addition and multiplication. This is equivalent to showing that:

$\text{Decode}(\sigma(m_1) + \sigma(m_2)) = m_1 + m_2$ ▷ where σ is the encoding function

$\text{Decode}(\sigma(m_1) \cdot \sigma(m_2)) = m_1 \cdot m_2$

Let $\sigma(m_1) = M_1(X)$ and $\sigma(m_2) = M_2(X)$. Then,

$\text{Decode}(\sigma(m_1) + \sigma(m_2)) = \text{Decode}(M_1(X) + M_2(X))$

$= \text{Decode}(M_{1+2}(X))$ ▷ where $M_{1+2}(X) = M_1(X) + M_2(X)$

$= M_{1+2}(2)$ ▷ since decoding is evaluating the polynomial at $X = 2$

$= M_1(2) + M_2(2)$ ▷ since evaluating $M_{1+2}(X)$ at $X = 2$ is computationally the same as splitting $M_{1+2}(X)$ into $M_1(X)$ and $M_2(X)$, evaluating $M_1(2)$ and $M_2(2)$ and summing them

Similarly, the decoding preserves correctness over multiplication as well:

$\text{Decode}(\sigma(m_1) \cdot \sigma(m_2)) = \text{Decode}(M_1(X) \cdot M_2(X))$

$= \text{Decode}(M_{1 \cdot 2}(X))$ ▷ where $M_{1 \cdot 2}(X) = M_1(X) \cdot M_2(X)$

$= M_{1 \cdot 2}(2)$ ▷ since decoding is evaluating the polynomial at $X = 2$

$= M_1(2) \cdot M_2(2)$ ▷ since evaluating $M_{1 \cdot 2}(X)$ at $X = 2$ is computationally the same as splitting $M_{1 \cdot 2}(X)$ into $M_1(X)$ and $M_2(X)$, evaluating $M_1(2)$ and $M_2(2)$ and multiplying them

Therefore these integer identities hold under the stated no-wrap and degree conditions; they are not unconditional identities in the quotient ring.

The encoding of Summary D-2.1 preserves integer semantics only if every resulting polynomial has degree strictly below n and its chosen integer coefficients do not wrap modulo t. At degree n, reduction by $X^n + 1$ already changes evaluation at 2. Due to these constraints, the single value encoding is not a good choice for fully homomorphic encryption. Also, the single value encoding is computationally inefficient, because each polynomial can encode only a single value even if it holds n coefficients.

D-2.2 Batch Encoding

For the scalar-slot formulas below, assume $n \geq 2$ is a power of two and t is prime with $2n \mid (t-1)$. Then $\omega = g^{(t-1)/(2n)}$ is a primitive $2n$-th root and the Vandermonde matrix is invertible. A prime-power modulus $t = p^r$ also supports these scalar slots when $p \equiv 1 \pmod{2n}$ and the roots are lifted to $\mathbb{Z}_{p^r}$; the exponent formula using $t-1$ is not valid in general for prime powers. Other parameters may give fewer slots over extension rings. In this chapter W^T is the row-wise evaluation Vandermonde matrix, so W is its transpose; this defines the matrix orientation explicitly.

While the single-value encoding scheme (§D-2.1) encodes & decodes each individual value one at a time, the batch encoding scheme does the same for a huge list of values simultaneously using a large dimensional vector. Therefore, batch encoding is more efficient than single-value encoding. Furthermore, batch-encoded values can be homomorphically added or multiplied simultaneously element-wise by vector-to-vector addition and Hadamard product. Therefore, the homomorphic operation of batch-encoded values can be processed more efficiently in a SIMD (single-instruction-multiple-data) manner than single-value encoded ones.

BFV's encoding converts an n-slot integer input vector $\vec{v} = (v_0, v_1, v_2, \cdots v_{n-1})$ modulo t into another n-slot vector $\vec{m} = (m_0, m_1, m_2, \cdots m_{n-1})$ modulo t, which are the coefficients of the encoded $(n-1)$-degree (or lesser-degree) polynomial $M(X) \in \mathbb{Z}_t[X]/(X^n+1)$.

D-2.2.1 $\mathsf{Encoding_1}$

In §A-11.6, we learned that an $(n-1)$-degree (or lesser degree) polynomial can be isomorphically mapped to an n-slot vector based on the mapping σ (we notate σ_c in §A-11.6 as σ for simplicity):

$\sigma : M(X) \in \mathbb{Z}_t[X]/(X^n+1) \longrightarrow (M(\omega), M(\omega^3), M(\omega^5), \cdots, M(\omega^{2n-1})) \in \mathbb{Z}_t^n$

, which evaluates the polynomial $M(X)$ at n distinct ($\mu = 2n$)-th primitive roots of unity: $\omega, \omega^3, \omega^5, \cdots, \omega^{2n-1}$. Let $\vec{m}$ be a vector that contains n coefficients of the polynomial $M(X)$. Then, we can express the mapping σ as follows:

$\vec{v} = W^T \cdot \vec{m}$

, where W^T is as follows:

$$W^T = \begin{bmatrix} 1 & (\omega) & (\omega)^2 & \cdots & (\omega)^{n-1} \\ 1 & (\omega^3) & (\omega^3)^2 & \cdots & (\omega^3)^{n-1} \\ 1 & (\omega^5) & (\omega^5)^2 & \cdots & (\omega^5)^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{2n-1}) & (\omega^{2n-1})^2 & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix} \quad \# \; W^T \text{ is a transpose of } W \text{ described in §A-11.6}$$

Note that the dot product between each row of W^T and $\vec{m}$ computes the evaluation of $M(X)$ at each $X = \{w, w^3, w^5, \cdots, w^{2n-1}\}$. In the BFV encoding scheme, the $\mathsf{Encoding_1}$ process encodes an n-slot input vector $\vec{v} \in \mathbb{Z}_t^n$ into a plaintext polynomial $M(X) \in \mathbb{Z}_t[X]/(X^n+1)$, and the $\mathsf{Decoding_2}$ process decodes $M(X)$ back to $\vec{v}$. Since $W^T \cdot \vec{m}$ gives us $\vec{v}$ which is a decoding of $M(X)$, we call W^T a decoding matrix. Meanwhile, the goal of $\mathsf{Encoding_1}$ is to encode $\vec{v}$ into $M(X)$ so that we can do homomorphic computations based on $M(X)$. Given the relation $\vec{v} = W^T \cdot \vec{m}$, the encoding formula can be derived as follows:

$(W^T)^{-1} \cdot \vec{v} = (W^T)^{-1} W^T \cdot \vec{m}$

$$\vec{m} = (W^T)^{-1} \cdot \vec{v}$$

Therefore, we need to find out what $(W^T)^{-1}$ is, the inverse of W^T as the encoding matrix. But we already learned from Theorem A-12.4 (in §A-12.4) that $V^{-1} = \frac{V^T \cdot I_n^R}{n}$, where $V = W^T$ and $V^T = W$. In other words, $(W^T)^{-1} = \frac{W \cdot I_n^R}{n}$. Therefore, we can express the BFV encoding formula as:

$$\vec{m} = (W^T)^{-1} \cdot \vec{v} = \frac{W \cdot I_n^R \cdot \vec{v}}{n}, \quad \text{where}$$

$$W = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ (\omega) & (\omega^3) & (\omega^5) & \cdots & (\omega^{2n-1}) \\ (\omega)^2 & (\omega^3)^2 & (\omega^5)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & (\omega^5)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & \cdots & 1 & 1 \\ (\omega) & (\omega^3) & \cdots & (\omega^{n-1}) & (\omega^{-(n-1)}) & \cdots & (\omega^{-3}) & (\omega^{-1}) \\ (\omega)^2 & (\omega^3)^2 & \cdots & (\omega^{n-1})^2 & (\omega^{-(n-1)})^2 & \cdots & (\omega^{-3})^2 & (\omega^{-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ (\omega)^{n-1} & (\omega^3)^{n-1} & \cdots & (\omega^{n-1})^{n-1} & (\omega^{-(n-1)})^{n-1} & \cdots & (\omega^{-3})^{n-1} & (\omega^{-1})^{n-1} \end{bmatrix}$$

, where ω is a primitive $2n$-th root of unity modulo t (which implies $t \equiv 1 \bmod 2n$). This implies that $\omega = g^{\frac{t-1}{2n}} \bmod t$ (g is a generator of $\mathbb{Z}_t^\times$ (see §A-11.6.1).

In §A-11.6, we learned that W is a valid basis of the n-dimensional vector space. Therefore, $\frac{W \cdot I_n^R \cdot \vec{v}}{n} = \vec{m}$ is guaranteed to be a unique vector corresponding to each $\vec{v}$ in the n-dimensional vector space $\mathbb{Z}_t^n$ (refer to Theorem A-11.4 in §A-11.4), and thereby the polynomial $M(X)$ comprising the n elements of $\vec{m}$ as coefficients is a unique polynomial bi-jective to $\vec{v}$.

Note that by computing $\frac{W \cdot I_n^R \cdot \vec{v}}{n}$, this is a linear bijection. The ring isomorphism identifies slotwise multiplication of $\vec{v}$ with negacyclic polynomial multiplication of M, not with componentwise multiplication of the coefficient vector $\vec{m}$.

D-2.2.2 Encoding$_2$

Once we have the n-slot vector $\vec{m}$, we scale (i.e., multiply) it by some scaling factor $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$, where q is the ciphertext modulus. We scale $\vec{m}$ by Δ and make it $\Delta\vec{m}$. The n integers in $\Delta\vec{m}$ will be used as n coefficients of the plaintext polynomial for RLWE encryption. The finally encoded plaintext polynomial $\Delta M = \sum_{i=0}^{n-1} \Delta m_i X^i$.

D-2.2.3 Decoding$_1$

The divisions by Δ in the encoding/decoding diagrams refer to the ideal, unreduced, noiseless encoded polynomial. For an actual ciphertext, first apply the scale-and-round decoder of §D-2.3; only then apply the slot evaluation matrix to the recovered polynomial in $\mathcal{R}_{\langle n,t \rangle}$.

Once an RLWE ciphertext is (first-half) decrypted to $\Delta M = \sum_{i=0}^{n-1} \Delta m_i X^i$, we compute $\frac{\Delta \vec{m}}{\Delta} = \vec{m}$.

D-2.2.4 Decoding$_2$

In §D-2.2.1, we already derived the decoding formula that transforms an $(n-1)$-degree polynomial having integer modulo t coefficients into an n-slot input vector as follows:
$\vec{v} = W^T \cdot \vec{m}$

D-2.2.5 Summary

⟨Summary D-2.2.5⟩ BFV's Encoding and Decoding

Input: An n-slot integer modulo t vector $\vec{v} = (v_0, v_1, \cdots, v_{n-1}) \in \mathbb{Z}_t^n$

Encoding

1. Convert $\vec{v} \in \mathbb{Z}_t^n$ into $\vec{m} \in \mathbb{Z}_t^n$ by applying the transformation $\vec{m} = n^{-1} \cdot W \cdot I_n^R \cdot \vec{v}$, where W is a basis of the n-dimensional vector space crafted as follows:

$$W = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ (\omega) & (\omega^3) & (\omega^5) & \cdots & (\omega^{2n-1}) \\ (\omega)^2 & (\omega^3)^2 & (\omega^5)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & (\omega^5)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & \cdots & 1 & 1 \\ (\omega) & (\omega^3) & \cdots & (\omega^{n-1}) & (\omega^{-(n-1)}) & \cdots & (\omega^{-3}) & (\omega^{-1}) \\ (\omega)^2 & (\omega^3)^2 & \cdots & (\omega^{n-1})^2 & (\omega^{-(n-1)})^2 & \cdots & (\omega^{-3})^2 & (\omega^{-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ (\omega)^{n-1} & (\omega^3)^{n-1} & \cdots & (\omega^{n-1})^{n-1} & (\omega^{-(n-1)})^{n-1} & \cdots & (\omega^{-3})^{n-1} & (\omega^{-1})^{n-1} \end{bmatrix}$$

, where ω is a primitive $2n$-th root of unity modulo t. This implies that $\omega = g^{\frac{t-1}{2n}} \bmod t$ (g is a generator of $\mathbb{Z}_t^\times$ (see §A-11.6.1).

2. Convert $\vec{m}$ into a scaled integer vector $\Delta \vec{m}$, where $\Delta = \lfloor \frac{q}{t} \rfloor$ is a scaling factor. If Δ is too close to 1 (i.e., t is too big), the **noise budget** will become too small (making decryption fail easily). If Δ is too close to q (i.e., t is too small), the **message capacity** will become too small (i.e., the plaintext modulus t limits the range of values that can be encoded). The finally encoded plaintext polynomial $\Delta M = \sum_{i=0}^{n-1} \Delta m_i X^i \in \mathbb{Z}_q[X]/(X^n+1)$.

Decoding: From the plaintext polynomial $\Delta M = \sum_{i=0}^{n-1} \Delta m_i X^i$, recover $\vec{m} = \frac{\Delta \vec{m}}{\Delta}$. Then, compute $\vec{v} = W^T \cdot \vec{m}$.

However, Summary D-2.2.5 is not the final version of BFV's batch encoding. In §D-2.9, we will explain how to homomorphically rotate the input vector slots without decrypting the ciphertext that encapsulates it. To support such homomorphic rotation, we will need to slightly update the encoding scheme explained

in Summary D-2.2.5. We will explain how to do this in §D-2.9, and BFV's final encoding scheme is summarized in Summary D-2.9.3 in §D-2.9.3.

D-2.3 Encryption and Decryption

We describe symmetric BFV encryption using the RLWE phase $B + AS$, the alternative sign convention of §B-4.4 with $k = 1$ (§B-3). Write $[U]_q$ for coefficientwise centered reduction and set $r_q = q - t\Delta$. The complete decoder Dec uses t/q. Elsewhere in the phase calculations, RLWE^{-1} denotes the intermediate noisy phase before this final scale-and-round step. Neither notation asserts fresh noise after a homomorphic operation.

⟨Summary D-2.3⟩ BFV Encryption and Decryption

Initial Setup: $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$ is a plaintext scaling factor for polynomial encoding, $S \xleftarrow{\$} \mathcal{R}_{\langle n,tern \rangle}$
The basic plaintext modulus may be any integer $t \geq 2$ with $q \gg t$. Batch encoding and the prime-power bootstrapping method impose the additional conditions stated in their respective sections. The coefficients of S here are ternary, in $\{-1, 0, 1\}$.

Encryption Input: $\Delta M \in \mathcal{R}_{\langle n,q \rangle}$, $A \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}$, $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$

1. Compute $B = -A \cdot S + \Delta M + E \ \in \mathcal{R}_{\langle n,q \rangle}$
2. $\mathsf{RLWE}_{S,\sigma}(\Delta M + E) = (A, B) \ \in \mathcal{R}^2_{\langle n,q \rangle}$

Decryption Input: $\mathsf{ct} = (A, B) \ \in \mathcal{R}^2_{\langle n,q \rangle}$

$$\mathsf{Dec}_S(A, B) = \left[\left\lceil \frac{t}{q} [B + AS]_q \right\rfloor \right]_t = M.$$

(The noise $E = \sum_{i=0}^{n-1} e_i X^i$ gets eliminated by the rounding process)

Conditions for Correct Decryption:
For a phase $\Delta M + E \pmod q$, a sufficient coefficientwise condition is $\|tE - r_q M\|_\infty < q/2$. In particular, $\|E\|_\infty < q/(2t) - r_q/2$ suffices when M is centered with $\|M\|_\infty \leq t/2$, when the right side is positive. Plaintext reduction after homomorphic operations contributes additional remainder errors, as derived in §D-2.4.1. This is the BFV scale-and-round condition; the alternative $1/\Delta$ convention discussed in §B-2.3.1 must not be substituted without changing its assumptions.

In this section, we will often write $\mathsf{RLWE}_{S,\sigma}(\Delta M + E)$ as $\mathsf{RLWE}_{S,\sigma}(\Delta M)$ for simplicity, because $\mathsf{RLWE}_{S,\sigma}(\Delta M + E) \approx \mathsf{RLWE}_{S,\sigma}(\Delta M)$ (i.e., they decrypt to the same message). Even in the case that we write $\mathsf{RLWE}_{S,\sigma}(\Delta M)$ instead of $\mathsf{RLWE}_{S,\sigma}(\Delta M + E)$, you should assume this as an encryption of $\Delta M + E$ (i.e., the noise is included inside the scaled message).

We will explain the conditions for BFV's correct decryption in more detail in §D-2.4.1.

D-2.4 Ciphertext-to-Ciphertext Addition

BFV's ciphertext-to-ciphertext addition uses RLWE's ciphertext-to-ciphertext addition scheme with the sign of the $A \cdot S$ term flipped in the encryption and decryption formula. Specifically, this is equivalent to

the alternative GLWE version's (§B-4.4) ciphertext-to-ciphertext addition scheme with $k = 1$.

⟨Summary D-2.4⟩ BFV Ciphertext-to-Ciphertext Addition

$$\begin{aligned}&\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle}) + \mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle})\\ &= (A^{\langle 1\rangle},\ B^{\langle 1\rangle}) + (A^{\langle 2\rangle},\ B^{\langle 2\rangle})\\ &= (A^{\langle 1\rangle} + A^{\langle 2\rangle},\ B^{\langle 1\rangle} + B^{\langle 2\rangle})\\ &= \mathsf{RLWE}_{S,\sigma}(\Delta(M^{\langle 1\rangle} + M^{\langle 2\rangle}) + E^{\langle 1\rangle} + E^{\langle 2\rangle})\end{aligned}$$

D-2.4.1 Noise Bound Analysis

Let $r_q = q - t\Delta$, and let the centered phase be $x = [\Delta M + E]_q = \Delta M + E - qK$. Multiplying by t/q gives the exact identity

$$\frac{t}{q}x = M + \frac{tE - r_q M}{q} - tK.$$

The integral multiple tK disappears after the final reduction modulo t. Therefore Summary D-2.3 in §D-2.3 decrypts correctly if

$$\|tE - r_q M\|_\infty < q/2.$$

This proof does not drop the inner reduction modulo q. Replacing t/q by $1/\Delta$ does not give the same identity when $r_q \neq 0$; it requires a separately specified decoder and representative convention.

For ciphertext addition, write $M^{\langle 1\rangle} + M^{\langle 2\rangle} = M^{\langle 1+2\rangle} + tH$, where $M^{\langle 1+2\rangle}$ is the chosen centered plaintext representative. Since $t\Delta = q - r_q$, the sum ciphertext has phase

$$\Delta M^{\langle 1+2\rangle} + E_{\text{add}} \pmod q, \qquad E_{\text{add}} = E^{\langle 1\rangle} + E^{\langle 2\rangle} - r_q H.$$

Its sufficient correctness condition is

$$\left\|t(E^{\langle 1\rangle} + E^{\langle 2\rangle} - r_q H) - r_q M^{\langle 1+2\rangle}\right\|_\infty < q/2.$$

The term $-r_q H$ records plaintext wraparound and cannot be omitted. The same reasoning applies after multiplication or rotation once the effective message and complete error have been identified. It is consistent with the need for explicit remainder terms in §B-2.3.1. A larger q/t may increase the correctness budget, but security must be assessed for the complete parameter set.

D-2.5 Ciphertext-to-Plaintext Addition

The added polynomial uses scale Δ. If $M + \Lambda = M_{\text{out}} + tH$, the effective error after plaintext reduction is $E - r_q H$, with $r_q = q - t\Delta$ as in §D-2.4.1.

BFV's ciphertext-to-plaintext addition uses RLWE's ciphertext-to-plaintext addition scheme with the sign of the $A \cdot S$ term flipped in the encryption and decryption formulas. Specifically, this is equivalent to the alternative GLWE version's (§B-4.4) ciphertext-to-plaintext addition scheme (§C-2) with $k = 1$.

⟨Summary D-2.5⟩ BFV Ciphertext-to-Plaintext Addition

$$\begin{aligned}&\mathsf{RLWE}_{S,\sigma}(\Delta M + E) + \Delta\Lambda\\ &= (A,\ B) + \Delta\Lambda\end{aligned}$$

$= (A,\ B + \Delta \cdot \Lambda)$
$= \mathsf{RLWE}_{S,\sigma}(\Delta(M + \Lambda) + E)$

D-2.6 Ciphertext-to-Plaintext Multiplication

Here Λ is an unscaled plaintext multiplier. If $M\Lambda = M_{\text{out}} + tH$, the effective error is $\Lambda E - r_q H$. For polynomial multiplication, $\|\Lambda E\|_\infty \le \|\Lambda\|_1 \|E\|_\infty$; both this convolution bound and the remainder term enter the decoder condition.

BFV's ciphertext-to-plaintext multiplication uses RLWE's ciphertext-to-plaintext multiplication scheme with the sign of the $A \cdot S$ term flipped in the encryption and decryption formula. Specifically, this is equivalent to the alternative GLWE version's (§B-4.4) ciphertext-to-plaintext multiplication scheme (§C-3) with $k = 1$.

⟨Summary D-2.6⟩ BFV Ciphertext-to-Plaintext Multiplication

$\mathsf{RLWE}_{S,\sigma}(\Delta M + E) \cdot \Lambda$
$= (A,\ B) \cdot \Lambda$
$= (A \cdot \Lambda,\ B \cdot \Lambda)$
$= \mathsf{RLWE}_{S,\sigma}(\Delta(M \cdot \Lambda) + \Lambda E)$

D-2.7 Ciphertext-to-Ciphertext Multiplication

The four-stage presentation below is an explicitly bounded teaching variant: it uses an auxiliary modulus $Q = q\Delta$ and rescales by $1/\Delta$. The original FV multiplication scales integer-lifted tensor-product components by t/q before relinearization; these factors coincide only when $t \mid q$. We keep this variant to explain modulus lifting, but retain its additional remainder and rounding errors. It is not a claim that every BFV implementation performs these four stages in this order.

- **Reference 1:** Introduction to the BFV encryption scheme [14]
- **Reference 2:** Somewhat Practical Fully Homomorphic Encryption [13]

Given two ciphertexts $\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 1 \rangle})$ and $\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 2 \rangle})$, the goal of ciphertext-to-ciphertext multiplication is to derive a new ciphertext whose decryption is $\Delta M^{\langle 1 \rangle} M^{\langle 2 \rangle}$. Ciphertext-to-ciphertext multiplication is more complex than ciphertext-to-plaintext multiplication. It comprises four steps: (1) ModRaise; (2) polynomial multiplication; (3) relinearization; and (4) rescaling.

For better understanding, we will explain BFV's ciphertext-to-ciphertext multiplication based on the alternate version of RLWE (Theorem B-4.4 in §B-4.4), where the sign of the AS term is flipped in the encryption and decryption formulas.

D-2.7.1 ModRaise

Choose integer coefficient representatives of (A, B) and write their unreduced phase in $\mathbb{Z}[X]/(X^n + 1)$ as

$$AS + B = \Delta M + E + qK.$$

Embedding these same coefficient representatives into modulus $Q = q\Delta$ gives phase $\Delta M + E + qK \pmod{Q}$, not an encryption of the unchanged ΔM with only error E. There is no general inequality

$(q-1)n + (q-1) < Q$: it would require an additional relation between n and Δ. Nor is a no-wrap condition needed for the congruence. All products and phases in the next two stages are interpreted modulo Q; integer lifts and their multiples of Q are accounted for explicitly when rescaling.

The first step of BFV's ciphertext-to-ciphertext multiplication is to *mod-raise* the two input ciphertexts $(A^{\langle 1\rangle}, B^{\langle 1\rangle}) \bmod q$ and $(A^{\langle 2\rangle}, B^{\langle 2\rangle}) \bmod q$ from $q \to Q$ (where $Q = q \cdot \Delta$) as follows:

$(A^{\langle 1\rangle}, B^{\langle 1\rangle}) \bmod Q$
$(A^{\langle 2\rangle}, B^{\langle 2\rangle}) \bmod Q$

After $\mathsf{ModRaise}$, the decryption of these two ciphertexts would be the following:

$A^{\langle 1\rangle} \cdot S + B^{\langle 1\rangle} = \Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1 q \pmod Q$
$A^{\langle 2\rangle} \cdot S + B^{\langle 2\rangle} = \Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2 q \pmod Q$

Therefore, the *mod-raised* ciphertexts have the following form:

$\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle} + K_1 q) = (A^{\langle 1\rangle}, B^{\langle 1\rangle}) \bmod Q$
$\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle} + K_2 q) = (A^{\langle 2\rangle}, B^{\langle 2\rangle}) \bmod Q$

D-2.7.2 Polynomial Multiplication

Our next goal is to derive a new ciphertext which encrypts $(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1 q) \cdot (\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2 q)$.

First, we can derive the following relation:

$$
\begin{aligned}
&(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1 q) \cdot (\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2 q) \\
&= (A^{\langle 1\rangle} \cdot S + B^{\langle 1\rangle}) \cdot (A^{\langle 2\rangle} \cdot S + B^{\langle 2\rangle}) \\
&= \underbrace{B^{\langle 1\rangle} B^{\langle 2\rangle}}_{D_0} + \underbrace{(B^{\langle 2\rangle} A^{\langle 1\rangle} + B^{\langle 1\rangle} A^{\langle 2\rangle})}_{D_1} \cdot S + \underbrace{(A^{\langle 1\rangle} \cdot A^{\langle 2\rangle})}_{D_2} \cdot S \cdot S \\
&= D_0 + D_1 \cdot S + D_2 \cdot S^2
\end{aligned}
$$

Meanwhile, we also have the following relations:

$\mathsf{RLWE}^{-1}_{S,\sigma}(\Delta M^{\langle 1\rangle} + K_1 q) = \Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1 q$
$\mathsf{RLWE}^{-1}_{S,\sigma}(\Delta M^{\langle 2\rangle} + K_2 q) = \Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2 q$

Combining all these, we reach the following relation:

$\mathsf{RLWE}^{-1}_{S,\sigma}(\Delta M^{\langle 1\rangle} + K_1 q) \cdot \mathsf{RLWE}^{-1}_{S,\sigma}(\Delta M^{\langle 2\rangle} + K_2 q) = D_0 + D_1 \cdot S + D_2 \cdot S^2$

Notice that D_0, D_1, and D_2 are known values as ciphertext components, whereas S is only known to the private key owner. Therefore, we can view $D_0 + D_1 \cdot S + D_2 \cdot S^2$ as a decryption formula such that given the ciphertext components D_0, D_1, D_2 and the private key S, one can derive $(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1 q) \cdot (\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2 q)$. In other words, we can let (D_0, D_1, D_2) be a new form of ciphertext which can be decrypted by S into $(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1 q) \cdot (\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2 q)$.

The triplet (D_0, D_1, D_2) is a degree-two ciphertext in the secret S, so decryption uses the square S^2, not a square root. Further multiplication adds secret degrees; repeated squaring doubles the degree at each step. Relinearization returns to two components and degree one, at the cost of evaluation-key noise.

D-2.7.3 Relinearization

For arbitrary Q, this subsection uses the integer gadget $g_i = \beta^{i-1}$ with $l = \lceil \log_\beta Q \rceil$ and exact coefficient decomposition $D_2 \equiv \sum_{i=1}^{l} D_{2,i} g_i \pmod Q$. This is the increasing-weight convention, rather than the

descending Q/β^i convention; its matching RLev rows encrypt $g_i S^2$. Let $E_{\mathrm{rl}} = \sum_i D_{2,i} E'_i$ denote their accumulated error. The relinearized phase equals the original phase plus E_{rl}, not exactly the original phase.

Relinearization turns the triplet $(D_0, D_1, D_2) \in \mathcal{R}^3_{\langle n,Q\rangle}$ into two components. With $\mathsf{ct}_\alpha = (D_1, D_0)$ and the gadget construction of ct_β, the relation is

$$\phi_S(\mathsf{ct}_\alpha + \mathsf{ct}_\beta) \equiv D_0 + D_1 S + D_2 S^2 + E_{\mathrm{rl}} \pmod{Q}.$$

In $D_0 + D_1 S + D_2 S^2$, the first two terms form the synthetic ciphertext $\mathsf{ct}_\alpha = (D_1, D_0)$. The gadget construction produces ct_β with phase $\phi_S(\mathsf{ct}_\beta) \equiv D_2 S^2 + E_{\mathrm{rl}} \pmod{Q}$. Publishing encryptions of functions of the secret, such as S^2, also requires the evaluation-key security assumption of the chosen construction; the algebra below is not a proof of that assumption.

A naive way of creating a ciphertext that encrypts $D_2 \cdot S^2$ is as follows: we encrypt S^2 into an RLWE ciphertext as $\mathsf{RLWE}_{S,\sigma}(S^2) = (A^{\langle s\rangle}, B^{\langle s\rangle})$ such that $A^{\langle s\rangle} \cdot S + B^{\langle s\rangle} = S^2 + E^{\langle s\rangle} \bmod Q$ (where the ciphertext modulus is Q and the plaintext scaling factor $\Delta = 1$). Then, we perform a ciphertext-to-plaintext multiplication (§C-3) with D_2, treating D_2 as a plaintext polynomial in modulo Q. However, this approach does not work in practice, because computing $D_2 \cdot \mathsf{RLWE}_{S,\sigma}(S^2)$ generates a huge noise as follows:

$D_2 \cdot (A^{\langle s\rangle}, B^{\langle s\rangle}) = (D_2 \cdot A^{\langle s\rangle}, D_2 \cdot B^{\langle s\rangle})$

, whose decryption is:

$D_2 \cdot A^{\langle s\rangle} \cdot S + D_2 \cdot B^{\langle s\rangle} = D_2 \cdot S^2 + D_2 \cdot E^{\langle s\rangle} \pmod{Q}$

. In the above decrypted expression $D_2 S^2 + D_2 E^{\langle s\rangle} \bmod Q$, the term $D_2 S^2$ is okay to be reduced modulo Q, because this term is originally allowed to be reduced modulo Q in the final decryption formula $D_0 + D_1 S + D_2 S^2 \bmod Q$ as well. However, the problematic term is the noise $D_2 \cdot E^{\langle s\rangle}$, because its coefficients can be any value in $[0, Q-1]$ (since each coefficient of polynomial $D_2 = A^{\langle 1\rangle} A^{\langle 2\rangle}$ can be any value in $[0, Q-1]$). Such a huge noise is not allowed for correct final decryption.

To avoid this noise issue, an improved solution is to express the RLWE ciphertext that encrypts $D_2 S^2$ as additions of multiple RLWE ciphertexts with small noises by using the gadget decomposition technique (§A-6.4). For this, we use an RLev ciphertext (§B-5) that encrypts S^2. Suppose our gadget vector is $\vec{g} = \left(\beta^0, \beta^1, \beta^2, \cdots, \beta^{l-1}\right)$. Remember that our goal is to find $\mathsf{ct}_\beta = \mathsf{RLWE}_{S,\sigma}(S^2 \cdot D_2)$ given known D_2, unknown S, and known $\mathsf{RLev}^{\beta,l}_{S,\sigma}(S^2) = \{\mathsf{RLWE}_{S,\sigma}\left(\beta^{i-1} \cdot S^2\right)\}_{i=1}^{l}$. Then, we can derive ct_β as follows:

$\mathsf{ct}_\beta = \mathsf{RLWE}_{S,\sigma}(S^2 \cdot D_2)$

$= \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot \left(D_{2,1}\beta^0 + D_{2,2}\beta^1 + \cdots D_{2,l}\beta^{l-1}\right)\right)$ ▷ by decomposing D_2

$= \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot D_{2,1} \cdot \beta^0\right) + \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot D_{2,2} \cdot \beta^1\right) + \cdots + \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot D_{2,l} \cdot \beta^{l-1}\right)$

$= D_{2,1} \cdot \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot \beta^0\right) + D_{2,2} \cdot \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot \beta^1\right) + \cdots + D_{2,l} \cdot \mathsf{RLWE}_{S,\sigma}\left(S^2 \cdot \beta^{l-1}\right)$ ▷ where each RLWE ciphertext is an encryption of $S^2\beta^0, S^2\beta^1, \cdots, S^2\beta^{l-1}$ as plaintext with the plaintext scaling factor $\Delta = 1$

$= \langle \mathsf{Decomp}^{\beta,l}(D_2),\ \mathsf{RLev}^{\beta,l}_{S,\sigma}(S^2)\rangle$ ▷ inner product of Decomp and RLev treating them as vectors

If we decrypt the above, we get the following:
$\mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}_\beta = \langle \mathsf{Decomp}^{\beta,l}(D_2),\ \mathsf{RLev}^{\beta,l}_{S,\sigma}(S^2)\rangle)$ $\triangleright$ the scaling factors of $\mathsf{RLev}^{\beta,l}_{S,\sigma}(S^2)$ are all 1
$= D_{2,1}\cdot(E'_1 + S^2\beta^0) + D_{2,2}\cdot(E'_2 + S^2\beta^1) + \cdots + D_{2,l}\cdot\left(E'_l + S^2\beta^{l-1}\right)$ # where each E'_i is a noise embedded in $\mathsf{RLWE}_{S,\sigma}\,(S^2\cdot\beta^{i-1})$

$= \sum_{i=1}^{l}(E'_i\cdot D_{2,i}) + S^2\cdot\left(D_{2,1}\beta^0 + D_{2,2}\beta^1 + \cdots + D_{2,l}\beta^{l-1}\right)$
$= \sum_{i=1}^{l}\epsilon_i + D_2\cdot S^2$ # where each $\epsilon_i = E'_i\cdot D_{2,i}$
$\equiv D_2S^2 + E_{\mathrm{rl}} \pmod{Q}$, with $\|E_{\mathrm{rl}}\|_\infty \le \sum_{i=1}^{l}\|D_{2,i}\|_1\|E'_i\|_\infty$. Small digits give a useful bound, not a strict comparison with every possible single-key noise realization.

Therefore, we get the following comprehensive relation:
$\mathsf{RLWE}^{-1}_{S,\sigma}(\Delta M^{\langle 1\rangle} + K_1q)\cdot\mathsf{RLWE}^{-1}_{S,\sigma}(\Delta M^{\langle 2\rangle} + K_2q) \bmod Q$
$= (\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1q)\cdot(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2q) \bmod Q$
$= (A^{\langle 1\rangle}\cdot S + B^{\langle 1\rangle})\cdot(A^{\langle 2\rangle}\cdot S + B^{\langle 2\rangle}) \bmod Q$
$= D_0 + D_1\cdot S + D_2\cdot S^2 \bmod Q$ $\triangleright$ $D_0 = B^{\langle 1\rangle}B^{\langle 2\rangle},\quad D_1 = A^{\langle 1\rangle}B^{\langle 2\rangle} + A^{\langle 2\rangle}B^{\langle 1\rangle},\quad D_2 = A^{\langle 1\rangle}A^{\langle 2\rangle}$
$= \mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}_\alpha) + \mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}_\beta) - \sum_{i=1}^{l}(E'_i\cdot D_{2,i}) \bmod Q$
$\triangleright$ $\mathsf{ct}_\alpha = (D_1, D_0) = (A_\alpha, B_\alpha),\quad \mathsf{ct}_\beta = \langle \mathsf{Decomp}^{\beta,l}(D_2), \mathsf{RLev}^{\beta,l}_{S,\sigma}(S^2)\rangle = (A_\beta, B_\beta)$

$= \mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}_\alpha + \mathsf{ct}_\beta) - \sum_{i=1}^{l}(E'_i\cdot D_{2,i}) \bmod Q$
$= \mathsf{RLWE}^{-1}_{S,\sigma}((A_{\alpha+\beta}, B_{\alpha+\beta})) - \sum_{i=1}^{l}(E'_i\cdot D_{2,i}) \bmod Q$ $\triangleright$ $A_{\alpha+\beta} = A_\alpha + A_\beta,\quad B_{\alpha+\beta} = B_\alpha + B_\beta$

From the above, we extract the following relation:
$(\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + K_1q)\cdot(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + K_2q) \bmod Q$
$= \Delta^2 M^{\langle 1\rangle}M^{\langle 2\rangle} + \Delta\cdot(M^{\langle 1\rangle}E^{\langle 2\rangle} + M^{\langle 2\rangle}E^{\langle 1\rangle}) + q\cdot(\Delta M^{\langle 1\rangle}K_2 + \Delta M^{\langle 2\rangle}K_1 + E^{\langle 1\rangle}K_2 + E^{\langle 2\rangle}K_1) + K_1K_2q^2 + E^{\langle 1\rangle}E^{\langle 2\rangle} \bmod Q$
$= \mathsf{RLWE}^{-1}_{S,\sigma}((A_{\alpha+\beta}, B_{\alpha+\beta})) - \sum_{i=1}^{l}(E'_i\cdot D_{2,i}) \bmod Q$

We can re-write the above relation as follows:
$\mathsf{RLWE}^{-1}_{S,\sigma}((A_{\alpha+\beta}, B_{\alpha+\beta})) = A_{\alpha+\beta}\cdot S + B_{\alpha+\beta} \bmod Q$
$= \Delta^2 M^{\langle 1\rangle}M^{\langle 2\rangle} + \Delta\cdot(M^{\langle 1\rangle}E^{\langle 2\rangle} + M^{\langle 2\rangle}E^{\langle 1\rangle}) + q\cdot(\Delta M^{\langle 1\rangle}K_2 + \Delta M^{\langle 2\rangle}K_1 + E^{\langle 1\rangle}K_2 + E^{\langle 2\rangle}K_1) + K_1K_2q^2 + E^{\langle 1\rangle}E^{\langle 2\rangle} + \sum_{i=1}^{l}(E'_i\cdot D_{2,i}) \bmod Q$

The expression contains the product message, the lift terms qK_i, the input-error products and the relinearization error. The next subsection follows all of these terms through the $1/\Delta$ rescaling of this teaching variant and applies the complete BFV decoder condition; approximate equality with a clean encryption is not assumed.

D-2.7.4 Rescaling

Let $(A_*, B_*) = (A_{\alpha+\beta}, B_{\alpha+\beta})$ be the relinearized ciphertext modulo $Q = q\Delta$, and choose integer coefficient representatives. Define

$$C_A = \left\lceil A_*/\Delta \right\rfloor, \quad C_B = \left\lceil B_*/\Delta \right\rfloor, \quad R_A = C_A - A_*/\Delta, \quad R_B = C_B - B_*/\Delta.$$

Each coefficient of R_A, R_B has absolute value at most $1/2$. Write $L_i = \Delta M^{\langle i \rangle} + E^{\langle i \rangle} + qK_i$ for the integer input phases, and retain E_{rl} from §D-2.7.3. Then

$$A_*S + B_* = L_1L_2 + E_{\mathrm{rl}} + QK_*, \qquad C_AS + C_B \equiv \frac{L_1L_2 + E_{\mathrm{rl}}}{\Delta} + R_AS + R_B \pmod{q}.$$

The equality uses $Q/\Delta = q$, so the discarded multiple of Q becomes an integral multiple of q. Set $r_q = q - t\Delta$. Expanding the product and removing integral multiples of q gives

$$C_AS + C_B \equiv \Delta M^{\langle 1 \rangle} M^{\langle 2 \rangle} + \eta \pmod{q},$$

$$\eta = M^{\langle 1 \rangle} E^{\langle 2 \rangle} + M^{\langle 2 \rangle} E^{\langle 1 \rangle} + \frac{q}{\Delta}(E^{\langle 1 \rangle} K_2 + E^{\langle 2 \rangle} K_1) + \frac{qr_q}{\Delta} K_1K_2 + \frac{E^{\langle 1 \rangle} E^{\langle 2 \rangle} + E_{\mathrm{rl}}}{\Delta} + R_AS + R_B.$$

In particular, $q^2/\Delta = qt + qr_q/\Delta$. The second term is not generally negligible and must not be discarded as an approximately zero error. The expression for η may use rational summands, but its complete value agrees with the integer output phase modulo q.

To decode, write $M^{\langle 1 \rangle} M^{\langle 2 \rangle} = M_{\mathrm{out}} + tH$ using the chosen centered plaintext representative. The effective output error is $E_{\mathrm{out}} = \eta - r_qH$. A sufficient correctness condition for the standard BFV decoder is

$$\|t(\eta - r_qH) - r_qM_{\mathrm{out}}\|_\infty < q/2.$$

When $t \mid q$, the r_q terms vanish, but message-noise products, evaluation-key error and component rounding remain.

Noise Analysis: All products above are negacyclic polynomial products. Use $\|UV\|_\infty \le \|U\|_1\|V\|_\infty \le n\|U\|_\infty\|V\|_\infty$, not a coefficientwise scalar bound. Also $\|R_AS + R_B\|_\infty \le (\|S\|_1 + 1)/2$. Bound each remaining summand of η with these inequalities and apply the triangle inequality before comparing to the final threshold. No unconditional successful-multiplication claim follows merely from $q \gg t$ or $Q = q\Delta$.

Modulus Switch v.s. Rescaling: In this particular construction, scaling components by $1/\Delta = q/Q$ changes the auxiliary modulus from Q back to q and reduces the message scale. The terms describe different goals and are not universally interchangeable. Standard FV multiplication can restore the message scale while retaining the external ciphertext modulus, by performing scale-and-round on unreduced integer products.

D-2.7.5 Summary

To put all things together, BFV's ciphertext-to-ciphertext multiplication is summarized as follows:

⟨Summary D-2.7.5⟩ BFV's Ciphertext-to-Ciphertext Multiplication

Suppose we have the following two RLWE ciphertexts:
$\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle}) = (A^{\langle 1\rangle}, B^{\langle 1\rangle}) \bmod q, \quad \text{where } B^{\langle 1\rangle} = -A^{\langle 1\rangle} \cdot S + \Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} \bmod q$
$\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle}) = (A^{\langle 2\rangle}, B^{\langle 2\rangle}) \bmod q, \quad \text{where } B^{\langle 2\rangle} = -A^{\langle 2\rangle} \cdot S + \Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} \bmod q$

Multiplication between these two ciphertexts is performed as follows:

1. **ModRaise**
   Forcibly raise the modulus of the ciphertexts $(A^{\langle 1\rangle}, B^{\langle 1\rangle}) \bmod q$ and $(A^{\langle 2\rangle}, B^{\langle 2\rangle}) \bmod q$ to Q (where $Q = q \cdot \Delta$) as follows:
   $(A^{\langle 1\rangle}, B^{\langle 1\rangle}) \bmod Q$
   $(A^{\langle 2\rangle}, B^{\langle 2\rangle}) \bmod Q$

2. **Multiplication**
   Compute the following polynomial multiplications in modulo Q:
   $D_0 = B^{\langle 1\rangle} B^{\langle 2\rangle} \bmod Q$
   $D_1 = B^{\langle 2\rangle} A^{\langle 1\rangle} + B^{\langle 1\rangle} A^{\langle 2\rangle} \bmod Q$
   $D_2 = A^{\langle 1\rangle} \cdot A^{\langle 2\rangle} \bmod Q$

3. **Relinearization**
   Compute the following:
   $\mathsf{ct}_\alpha = (D_1, D_0)$
   $\mathsf{ct}_\beta = \langle \mathsf{Decomp}^{\beta,l}(D_2),\ \mathsf{RLev}^{\beta,l}_{S,\sigma}(S^2) \rangle$.
   $\mathsf{ct}_{\alpha+\beta} = \mathsf{ct}_\alpha + \mathsf{ct}_\beta$

   Then, the following property holds:
   $\phi_S(\mathsf{ct}_{\alpha+\beta}) \equiv (\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle} + qK_1)(\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle} + qK_2) + E_{\mathrm{rl}} \pmod Q$.

4. **Rescaling**
   Update $\mathsf{ct}_{\alpha+\beta} = (A_{\alpha+\beta}, B_{\alpha+\beta}) \bmod Q$ to $\mathsf{ct'}_{\alpha+\beta} = \left(\left\lceil \frac{A_{\alpha+\beta}}{\Delta} \right\rfloor, \left\lceil \frac{B_{\alpha+\beta}}{\Delta} \right\rfloor \right) \bmod q$.

   This plaintext rescaling process can be also viewed as a modulus switch of the ciphertext $\mathsf{ct}_{\alpha+\beta}$ from $Q \to q$.

Note that after the ciphertext-to-ciphertext multiplication, the plaintext scaling factor $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$, the ciphertext modulus q, and the private key S stay the same as before.

The Purpose of ModRaise: Reducing a tensor product modulo q before scaling can destroy information needed for scale-and-round. This teaching variant instead computes modulo $Q = q\Delta$ because a discarded Q multiple becomes a q multiple after division by Δ. It does not require every plaintext product to lie below Q; that claim would in general fail for polynomial convolution and large plaintexts. The lifting terms qK_i and the complete error derived in §D-2.7.4 must still be included.

Swapping the Order of Relinearization and Rescaling: Both orders can be designed, but their evaluation keys, moduli and rounding-error bounds differ. They are not a literal swap using the same keys and error proof. Standard FV commonly scales the degree-two integer product before relinearizing at the target modulus.

D-2.7.6 Application to TFHE's Ciphertext-to-Ciphertext Multiplication

This subsection deliberately uses the alternative LWE phase $b+\langle\vec{a},\vec{s}\rangle$. To apply it to the TFHE convention $b-\langle\vec{a},\vec{s}\rangle$ of §D-1.1, first negate the public mask and negate it back on output. The tensor-product construction is a leveled LWE multiplication alternative, not the external product used by TFHE bootstrapping. It needs evaluation keys for the products $s_i s_j$, appropriate parameters and the full rescaling error bounds; it does not make arbitrary LWE multiplication noise-free.

In §D-1.6, the external product takes an LWE ciphertext and a GSW ciphertext. Obtaining the latter from an arbitrary LWE ciphertext may require circuit bootstrapping. The tensor construction below instead requires a different, quadratic-size evaluation key and a rescaling budget. This establishes an alternative operation, without asserting a performance advantage over TFHE's external product or programmable bootstrapping. We impose $t \mid q$ and $\Delta = q/t$, as in the TFHE encoding of this book. Given two LWE ciphertexts, their integer lifts satisfy:

$$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m^{\langle 1\rangle}+e^{\langle 1\rangle})=(\vec{a}^{\langle 1\rangle},b^{\langle 1\rangle}) \Rightarrow \ \vec{a}^{\langle 1\rangle}\cdot\vec{s}+b^{\langle 1\rangle}=\Delta m^{\langle 1\rangle}+e^{\langle 1\rangle}+k_1 q$$
$$\mathsf{LWE}_{\vec{s},\sigma}(\Delta m^{\langle 2\rangle}+e^{\langle 2\rangle})=(\vec{a}^{\langle 2\rangle},b^{\langle 2\rangle}) \Rightarrow \ \vec{a}^{\langle 2\rangle}\cdot\vec{s}+b^{\langle 2\rangle}=\Delta m^{\langle 2\rangle}+e^{\langle 2\rangle}+k_2 q$$

Multiplying these two equations yields the following relation:

$$\begin{aligned}
&(\Delta m^{\langle 1\rangle}+e^{\langle 1\rangle}+k_1 q)\cdot(\Delta m^{\langle 2\rangle}+e^{\langle 2\rangle}+k_2 q)\\
&=(\vec{a}^{\langle 1\rangle}\cdot\vec{s}+b^{\langle 1\rangle})\cdot(\vec{a}^{\langle 2\rangle}\cdot\vec{s}+b^{\langle 2\rangle})\\
&=b^{\langle 1\rangle}b^{\langle 2\rangle}+(b^{\langle 2\rangle}\vec{a}^{\langle 1\rangle}+b^{\langle 1\rangle}\vec{a}^{\langle 2\rangle})\cdot\vec{s}+(\vec{a}^{\langle 1\rangle}\cdot\vec{s})\cdot(\vec{a}^{\langle 2\rangle}\cdot\vec{s})\\
&=\underbrace{b^{\langle 1\rangle}b^{\langle 2\rangle}}_{d_0}+\underbrace{(b^{\langle 2\rangle}\vec{a}^{\langle 1\rangle}+b^{\langle 1\rangle}\vec{a}^{\langle 2\rangle})}_{d_1}\cdot\vec{s}+\underbrace{(\vec{a}^{\langle 1\rangle}\otimes\vec{a}^{\langle 2\rangle})}_{\vec{d}_2}\cdot(\vec{s}\otimes\vec{s})\\
&=d_0+d_1\cdot\vec{s}+\vec{d}_2\cdot\vec{s}^{\otimes} \ \triangleright \text{ where } \vec{s}^{\otimes}=\vec{s}\otimes\vec{s}
\end{aligned}$$

, where $\otimes$ denotes an outer product of two vectors. For example, given two n-length vectors $\vec{v}$ and $\vec{u}$, $\vec{v}\otimes\vec{u}$ is equivalent to an n^2-length vector that concatenates the following n distinct n-length vectors: $v_0\cdot\vec{u},\ v_1\cdot\vec{u},\ \cdots v_{n-1}\cdot\vec{u}$. Notice that the above relation is similar to that we derived in BFV's ciphertext-to-ciphertext multiplication. Therefore, similar to the remaining steps in BFV, we can relinearize and rescale this LWE term as follows:

⟨Summary D-2.7.6⟩ Leveled LWE Tensor Multiplication

Suppose we have the following two LWE ciphertexts:

$$\mathsf{LWE}_{\vec{s},\sigma}(\Delta M^{\langle 1\rangle})=(\vec{a}^{\langle 1\rangle},b^{\langle 1\rangle}) \bmod q, \quad \text{where } b^{\langle 1\rangle}=-\vec{a}^{\langle 1\rangle}\cdot\vec{s}+\Delta m^{\langle 1\rangle}+e^{\langle 1\rangle} \bmod q$$
$$\mathsf{LWE}_{\vec{s},\sigma}(\Delta M^{\langle 2\rangle})=(\vec{a}^{\langle 2\rangle},b^{\langle 2\rangle}) \bmod q, \quad \text{where } b^{\langle 2\rangle}=-\vec{a}^{\langle 2\rangle}\cdot\vec{s}+\Delta m^{\langle 2\rangle}+e^{\langle 2\rangle} \bmod q$$

Multiplication between these two LWE ciphertexts is performed as follows:

1. **ModRaise**
   Interpret the chosen integer component lifts of $(\vec{a}^{\langle 1\rangle},b^{\langle 1\rangle})$ and $(\vec{a}^{\langle 2\rangle},b^{\langle 2\rangle})$ modulo $Q=q\Delta$.

Their phases still include the terms qk_1 and qk_2; this is not fresh encryption of the same scaled messages at the larger modulus:
$(\vec{a}^{\langle 1\rangle}, b^{\langle 1\rangle}) \bmod Q$
$(\vec{a}^{\langle 2\rangle}, b^{\langle 2\rangle}) \bmod Q$

2. **Multiplication**
Compute the following polynomial multiplications in modulo Q:
$d_0 = b^{\langle 1\rangle} b^{\langle 2\rangle} \bmod Q$
$d_1 = b^{\langle 2\rangle} \vec{a}^{\langle 1\rangle} + b^{\langle 1\rangle} \vec{a}^{\langle 2\rangle} \bmod Q$
$\vec{d_2} = \vec{a}^{\langle 1\rangle} \otimes \vec{a}^{\langle 2\rangle} \bmod Q$

3. **Relinearization**
Compute the following:
$\mathsf{ct}_\alpha = (d_1, d_0)$
$\mathsf{ct}_\beta = \langle \mathsf{Decomp}^{\beta,l}(\vec{d_2}),\ \mathsf{Lev}^{\beta,l}_{\vec{s},\sigma}(\vec{s}^{\otimes})\rangle$.
$\mathsf{ct}_{\alpha+\beta} = \mathsf{ct}_\alpha + \mathsf{ct}_\beta$

Then, the following property holds:
$\phi_+(\mathsf{ct}_{\alpha+\beta}) \equiv (\Delta m^{\langle 1\rangle} + e^{\langle 1\rangle} + qk_1)(\Delta m^{\langle 2\rangle} + e^{\langle 2\rangle} + qk_2) + e_{\mathrm{rl}} \pmod Q$, where $\phi_+(\vec{a}, b) = b + \langle \vec{a}, \vec{s}\rangle$.

4. **Rescaling**
Update $\mathsf{ct}_{\alpha+\beta} = (\vec{a}_{\alpha+\beta}, b_{\alpha+\beta}) \bmod Q$ to $\mathsf{ct'}_{\alpha+\beta} = \left(\left\lceil \frac{\vec{a}_{\alpha+\beta}}{\Delta} \right\rfloor, \left\lceil \frac{b_{\alpha+\beta}}{\Delta} \right\rfloor\right) \bmod q$.

This plaintext rescaling process can be also viewed as a modulus switch of the ciphertext $\mathsf{ct}_{\alpha+\beta}$ from $Q \to q$.

Note that after the ciphertext-to-ciphertext multiplication, the plaintext scaling factor $\Delta = \frac{q}{t}$, the ciphertext modulus q, and the private key $\vec{s}$ stay the same as before.

Evaluation key and conditional correctness. Let the LWE dimension be k, choose integer gadget weights $g_j = \beta^j$ for $0 \le j < L$, and require $\beta^L \ge Q$. For every ordered pair (u, v), publish rows with phase $g_j s_u s_v + e^{\mathrm{key}}_{u,v,j} \pmod Q$. Decompose each component of $\vec{d_2}$ exactly modulo Q into digits bounded by D_g. If $|e^{\mathrm{key}}_{u,v,j}| \le B_{\mathrm{key}}$, the resulting relinearization error satisfies

$$|e_{\mathrm{rl}}| \le k^2 L D_g B_{\mathrm{key}}.$$

This key is distinct from TFHE's GGSW bootstrapping key. Use centered input components and messages, put $h = \|\vec{s}\|_1$, and suppose $|e_i| \le B_i$. Then the integer phase carries obey

$$|k_i| \le \kappa_i := \frac{h+2}{2} + \frac{B_i}{q}.$$

If R_a, R_b are the component rounding errors in the final division by Δ, direct expansion gives

$$\phi_+(\mathsf{ct}_{\text{out}}) \equiv \Delta m_1 m_2 + \eta \pmod{q},$$
$$\eta = m_1 e_2 + m_2 e_1 + t(e_1 k_2 + e_2 k_1) + \frac{e_1 e_2 + e_{\text{rl}}}{\Delta} + \langle R_a, \vec{s} \rangle + R_b.$$

The terms $qk_1 m_2$, $qk_2 m_1$ and $qtk_1 k_2$ vanish modulo q. In particular, the dangerous modulus-remainder term of the general BFV teaching variant vanishes because $t \mid q$. A sufficient decoder condition is

$$\frac{t}{2}(B_1 + B_2) + t(B_1\kappa_2 + B_2\kappa_1) + \frac{B_1 B_2 + k^2 L D_g B_{\text{key}}}{\Delta} + \frac{h+1}{2} < \frac{\Delta}{2}.$$

Message reduction modulo t adds a multiple of q and no further error. This proves the displayed order conditionally. If one instead scales the integer tensor first by $1/\Delta = t/q$ and then relinearizes modulo q, the key modulus is q, the key-error term is no longer divided by Δ, and tensor rounding is bounded by $(1 + h + h^2)/2$. These are two separately justified orders, not interchangeable keys. The accompanying exact checks cover both orders with nonzero input and evaluation-key errors; they do not establish secure parameter sizes or a speed improvement.

D-2.8 Homomorphic Key Switching

The gadget and its encrypted rows must use the same convention. For a general modulus q, one may use integer weights β^{i-1} and enough digits for exact decomposition; a truncated decomposition adds its residual multiplied by the old secret to the key-switching error.

BFV's key switching scheme changes an RLWE ciphertext's secret key from S to S'. This scheme is essentially RLWE's key switching scheme with the sign of the $A \cdot S$ term flipped in the encryption and decryption formula. Specifically, this is equivalent to the alternative GLWE version's (§B-4.4) key switching scheme (§C-5) with $k = 1$ as follows:

⟨Summary D-2.8⟩ BFV's Key Switching

$$\mathsf{RLWE}_{S',\sigma}(\Delta M) = (0, B) + \langle \mathsf{Decomp}^{\beta,l}(A),\ \mathsf{RLev}^{\beta,l}_{S',\sigma}(S) \rangle$$

D-2.9 Homomorphic Rotation of Input Vector Slots

In this section, we will explain how to homomorphically rotate the elements of an input vector $\vec{v}$ after it is already encoded as a polynomial and encrypted as an RLWE ciphertext. In §A-5.2, we learned how to rotate the coefficients of a polynomial. However, rotating the plaintext polynomial $M(X)$ or RLWE ciphertext polynomials $(A(X), B(X))$ does not necessarily rotate the input vector, which is the source of them.

The key requirement of homomorphic rotation of input vector slots (i.e., input vector) is that this operation should be performed on the RLWE ciphertext such that after this operation, if we decrypt the RLWE ciphertext and decode it, the recovered input vector will be in a rotated state as we expect. We will divide this task into the following two sub-problems:

1. How to indirectly rotate the input vector by updating the plaintext polynomial M to M'?
2. How to indirectly update the plaintext polynomial M to M' by updating the RLWE ciphertext polynomials (A, B) to (A', B')?

D-2.9.1 Rotating Input Vector Slots by Updating the Plaintext Polynomial

In this task, our goal is to modify the plaintext polynomial $M(X)$ such that the first-half elements of the input vector $\vec{v}$ are shifted to the left by h positions in a wrapping manner among them, and the second-half elements of $\vec{v}$ are also shifted to the left by h positions in a wrapping manner among them. Specifically, if $\vec{v}$ is defined as follows:

$\vec{v} = (v_0, v_1, \cdots, v_{n-1})$

Then, we will denote the h-shifted vector $\vec{v}^{\langle h \rangle}$ as follows:

$$\vec{v}^{\langle h \rangle} = (\underbrace{v_h, v_{h+1}, \cdots, v_0, v_1, \cdots, v_{h-2}, v_{h-1}}_{\text{The first-half } \frac{n}{2} \text{ elements } h\text{-rotated to the left}}, \underbrace{v_{\frac{n}{2}+h}, v_{\frac{n}{2}+h+1}, \cdots, v_{\frac{n}{2}+h-2}, v_{\frac{n}{2}+h-1}}_{\text{The second-half } \frac{n}{2} \text{ elements } h\text{-rotated to the left}})$$

Remember from §D-2.2 that the BFV encoding scheme's components are as follows:

$\vec{v} = (v_0, v_1, v_2, \cdots, v_{n-1})$ ▷ n-slot input vector

$$W = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ (\omega) & (\omega^3) & (\omega^5) & \cdots & (\omega^{2n-1}) \\ (\omega)^2 & (\omega^3)^2 & (\omega^5)^2 & \cdots & (\omega^{2n-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega)^{n-1} & (\omega^3)^{n-1} & (\omega^5)^{n-1} & \cdots & (\omega^{2n-1})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & \cdots & 1 & 1 \\ (\omega) & (\omega^3) & \cdots & (\omega^{n-1}) & (\omega^{-(n-1)}) & \cdots & (\omega^{-3}) & (\omega^{-1}) \\ (\omega)^2 & (\omega^3)^2 & \cdots & (\omega^{n-1})^2 & (\omega^{-(n-1)})^2 & \cdots & (\omega^{-3})^2 & (\omega^{-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ (\omega)^{n-1} & (\omega^3)^{n-1} & \cdots & (\omega^{n-1})^{n-1} & (\omega^{-(n-1)})^{n-1} & \cdots & (\omega^{-3})^{n-1} & (\omega^{-1})^{n-1} \end{bmatrix}$$

, where $\omega = g^{\frac{t-1}{2n}} \bmod t$ (g is a generator of $\mathbb{Z}_t^\times$)

▷ The encoding matrix that converts $\vec{v}$ into $\vec{m}$ (i.e., n coefficients of the plaintext polynomial $M(X)$)

$\Delta\vec{m} = n^{-1} \cdot \Delta W \cdot I_n^R \cdot \vec{v}$

▷ A vector containing the scaled n integer coefficients of the plaintext polynomial

$\Delta M = \sum_{i=0}^{n-1} (\Delta m_i X^i)$

▷ The integer polynomial that isomorphically encodes the input vector $\vec{v}$

We learned from §A-11.6 that decoding the polynomial $M(X)$ $\vec{v}$ is equivalent to evaluating $M(X)$ at the following n distinct primitive ($\mu = 2n$)-th root of unity: $\{\omega, \omega^3, \omega^5, \cdots, \omega^{2n-3}, \omega^{2n-1}\}$. Thus, the above decoding process is equivalent to the following:

$$\vec{v} = \frac{W^T \Delta\vec{m}}{\Delta} = (W_0^T \cdot \vec{m},\ W_1^T \cdot \vec{m},\ W_2^T \cdot \vec{m},\ \cdots,\ W_{n-1}^T \cdot \vec{m})$$
$$= (\ M(\omega),\ M(\omega^3),\ M(\omega^5), \cdots, M(\omega^{2n-3}),\ M(\omega^{2n-1})\ )$$
$$= (\ M(\omega),\ M(\omega^3),\ M(\omega^5), \cdots, M(\omega^{n-3}),\ M(\omega^{n-1}),\ M(\omega^{-(n-1)}),\ M(\omega^{-(n-3)}), \cdots, M(\omega^{-3}),\ M(\omega^{-1})\ )$$

Now, our next task is to modify $M(X)$ to $M'(X)$ such that decoding $M'(X)$ will give us a modified input vector $\vec{v}^{\langle h \rangle}$ that is a rotation of the first half elements of $\vec{v}$ by h positions to the left (in a wrapping

manner among them), and the second half elements of it also rotated by h positions to the left (in a wrapping manner among them). To accomplish this rotation, we will take a 2-step solution:

1. To convert $M(X)$ into $M'(X)$, we will define the new mapping σ_M as follows:
   $\sigma_M : (M(X), h) \in (\mathcal{R}_{\langle n,t \rangle}, \mathbb{Z}_n) \longrightarrow M'(X) \in \mathcal{R}_{\langle n,t \rangle}$
   , where h is the number of rotation positions to be applied to $\vec{v}$.
2. To decode $M'(X)$ into the rotated input vector $\vec{v}^{\langle h \rangle}$, we need to re-design our decoding scheme by modifying $\mathsf{Encoding}_1$'s (§D-2.2.1) isomorphic mapping $\sigma : M(X) \in \mathcal{R}_{\langle n,t \rangle} \longrightarrow \vec{v} \in \mathbb{Z}^n$

Converting $M(X)$ into $M'(X)$: Our first task is to convert $M(X)$ into $M'(X)$, which is equivalent to applying our new mapping $\sigma_M : (M(X), h) \in (\mathcal{R}_{\langle n,t \rangle}, \mathbb{Z}_n) \longrightarrow M'(X) \in \mathcal{R}_{\langle n,t \rangle}$, such that decoding $M'(X)$ gives a rotated input vector $\vec{v}^{\langle h \rangle}$ whose first half of the elements in $\vec{v}$ are rotated by h positions to the left (in a wrapping manner among them), and the second half of the elements are also rotated by h positions to the left (in a wrapping manner among them). To design σ_M that satisfies this requirement, we will use the number 5^j which has the following two special properties (based on number theory):

- $(5^j \bmod 2n)$ and $(-5^j \bmod 2n)$ generate all odd numbers between $[0, 2n)$ for the integer j where $0 \leq j < \frac{n}{2}$.
- For each integer j where $0 \leq j < \frac{n}{2}$, $(5^j \bmod 2n) + (-5^j \bmod 2n) \equiv 0 \bmod 2n$.

For example, suppose the modulus $2n = 16$. Then,

$5^0 \bmod 16 = 1$	$-(5)^0 \bmod 16 = 15$
$5^1 \bmod 16 = 5$	$-(5)^1 \bmod 16 = 11$
$5^2 \bmod 16 = 9$	$-(5)^2 \bmod 16 = 7$
$5^3 \bmod 16 = 13$	$-(5)^3 \bmod 16 = 3$

As shown above, $0 \leq j < 4$ generate all odd numbers between $[0, 16)$. Also, for each j in $0 \leq j < 4$, $(5^j \bmod 16) + (-5^j \bmod 16) = 16$.

Let's define $J(h) = 5^h \bmod 2n$, and $J_*(h) = -5^h \bmod 2n$. Based on $J(h)$ and $J_*(h)$, we will define the mapping $\sigma_M : M(X) \to M'(X)$ as follows:
$\sigma_M : M(X) \to M(X^{J(h)})$

Given a plaintext polynomial $M(X)$, in order to give its decoded version of input vector $\vec{v}$ the effect of the first half of the elements being rotated by h positions to the left (in a wrapping manner) and the second half of the elements also being rotated by h positions to the left (in a wrapping manner), we update the current plaintext polynomial $M(X)$ to a new polynomial $M'(X) = M(X^{J(h)}) = M(X^{5^h})$ by applying the σ_M mapping, where h is the number of positions for left rotations for the first half and second half of the elements of $\vec{v}$.

Decoding $M'(X)$ into $\vec{v}^{\langle h \rangle}$: Our second task is to modify our original decoding scheme in order to successfully decode $M'(X)$ into the rotated input vector $\vec{v}^{\langle h \rangle}$. For this, we will modify our original isomorphic mapping $\sigma : M(X) \longrightarrow \vec{v}$, from:
$\sigma : M(X) \in \mathcal{R}_{\langle n,q \rangle} \longrightarrow \big(M(\omega), M(\omega^3), M(\omega^5), \cdots, M(\omega^{2n-1})\big) \in \mathbb{Z}^n$ # designed in §A-11.6

, to the following:

$$\sigma_J : M(X) \in \mathcal{R}_{\langle n,t \rangle} \longrightarrow (M(\omega^{J(0)}), M(\omega^{J(1)}), M(\omega^{J(2)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),$$
$$M(\omega^{J_*(0)}),\ M(\omega^{J_*(1)}),\ M(\omega^{J_*(2)}), \cdots, M(\omega^{J_*(\frac{n}{2}-1)})\ ) \in \mathbb{Z}^n$$

The common aspect between σ and σ_J is that they both evaluate the polynomial $M(X)$ at n distinct primitive $(\mu = 2n)$-th roots of unity (i.e., w^i for all odd i between $[0, 2n]$). In the case of the σ_J mapping, note that $J(j) = 5^j \bmod 2n$ and $J_*(j) = -5^j \bmod 2n$ for each j in $0 \leq j < \dfrac{n}{2}$ cover all odd numbers between $[0, 2n]$. Therefore, $\omega^{J(j)}$ and $\omega^{J_*(j)}$ between $0 \leq j < \frac{n}{2}$ cover all n distinct primitive $(\mu = 2n)$-th roots of unity.

Meanwhile, the difference between σ and σ_J is the order of the output vector elements. In the σ mapping, the order of evaluated coordinates for $M(X)$ is $\omega, \omega^3, \cdots, \omega^{2n-1}$, whereas in the σ_J mapping, the order of evaluated coordinates is $\omega^{J(0)}, \omega^{J(1)}, \cdots, \omega^{J(\frac{n}{2}-1)}, \omega^{J_*(0)}, \omega^{J_*(1)}, \cdots, \omega^{J_*(\frac{n}{2}-1)}$. We will later explain why we modified the ordering like this.

In the original $\mathsf{Decoding}_2$ process (§D-2.2), applying the σ mapping to a plaintext polynomial $M(X)$ was equivalent to computing the following:

$\vec{v'} = (\ M(\omega),\ M(\omega^3),\ M(\omega^5), \cdots, M(\omega^{2n-3}),\ M(\omega^{2n-1}))$

$= (\ M(\omega),\ M(\omega^3),\ M(\omega^5), \cdots, M(\omega^{n-1}),\ M(\omega^{-(n-1)}), \cdots,\ M(\omega^{-3}),\ M(\omega^{-1})\ )$

$= (\ W_0^T \cdot \vec{m},\ W_1^T \cdot \vec{m},\ W_2^T \cdot \vec{m},\ \cdots,\ W_{n-1}^T \cdot \vec{m}\ )$ ▷ where W_i^T is the $(i+1)$-th row of W^T

$= W^T \vec{m}$

Similarly, the modified σ_J mapping to the plaintext polynomial $M(X)$ is equivalent to computing the following:

$\vec{v'} = (\ M(\omega^{J(0)}),\ M(\omega^{J(1)}),\ M(\omega^{J(2)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),\ M(\omega^{J_*(0)}),\ M(\omega^{J_*(1)}), \cdots, M(\omega^{J_*(\frac{n}{2}-1)})\ )$

$= (\ \tilde{W}_0^* \cdot \vec{m},\ \tilde{W}_1^* \cdot \vec{m},\ \tilde{W}_2^* \cdot \vec{m},\ \cdots,\ \tilde{W}_{n-1}^* \cdot \vec{m}\ )$

$= \tilde{W}^* \vec{m}$, where

$$\tilde{W}^* = \begin{bmatrix}
1 & (\omega^{J(0)}) & (\omega^{J(0)})^2 & \cdots & (\omega^{J(0)})^{n-1} \\
1 & (\omega^{J(1)}) & (\omega^{J(1)})^2 & \cdots & (\omega^{J(1)})^{n-1} \\
1 & (\omega^{J(2)}) & (\omega^{J(2)})^2 & \cdots & (\omega^{J(2)})^{n-1} \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
1 & (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J(\frac{n}{2}-1)})^{n-1} \\
1 & (\omega^{J_*(0)}) & (\omega^{J_*(0)})^2 & \cdots & (\omega^{J_*(0)})^{n-1} \\
1 & (\omega^{J_*(1)}) & (\omega^{J_*(1)})^2 & \cdots & (\omega^{J_*(1)})^{n-1} \\
1 & (\omega^{J_*(2)}) & (\omega^{J_*(2)})^2 & \cdots & (\omega^{J_*(2)})^{n-1} \\
\vdots & \vdots & \vdots & \ddots & \vdots \\
1 & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J_*(\frac{n}{2}-1)})^{n-1}
\end{bmatrix}$$

$$\tilde{W} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-2)}) & \cdots & (\omega^{J(0)}) & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-2)}) & \cdots & (\omega^{J_*(0)}) \\ (\omega^{J(\frac{n}{2}-1)})^2 & (\omega^{J(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J(0)})^2 & (\omega^{J_*(\frac{n}{2}-1)})^2 & (\omega^{J_*(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J_*(0)})^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\ (\omega^{J(\frac{n}{2}-1)})^{n-1} & (\omega^{J(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J(0)})^{n-1} & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} & (\omega^{J_*(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J_*(0)})^{n-1} \end{bmatrix}$$

From this point, we will replace W in the $\mathsf{Encoding}_1$ process (§D-2.2) by $\tilde{W}$, and W^T in the $\mathsf{Decoding}_2$ process by $\tilde{W}^*$.

To demonstrate that $\tilde{W}$ is a valid encoding matrix like W and $\tilde{W}^*$ is a valid decoding matrix like W^T, we need to prove the following 2 aspects:

- $\boldsymbol{\tilde{W}}$ **is a basis of the** $\boldsymbol{n}$**-dimensional vector space:** This is true, because $\tilde{W}$ is simply a column-wise re-ordering of W, which is still a basis of the $\boldsymbol{n}$-dimensional vector space.
- $\boldsymbol{\tilde{W}^* \cdot \tilde{W} = n \cdot I_n^R}$ (to satisfy Theorem A-12.4 in §A-12.4): This proof is split into 2 sub-proofs:
  1. $\boldsymbol{\tilde{W}^* \cdot \tilde{W}}$ has value $\boldsymbol{n}$ along the anti-diagonal line: Each element along the anti-diagonal line of $\boldsymbol{\tilde{W}^* \cdot \tilde{W}}$ is computed as $\boldsymbol{\sum_{i=0}^{n-1} \omega^{2nik} = n}$ where $\boldsymbol{k}$ is some integer.
  2. $\boldsymbol{\tilde{W}^* \cdot \tilde{W}}$ has value 0 at all other coordinates: All other elements except for the ones along the anti-diagonal lines are $\boldsymbol{\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1} = 0 \quad (r = \omega^{a+b} \neq 1)}$ (by Geometric Sum).

  We provide the Python script that empirically demonstrates this.

Therefore, $\tilde{W}^*$ and $\tilde{W}$ are valid encoding & decoding matrices that transform $\vec{v}$ into $M(X)$.

Now, let's think about what will be the structure of $\vec{v}^{\langle h \rangle}$ (i.e., the first-half elements of $\vec{v}$ being rotated h positions to the left in a wrapping manner among them and the second-half elements of it also being rotated h positions to the left in a wrapping manner among them). Remember that $\vec{v}$ is as follows:

$$\begin{aligned} \vec{v} &= (\ M(\omega^{J(0)}),\ M(\omega^{J(1)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),\ M(\omega^{J_*(0)}),\ M(\omega^{J_*(1)}), \cdots, M(\omega^{J_*(\frac{n}{2}-1)})) \\ &= (\ \tilde{W}_0^* \cdot \vec{m},\ \tilde{W}_1^* \cdot \vec{m},\ \tilde{W}_2^* \cdot \vec{m},\ \cdots,\ \tilde{W}_{n-1}^* \cdot \vec{m}\ ) \end{aligned}$$

Thus, the state of $\vec{v}^{\langle h \rangle}$ which is equivalent to rotating $\vec{v}$ by h positions to the left for the first-half and second-half element groups will be the following:

$$\begin{aligned} \vec{v}^{\langle h \rangle} = (\ & M(\omega^{J(h)}),\ M(\omega^{J(h+1)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),\ M(\omega^{J(0)}),\ M(\omega^{J(1)}), \cdots,\ M(\omega^{J(h-2)}),\ M(\omega^{J(h-1)}), \\ & M(\omega^{J_*(h)}),\ M(\omega^{J_*(h+1)}), \cdots, M(\omega^{J_*(\frac{n}{2}-1)}),\ M(\omega^{J_*(0)}),\ M(\omega^{J_*(1)}), \cdots,\ M(\omega^{J_*(h-2)}),\ M(\omega^{J_*(h-1)})\ ) \end{aligned}$$

Notice that the above computation of $\vec{v}^{\langle h \rangle}$ is equivalent to vertically rotating the upper $\frac{n}{2}$ rows of $\tilde{W}^*$ by h positions upward (in a wrapping manner among them), rotating the lower $\frac{n}{2}$ rows of $\tilde{W}^*$ by h positions upward (in a wrapping manner among them), and multiplying the resulting matrix with $\vec{m}$. However, it is not desirable to directly modify the decoding matrix $\tilde{W}^*$ like this in practice, because then the decoding matrix loses its consistency. Therefore, instead of directly modifying $\tilde{W}^*$, we will modify $\vec{m}$ to $\vec{m}'$ (i.e., modify $M(X)$ to $M'(X)$) such that the relation $\vec{v}^{\langle h \rangle} = \tilde{W}^* \cdot \vec{m}'$ holds. Let's extract the upper-half rows of $\tilde{W}^*$ and denote this $\frac{n}{2} \times n$ matrix as $\tilde{H}_1^*$. Then, $\tilde{H}_1^*$ is equivalent to a Vandermonde matrix (Definition A-11.2 in §A-11.2) in the form of $V(\omega^{J(0)}, \omega^{J(1)}, \cdots, \omega^{J(\frac{n}{2}-1)})$. Similarly, let's extract the lower-half rows of $\tilde{W}^*$ and denote this $\frac{n}{2} \times n$ matrix as $\tilde{H}_2^*$. Then, $\tilde{H}_2^*$ is equivalent to a Vandermonde

matrix (Definition A-11.2 in §A-11.2) in the form of $V(\omega^{J_*(0)}, \omega^{J_*(1)}, \cdots, \omega^{J_*(\frac{n}{2}-1)})$.

Now, let's vertically rotate the rows of $\tilde{H}_1^*$ by h positions upward and denote it as $\tilde{H}_1^{*\langle h\rangle}$; and vertically rotate the rows of $\tilde{H}_2^*$ by h positions upward and denote it as $\tilde{H}_2^{*\langle h\rangle}$. And let's denote the $\frac{n}{2}$-dimensional vector comprising the first-half elements of $\vec{v}^{\langle h\rangle}$ as $\vec{v}_1^{\langle h\rangle}$, and the $\frac{n}{2}$-dimensional vector comprising the second-half elements of $\vec{v}^{\langle h\rangle}$ as $\vec{v}_2^{\langle h\rangle}$. Then, computing (i.e., decoding) $\vec{v}_1^{\langle h\rangle} = \tilde{H}_1^{*\langle h\rangle} \cdot \vec{m}$ is equivalent to modifying $M(X)$ to $M'(X) = M(X^{J(h)})$ (whose coefficient vector is $\vec{m}^{\langle h\rangle}$) and then computing (i.e., decoding) $\vec{v}_1^{\langle h\rangle} = \tilde{H}_1^* \cdot \vec{m}^{\langle h\rangle}$. This is because:

$\vec{v}_1^{\langle h\rangle} = \tilde{H}_1^{*\langle h\rangle} \cdot \vec{m}$

$= (\ \tilde{W}_h^* \cdot \vec{m},\ \tilde{W}_{h+1}^* \cdot \vec{m},\ \tilde{W}_{h+2}^* \cdot \vec{m}, \cdots, \tilde{W}_{\frac{n}{2}-1}^* \cdot \vec{m},\ \tilde{W}_0^* \cdot \vec{m},\ \tilde{W}_1^* \cdot \vec{m}, \cdots,\ \tilde{W}_{h-2}^* \cdot \vec{m},\ \tilde{W}_{h-1}^* \cdot \vec{m})$

$= (\ M((\omega^{J(h)})^{J(0)}),\ M((\omega^{J(h)})^{J(1)}),\ M((\omega^{J(h)})^{J(2)}), \cdots, M((\omega^{J(h)})^{J(\frac{n}{2}-1)})\ )$

▷ This is equivalent to evaluating the polynomial $M(X^{J(h)})$ at the following $\frac{n}{2}$ distinct $(\mu = 2n)$-th roots of unity: $\omega^{J(0)}, \omega^{J(1)}, \omega^{J(2)}, \cdots, \omega^{J(\frac{n}{2}-1)}$

$= (\ M(\omega^{J(h)\cdot J(0)}),\ M(\omega^{J(h)\cdot J(1)}),\ M(\omega^{J(h)\cdot J(2)}), \cdots, M(\omega^{J(h)\cdot J(\frac{n}{2}-1)})\ )$

$= (\ M(\omega^{5^h \cdot 5^0}),\ M(\omega^{5^h \cdot 5^1}),\ M(\omega^{5^h \cdot 5^2}), \cdots, M(\omega^{5^h \cdot 5^{n/2-1}})\ )$

$= (\ M(\omega^{5^h}),\ M(\omega^{5^{h+1}}),\ M(\omega^{5^{h+2}}), \cdots, M(\omega^{5^{h+n/2-1}})\ )$ ▷ note that $5^{\frac{n}{2}} \bmod 2n = 1$

$= (\ M(\omega^{J(h)}),\ M(\omega^{J(h+1)}),\ M(\omega^{J(h+2)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),\ M(\omega^{J(0)}),\ M(\omega^{J(1)}),$
$\cdots,\ M(\omega^{J(h-2)}),\ M(\omega^{J(h-1)}))$

$= \tilde{H}_1^* \cdot \vec{m}^{\langle h\rangle}$ # where $\vec{m}^{\langle h\rangle}$ contains the n coefficients of the polynomial $M(X^{J(h)})$

Similarly, computing (i.e., decoding) $\vec{v}_2^{\langle h\rangle} = \tilde{H}_2^{*\langle h\rangle} \cdot \vec{m}$ is equivalent to modifying $M(X)$ to $M'(X) = M(X^{J(h)})$ (whose coefficient vector is $\vec{m}^{\langle h\rangle}$) and then computing (i.e., decoding) $\vec{v}_2^{\langle h\rangle} = \tilde{H}_2^* \cdot \vec{m}^{\langle h\rangle}$. This is because,

$\vec{v}_2^{\langle h\rangle} = \tilde{H}_2^{*\langle h\rangle} \cdot \vec{m}$

$= (\ \tilde{W}_{\frac{n}{2}+h}^* \cdot \vec{m},\ \tilde{W}_{\frac{n}{2}+h+1}^* \cdot \vec{m},\ \tilde{W}_{\frac{n}{2}+h+2}^* \cdot \vec{m}, \cdots, \tilde{W}_{n-1}^* \cdot \vec{m},\ \tilde{W}_{\frac{n}{2}}^* \cdot \vec{m},\ \tilde{W}_{\frac{n}{2}+1}^* \cdot \vec{m}, \cdots,$
$\tilde{W}_{\frac{n}{2}+h-2}^* \cdot \vec{m},\ \tilde{W}_{\frac{n}{2}+h-1}^* \cdot \vec{m})$

$= (\ M((\omega^{J(h)})^{J_*(0)}),\ M((\omega^{J(h)})^{J_*(1)}),\ M((\omega^{J(h)})^{J_*(2)}), \cdots, M((\omega^{J(h)})^{J_*(\frac{n}{2}-1)})\ )$

▷ This is equivalent to evaluating the polynomial $M(X^{J(h)})$ at the following $\frac{n}{2}$ distinct $(\mu = 2n)$-th roots of unity: $\omega^{J_*(0)}, \omega^{J_*(1)}, \omega^{J_*(2)}, \cdots, \omega^{J_*(\frac{n}{2}-1)}$

$= (\ M(\omega^{J(h)\cdot J_*(0)}),\ M(\omega^{J(h)\cdot J_*(1)}),\ M(\omega^{J(h)\cdot J_*(2)}), \cdots, M(\omega^{J(h)\cdot J_*(\frac{n}{2}-1)})\ )$

$= (\ M(\omega^{5^h \cdot -5^0}),\ M(\omega^{5^h \cdot -5^1}),\ M(\omega^{5^h \cdot -5^2}), \cdots, M(\omega^{5^h \cdot -5^{n/2-1}})\ )$

$= (\ M(\omega^{-5^h}),\ M(\omega^{-5^{h+1}}),\ M(\omega^{-5^{h+2}}), \cdots, M(\omega^{-5^{h+n/2-1}})\ )$ ▷ note that $-(5^{\frac{n}{2}} \bmod 2n) = -1$

$= (\ M(\omega^{J_*(h)}),\ M(\omega^{J_*(h+1)}),\ M(\omega^{J_*(h+2)}), \cdots, M(\omega^{J_*(\frac{n}{2}-1)}),\ M(\omega^{J_*(0)}),\ M(\omega^{J_*(1)}),$
$\cdots,\ M(\omega^{J_*(h-2)}),\ M(\omega^{J_*(h-1)}))$

$= \tilde{H}_2^* \cdot \vec{m}^{\langle h \rangle}$

The above derivations demonstrate that $\vec{v}_1^{\langle h \rangle} = \tilde{H}_1^* \cdot \vec{m}^{\langle h \rangle}$, and $\vec{v}_2^{\langle h \rangle} = \tilde{H}_2^* \cdot \vec{m}^{\langle h \rangle}$. Combining these two findings, we reach the conclusion that $\vec{v}^{\langle h \rangle} = \tilde{W}^* \cdot \vec{m}^{\langle h \rangle}$: rotating the first-half elements of the input vector $\vec{v}$ by h positions to the left and the second-half elements of it by h positions also to the left is equivalent to updating the plaintext polynomial $M(X)$ to $M(X^{J(h)})$ and then decoding it with the decoding matrix $\tilde{W}^*$.

However, now a new problem is that we cannot directly update the plaintext $M(X)$ to $M(X^{J(h)})$, because $M(X)$ is encrypted as an RLWE ciphertext. Therefore, we need to instead update the RLWE ciphertext components (A, B) to *indirectly* by updating $M(X)$ to $M(X^{J(h)})$. We will explain this in the next subsection.

D-2.9.2 Updating the Plaintext Polynomial by Updating the Ciphertext Polynomials

Given an RLWE ciphertext $\mathsf{ct} = (A, B)$, our goal is to update $\mathsf{ct} = (A, B)$ to $C^{\langle h \rangle} = (A^{\langle h \rangle}, B^{\langle h \rangle})$ such that decrypting it gives the input vector $\vec{v}^{\langle h \rangle}$. That is, the following relation should hold:

$\mathsf{RLWE}_{S,\sigma}^{-1}\big(\, C^{\langle h \rangle} = (A^{\langle h \rangle}, B^{\langle h \rangle}) \,\big) = \Delta M(X^{J(h)}) + E'$

Remember that in the RLWE cryptosystem (§B-3)'s alternative version (§B-4.4), the plaintext and ciphertext pair have the following relation:

$\Delta M(X) + E(X) = A(X) \cdot S(X) + B(X) \approx \Delta M(X)$

If we apply $X = X^{J(h)}$ in the above relation, we can derive the following relation:

$\Delta M(X^{J(h)}) + E(X^{J(h)}) = A(X^{J(h)}) \cdot S(X^{J(h)}) + B(X^{J(h)}) \approx \Delta M(X^{J(h)})$

This relation implies that if we decrypt the ciphertext $C^{\langle h \rangle} = (A(X^{J(h)}), B(X^{J(h)}))$ with $S(X^{J(h)})$ as the secret key, then we get $\Delta M(X^{J(h)})$. Therefore, $C^{\langle h \rangle} = (A(X^{J(h)}), B(X^{J(h)}))$ is the RLWE ciphertext we are looking for, because decrypting it and then decoding its plaintext $M(X^{J(h)})$ will give us the input vector $\vec{v}^{\langle h \rangle}$.

Apply the automorphism componentwise to obtain $C^{\langle h \rangle} = (A(X^{J(h)}), B(X^{J(h)}))$, whose secret is $S(X^{J(h)})$. Then key-switch to the original secret $S(X)$ using Summary D-2.8 from §D-2.8. This produces new ciphertext components and adds key-switching error; the same unchanged pair is not simultaneously an encryption under both secrets. The following expression suppresses that error only in the message notation:

$$\underbrace{\mathsf{RLWE}_{S(X),\sigma}(\Delta M(X^{J(h)}))}_{\substack{\text{the result of}\\ \text{plaintext-to-ciphertext addition}}} = \underbrace{(\, 0, B(X^{J(h)}) \,)}_{\substack{\text{the plaintext } B(X^{J(h)})\\ \text{(trivial ciphertext)}}} + \langle\, \underbrace{\mathsf{Decomp}^{\beta,l}(A(X^{J(h)})),\ \mathsf{RLev}^{\beta,l}_{S(X),\sigma}(S(X^{J(h)}))}_{\substack{\text{an RLWE ciphertext encrypting } A(X^{J(h)}) \cdot S(X^{J(h)})\\ \text{which is key-switched from } S(X^{J(h)}) \to S(X)}} \,\rangle$$

D-2.9.3 Summary

We summarize the procedure of rotating the BFV input vectors as follows:

⟨Summary D-2.9⟩ BFV's Homomorphic Rotation of Input Vector Slots

To support input vector slot rotation, we update the original encoding matrix in $\mathsf{Encoding}_1$ as follows:

$$\tilde{W} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-2)}) & \cdots & (\omega^{J(0)}) & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-2)}) & \cdots & (\omega^{J_*(0)}) \\ (\omega^{J(\frac{n}{2}-1)})^2 & (\omega^{J(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J(0)})^2 & (\omega^{J_*(\frac{n}{2}-1)})^2 & (\omega^{J_*(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J_*(0)})^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega^{J(\frac{n}{2}-1)})^{n-1} & (\omega^{J(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J(0)})^{n-1} & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} & (\omega^{J_*(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J_*(0)})^{n-1} \end{bmatrix}$$

, and update the original decoding matrix in $\mathsf{Decoding}_2$ as follows:

$$\tilde{W}^* = \begin{bmatrix} 1 & (\omega^{J(0)}) & (\omega^{J(0)})^2 & \cdots & (\omega^{J(0)})^{n-1} \\ 1 & (\omega^{J(1)}) & (\omega^{J(1)})^2 & \cdots & (\omega^{J(1)})^{n-1} \\ 1 & (\omega^{J(2)}) & (\omega^{J(2)})^2 & \cdots & (\omega^{J(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J(\frac{n}{2}-1)})^{n-1} \\ 1 & (\omega^{J_*(0)}) & (\omega^{J_*(0)})^2 & \cdots & (\omega^{J_*(0)})^{n-1} \\ 1 & (\omega^{J_*(1)}) & (\omega^{J_*(1)})^2 & \cdots & (\omega^{J_*(1)})^{n-1} \\ 1 & (\omega^{J_*(2)}) & (\omega^{J_*(2)})^2 & \cdots & (\omega^{J_*(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} \end{bmatrix}$$

, where $J(h)$ is the *rotation helper formula*: $J(h) = 5^h \bmod 2n, \quad J_*(h) = -5^h \bmod 2n.$

Using $\tilde{W}$, the encoding is perform as: $\vec{m} = n^{-1} \cdot \tilde{W} \cdot I_n^R \cdot \vec{v}$. Using $\tilde{W}^*$, the decoding is performed as $\vec{v} = \tilde{W}^* \cdot \vec{m}$

Suppose we have an RLWE ciphertext and a key-switching key as follows:
$\mathsf{RLWE}_{S,\sigma}(\Delta M) = (A, B), \quad \mathsf{RLev}_{S,\sigma}^{\beta,l}(S(X^{J(h)}))$

Then, the procedure of rotating the first-half elements of the ciphertext's original input vector $\vec{v}$ by h positions to the left (in a wrapping manner among them) and the second-half elements of $\vec{v}$ by h positions to the left (in a wrapping manner among them) is as follows:
1. Update $A(X)$, $B(X)$ to $A(X^{J(h)})$, $B(X^{J(h)})$.
2. Perform the following key switching (§D-2.8) from $S(X^{J(h)})$ to $S(X)$:
   $\mathsf{RLWE}_{S(X),\sigma}(\Delta M(X^{J(h)})) = (0, B(X^{J(h)})) + \langle \mathsf{Decomp}^{\beta,l}(A(X^{J(h)})),\ \mathsf{RLev}_{S(X),\sigma}^{\beta,l}(S(X^{J(h)}))\rangle$

D-2.9.4 Encoding Example

In the above example, we use the unsigned modulo representation (e.g., $[0, t-1], [0, q]$) when computing mod$t$ and mod$q$. However, the correctness of the result holds even if we use the centered modulo representation (e.g., $[-\frac{t}{2}, \frac{t}{2}), [-\frac{q}{2}, \frac{q}{2})$). In many actual FHE libraries, centered modulo arithmetic is

often used to manage noise growth more effectively.

Suppose we have the following setup:

$\mu = 8,\ n = \dfrac{\mu}{2} = 4,\ t = 17,\ q = 2^8 = 256,\ n^{-1} = 13,\ \Delta = \left\lfloor \dfrac{q}{t} \right\rfloor = 15$

$\mathcal{R}_{\langle 4,17 \rangle} = \mathbb{Z}_{17}[X]/(X^4 + 1)$

The roots of $X^4 + 1 \pmod{17}$ are $X = \{2, 8, 15, 9\}$, as demonstrated as follows:
$2^4 \equiv 8^4 \equiv 15^4 \equiv 9^4 \equiv 16 \equiv -1 \bmod 17$

Definition A-8.1 (in §A-8.1) states that the roots of the μ-th cyclotomic polynomial are the primitive μ-th roots of unity. And Definition A-7.1 (in §A-7.1) states that the order of the primitive μ-th roots of unity is μ. These definitions apply to both the cyclotomic polynomials over $X \in \mathbb{C}$ (complex numbers) and the cyclotomic polynomials over $X \in \mathbb{Z}_t$ (ring).

Since $\{2, 8, 15, 9\}$ are the roots of the $(\mu = 8)$-th cyclotomic polynomial $X^4 + 1$ over the ring $\mathbb{Z}_{17}$, they are also the $(\mu = 8)$-th primitive roots of unity of $\mathbb{Z}_{17}$. Therefore, their order (§A-4.1) is $\mu = 8$ as demonstrated as follows:
$2^8 \equiv 8^8 \equiv 15^8 \equiv 9^8 \equiv 1 \bmod 17$
$2^4 \equiv 8^4 \equiv 15^4 \equiv 9^4 \equiv 16 \not\equiv 1 \bmod 17$

Definition A-8.1 (in §A-8.1) and Theorem A-8.2 (§A-8.2) also state that for each primitive μ-th root of unity ω, $\{\omega^k\}_{\gcd(k,\mu)=1}$ generates all roots of the μ-th cyclotomic polynomial. Notice that in the case of the $(\mu = 8)$-th cyclotomic polynomial $X^4 + 1$, its roots $\{2, 8, 15, 9\}$ generate all $(\mu = 8)$-th roots of unity as follows:
$\{2^1, 2^3, 2^5, 2^7\} \equiv \{8^1, 8^3, 8^5, 8^7\} \equiv \{15^1, 15^3, 15^5, 15^7\} \equiv \{9^1, 9^3, 9^5, 9^7\} \equiv \{2, 8, 15, 9\} \bmod 17$

Among $\{2, 8, 15, 9\}$ as the roots of $X^4 + 1$, let's choose $\omega = 9$ as the base root to construct the encoding matrix $\tilde{W}$ and the decoding matrix $\tilde{W}^*$ as follows:

$$\tilde{W} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \omega^{J(1)} & \omega^{J(0)} & \omega^{J_*(1)} & \omega^{J_*(0)} \\ (\omega^{J(1)})^2 & (\omega^{J(0)})^2 & (\omega^{J_*(1)})^2 & (\omega^{J_*(0)})^2 \\ (\omega^{J(1)})^3 & (\omega^{J(0)})^3 & (\omega^{J_*(1)})^3 & (\omega^{J_*(0)})^3 \end{bmatrix} \triangleright \text{ where } J(h) = 5^h \bmod 8$$

$$= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 9^5 & 9^1 & 9^3 & 9^7 \\ (9^5)^2 & (9^1)^2 & (9^3)^2 & (9^7)^2 \\ (9^5)^3 & (9^1)^3 & (9^3)^3 & (9^7)^3 \end{bmatrix} \equiv \begin{bmatrix} 1 & 1 & 1 & 1 \\ 8 & 9 & 15 & 2 \\ 13 & 13 & 4 & 4 \\ 2 & 15 & 9 & 8 \end{bmatrix} \bmod 17$$

$$\tilde{W}^* = \begin{bmatrix} 1 & \omega^{J(0)} & (\omega^{J(0)})^2 & (\omega^{J(0)})^3 \\ 1 & \omega^{J(1)} & (\omega^{J(1)})^2 & (\omega^{J(1)})^3 \\ 1 & \omega^{J_*(0)} & (\omega^{J_*(0)})^2 & (\omega^{J_*(0)})^3 \\ 1 & \omega^{J_*(1)} & (\omega^{J_*(1)})^2 & (\omega^{J_*(1)})^3 \end{bmatrix} \equiv \begin{bmatrix} 1 & 9 & 13 & 15 \\ 1 & 8 & 13 & 2 \\ 1 & 2 & 4 & 8 \\ 1 & 15 & 4 & 9 \end{bmatrix} \bmod 17$$

Notice that Theorem A-12.5 (in §A-12.5) is demonstrated as follows:

$$\tilde{W}^* \cdot \tilde{W} = \begin{bmatrix} 1 & 9 & 13 & 15 \\ 1 & 8 & 13 & 2 \\ 1 & 2 & 4 & 8 \\ 1 & 15 & 4 & 9 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ 8 & 9 & 15 & 2 \\ 13 & 13 & 4 & 4 \\ 2 & 15 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 4 \\ 0 & 0 & 4 & 0 \\ 0 & 4 & 0 & 0 \\ 4 & 0 & 0 & 0 \end{bmatrix} = nI_n^R \pmod{17}$$

Now suppose that we encode the following two input vectors (i.e., input vector slots) in $\mathbb{Z}_{17}$:
$\vec{v}_1 = (10, 3, 5, 13)$
$\vec{v}_2 = (2, 4, 3, 6)$
$\vec{v}_1 + \vec{v}_2 = (10, 3, 5, 13) + (2, 4, 3, 6) \equiv (12, 7, 8, 2) \bmod 17$

These two vectors are encoded as follows:

$$\vec{m}_1 = n^{-1}\tilde{W} \cdot I_n^R \cdot \vec{v} = 13 \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ 8 & 9 & 15 & 2 \\ 13 & 13 & 4 & 4 \\ 2 & 15 & 9 & 8 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 10 \\ 3 \\ 5 \\ 13 \end{bmatrix} \equiv (12, 11, 12, 1) \bmod 17$$

$$\vec{m}_2 = n^{-1}\tilde{W} \cdot I_n^R \cdot \vec{v} = 13 \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ 8 & 9 & 15 & 2 \\ 13 & 13 & 4 & 4 \\ 2 & 15 & 9 & 8 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 2 \\ 4 \\ 3 \\ 6 \end{bmatrix} \equiv (8, 5, 14, 6) \bmod 17$$

$\Delta M_1(X) = 15 \cdot (12 + 11X + 12X^2 + 1X^3) = 180 + 165X + 180X^2 + 15X^3 \pmod q$ ▷ where $q = 256$
$\Delta M_2(X) = 15 \cdot (8 + 5X + 14X^2 + 6X^3) = 120 + 75X + 210X^2 + 90X^3 \pmod q$
$\Delta M_{1+2}(X) = \Delta M_1(X) + \Delta M_2(X) = \Delta(M_1(X) + M_2(X)) = 44 + 240X + 134X^2 + 105X^3 \pmod q$
$M_{1+2}(X) = \left\lceil \frac{17}{256}(44 + 240X + 134X^2 + 105X^3) \right\rfloor \bmod 17$
$= \lceil 2.922 + 15.938X + 8.898X^2 + 6.973X^3 \rfloor \bmod 17$
$3 + 16X + 9X^2 + 7X^3 \bmod 17$

This polynomial matches the value of $\vec{m}_{1+2}$ as follows:
$\vec{m}_1 + \vec{m}_2 = (12, 11, 12, 1) + (8, 5, 14, 6) = (20, 16, 26, 7) \equiv (3, 16, 9, 7) \bmod 17$

Finally, we decode $\vec{m}_{1+2}$ as follows:

$$\vec{v}_{1+2} = \tilde{W}^* \cdot \vec{m}_{1+2} = \begin{bmatrix} 1 & 9 & 13 & 15 \\ 1 & 8 & 13 & 2 \\ 1 & 2 & 4 & 8 \\ 1 & 15 & 4 & 9 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 16 \\ 9 \\ 7 \end{bmatrix} = (12, 7, 8, 2) \bmod 17$$

This result matches the expected vector sum $\vec{v}_1 + \vec{v}_2$.

D-2.9.5 Rotation Example

Also in the above example, we use the unsigned modulo representation (e.g., $[0, t-1], [0, q]$) when computing mod$t$ and mod$q$. However, the correctness of the result holds even if we use the centered modulo representation (e.g., $[-\frac{t}{2}, \frac{t}{2}), [-\frac{q}{2}, \frac{q}{2})$).

Suppose we have the following setup:
$\mu = 16, n = \frac{\mu}{2} = 8,\ t = 17,\ q = 2^8 = 256,\ n^{-1} = 15,\ \Delta = 15$

$\mathcal{R}_{\langle 8,17 \rangle} = \mathbb{Z}_{17}[X]/(X^8 + 1)$

The roots of $X^8 + 1 \pmod{17}$ are $X = \{3, 5, 6, 7, 10, 11, 12, 14\}$, as demonstrated as follows:

$3^8 \equiv 5^8 \equiv 6^8 \equiv 7^8 \equiv 10^8 \equiv 11^8 \equiv 12^8 \equiv 14^8 \equiv 16 \equiv -1 \bmod 17$

Among $\{3, 5, 6, 7, 10, 11, 12, 14\}$ as the roots of $X^8 + 1$, let's choose $\omega = 3$ as the base root to construct the encoding matrix $\tilde{W}$ and the decoding matrix $\tilde{W}^*$ as follows:

$$\tilde{W} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ \omega^{J(3)} & \omega^{J(2)} & \omega^{J(1)} & \omega^{J(0)} & \omega^{J_*(3)} & \omega^{J_*(2)} & \omega^{J_*(1)} & \omega^{J_*(0)} \\ (\omega^{J(3)})^2 & (\omega^{J(2)})^2 & (\omega^{J(1)})^2 & (\omega^{J(0)})^2 & (\omega^{J_*(3)})^2 & (\omega^{J_*(2)})^2 & (\omega^{J_*(1)})^2 & (\omega^{J_*(0)})^2 \\ (\omega^{J(3)})^3 & (\omega^{J(2)})^3 & (\omega^{J(1)})^3 & (\omega^{J(0)})^3 & (\omega^{J_*(3)})^3 & (\omega^{J_*(2)})^3 & (\omega^{J_*(1)})^3 & (\omega^{J_*(0)})^3 \\ (\omega^{J(3)})^4 & (\omega^{J(2)})^4 & (\omega^{J(1)})^4 & (\omega^{J(0)})^4 & (\omega^{J_*(3)})^4 & (\omega^{J_*(2)})^4 & (\omega^{J_*(1)})^4 & (\omega^{J_*(0)})^4 \\ (\omega^{J(3)})^5 & (\omega^{J(2)})^5 & (\omega^{J(1)})^5 & (\omega^{J(0)})^5 & (\omega^{J_*(3)})^5 & (\omega^{J_*(2)})^5 & (\omega^{J_*(1)})^5 & (\omega^{J_*(0)})^5 \\ (\omega^{J(3)})^6 & (\omega^{J(2)})^6 & (\omega^{J(1)})^6 & (\omega^{J(0)})^6 & (\omega^{J_*(3)})^6 & (\omega^{J_*(2)})^6 & (\omega^{J_*(1)})^6 & (\omega^{J_*(0)})^6 \\ (\omega^{J(3)})^7 & (\omega^{J(2)})^7 & (\omega^{J(1)})^7 & (\omega^{J(0)})^7 & (\omega^{J_*(3)})^7 & (\omega^{J_*(2)})^7 & (\omega^{J_*(1)})^7 & (\omega^{J_*(0)})^7 \end{bmatrix}$$

$$\equiv \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 12 & 14 & 5 & 3 & 10 & 11 & 7 & 6 \\ 8 & 9 & 8 & 9 & 15 & 2 & 15 & 2 \\ 11 & 7 & 6 & 10 & 14 & 5 & 3 & 12 \\ 13 & 13 & 13 & 13 & 4 & 4 & 4 & 4 \\ 3 & 12 & 14 & 5 & 6 & 10 & 11 & 7 \\ 2 & 15 & 2 & 15 & 9 & 8 & 9 & 8 \\ 7 & 6 & 10 & 11 & 5 & 3 & 12 & 14 \end{bmatrix} \bmod 17$$

$$\tilde{W}^* = \begin{bmatrix} 1 & \omega^{J(0)} & (\omega^{J(0)})^2 & (\omega^{J(0)})^3 & (\omega^{J(0)})^4 & (\omega^{J(0)})^5 & (\omega^{J(0)})^6 & (\omega^{J(0)})^7 \\ 1 & \omega^{J(1)} & (\omega^{J(1)})^2 & (\omega^{J(1)})^3 & (\omega^{J(1)})^4 & (\omega^{J(1)})^5 & (\omega^{J(1)})^6 & (\omega^{J(1)})^7 \\ 1 & \omega^{J(2)} & (\omega^{J(2)})^2 & (\omega^{J(2)})^3 & (\omega^{J(2)})^4 & (\omega^{J(2)})^5 & (\omega^{J(2)})^6 & (\omega^{J(2)})^7 \\ 1 & \omega^{J(3)} & (\omega^{J(3)})^2 & (\omega^{J(3)})^3 & (\omega^{J(3)})^4 & (\omega^{J(3)})^5 & (\omega^{J(3)})^6 & (\omega^{J(3)})^7 \\ 1 & \omega^{J_*(0)} & (\omega^{J_*(0)})^2 & (\omega^{J_*(0)})^3 & (\omega^{J_*(0)})^4 & (\omega^{J_*(0)})^5 & (\omega^{J_*(0)})^6 & (\omega^{J_*(0)})^7 \\ 1 & \omega^{J_*(1)} & (\omega^{J_*(1)})^2 & (\omega^{J_*(1)})^3 & (\omega^{J_*(1)})^4 & (\omega^{J_*(1)})^5 & (\omega^{J_*(1)})^6 & (\omega^{J_*(1)})^7 \\ 1 & \omega^{J_*(2)} & (\omega^{J_*(2)})^2 & (\omega^{J_*(2)})^3 & (\omega^{J_*(2)})^4 & (\omega^{J_*(2)})^5 & (\omega^{J_*(2)})^6 & (\omega^{J_*(2)})^7 \\ 1 & \omega^{J_*(3)} & (\omega^{J_*(3)})^2 & (\omega^{J_*(3)})^3 & (\omega^{J_*(3)})^4 & (\omega^{J_*(3)})^5 & (\omega^{J_*(3)})^6 & (\omega^{J_*(3)})^7 \end{bmatrix}$$

$$\equiv \begin{bmatrix} 1 & 3 & 9 & 10 & 13 & 5 & 15 & 11 \\ 1 & 5 & 8 & 6 & 13 & 14 & 2 & 10 \\ 1 & 14 & 9 & 7 & 13 & 12 & 15 & 6 \\ 1 & 12 & 8 & 11 & 13 & 3 & 2 & 7 \\ 1 & 6 & 2 & 12 & 4 & 7 & 8 & 14 \\ 1 & 7 & 15 & 3 & 4 & 11 & 9 & 12 \\ 1 & 11 & 2 & 5 & 4 & 10 & 8 & 3 \\ 1 & 10 & 15 & 14 & 4 & 6 & 9 & 5 \end{bmatrix} \bmod 17$$

Notice that Theorem A-12.5 (in §A-12.5) is demonstrated as follows:

$$\tilde{W}^* \cdot \tilde{W} = \begin{bmatrix} 1 & 3 & 9 & 10 & 13 & 5 & 15 & 11 \\ 1 & 5 & 8 & 6 & 13 & 14 & 2 & 10 \\ 1 & 14 & 9 & 7 & 13 & 12 & 15 & 6 \\ 1 & 12 & 8 & 11 & 13 & 3 & 2 & 7 \\ 1 & 6 & 2 & 12 & 4 & 7 & 8 & 14 \\ 1 & 7 & 15 & 3 & 4 & 11 & 9 & 12 \\ 1 & 11 & 2 & 5 & 4 & 10 & 8 & 3 \\ 1 & 10 & 15 & 14 & 4 & 6 & 9 & 5 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 12 & 14 & 5 & 3 & 10 & 11 & 7 & 6 \\ 8 & 9 & 8 & 9 & 15 & 2 & 15 & 2 \\ 11 & 7 & 6 & 10 & 14 & 5 & 3 & 12 \\ 13 & 13 & 13 & 13 & 4 & 4 & 4 & 4 \\ 3 & 12 & 14 & 5 & 6 & 10 & 11 & 7 \\ 2 & 15 & 2 & 15 & 9 & 8 & 9 & 8 \\ 7 & 6 & 10 & 11 & 5 & 3 & 12 & 14 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 8 \\ 0 & 0 & 0 & 0 & 0 & 0 & 8 & 0 \\ 0 & 0 & 0 & 0 & 0 & 8 & 0 & 0 \\ 0 & 0 & 0 & 0 & 8 & 0 & 0 & 0 \\ 0 & 0 & 0 & 8 & 0 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 0 & 0 & 0 & 0 \\ 0 & 8 & 0 & 0 & 0 & 0 & 0 & 0 \\ 8 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = nI_n^R \pmod{17}$$

Now suppose that we encode the following input vector (i.e., input vector slots) in $\mathbb{Z}_{17}$:
$\vec{v} = (1, 2, 3, 4, 5, 6, 7, 8)$
By rotating this vector by 3 positions to the left (i.e., the first-half slots and the second-half slots separately wrapping around within their own group), we get a new vector:
$\vec{v}_r = (4, 1, 2, 3, 8, 5, 6, 7) \bmod 17$

$\vec{v}$ is encoded as follows:

$\vec{m} = n^{-1}\tilde{W} \cdot I_n^R \cdot \vec{v}$

$$= 15 \cdot \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 12 & 14 & 5 & 3 & 10 & 11 & 7 & 6 \\ 8 & 9 & 8 & 9 & 15 & 2 & 15 & 2 \\ 11 & 7 & 6 & 10 & 14 & 5 & 3 & 12 \\ 13 & 13 & 13 & 13 & 4 & 4 & 4 & 4 \\ 3 & 12 & 14 & 5 & 6 & 10 & 11 & 7 \\ 2 & 15 & 2 & 15 & 9 & 8 & 9 & 8 \\ 7 & 6 & 10 & 11 & 5 & 3 & 12 & 14 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \end{bmatrix}$$

$\equiv (13, 16, 10, 5, 9, 12, 7, 1) \bmod 17$

$\Delta M(X) = 15 \cdot (13 + 16X + 10X^2 + 5X^3 + 9X^4 + 12X^5 + 7X^6 + X^7)$
$= 195 + 240X + 150X^2 + 75X^3 + 135X^4 + 180X^5 + 105X^6 + 15X^7 \pmod{q}$ ▷ where $q = 256$

This polynomial matches the value of $\Delta M(X^{J(3)})$ as follows:
$\Delta M(X^{J(3)}) = \Delta M(X^{13})$
$= 195 + 180X - 150X^2 - 15X^3 + 135X^4 - 240X^5 - 105X^6 + 75X^7 \pmod{q}$

Now, we decode $M(X^{J(3)})$ as follows:
$$\vec{m}_{J(3)} = \frac{\Delta \vec{m}_{J(3)}}{\Delta} = \frac{(195, 180, -150, -15, 135, -240, -105, 75)}{15} = (13, 12, -10, -1, 9, -16, -7, 5)$$

(mod 17)

$$\vec{v}_{J(3)} = \tilde{W}^* \cdot \vec{m}_{J(3)} = \begin{bmatrix} 1 & 3 & 9 & 10 & 13 & 5 & 15 & 11 \\ 1 & 5 & 8 & 6 & 13 & 14 & 2 & 10 \\ 1 & 14 & 9 & 7 & 13 & 12 & 15 & 6 \\ 1 & 12 & 8 & 11 & 13 & 3 & 2 & 7 \\ 1 & 6 & 2 & 12 & 4 & 7 & 8 & 14 \\ 1 & 7 & 15 & 3 & 4 & 11 & 9 & 12 \\ 1 & 11 & 2 & 5 & 4 & 10 & 8 & 3 \\ 1 & 10 & 15 & 14 & 4 & 6 & 9 & 5 \end{bmatrix} \cdot \begin{bmatrix} 13 \\ 12 \\ -10 \\ -1 \\ 9 \\ -16 \\ -7 \\ 5 \end{bmatrix}$$
$$= (4, 1, 2, 3, 8, 5, 6, 7) \pmod{17}$$
$$= \vec{v}_r$$

The decoded $\vec{v}_{J(3)}$ matches the expected rotated input vector $\vec{v}_r$.

In practice, apply the automorphism to the encrypted components, producing

$$\mathsf{ct}^{\langle h=3 \rangle} = (A(X^{J(3)}), B(X^{J(3)}))$$

. Its decryption secret is $S(X^{J(3)})$. Switch back to $S(X)$ with the corresponding rotation key before ordinary decryption; provided the transformed and key-switching errors remain within the bound, decoding yields $\vec{v}_r$.

Source Code: Examples of BFV's batch encoding and homomorphic input vector rotation can be executed by running this Python script (BFV addition) and this one (BFV rotation only).

D-2.10 Application: Matrix Multiplication

BFV supports packed inner products by multiplying slots and summing them with rotations and additions. A doubling schedule uses logarithmically many rotations within a power-of-two slot block; summing both rows additionally needs a row exchange. For matrix-vector multiplication, the diagonal method below assumes $m \leq n$ and a logical cyclic rotation ρ of all n active slots. The previous subsection implements two independent half-row rotations, so it cannot by itself realize this full ρ. Either place the logical vector in one cyclic row of a larger ring, or combine row rotations, the row-swap automorphism $X \mapsto X^{-1}$ and plaintext masks. Given a matrix A and vector $\vec{x}$ (Definition A-11.3 in §A-11.3):

$$A = \begin{bmatrix} a_{\langle 0,0 \rangle} & a_{\langle 0,1 \rangle} & a_{\langle 0,2 \rangle} & \cdots & a_{\langle 0,n-1 \rangle} \\ a_{\langle 1,0 \rangle} & a_{\langle 1,1 \rangle} & a_{\langle 1,2 \rangle} & \cdots & a_{\langle 1,n-1 \rangle} \\ a_{\langle 2,0 \rangle} & a_{\langle 2,1 \rangle} & a_{\langle 2,2 \rangle} & \cdots & a_{\langle 2,n-1 \rangle} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{\langle m-1,0 \rangle} & a_{\langle m-1,1 \rangle} & a_{\langle m-1,2 \rangle} & \cdots & a_{\langle m-1,n-1 \rangle} \end{bmatrix} = \begin{bmatrix} \vec{a}_{\langle 0,* \rangle} \\ \vec{a}_{\langle 1,* \rangle} \\ \vec{a}_{\langle 2,* \rangle} \\ \vdots \\ \vec{a}_{\langle m-1,* \rangle} \end{bmatrix}, \ \vec{x} = (x_0, x_1, \cdots, x_{n-1})$$

The result of $A \cdot \vec{x}$ is an m-dimensional vector computed as:

$$A \cdot \vec{x} = \left(\vec{a}_{\langle 0,* \rangle} \cdot \vec{x}, \ \vec{a}_{\langle 1,* \rangle} \cdot \vec{x}, \ \cdots, \ \vec{a}_{\langle m-1,* \rangle} \cdot \vec{x} \right) = \left(\sum_{i=0}^{n-1} a_{0,i} \cdot x_i, \ \sum_{i=0}^{n-1} a_{1,i} \cdot x_i, \cdots, \ \sum_{i=0}^{n-1} a_{m-1,i} \cdot x_i \right)$$

Let's define $\rho(\vec{v}, h)$ as the rotation of $\vec{v}$ by h positions to the left. And remember that the Hadamard dot product (Definition A-11.1 in §A-11.1) is defined as slot-wise multiplication of two vectors:

$\vec{a} \odot \vec{b} = (a_0 b_0,\ a_1 b_1,\ \cdots,\ a_{n-1} b_{n-1})$

Let's define n distinct diagonal vector $\vec{u}_i$ extracted from matrix A as follows:
$\vec{u}_i = \{a_{\langle j \bmod m,\ (i+j) \bmod n \rangle}\}_{j=0}^{n-1}$

Then, the original matrix-to-vector multiplication formula can be equivalently constructed as follows:
$A \cdot \vec{x} = \left(\sum_{i=0}^{n-1} \vec{u}_i \odot \rho(\vec{x}, i)\right)_{0:m}$
, whose computation result is equivalent to $A \cdot \vec{x}$. The above formula is compatible with homomorphic computation, because BFV supports Hadamard dot product between input vectors as a ciphertext-to-plaintext multiplication between their polynomial-encoded forms (§D-2.6), and BFV also supports $\rho(\vec{v}, h)$ as homomorphic input vector slot rotation (§D-2.9). After homomorphically computing the above formula, we can consider only the first m (out of n) resulting input vector slots to store the result of $A \cdot \vec{x}$.

D-2.11 Noise Bootstrapping

- **Reference 1:** Bootstrapping for BGV and BFV Revisited [15]
- **Reference 2:** Bootstrapping for HELib [16]
- **Reference 3:** A Note on Lower Digits Extraction Polynomial for Bootstrapping [17]
- **Reference 4:** Fully Homomorphic Encryption for Cyclotomic Prime Moduli [18]

In BFV, continuous ciphertext-to-ciphertext multiplication increases the noise in a multiplicative manner, and once the noise overflows the message bits, then the message gets corrupted. Bootstrapping is a process of resetting the grown noise.

D-2.11.1 High-level Idea

Let $t = p^r$, $\varepsilon > r$ and $P = p^\varepsilon$. We decompose the decryption function, give a general coefficient/slot construction, and state an explicit gate-by-gate sufficient correctness test in §D-2.11.6. The homomorphic polynomial evaluations in that test use standard FV integer tensor multiplication followed by T/q scaling at the current plaintext modulus T, not the $1/\Delta$ teaching variant when $T \nmid q$. This is a conditional correctness result, not a secure or optimized parameter selection. Encrypting a secret under itself additionally needs the evaluation-key and circular-security assumptions of the cited construction [15].

For a current plaintext modulus T, it is useful to measure error against the exact rational scale $\mu_T = q/T$. With $r_T = q - T\lfloor q/T \rceil$, the same ciphertext satisfies

$$\phi_S(C) \equiv \mu_T M_T + \mathcal{E}_T \pmod q, \qquad \mathcal{E}_T = E_T - \frac{r_T}{T} M_T.$$

Here M_T is the centered coefficient representative. The decoder is correct if $\|\mathcal{E}_T\|_\infty < q/(2T)$. This change of error coordinates neither changes the ciphertext nor introduces an approximate plaintext scheme; it is exactly the decoder condition from §D-2.3.

For a ciphertext (A, B) with phase $AS + B = \Delta M + E \pmod q$, the target plaintext identity is

$$A' = \left[\left\lceil (P/q)A \right\rfloor\right]_P, \quad B' = \left[\left\lceil (P/q)B \right\rfloor\right]_P, \quad Z = [A'S + B']_P, \qquad M = \left[\left\lceil Z/p^{\varepsilon - r} \right\rfloor\right]_{p^r}.$$

1. **Modulus Switch:** Change the input ciphertext modulus from q to P. This alone does not change the original plaintext modulus p^r. For suitable parameters the switched phase is $p^{\varepsilon - r} M + E' \pmod P$, with $\|E'\|_\infty < p^{\varepsilon - r}/2$.

2. **Homomorphic Decryption:** Use an evaluation ciphertext encrypting S at a large ciphertext modulus q, now with plaintext modulus P and scale $\Delta' = \lfloor q/P \rfloor$. Compute $A' \cdot \mathsf{RLWE}_{S,\sigma}(\Delta' S) + (0, \Delta' B')$. The result encrypts the phase $Z \pmod P$, with additional evaluation noise. The public constant B' must be encoded at scale Δ'.
3. **CoeffToSlot:** Apply the homomorphic linear transformation that places the coefficients of Z into slots. Under the full scalar-splitting conditions of §D-2.9.3, the transformation on the slot vector is $n^{-1}\tilde{W} I_n^R$.
4. **Digit Extraction:** Evaluate polynomial low-digit extraction and exact-division steps to compute $[\lceil z_i/p^{\varepsilon-r} \rfloor]_{p^r}$ in each slot. For odd p, use balanced digits. For $p = 2$, unsigned digit deletion is flooring, so a half-unit offset is required before deletion. The digit identities are explained below; their use in a complete bootstrap is subject to the CRT-domain restriction of §D-2.11.4.
5. **SlotToCoeff:** Apply the inverse linear transformation $\tilde{W}^*$ to put the recovered coefficients back into a polynomial. The result encrypts M with a new error $E^{\langle b \rangle}$ generated by the evaluation. Its bound depends on the parameters and circuit; it is not automatically smaller than an arbitrary input error E or the differently scaled E'.

The old E' is removed as part of evaluating the rounding function, while the errors of the homomorphic evaluation remain. A usable bootstrap must leave enough output noise budget for subsequent operations. Repetition supports arbitrary prescribed finite circuit depth only with suitable parameters and an acceptable total failure probability.

D-2.11.2 Modulus Switch

Let $P = p^\varepsilon$ and define the component rounding polynomials $R_A = A' - (P/q)A$ and $R_B = B' - (P/q)B$, using compatible integer lifts before reduction. Their coefficient magnitudes are at most $1/2$. The exact switched phase is

$$A'S + B' \equiv p^{\varepsilon-r} M + E' \pmod P, \quad E' = \frac{P}{q} E + \left(\frac{P}{q} \Delta - p^{\varepsilon-r} \right) M + R_A S + R_B.$$

Thus a sufficient condition for the target rounding is $\|E'\|_\infty < p^{\varepsilon-r}/2$. The component-rounding term cannot be replaced by rounding the old error alone. Its deterministic bound is $(\|S\|_1 + 1)/2$, in addition to the scaled old error and the scale mismatch term.

D-2.11.3 Homomorphic Decryption

Write $Z = [A'S + B']_P = [p^{\varepsilon-r} M + E']_P$. With $\Delta' = \lfloor q/P \rfloor$, let the evaluation ciphertext (A_s, B_s) satisfy

$$A_s S + B_s = \Delta' S + E_s \pmod q.$$

The homomorphic inner product is the ciphertext

$$C = (A' A_s,\ A' B_s + \Delta' B'), \qquad \phi_S(C) = \Delta'(A'S + B') + A' E_s \pmod q.$$

This explicitly distinguishes the encrypted message $\Delta' S$ from the encryption secret S. If $A'S + B' = Z + PK$, then

$$\phi_S(C) \equiv \Delta' Z + A' E_s - r_P K \pmod q, \qquad r_P = q - P\Delta'.$$

The plaintext reduction modulo P is therefore represented by an additional error $-r_P K$ when $P \nmid q$; it does not require a separate ciphertext-side modulo gate. Also $\|A' E_s\|_\infty \le \|A'\|_1 \|E_s\|_\infty$. The loose inequality $q \gg P$ alone does not ensure that this error fits a decoding budget of order q/P.

The desired coefficient operation is $[\lceil z_i/p^{\varepsilon-r}\rfloor]_{p^r}$. It is implemented by a sequence of polynomial evaluations and exact divisions, not by one degree-p polynomial for the entire rounding function. To evaluate it coefficientwise, first perform CoeffToSlot, then the digit-extraction procedure of §D-2.11.5, and finally SlotToCoeff. The homomorphic evaluation errors remain separate from the old error E' inside the encrypted integer z_i.

D-2.11.4 CoeffToSlot and SlotToCoeff

The displayed n-scalar-slot matrices assume $p \equiv 1 \pmod{2n}$, with a primitive root Hensel-lifted to $\mathbb{Z}_{p^\varepsilon}$ and compatibly reduced at each later plaintext modulus. In general, when $\gcd(p, 2n) = 1$ and $d = \mathrm{ord}_{2n}(p)$, there are n/d extension-ring slots. The trace-dual construction below packs all coefficients into scalar entries of d ciphertexts, so complete scalar splitting is not necessary. Matrix multiplication denotes a homomorphic linear circuit using automorphisms, key switching and plaintext multiplications. This CRT route still requires p odd in this chapter's power-of-two ring: at $p = 2$, $X^n + 1 = (X + 1)^n \pmod 2$ has repeated factors. A binary CRT bootstrap must instead use a cyclotomic index coprime to 2, or a separately justified construction.

A general realizable linear transform. To include the alternative binary ring, temporarily write $R_j = (\mathbb{Z}/p^j\mathbb{Z})[X]/\Phi_{m_c}(X)$ with $p \nmid m_c$, $n = \varphi(m_c)$, $d = \mathrm{ord}_{m_c}(p)$ and $\ell = n/d$. This chapter's ring has $m_c = 2n$. Choose compatible monic factors of Φ_{m_c} modulo p^ε and reduce them modulo p^j; the CRT gives ℓ unramified degree-d extension-ring slots.

Let $\tau_a(X) = X^a$ for $a \in (\mathbb{Z}/m_c\mathbb{Z})^\times$. The trace pairing is nondegenerate modulo p, since the algebra is a product of finite fields; its Gram determinant is consequently a unit modulo every p^j. There is a trace-dual basis $b_0, \ldots, b_{n-1}$ such that

$$\mathrm{Tr}(b_u X^v) = \delta_{uv}, \qquad \mathrm{Tr}(W) = \sum_a \tau_a(W), \qquad z_u = \mathrm{Tr}(b_u Z).$$

Every base-ring-linear map $\mathcal{T} : R_j \to R_j$ therefore has the explicit realization

$$\mathcal{T}(Z) = \sum_a \kappa_a \tau_a(Z), \qquad \kappa_a = \sum_{u=0}^{n-1} \mathcal{T}(X^u)\tau_a(b_u).$$

Indeed, substituting the trace expansion gives $\sum_u \mathcal{T}(X^u)z_u$. Each term uses a public polynomial multiplier and an automorphism followed by key switching. No decryption or division by the extension degree is involved. In particular, when $p \mid d$, dividing the slot trace by d is invalid. The lifted Frobenius is the automorphism $X \mapsto X^p$ acting on generators; it is not the map $W \mapsto W^p$ on arbitrary elements modulo p^j when $j > 1$.

For $0 \le u < d$, define $\mathcal{T}_u$ by giving its CRT residues: slot h contains the scalar coefficient $z_{u\ell+h}$ and no nonconstant extension coordinates. An inverse map $\mathcal{U}_u$ reads each slot's scalar coordinate and writes it to coefficient position $u\ell + h$, filling other positions with zero. CRT matrix inversion constructs these maps, and the trace-dual formula compiles both into homomorphic operations. They satisfy

$$\sum_{u=0}^{d-1} \mathcal{U}_u(\mathcal{T}_u(Z)) = Z.$$

Use the packing maps modulo p^ε and the compatible unpacking maps modulo p^r. Digit polynomials act on the scalar entries, not on arbitrary extension elements. If every slot of their numerator is divisible by p,

the CRT implies that the entire polynomial lies in pR_j; thus the subsequent coefficientwise exact division is well defined. This supplies a general, unoptimized construction for the transforms used below [15].

The familiar completely split matrices are the special case $d = 1$:

The goal of the CoeffToSlot step is to homomorphically move polynomial Z's coefficients z_i to input vector slots.

In Summary D-2.9.3 (in §D-2.9.3), we learned that the encoding formula for converting a vector of input slots $\vec{v}$ into a vector of polynomial coefficients $\vec{m}$ is: $\vec{m} = n^{-1} \cdot \tilde{W} \cdot I_n^R \cdot \vec{v}$, where $\tilde{W}$ is a basis of the n-dimensional vector space crafted as follows:

$$\tilde{W} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-2)}) & \cdots & (\omega^{J(0)}) & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-2)}) & \cdots & (\omega^{J_*(0)}) \\ (\omega^{J(\frac{n}{2}-1)})^2 & (\omega^{J(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J(0)})^2 & (\omega^{J_*(\frac{n}{2}-1)})^2 & (\omega^{J_*(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J_*(0)})^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ (\omega^{J(\frac{n}{2}-1)})^{n-1} & (\omega^{J(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J(0)})^{n-1} & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} & (\omega^{J_*(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J_*(0)})^{n-1} \end{bmatrix}$$

▷ where the rotation helper function $J(h) = 5^h \bmod 2n$

Therefore, given the input ciphertext $\mathsf{ct} = \mathsf{RLWE}_{S,\sigma}(\Delta' Z) \bmod q$, we can understand its input vector slots as storing some values such that multiplying $n^{-1} \cdot \tilde{W} \cdot I_n^R$ to each of them turns them into a coefficient z_i of polynomial Z. This implies that if we *homomorphically* multiply $n^{-1} \cdot \tilde{W} \cdot I_n^R$ to the input vector slots of $\mathsf{RLWE}_{S,\sigma}(\Delta' Z)$, then the resulting ciphertext's n-slot input vector slots will contain the n coefficients of Z, which is equivalent to moving the coefficients of Z to the input vector slots. Therefore, the CoeffToSlot step is equivalent to homomorphically computing $n^{-1} \cdot \tilde{W} \cdot I_n^R \cdot \mathsf{RLWE}_{S,\sigma}(\Delta' Z)$. We can homomorphically compute matrix-vector multiplication by using the technique explained in §D-2.10.

After the CoeffToSlot step, we can homomorphically eliminate the noise in the lower (base-p) $\varepsilon - r$ digits of each z_i by homomorphically evaluating the noise-removing polynomial (to be explained in the next subsection).

After recovering the coefficients of M in the plaintext slots, we need to move them back to their original coefficient positions. The ciphertext still contains bounded evaluation error. This step is called SlotToCoeff, which is an exact inverse procedure of CoeffToSlot. We also learned in Summary D-2.9.3 (in §D-2.9.3) that the inverse matrix of $n^{-1} \cdot \tilde{W} \cdot I_n^R$ is $\tilde{W}^*$, where:

$$\tilde{W}^* = \begin{bmatrix} 1 & (\omega^{J(0)}) & (\omega^{J(0)})^2 & \cdots & (\omega^{J(0)})^{n-1} \\ 1 & (\omega^{J(1)}) & (\omega^{J(1)})^2 & \cdots & (\omega^{J(1)})^{n-1} \\ 1 & (\omega^{J(2)}) & (\omega^{J(2)})^2 & \cdots & (\omega^{J(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J(\frac{n}{2}-1)})^{n-1} \\ 1 & (\omega^{J_*(0)}) & (\omega^{J_*(0)})^2 & \cdots & (\omega^{J_*(0)})^{n-1} \\ 1 & (\omega^{J_*(1)}) & (\omega^{J_*(1)})^2 & \cdots & (\omega^{J_*(1)})^{n-1} \\ 1 & (\omega^{J_*(2)}) & (\omega^{J_*(2)})^2 & \cdots & (\omega^{J_*(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} \end{bmatrix}$$

Therefore, the SlotToCoeff step is equivalent to homomorphically multiplying $\tilde{W}^*$ to the output of the noise-eliminating polynomial evaluation.

In the next subsection, we will learn how to design the core algorithm of BFV, the noise elimination polynomial, based on the digit extraction polynomial $G_\varepsilon(x)$.

D-2.11.5 Digit Extraction

For odd p, use balanced base-p digits $d \in \{-(p-1)/2, \ldots, (p-1)/2\}$. Write an input coefficient as $z = \sum_{j=0}^{\varepsilon-1} d_j p^j \pmod{p^\varepsilon}$. Deleting its lowest $v = \varepsilon - r$ balanced digits gives $\lceil z/p^v \rfloor \pmod{p^r}$, as required by decryption. Ordinary nonnegative digits would instead give flooring: for example $p = 3, \varepsilon = 3, m = 1, e' = -1$ gives $z = 8$, and $\lfloor 8/9 \rfloor = 0 \neq 1$ despite $|e'| < 9/2$. For $p = 2$, use nonnegative digits and add 2^{v-1} before deleting v bits, implementing $\lceil z/2^v \rfloor = \lfloor (z + 2^{v-1})/2^v \rfloor$ away from ties. The $p = 2$ formula is only a scalar rounding identity here. It does not extend the cited CRT bootstrapping route to this chapter's power-of-two ring; that route requires a cyclotomic index coprime to p, as explained in §D-2.11.4.

For $j \geq 1$, a lifting polynomial F_j satisfies

$$F_j(d + p^j u) \equiv d \pmod{p^{j+1}}$$

for each allowed lowest digit d and integer u. The index j denotes a known precision in the algorithm; it need not be the first nonzero higher digit of a secret input. Let $L_\varepsilon = F_{\varepsilon-1} \circ \cdots \circ F_1$.

1st Recursion: $F_1(z) \equiv d_0 \pmod{p^2}$.

2nd Recursion: $F_2 \circ F_1(z) \equiv d_0 \pmod{p^3}$.

3rd Recursion: $F_3 \circ F_2 \circ F_1(z) \equiv d_0 \pmod{p^4}$.

$(\varepsilon - 1)$-th Recursion: $L_\varepsilon(z) \equiv d_0 \pmod{p^\varepsilon}$.

The digit-removal operation is therefore

$$G_\varepsilon(z) = \frac{z - L_\varepsilon(z)}{p} \pmod{p^{\varepsilon-1}}.$$

This is exact integer division of a divisible representative, with the output modulus reduced by one power of p. It is not multiplication by an inverse of p in $\mathbb{Z}_{p^\varepsilon}$, where such an inverse does not exist. Even if $p^{-1} \bmod q$ exists, multiplying ciphertext components by it is a different operation and can amplify noise. Thus G_ε denotes a polynomial-evaluation-and-division procedure, not a single polynomial over $\mathbb{Z}_{p^\varepsilon}$. Repeating

$$G_{r+1} \circ \cdots \circ G_{\varepsilon-1} \circ G_\varepsilon(z)$$

deletes $\varepsilon - r$ digits. Each invocation of G_j computes its own lowest digit using $F_1, \ldots, F_{j-1}$, starting at F_1 even when $r > 1$.

Scaling Factor Re-interpretation: Suppose a homomorphic subtraction leaves a plaintext representative $u = pw \pmod{p^j}$ and ciphertext phase $\Delta_j u + E_j$, with $\Delta_j = \lfloor q/p^j \rfloor$. With a consistent divisible centered representative, the same ciphertext can be interpreted at plaintext modulus p^{j-1} as

$$\Delta_j pw + E_j = \Delta_{j-1} w + (E_j - \delta_j w), \qquad \delta_j = \Delta_{j-1} - p\Delta_j \in \{0, \ldots, p-1\}.$$

This avoids multiplying ciphertext components by $p^{-1} \bmod q$. The term $-\delta_j w$ is a change in the error measured against the floored scale. Against the exact scale it cancels:

$$\frac{q}{p^j}(pw) + \mathcal{E}_j = \frac{q}{p^{j-1}} w + \mathcal{E}_j.$$

Thus exact division adds no natural error and increases the decoder tolerance. Both descriptions are equivalent: one must not add the floor-scale drift again to a bound already expressed in natural-error

coordinates. Consistent divisible centered coefficient representatives make the identity exact, including when $p^j \nmid q$.

The floored scales are $\lfloor q/p^{\varepsilon} \rfloor, \lfloor q/p^{\varepsilon-1} \rfloor, \ldots, \lfloor q/p^{r} \rfloor$. A plaintext added to a ciphertext must use the current scale, while ciphertext-to-plaintext multiplication uses an unscaled multiplier as in §D-2.6. Polynomial evaluation changes the natural error; exact division preserves it while changing the plaintext modulus. The explicit recurrence in §D-2.11.6 checks every intermediate decoder condition.

The remaining local algebraic task is to construct the lifting polynomials.

Designing $F_{\varepsilon'}(z_i)$: We give a precision-aware interpolation construction. Let $j = \varepsilon' \geq 1$ and let D be the chosen set of p consecutive digit representatives. For every $d \in D$, Fermat's theorem makes $(d^p - d)/p$ an integer. Interpolate a polynomial H_j of degree at most $p-1$ over $\mathbb{Z}_{p^j}$ with

$$H_j(d) \equiv (d^p - d)/p \pmod{p^j} \quad (d \in D).$$

Every difference between distinct digit nodes is a unit modulo p^j, so Lagrange interpolation is valid. Choose integer lifts of the coefficients and define

$$F_j(X) = X^p - pH_j(X).$$

Proof. For $z = d + p^j u$, the binomial theorem gives $z^p \equiv d^p \pmod{p^{j+1}}$. Integer-coefficient polynomial evaluation also gives $H_j(z) \equiv H_j(d) \pmod{p^j}$. Hence

$$F_j(z) \equiv d^p - pH_j(d) \equiv d \pmod{p^{j+1}}.$$

□

This proves the lifting property used in each recursion. The degree is at most p, rather than $p-1$. Interpolating only the separate digit functions over $\mathbb{F}_p$ and reusing arbitrary integer coefficient lifts is insufficient at higher powers: carries affect the higher digits. The coefficients must be constructed at the required precision, as above. This local existence proof does not by itself bound the depth or noise of an optimized homomorphic implementation.

D-2.11.6 Summary

The following summarizes the generalized plaintext transformation. The recurrence after the box makes its correctness conditions explicit: it accounts for the low-modulus rounding, the general linear transforms, all homomorphic gates and the final decoder. Security and optimization are separate questions.

⟨Summary D-2.11.6⟩ BFV Bootstrapping

Let $t = p^r$, $\varepsilon > r$, $P = p^{\varepsilon}$, $\Delta = \lfloor q/p^r \rfloor$ and $\Delta' = \lfloor q/P \rfloor$. The complete CRT procedure assumes the compatible ring conditions in §D-2.11.4; in this chapter's power-of-two ring, p is odd.

1. **Modulus Switch:** Compute $A' = [\lceil (P/q)A \rfloor]_P$ and $B' = [\lceil (P/q)B \rfloor]_P$. Their phase satisfies $A'S + B' \equiv p^{\varepsilon-r}M + E' \pmod{P}$, with the complete E' from §D-2.11.2. Require $\|E'\|_\infty < p^{\varepsilon-r}/2$.
2. **Homomorphic Decryption:** With an encryption of $\Delta' S$ at modulus q, compute $A' \cdot \mathsf{RLWE}_{S,\sigma}(\Delta' S) + (0, \Delta' B')$. Track both the key-noise term and the plaintext-reduction remainder as in §D-2.11.3.

3. **CoeffToSlot:** Homomorphically place the coefficients of $Z = [p^{\varepsilon-r}M + E']_P$ into slots. In the scalar-split case the slot transformation is $n^{-1}\tilde{W}I_n^R$ (§D-2.9.3); for extension slots, use the d maps $\mathcal{T}_u$ constructed in §D-2.11.4.
4. **Digit Extraction:** For each current precision $j = \varepsilon, \varepsilon-1, \ldots, r+1$, extract its lowest digit with $L_j = F_{j-1} \circ \cdots \circ F_1$ and implement the exact quotient $G_j(z) = (z - L_j(z))/p \pmod{p^{j-1}}$. Use balanced digits for odd p, or the stated rounding offset for $p = 2$. The final slot is $m_i \pmod{p^r}$, so any unreduced representative differs by $k_i'p^r$, not by $k_i'p$ when $r > 1$. **Scaling Factor Re-interpretation:** At each division, reinterpret the scale from $\lfloor q/p^j \rfloor$ to $\lfloor q/p^{j-1} \rfloor$ and include the drift $-\delta_j w$ derived above. The final scale is $\lfloor q/p^r \rfloor = \Delta$. No modular inverse of p is assumed.
5. **SlotToCoeff:** Apply the inverse linear transform $\tilde{W}^*$ in the scalar-split case, or sum the $\mathcal{U}_u$ outputs in the extension-slot case, recovering an encryption with phase $\Delta M + E^{\langle b \rangle} \pmod{q}$. The complete output error includes all evaluation, scale-drift and plaintext-reduction terms and must satisfy the BFV decoder bound.

An explicit conditional correctness certificate. Use the natural error $\mathcal{E}_T$ defined in §D-2.11.1. Let C_R satisfy $\|UV\|_\infty \le C_R\|U\|_\infty\|V\|_\infty$, and let h_S bound the infinity operator norm of multiplication by the secret. In the negacyclic ring, one may take $C_R = n$ and $h_S = \|S\|_1$. For a public polynomial C or automorphism τ_a, let λ_C or λ_a be its coefficient infinity operator norm. These are computable matrix norms, also for general cyclotomic rings.

Use exact integer gadgets with L digits bounded by D_g, and raw evaluation-key row phases $g_iS_{\text{old}} + E_i^{\text{key}}$ under the output secret, with $\|E_i^{\text{key}}\|_\infty \le B_{\text{key}}$. A key switch adds at most

$$B_{\text{ks}} = C_R L D_g B_{\text{key}}$$

to the natural-error bound. No independence assumption is needed. If input bounds are B_1, B_2, the following valid bounds can be propagated, rounding them upward if desired:

$$\begin{aligned} B_{\text{add/sub}} &= B_1 + B_2, \qquad B_{\text{plain mult}} = \lambda_C B_1, \\ B_{\text{plain add}} &= B_1 + (r_T/T)\|C\|_\infty, \\ B_{\text{auto}} &= \lambda_a B_1 + B_{\text{ks}}, \qquad B_{\text{exact div}} = B_1, \\ \kappa_i &= (h_S + 2)/2 + B_i/q. \end{aligned}$$

The plaintext addition uses $\lfloor q/T \rfloor C$; exact division requires a plaintext divisible by p and changes T to T/p. Standard FV multiplication gives

$$\begin{aligned} B_{\text{mult}} = C_R \left[\frac{T}{2}(B_1 + B_2) + T(\kappa_1 B_2 + \kappa_2 B_1) + \frac{T}{q} B_1 B_2 \right] \\ + \frac{1 + h_S + h_S^2}{2} + B_{\text{ks}}. \end{aligned}$$

To verify this bound, write centered component lifts as $AS + B = (q/T)M + \mathcal{E} + qK$. Their norms give $\|K\|_\infty \le \kappa$. Expanding the integer tensor product and multiplying by T/q leaves the natural-error terms

$$\begin{aligned} \mathcal{E}_{\text{mult}} = M_1\mathcal{E}_2 + M_2\mathcal{E}_1 + T(\mathcal{E}_1 K_2 + \mathcal{E}_2 K_1) + \frac{T}{q}\mathcal{E}_1\mathcal{E}_2 \\ + \rho_0 + \rho_1 S + \rho_2 S^2 + E_{\text{ks}} \pmod{q}. \end{aligned}$$

All omitted carry terms are integer multiples of q. Each tensor-component rounding error ρ_i has norm at most $1/2$, giving the displayed rounding bound. Reducing the message modulo T also changes the exact scaled message only by multiples of q. This proves the recurrence, rather than assuming that multiplication is correct.

For the first modulus switch, an input natural-error bound B_{in} is sufficient if

$$\frac{P}{q}B_{\mathrm{in}} + \frac{1+h_{\mathrm{in}}}{2} < \frac{p^{\varepsilon-r}}{2}.$$

Here h_{in} bounds multiplication by the input secret. The old error is now part of the integer plaintext Z; it is not the error of the new evaluation ciphertext. If the encryption of the input secret has natural-error bound B_s, the homomorphic decryption starts the evaluation ledger with

$$B_Z \le \lambda_{A'} B_s + (r_P/P)\|B'\|_\infty.$$

For a floor-scaled bootstrapping-key encryption with integer error bounded by B_{key}, one may use $B_s = B_{\mathrm{key}} + (r_P/P)\|S_{\mathrm{in}}\|_\infty$. The evaluation modulus may be chosen independently of the input modulus; the displayed formulas use the same q for simplicity.

Require the bound $B_C < q/(2T_C)$ at every ciphertext node of the explicit circuit. The modulus-switch inequality selects the correct rounded message; the trace-dual maps place its coefficients in scalar slots; the lifting polynomials and exact divisions recover those coefficients modulo p^r; the inverse maps reconstruct M. Induction with the above recurrence proves that the final ciphertext decrypts to M. In floor-scale notation its output error is $E^{\langle b\rangle} = \mathcal{E}_{p^r} + (r_{p^r}/p^r)M$. The new bound need not be smaller than every possible input error; it must fit the intended subsequent computation.

The exact public-evaluation fixtures in `audit/check_bootstrap.py` cover $\Phi_4, p = 5$ with scalar splitting, $\Phi_8, p = 3$ with degree-two extension slots at $r = 1$ and $r = 2$, and the coprime binary alternative $\Phi_3, p = 2$. Their ciphertext moduli are not divisible by the plaintext moduli, and their input and evaluation-key errors are nonzero. A separate checker audits every intermediate ciphertext after public evaluation; no secret or decryption callback enters the evaluator. These tiny-ring fixtures verify the implementation and algebra, not cryptographic security or the optimized thin-bootstrapping algorithms of the reference.

The purpose of Homomorphic Decryption: It converts a low-modulus ciphertext's secret-dependent phase into an encrypted plaintext at a fresh evaluation modulus and suitable scale. This makes the remaining rounding circuit evaluable without revealing the secret or message. Scale reinterpretation is one component of the digit-removal implementation, not by itself a complete explanation or correctness proof of bootstrapping.

D-3 CKKS Scheme

The CKKS scheme supports SIMD approximate arithmetic on complex numbers, including real numbers as a special case. It is useful for numerical workloads such as machine learning. BFV, BGV, and TFHE have exact finite plaintext spaces, but can also implement fixed-point or floating-point arithmetic through suitable encodings or circuits; the distinction is their native arithmetic and error semantics. [19]

For a power-of-two $n \geq 2$, distinguish the real algebra $\mathcal{R}_{\langle n \rangle} = \mathbb{R}[X]/(X^n + 1)$, the integer ring $R = \mathbb{Z}[X]/(X^n + 1)$, and the ciphertext ring $R_q = R/qR$. The inverse canonical embedding maps $n/2$ complex slots to a polynomial $M \in \mathcal{R}_{\langle n \rangle}$. Encoding then scales its coefficients by $\Delta > 0$ and rounds them to obtain an integer polynomial in R, which is encrypted modulo q.

Overall, CKKS's encryption procedure is as follows:
1. $\underline{\mathsf{Encoding}_1}$: Encode the targeted input complex number as a real number
2. $\underline{\mathsf{Encoding}_2}$: Encode the real number as an integer
3. $\underline{\mathsf{Encryption}}$: Encrypt the integer using RLWE

The encrypted RLWE ciphertext supports homomorphic addition and multiplication.

At the end of all homomorphic operations, CKKS's decryption procedure is as follows:
1. $\underline{\mathsf{Decryption}}$: Decrypt the RLWE ciphertext into a plaintext integer
2. $\underline{\mathsf{Decoding}_1}$: Decode the integer to a real number
3. $\underline{\mathsf{Decoding}_2}$: Decode the real number to a complex number

CKKS generally introduces encoding error when the scaled inverse embedding is rounded to the integer lattice. Encryption noise and subsequent arithmetic, key-switching, rescaling, and approximation errors also contribute to the decoded error. This is not specifically an error from square roots, and some inputs can be encoded exactly. Unlike exact plaintext decryption in BFV, general CKKS decryption retains the numerical error.

CKKS and BFV share ring arithmetic and key-switching techniques, but their encoding, decoding, scale management, multiplication, and bootstrapping must respect different plaintext semantics. Both use rings: $\mathbb{C}$ is itself a field, hence a ring. In CKKS, the relevant distinction is approximate arithmetic in characteristic zero instead of exact arithmetic modulo a plaintext modulus.

Required Background

- §A-1: Modulo Arithmetic
- §A-2: Group
- §A-3: Field
- §A-4: Order
- §A-5: Polynomial Ring
- §A-6: Decomposition
- §A-7: Roots of Unity
- §A-8: Cyclotomic Polynomial
- §A-11: Vector and Matrix
- §A-12: Euler's Formula
- §A-13: Modulo Rescaling

- §A-14: Chinese Remainder Theorem
- §A-15: Taylor Series
- §A-10: Lagrange's Polynomial Interpolation
- §A-16: Efficient Polynomial Multiplication by FFT and NTT
- §B-1: Lattice-based Cryptography
- §B-3: RLWE Cryptosystem
- §B-4: GLWE Cryptosystem
- §B-5: GLev
- §C-1: GLWE Ciphertext-to-Ciphertext Addition
- §C-2: GLWE Ciphertext-to-Plaintext Addition
- §C-3: GLWE Ciphertext-to-Plaintext Multiplication
- §C-4.4: RLWE Modulus Switching
- §C-5: GLWE Key Switching
- §D-2: BFV Scheme

D-3.1 Encoding and Decoding

CKKS's encoding and decoding is fundamentally very similar to BFV's batch encoding scheme. BFV designs its batch encoding scheme (Summary D-2.3 in §D-2.3) based on the updated $\tilde{W}$ and $\tilde{W}^*$ matrices (Summary D-2.9.3 in §D-2.9). That is, BFV decodes a polynomial into an input slot vector by evaluating the polynomial at each root of X^n+1, which is the primitive ($\mu=2n$)-th root of unity (i.e., $\vec{v}=\tilde{W}^*\cdot\vec{m}$), and encodes an input slot vector into a polynomial by inversing this operation (i.e., $\vec{m}=n^{-1}\cdot\tilde{W}\cdot I_n^R\cdot\vec{v}$). This encoding and decoding scheme is designed based on Summary A-11.7 (§A-11.7) which designs the isomorphic mapping between n-slot vectors in a ring (finite field) and $(n-1)$-degree (or lesser degree) polynomials as follows:

$\sigma: f(x)\in\mathbb{Z}_t[X]/F(X) \longrightarrow (f(\omega^1), f(\omega^3)), \cdots, f(\omega^{2n-1}))\in\mathbb{Z}_t^n$

, where $\omega=g^{\frac{t-1}{2n}}$ is a root of (i.e., primitive ($\mu=2n$)-th root of unity) of the ($\mu=2n$)-th cyclotomic polynomial X^n+1 defined over a prime modulo t ring.

CKKS uses the same evaluation/interpolation principle over complex roots of unity. Complex numbers also form a ring; here the full evaluation vector lies in the real algebra $\hat{\mathbb{C}}^n$ with conjugate symmetry. Summary A-11.7 (§A-11.7) gives the following canonical embedding:

$\sigma_c: f(X)\in\mathbb{R}[X]/(X^n+1)\longrightarrow (f(\omega), f(\omega^3), f(\omega^5), \cdots, f(\omega^{2n-1}))\in\hat{\mathbb{C}}^n\ (\longrightarrow\mathbb{C}^{\frac{n}{2}})$

, where $\omega=e^{i\pi/n}$ is a root (i.e., the primitive ($\mu=2n$)-th root) of the ($\mu=2n$)-th cyclotomic polynomial X^n+1 defined over complex numbers, and $\hat{\mathbb{C}}^n$ is n-dimensional complex special vector space whose second-half elements are reverse-ordered conjugates of the first-half elements. And $\hat{\mathbb{C}}^n$ is isomorphic to $\mathbb{C}^{\frac{n}{2}}$, because the second-half elements of $\hat{\mathbb{C}}^n$ are automatically determined by its first-half elements. Therefore, the σ_c mapping is essentially an isomorphism between $\frac{n}{2}$-slot complex vectors $\vec{v}\in\mathbb{C}^{\frac{n}{2}}$ and $(n-1)$-degree (or lesser degree) real-number polynomials $\mathbb{R}[X]/(X^n+1)$. Therefore, CKKS' batch encoding scheme encodes an $\frac{n}{2}$-slot complex input vector into an $(n-1)$-degree (or lesser degree) real-number polynomial, and the decoding process is a reverse of this.

In addition, remember that in BFV, we updated W and W^T to $\tilde{W}$ and $\tilde{W}^*$ (Summary D-2.9.3 in §D-2.9.3) to support homomorphic rotation of input vector slots. Likewise, the CKKS batch encoding scheme uses $\tilde{W}$ and $\tilde{W}^*$ instead of W and W^T in order to support homomorphic rotation. Therefore, the CKKS batch encoding scheme's isomorphic mapping is updated as follows:

$\sigma_c: f(X)\in\mathbb{R}[X]/(X^n+1)\longrightarrow \big(f(\omega^{J(0)}), f(\omega^{J(1)}), f(\omega^{J(2)}), \cdots, f(\omega^{J(\frac{n}{2}-1)}),$
$\cdots, f(\omega^{J_*(0)}), f(\omega^{J_*(1)}), f(\omega^{J_*(2)}), \cdots, f(\omega^{J_*(\frac{n}{2}-1)})\big)\in\hat{\mathbb{C}}^n\ (\longrightarrow\mathbb{C}^{\frac{n}{2}})$

, where $J(h)=5^h \bmod 2n$, a rotation helper formula.

The encoding schemes of BFV and CKKS have the following differences:

- **Type of Input Slot Values:** BFV's input slot values are n integers modulo t, which are encoded into n polynomial coefficients (i.e., modulo-t integers). On the other hand, CKKS's input slot values are $\frac{n}{2}$ complex numbers, which are encoded into n polynomial coefficients (i.e., real numbers).
- **Type of Polynomial Coefficients:** BFV encodes into $\mathbb{Z}_t[X]/(X^n+1)$. CKKS first interpolates real coefficients and then rounds the scaled coefficients to integers. The encrypted polynomial has integer coefficients in both schemes.
- **Scaling Factor:** BFV uses $\Delta=\lfloor q/t\rfloor$ at ciphertext modulus q. CKKS chooses Δ according to the desired precision, message bounds, and modulus chain. Correct interpretation requires a centered no-wrap bound on the whole decryption phase, including all accumulated errors; zero and negative

coefficients are allowed. At every level, require $\|\Delta M + E\|_\infty < q_l/2$.

- **Encoding Precision:** In the case of BFV, during its decoding process, BFV's down-scaled polynomial coefficients $\frac{\Delta \cdot \vec{m}}{\Delta}$ preserve the precision of input values . On the other hand, CKKS's down-scaled polynomial coefficients may lose their precision if their original input values have too many decimal digits so that the scaling factor cannot left-shift all of them to make them part of the integer domain, which means that some lower decimal digits of the input value may be rounded off, which loses precision of the original input. For example, suppose the polynomial coefficient $m_i = \frac{1}{3} = 0.33333\cdots$, and the scaling factor $\Delta = 100$. Then, the scaled coefficient $\lceil \Delta m_i \rfloor = 33$, and down-scaling it gives $\frac{33}{100} = 0.33$. Since $0.33 \neq 0.33333\cdots$, CKKS's encoding and decoding process does not always guarantee exact precision. Due to this encoding error, CKKS is called an *approximate* encryption scheme. The impact of this encoding error can grow over homomorphic operations which increases the magnitude of error and the decoded result would gradually become more deviated from the expected exact value. One way to reduce CKKS's encoding error is to increase Δ, and thereby left-shift more decimal digits to make them part of the scaled integer digits.

Structure of $\vec{v}' \in \hat{\mathbb{C}}^n$: Note that the original decoding scheme for $\vec{v}'$ described in Summary A-11.7 (§A-11.7) was:

$\vec{v}' = (\ M(\omega),\ M(\omega^3),\ M(\omega^5), \cdots, M(\omega^{2n-3}),\ M(\omega^{2n-1}))$

, which decodes to a Hermitian vector:

$\vec{v}' = (v_0, v_1, \cdots, v_{\frac{n}{2}-1}, \overline{v}_{\frac{n}{2}-1}, \cdots, \overline{v}_1, \overline{v}_0)$

, whose second-half elements are reverse-ordered conjugates of the first-half elements.

However, by replacing W and W^T with $\tilde{W}$ and $\tilde{W}^*$, we changed the above decoding scheme to the following that supports homomorphic rotation:

$\vec{v}' = (\ M(\omega^{J(0)}),\ M(\omega^{J(1)}),\ M(\omega^{J(2)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),\ M(\omega^{J_*(0)}),\ M(\omega^{J_*(1)}), \cdots, M(\omega^{J_*(\frac{n}{2}-1)})\ )$

$= (\ M(\omega^{J(0)}),\ M(\omega^{J(1)}),\ M(\omega^{J(2)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}),\ M(\overline{\omega}^{J(0)}),\ M(\overline{\omega}^{J(1)}), \cdots, M(\overline{\omega}^{J(\frac{n}{2}-1)})\ )$

$\triangleright$ because $\omega^{-1} = (e^{\frac{i\pi}{n}})^{-1} = e^{\frac{-i\pi}{n}} = \overline{\omega}$, given $J(h) = 5^h \bmod 2n$, and $J_*(h) = -5^h \bmod 2n$

, which decodes to a *forward-ordered* (not reverse-ordered) Hermitian vector as follows:

$\vec{v}' = (v_0, v_1, \cdots, v_{\frac{n}{2}-1}, \overline{v}_0, \overline{v}_1, \cdots, \overline{v}_{\frac{n}{2}-1})$

, whose second-half elements are conjugates of the first-half elements with the same order. Upon homomorphic rotation (which will be explained in §D-3.9), just like in BFV's homomorphic rotation, the first-half elements and the second-half elements of $\vec{v}'$ rotate within their own group in a wrapping manner.

We summarize CKKS's encoding and decoding procedure as follows, which is similar to BFV's encoding and decoding procedure (described in Summary D-2.2.5 in §D-2.2.5):

⟨Summary D-3.1⟩ CKKS's Encoding and Decoding

Input: An $\frac{n}{2}$-slot complex vector $\vec{v} = (v_0, v_1, \cdots, v_{\frac{n}{2}-1}) \in \mathbb{C}^{\frac{n}{2}}$

Encoding:

1. Convert (i.e., isomorphically transform) $\vec{v}$ into an n-slot *forward-ordered* Hermitian vector $\vec{v}'$

as follows:
$\vec{v}' = (v_0, v_1, \cdots, v_{\frac{n}{2}-1}, \overline{v}_0, \overline{v}_1, \cdots, \overline{v}_{\frac{n}{2}-1}) \in \hat{\mathbb{C}}^n$

2. Convert $\vec{v}'$ into a real number vector $\vec{m}$ by applying the transformation $\vec{m} = \dfrac{\tilde{W} \cdot I_n^R \cdot \vec{v}'}{n}$, where $\tilde{W}$ is a basis of the n-dimensional vector space crafted as follows:

$$\tilde{W} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-2)}) & \cdots & (\omega^{J(0)}) & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-2)}) & \cdots & (\omega^{J_*(0)}) \\ (\omega^{J(\frac{n}{2}-1)})^2 & (\omega^{J(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J(0)})^2 & (\omega^{J_*(\frac{n}{2}-1)})^2 & (\omega^{J_*(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J_*(0)})^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ (\omega^{J(\frac{n}{2}-1)})^{n-1} & (\omega^{J(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J(0)})^{n-1} & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} & (\omega^{J_*(\frac{n}{2}-2)})^{n-1} & \vdots & (\omega^{J_*(0)})^{n-1} \end{bmatrix}$$

▷ where $\omega = e^{i\pi/n} = \cos\left(\dfrac{\pi}{n}\right) + i \sin\left(\dfrac{\pi}{n}\right)$, $J(h) = 5^h \bmod 2n$, and $J_*(h) = -5^h \bmod 2n$

$$= \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-2)}) & \cdots & (\omega^{J(0)}) & (\overline{\omega}^{J(\frac{n}{2}-1)}) & (\overline{\omega}^{J(\frac{n}{2}-2)}) & \cdots & (\overline{\omega}^{J(0)}) \\ (\omega^{J(\frac{n}{2}-1)})^2 & (\omega^{J(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J(0)})^2 & (\overline{\omega}^{J(\frac{n}{2}-1)})^2 & (\overline{\omega}^{J(\frac{n}{2}-2)})^2 & \cdots & (\overline{\omega}^{J(0)})^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ (\omega^{J(\frac{n}{2}-1)})^{n-1} & (\omega^{J(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J(0)})^{n-1} & (\overline{\omega}^{J(\frac{n}{2}-1)})^{n-1} & (\overline{\omega}^{J(\frac{n}{2}-2)})^{n-1} & \vdots & (\overline{\omega}^{J(0)})^{n-1} \end{bmatrix}$$

▷ because $\omega^{-1} = e^{\frac{-i\pi}{n}} = \overline{e^{\frac{i\pi}{n}}} = \overline{\omega}$

3. Convert $\vec{m}$ into a scaled integer vector $\lceil \Delta\vec{m} \rfloor \approx \Delta\vec{m}$, where Δ is a scaling factor bigger than 1 such that Δm_i never overflows or underflows q_0 (i.e., $0 \leq \Delta m_i < q_0$ or $-\dfrac{q_0}{2} \leq \Delta m_i < \dfrac{q_0}{2}$) in all cases, even across all homomorphic operations. The finally encoded plaintext polynomial is $\Delta M = \sum_{i=0}^{n-1} \lceil \Delta m_i \rfloor X^i \in \mathbb{Z}_q[X]/(X^n + 1)$. The rounding process of $\lceil \Delta\vec{m} \rfloor$ during the encoding process causes an encoding error, which makes CKKS an approximate encryption scheme.

Decoding: From the plaintext polynomial $\Delta M = \sum_{i=0}^{n-1} \Delta m_i X^i$, recover $\vec{m} = \dfrac{\Delta\vec{m}}{\Delta}$. Then, compute $\vec{v}' = \tilde{W}^* \cdot \vec{m}$, where:

$$\tilde{W}^* = \begin{bmatrix} 1 & (\omega^{J(0)}) & (\omega^{J(0)})^2 & \cdots & (\omega^{J(0)})^{n-1} \\ 1 & (\omega^{J(1)}) & (\omega^{J(1)})^2 & \cdots & (\omega^{J(1)})^{n-1} \\ 1 & (\omega^{J(2)}) & (\omega^{J(2)})^2 & \cdots & (\omega^{J(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J(\frac{n}{2}-1)})^{n-1} \\ 1 & (\omega^{J_*(0)}) & (\omega^{J_*(0)})^2 & \cdots & (\omega^{J_*(0)})^{n-1} \\ 1 & (\omega^{J_*(1)}) & (\omega^{J_*(1)})^2 & \cdots & (\omega^{J_*(1)})^{n-1} \\ 1 & (\omega^{J_*(2)}) & (\omega^{J_*(2)})^2 & \cdots & (\omega^{J_*(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} 1 & (\omega^{J(0)}) & (\omega^{J(0)})^2 & \cdots & (\omega^{J(0)})^{n-1} \\ 1 & (\omega^{J(1)}) & (\omega^{J(1)})^2 & \cdots & (\omega^{J(1)})^{n-1} \\ 1 & (\omega^{J(2)}) & (\omega^{J(2)})^2 & \cdots & (\omega^{J(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J(\frac{n}{2}-1)})^{n-1} \\ 1 & (\overline{\omega}^{J(0)}) & (\overline{\omega}^{J(0)})^2 & \cdots & (\overline{\omega}^{J(0)})^{n-1} \\ 1 & (\overline{\omega}^{J(1)}) & (\overline{\omega}^{J(1)})^2 & \cdots & (\overline{\omega}^{J(1)})^{n-1} \\ 1 & (\overline{\omega}^{J(2)}) & (\overline{\omega}^{J(2)})^2 & \cdots & (\overline{\omega}^{J(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\overline{\omega}^{J(\frac{n}{2}-1)}) & (\overline{\omega}^{J(\frac{n}{2}-1)})^2 & \cdots & (\overline{\omega}^{J(\frac{n}{2}-1)})^{n-1} \end{bmatrix}$$

, and extract only the first $\frac{n}{2}$ elements in the *forward-ordered* Hermitian vector $\vec{v}_r$ to recover the input vector $\vec{v}$.

CKKS's Approximation Property: Let $P = \lceil \Delta M \rfloor$ be coefficient-wise nearest-integer rounding. Then $P = \Delta M + E_{\text{enc}}$ with $\|E_{\text{enc}}\|_\infty \leq 1/2$. Each decoded slot has encoding error at most $n/(2\Delta)$ by the triangle inequality at unit-modulus roots. This conservative coefficient-norm bound is distinct from the canonical embedding norm used in CKKS analyses. Encryption and later computation add further errors; approximate decoding does not automatically remove them.

D-3.1.1 Example

Suppose our input complex vector's dimension $\frac{n}{2} = 2$, the bounding polynomial degree $n = 4$, and the scaling factor $\Delta = 1024$.

Our basis of the n-dimensional vector space

$$\tilde{W} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \omega^{J(1)} & \omega^{J(0)} & \overline{\omega}^{J(1)} & \overline{\omega}^{J(0)} \\ (\omega^{J(1)})^2 & (\omega^{J(0)})^2 & (\overline{\omega}^{J(1)})^2 & (\overline{\omega}^{J(0)})^2 \\ (\omega^{J(1)})^3 & (\omega^{J(0)})^3 & (\overline{\omega}^{J(1)})^3 & (\overline{\omega}^{J(0)})^3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \omega^5 & \omega & \overline{\omega}^5 & \overline{\omega} \\ \omega^2 & \omega^2 & \overline{\omega^2} & \overline{\omega}^2 \\ \omega^7 & \omega^3 & \overline{\omega}^7 & \overline{\omega}^3 \end{bmatrix}$$

, where $\omega = e^{i\pi/n} = \cos\left(\frac{\pi}{n}\right) + i \sin\left(\frac{\pi}{n}\right)$

Given this setup, suppose we have the input complex vector $\vec{v} = (1.1 + 4.3i,\ 3.5 - 1.4i)$ to encode.

First, construct the forward-ordered Hermitian vector $\vec{v}_r = (1.1+4.3i,\ 3.5-1.4i,\ 1.1-4.3i,\ 3.5+1.4i)$.

Next, convert the complex vector $\vec{v}_r$ into a real number vector $\vec{m}$ by applying the transformation:

$$\vec{m} = \frac{\tilde{W} \cdot I_n^R \cdot \vec{v}_r}{n} = \frac{1}{4} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ \omega^5 & \omega & \overline{\omega}^5 & \overline{\omega} \\ \omega^2 & \omega^2 & \overline{\omega}^2 & \overline{\omega}^2 \\ \omega^7 & \omega^3 & \overline{\omega}^7 & \overline{\omega}^3 \end{bmatrix} \cdot \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1.1 + 4.3i \\ 3.5 - 1.4i \\ 1.1 - 4.3i \\ 3.5 + 1.4i \end{bmatrix}$$

$$= \frac{\tilde{W} \cdot I_n^R \cdot \vec{v}\prime}{n} = \frac{1}{4} \cdot \begin{bmatrix} 1 & 1 & 1 & 1 \\ \overline{\omega} & \overline{\omega}^5 & \omega & \omega^5 \\ \overline{\omega}^2 & \overline{\omega}^2 & \omega^2 & \omega^2 \\ \overline{\omega}^3 & \overline{\omega}^7 & \omega^3 & \omega^7 \end{bmatrix} \cdot \begin{bmatrix} 1.1+4.3i \\ 3.5-1.4i \\ 1.1-4.3i \\ 3.5+1.4i \end{bmatrix}$$

$$= \frac{1}{4} \cdot \begin{bmatrix} (1.1+4.3i)+(3.5-1.4i)+(1.1-4.3i)+(3.5+1.4i) \\ (1.1+4.3i)\overline{\omega}+(3.5-1.4i)\overline{\omega}^5+(1.1-4.3i)\omega+(3.5+1.4i)\omega^5 \\ (1.1+4.3i)\overline{\omega^2}+(3.5-1.4i)\overline{\omega^2}+(1.1-4.3i)\omega^2+(3.5+1.4i)\omega^2 \\ (1.1+4.3i)\overline{\omega}^3+(3.5-1.4i)\overline{\omega}^7+(1.1-4.3i)\omega^3+(3.5+1.4i)\omega^7 \end{bmatrix}$$

$$= \frac{1}{4} \cdot \begin{bmatrix} (1.1+4.3i)+(3.5-1.4i)+(1.1-4.3i)+(3.5+1.4i) \\ (1.1+4.3i)\overline{\omega}+(3.5-1.4i)\overline{\omega}^5+(1.1-4.3i)\omega+(3.5+1.4i)\omega^5 \\ (1.1+4.3i)\overline{\omega^2}+(3.5-1.4i)\overline{\omega^2}+(1.1-4.3i)\omega^2+(3.5+1.4i)\omega^2 \\ (1.1+4.3i)\overline{\omega}^3+(3.5-1.4i)\overline{\omega}^7+(1.1-4.3i)\omega^3+(3.5+1.4i)\omega^7 \end{bmatrix}$$

$$= \frac{1}{4} \cdot \begin{bmatrix} 9.2 \\ 1.1(\overline{\omega}+\omega)+4.3i(\overline{\omega}-\omega)+3.5(\overline{\omega}^5+\omega^5)-1.4i(\overline{\omega}^5-\omega^5) \\ 1.1(\overline{\omega}^2+\omega^2)+4.3i(\overline{\omega}^2-\omega^2)+3.5(\overline{\omega}^2+\omega^2)-1.4i(\overline{\omega}^2-\omega^2) \\ 1.1(\overline{\omega}^3+\omega^3)+4.3i(\overline{\omega}^3-\omega^3)+3.5(\overline{\omega}^7+\omega^7)-1.4i(\overline{\omega}^7-\omega^7) \end{bmatrix}$$

$$= \frac{1}{4} \cdot \begin{bmatrix} 9.2 \\ 1.1\left(2\cos\frac{\pi}{4}\right)-4.3i\left(2i\sin\frac{\pi}{4}\right)+3.5\left(2\cos\frac{5\pi}{4}\right)+1.4i\left(2i\sin\frac{5\pi}{4}\right) \\ 1.1\left(2\cos\frac{\pi}{2}\right)-4.3i\left(2i\sin\frac{\pi}{2}\right)+3.5\left(2\cos\frac{\pi}{2}\right)+1.4i\left(2i\sin\frac{\pi}{2}\right) \\ 1.1\left(2\cos\frac{3\pi}{4}\right)-4.3i\left(2i\sin\frac{3\pi}{4}\right)+3.5\left(2\cos\frac{7\pi}{4}\right)+1.4i\left(2i\sin\frac{7\pi}{4}\right) \end{bmatrix}$$

$$= \frac{1}{4} \cdot \begin{bmatrix} 9.2 \\ 1.1\left(2\frac{\sqrt{2}}{2}\right)+4.3\left(2\frac{\sqrt{2}}{2}\right)+3.5\left(-2\frac{\sqrt{2}}{2}\right)-1.4\left(-2\frac{\sqrt{2}}{2}\right) \\ 1.1(2\cdot 0)+4.3(2\cdot 1)+3.5(2\cdot 0)-1.4(2\cdot 1) \\ 1.1\left(-2\frac{\sqrt{2}}{2}\right)+4.3\left(2\frac{\sqrt{2}}{2}\right)+3.5\left(2\frac{\sqrt{2}}{2}\right)-1.4\left(-2\frac{\sqrt{2}}{2}\right) \end{bmatrix}$$

$$= 0.25 \cdot \begin{bmatrix} 9.2 \\ 1.1\sqrt{2}+4.3\sqrt{2}-3.5\sqrt{2}+1.4\sqrt{2} \\ 1.1(0)+4.3(2)-3.5(0)-1.4(2) \\ -1.1\sqrt{2}+4.3\sqrt{2}+3.5\sqrt{2}+1.4\sqrt{2} \end{bmatrix} = \begin{bmatrix} 2.3 \\ 0.825\sqrt{2} \\ 1.45 \\ 2.025\sqrt{2} \end{bmatrix} \approx (2.3,\ 1.1667,\ 1.45,\ 2.8638)$$

Convert the real number vector $\vec{m}$ into a scaled integer vector $\Delta\vec{m}$ by Δ-scaling and rounding as follows:

$\Delta\vec{m} \approx \lceil \Delta\vec{m} \rfloor = \lceil 1024 \cdot (2.3,\ 1.1667,\ 1.45,\ 2.8638) \rfloor = (2355,\ 1195,\ 1485,\ 2933)$

Finally, $\vec{v} = (1.1+4.3i,\ 3.5-1.4i)$ has been encoded into the plaintext polynomial $M(X)$ as follows:
$\Delta M(X) = 2355 + 1195X + 1485X^2 + 2933X^3 \in \mathcal{R}_{\langle 4 \rangle}$

To decode $\vec{m}$, we compute:

$$\vec{v'} = \frac{\tilde{W}^* \cdot \Delta\vec{m}}{\Delta} = \begin{bmatrix} 1, \omega, \omega^2, \omega^3 \\ 1, \omega^5, \omega^2, \omega^7 \\ 1, \overline{\omega}, \overline{\omega^2}, \overline{\omega^3} \\ 1, \overline{\omega^5}, \overline{\omega^2}, \overline{\omega^7} \end{bmatrix} \cdot \begin{bmatrix} 2355 \\ 1195 \\ 1485 \\ 2933 \end{bmatrix} \cdot \frac{1}{1024}$$

$$= \begin{bmatrix} 1, \frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2}, i, -\frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2} \\ 1, -\frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2}, i, \frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2} \\ 1, \frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2}, -i, -\frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2} \\ 1, -\frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2}, -i, \frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2} \end{bmatrix} \cdot \begin{bmatrix} 2.2998046875 \\ 1.1669921875 \\ 1.4501953125 \\ 2.8642578125 \end{bmatrix}$$

$$\approx (1.0997 + 4.3007i,\ 3.5000 - 1.4003i,\ 1.0997 - 4.3007i,\ 3.5000 + 1.4003i)$$

Extract the first $\frac{n}{2} = 2$ elements in the Hermitian vector $\vec{v'}$ to recover the input vector:
$(1.0997 + 4.3007i,\ 3.5000 - 1.4003i)$
$\approx (1.1 + 4.3i,\ 3.5 - 1.4i) = \vec{v}$ ▷ The original input vector

The integer rounding of the scaled inverse embedding produces the small difference between the original and decoded slots. The displayed decimal intermediate coefficients are approximations; the integer encoding is computed from the exact expressions involving $\sqrt{2}$.

Source Code: Examples of CKKS encoding can be executed by running this Python script.

D-3.2 Encryption and Decryption

CKKS encryption operates on $P = \lceil \Delta M \rfloor \in R$, not on arbitrary real coefficients directly. The following is the symmetric-key RLWE form; public-key CKKS encryption uses a public RLWE sample and has a corresponding combined error.

⟨Summary D-3.2⟩ CKKS Encryption and Decryption

Choose a small secret $S \in R$, sample $A \leftarrow R_q$ uniformly and a small integer error E_0, and output

$$B = [-AS + P + E_0]_q, \qquad \mathsf{ct} = (A, B) \in R_q^2.$$

Define the raw decryption phase by

$$\mathsf{Phase}_S(\mathsf{ct}) = [B + AS]_q = P + E_0 = \Delta M + E, \qquad E = E_{\text{enc}} + E_0,$$

provided $\|\Delta M + E\|_\infty < q/2$. Here $[\cdot]_q$ selects centered coefficient representatives. Decode by evaluating the phase at the selected roots and dividing by Δ:

$$\widehat{v}_j = \Delta^{-1}\mathsf{Phase}_S(\mathsf{ct})(\omega^{J(j)}) = v_j + \Delta^{-1}E(\omega^{J(j)}).$$

Thus $|\widehat{v}_j - v_j| \le \|E\|_{\text{can},\infty}/\Delta \le n\|E\|_\infty/\Delta$. The condition $|e_i| < \Delta/2$ alone does not imply slot precision $1/\Delta$. Rounding to multiples of $1/\Delta$ cannot remove general CKKS noise.

Below, RLWE^{-1} denotes this raw phase when used in algebraic derivations; division by the tracked scale and slot decoding are separate operations. $\mathsf{RLWE}_{S,\sigma}(P)$ denotes an encryption relation, not a claim that derived ciphertexts retain the original independent error distribution χ_σ.

D-3.3 Ciphertext-to-Ciphertext Addition

CKKS adds ciphertext components as in BFV (Summary D-2.4, §D-2.4). The ciphertexts must have compatible keys, moduli, and scales. The resulting phase error is $E^{\langle 1\rangle} + E^{\langle 2\rangle}$, and the summed phase must satisfy the no-wrap bound.

⟨Summary D-3.3⟩ CKKS Ciphertext-to-Ciphertext Addition

$$\begin{aligned}
&\mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 1\rangle}) + \mathsf{RLWE}_{S,\sigma}(\Delta M^{\langle 2\rangle}) \\
&= (A^{\langle 1\rangle},\ B^{\langle 1\rangle}) + (A^{\langle 2\rangle},\ B^{\langle 2\rangle}) \\
&= (A^{\langle 1\rangle} + A^{\langle 2\rangle},\ B^{\langle 1\rangle} + B^{\langle 2\rangle}) \\
&= \mathsf{RLWE}_{S,\sigma}(\Delta(M^{\langle 1\rangle} + M^{\langle 2\rangle}))
\end{aligned}$$

D-3.4 Ciphertext-to-Plaintext Addition

CKKS adds a plaintext to the B component as in BFV (Summary D-2.5, §D-2.5). Encode the plaintext at the ciphertext's actual scale Δ, and include its encoding error in the error budget. In the following shorthand, $\Delta\Lambda$ denotes the rounded integer encoding.

⟨Summary D-3.4⟩ CKKS Ciphertext-to-Plaintext Addition

$$\begin{aligned}
&\mathsf{RLWE}_{S,\sigma}(\Delta M) + \Delta\Lambda \\
&= (A,\ B) + \Delta\Lambda \\
&= (A,\ B + \Delta \cdot \Lambda) \\
&= \mathsf{RLWE}_{S,\sigma}(\Delta(M + \Lambda))
\end{aligned}$$

D-3.5 Ciphertext-to-Ciphertext Multiplication

CKKS multiplication first tensors ciphertexts at their current modulus, then usually relinearizes and rescales. Tensoring changes the scale from Δ_1, Δ_2 to $\Delta_1\Delta_2$; rescaling by $w_l = q_l/q_{l-1}$ changes it to $\Delta_1\Delta_2/w_l$ and consumes one level. The following derivation uses the sign convention $B = -AS + \Delta M + E \pmod q$, as in Theorem B-4.4 (§B-4.4). We first derive the exact noisy phase, then explain two relinearization methods and rescaling.

D-3.5.1 Synthetic Ciphertext Derivation

For two ciphertexts at the same modulus and secret key, let their phases be $\Delta M^{\langle 1\rangle} + E^{\langle 1\rangle}$ and $\Delta M^{\langle 2\rangle} + E^{\langle 2\rangle}$. Define, modulo q,

$$D_0 = B^{\langle 1\rangle}B^{\langle 2\rangle}, \quad D_1 = A^{\langle 1\rangle}B^{\langle 2\rangle} + A^{\langle 2\rangle}B^{\langle 1\rangle}, \quad D_2 = A^{\langle 1\rangle}A^{\langle 2\rangle}.$$

The exact phase identity is

$$D_0 + D_1 S + D_2 S^2 \equiv \Delta^2 M^{\langle 1\rangle} M^{\langle 2\rangle} + F \pmod q,$$

$$F = \Delta M^{\langle 1\rangle} E^{\langle 2\rangle} + \Delta M^{\langle 2\rangle} E^{\langle 1\rangle} + E^{\langle 1\rangle} E^{\langle 2\rangle}.$$

CKKS decoding retains these error terms. Relinearization forms $C_\alpha = (D_1, D_0)$ and a pair C_β with phase $D_2 S^2 + E_{\mathrm{ks}}$. Then $C_\alpha + C_\beta \in R_q^2$ has phase $\Delta^2 M^{\langle 1\rangle} M^{\langle 2\rangle} + F + E_{\mathrm{ks}}$. It does not restore the scale to Δ; that is the purpose of the later rescaling step. Without relinearization, ciphertext length grows with the degree in S; repeated squaring doubles that degree at every step.

D-3.5.2 Relinearization Method 1 – Ciphertext Decomposition

Use an explicit integer gadget for arbitrary q: choose an integer $\beta \geq 2$, let $l = \lceil \log_\beta q \rceil$, and set $g_i = \beta^{i-1}$ for $1 \leq i \leq l$. Decompose each coefficient of D_2 using its unsigned representative into digits $d_i \in \{0, \ldots, \beta - 1\}$, so

$$D_2 \equiv \sum_{i=1}^{l} d_i g_i \pmod q.$$

This ascending gadget is a separate convention from the descending q/β^i weights: the same digit vector cannot be used with both conventions. Generate evaluation-key pairs $K_i = (A_i, B_i)$ satisfying

$$B_i + A_i S \equiv g_i S^2 + E_i \pmod q.$$

Then

$$C_\beta = \sum_{i=1}^{l} d_i K_i, \qquad \mathsf{Phase}_S(C_\beta) \equiv D_2 S^2 + \sum_{i=1}^{l} d_i E_i \pmod q.$$

For the negacyclic coefficient norm, $\| \sum_i d_i E_i \|_\infty \leq n(\beta - 1) \sum_i \|E_i\|_\infty$. Balanced digits or a truncated decomposition require their corresponding digit and residual bounds. The evaluation keys encrypt gadget multiples of S^2, without the CKKS message scale Δ.

D-3.5.3 Relinearization Method 2 – Ciphertext Modulus Switch

An alternative relinearization method uses an auxiliary positive integer g and an evaluation key at modulus gq. Choose g according to the key-switching error budget; it need not be random or a power of two. Sample A' uniformly in R_{gq} and a small integer error E', and set

$$evk_g = (A', B'), \qquad B' = [-A'S + gS^2 + E']_{gq}.$$

Lift the coefficients of $D_2 \in R_q$ to fixed centered integers. Compute the pair $Z = [D_2 evk_g]_{gq}$ and set

$$C_\beta = [\lceil Z/g \rfloor]_q .$$

Write $R_A = \lceil Z_A/g \rfloor - Z_A/g$ and $R_B = \lceil Z_B/g \rfloor - Z_B/g$. Each coefficient has magnitude at most $1/2$. Modulo q, the output phase is

$$\mathsf{Phase}_S(C_\beta) \equiv D_2 S^2 + D_2 E'/g + R_B + R_A S \pmod q.$$

Hence a coefficient-norm bound on the added error is

$$\|E_{\mathrm{ks}}\|_\infty \le \frac{n\|D_2\|_\infty\|E'\|_\infty}{g} + \frac{1+\|S\|_1}{2}.$$

The multiple of gq introduced by reducing Z becomes a multiple of q after division, which disappears modulo q. Rounding does not distribute over sums or commute with multiplication by S; its contribution is the explicit term $R_B + R_A S$. Large g suppresses the evaluation-key error but does not eliminate this rounding term. The choice of g also enters the security parameters of the extended-modulus evaluation key.

D-3.5.4 Rescaling

Let $q_l = w_l q_{l-1}$ and let a ciphertext before rescaling have phase $\Delta_{\mathrm{in}} M + F$. Rescaling is coefficient-wise nearest-integer division of centered ciphertext representatives by w_l:

$$\widehat{A} = [\lceil A/w_l \rfloor]_{q_{l-1}}, \qquad \widehat{B} = [\lceil B/w_l \rfloor]_{q_{l-1}}.$$

With $R_A = \lceil A/w_l \rfloor - A/w_l$ and $R_B = \lceil B/w_l \rfloor - B/w_l$,

$$\widehat{B} + \widehat{A}S \equiv \frac{\Delta_{\mathrm{in}}}{w_l} M + \frac{F}{w_l} + R_B + R_A S \pmod{q_{l-1}}.$$

Therefore the actual output scale is $\Delta_{\mathrm{out}} = \Delta_{\mathrm{in}}/w_l$, without rounding the scale metadata. After multiplying ciphertexts of scales Δ_1, Δ_2, this becomes $\Delta_{\mathrm{out}} = \Delta_1\Delta_2/w_l$. If $\Delta_1 = \Delta_2 = \Delta$ and $w_l \approx \Delta$, the result is close to Δ, but generally is not equal to it. Relabeling the scale as exactly Δ introduces a message-dependent error and requires a separate accuracy analysis.

The new phase error is $\widehat{F} = F/w_l + R_B + R_A S$, not coefficient-wise $\lceil F/w_l \rfloor$. A deterministic bound is

$$\|\widehat{F}\|_\infty \le \|F\|_\infty/w_l + (1+\|S\|_1)/2.$$

Correct interpretation also requires the complete output phase to lie in the centered interval modulo q_{l-1}.

For an RNS chain $q_l = \prod_{i=0}^{l} w_i$, the primes are chosen for efficient arithmetic and the desired scale schedule. RNS changes the representation, not the total information in q_l. RLWE security depends on the dimension, total modulus, secret distribution, error distribution, and evaluation-key parameters; it is not determined only by the smallest and largest RNS primes.

Initial Scaling Factor Δ Upon Encryption: Choose Δ for the desired precision and available modulus headroom, with q_0 large enough for the intended final message and error. Setting Δ near the rescaling primes is convenient, but $\Delta = w_L$ is not mandatory.

Noise Growth: Multiplication propagates both message-error cross terms and the error product, and rescaling adds a new rounding error. Bounds depend on message and error magnitudes; the error need not increase for every input, for example when multiplying by zero. Rescaling alone does not restore lost numerical precision.

D-3.5.5 Comparing BFV and CKKS Bootstrapping

Both schemes refresh a ciphertext by evaluating a suitable decryption-related function, but their exact steps and plaintext semantics differ. In the CKKS procedure below, lifting the ciphertext coefficients

exposes the raw phase $\Delta M + E + q_0 K$ at a larger modulus; no separate encrypted-secret inner product is needed. Linear coefficient/slot transforms and an approximate modular-reduction function remove the $q_0 K$ contribution approximately. BFV bootstrapping instead evaluates an exact finite-plaintext decryption function, with its parameter and digit-extraction conditions.

Multiplicative Level: BFV at a fixed ciphertext modulus still has a finite noise budget. In the standard CKKS multiply–relinearize–rescale schedule, rescaling consumes a modulus-chain level. Reaching q_0 means that no further scheduled rescaling is available; it is not an algebraic prohibition on every multiplication. Bootstrapping can replenish the available levels, while consuming some levels itself (§D-3.13).

Limitation of Noise Handling: Rescaling divides the existing phase error and modulus by the same factor, and adds rounding error. It therefore does not by itself improve the old error-to-modulus ratio or recover lost numerical precision. CKKS bootstrapping raises the available modulus but retains the previous numerical error and adds its own approximation and evaluation errors.

D-3.5.6 Summary

⟨Summary D-3.5.6⟩ CKKS Ciphertext-to-Ciphertext Multiplication

For ciphertexts with the same key and modulus q_l, and scales Δ_1, Δ_2:

1. Compute $D_0 = B_1 B_2$, $D_1 = A_1 B_2 + A_2 B_1$, and $D_2 = A_1 A_2$ modulo q_l. Their phase is the product of the two input phases.
2. Relinearize using either of the preceding methods, obtaining $(A, B) = (D_1, D_0) + C_\beta$ with phase $\Delta_1 \Delta_2 M_1 M_2 + F$; F includes multiplication and key-switching errors.
3. If the schedule calls for rescaling, set $q_{l-1} = q_l / w_l$ and

$$(\widehat{A}, \widehat{B}) = [(\lceil A/w_l \rfloor, \lceil B/w_l \rfloor)]_{q_{l-1}}, \quad \Delta_{\text{out}} = \Delta_1 \Delta_2 / w_l.$$

The output error is $F/w_l + R_B + R_A S$, subject to the no-wrap and precision bounds above.

Swapping the Order of Relinearization and Rescaling: It is possible to rescale the three-component ciphertext first, using rounding error $R_0 + R_1 S + R_2 S^2$, and then relinearize with a key valid at the lower modulus. The two schedules are not identical ciphertext computations and have different error and cost bounds.

D-3.6 Ciphertext-to-Plaintext Multiplication

Encode a plaintext multiplier Λ at a chosen scale Δ_Λ, giving $P_\Lambda = \lceil \Delta_\Lambda \Lambda \rfloor$. For a ciphertext (A, B) at scale Δ, compute

$$(A', B') = (AP_\Lambda, BP_\Lambda) \pmod q, \qquad \Delta' = \Delta \Delta_\Lambda.$$

The phase includes both the input ciphertext error multiplied by P_Λ and the plaintext encoding error. If rescaling by w_l is needed, use the procedure in §D-3.5.6, with output scale $\Delta\Delta_\Lambda / w_l$ and its rounding error. This common schedule consumes one level. It is not mandatory for every plaintext multiplication: an exactly represented small integer constant may use $\Delta_\Lambda = 1$, and rescaling can sometimes be delayed when the modulus and precision budgets allow.

D-3.7 ModDrop

For $q_{l-1} \mid q_l$, define

$$(A', B') = ([A]_{q_{l-1}}, [B]_{q_{l-1}}).$$

If $A' = A - K_A q_{l-1}$ and $B' = B - K_B q_{l-1}$, then

$$A'S + B' = AS + B - (K_A S + K_B) q_{l-1}.$$

Consequently the new phase is congruent to the old phase modulo q_{l-1}. If $\|\Delta M + E\|_\infty < q_{l-1}/2$, its centered representative remains exactly $\Delta M + E$; otherwise the phase can wrap and change the decoded value.

⟨Summary D-3.7⟩ CKKS ModDrop

Reduce each ciphertext component modulo a divisor of its current modulus. The tracked scale and the existing phase error are unchanged only when the complete phase fits the new centered interval. The modulus headroom decreases.

D-3.7.1 Difference between Modulus Switch and ModDrop

Both operations lower the ciphertext modulus. Rescaling also divides the tracked scale and old error by q_l/q_{l-1} and introduces rounding error. ModDrop leaves the scale unchanged and preserves the phase only under its no-wrap condition. It is also used to align ciphertexts at different levels before addition or multiplication, not only when no future multiplication remains.

D-3.8 Homomorphic Key Switching

Choose an exact integer gadget g_i as in §D-3.5.2, and decompose $A \equiv \sum_i d_i g_i \pmod q$. Evaluation keys (a_i, b_i) under the new secret S' satisfy $b_i + a_i S' \equiv g_i S + E_i \pmod q$. Then

⟨Summary D-3.8⟩ CKKS Key Switching

$$(A', B') = (0, B) + \sum_i d_i (a_i, b_i), \qquad B' + A'S' \equiv B + AS + \sum_i d_i E_i \pmod q.$$

The scale is unchanged and the error increases by the stated key-switching term. Correct interpretation requires the new phase to fit the modulus. This is the same underlying gadget technique used in §D-2.8, with CKKS's approximate decoding semantics.

D-3.9 Homomorphic Rotation of Input Vector Slots

The same root-exponent permutation underlies BFV and CKKS slot rotation, but BFV evaluates at roots in its plaintext ring and CKKS at complex roots. In the encoding convention of §D-3.1, the inverse transform is $n^{-1}\tilde{W} I_n^R$, and the decoding transform is $\tilde{W}^*$. The matrix order matters.

For $J(j) = 5^j \bmod 2n$, the automorphism $M(X) \mapsto M(X^{J(h)})$ sends slot j to the old slot $j + h \bmod (n/2)$, because $5^j 5^h = 5^{j+h}$. With $J_*(j) = -5^j \bmod 2n$, the conjugate half rotates in the same direction.

⟨Summary D-3.9⟩ CKKS Homomorphic Rotation of Input Vector Slots

1. Apply $X \mapsto X^{J(h)}$ to both A and B, reducing modulo $X^n + 1$. The secret becomes $S(X^{J(h)})$.
2. Use the exact gadget key switching in §D-3.8 from $S(X^{J(h)})$ to $S(X)$. Include the added key-switching error in the output phase.

The result rotates the $n/2$ independent complex slots left by h positions.

Rotation within Half Slots: A single 5^h automorphism rotates each of the two orbits separately. This does not prohibit other permutations constructed using masks and several automorphisms. CKKS's second half is determined by conjugation, while fully split BFV batching allows all n slots to be independently chosen.

D-3.9.1 Example

In this subsection, we will show the following 2 examples:

1. Encode an input vector $\vec{v}$ into a plaintext polynomial $M(X)$ based on our updated updated encoding & decoding matrices $\tilde{W}$ and $\tilde{W}^*$
2. Rotate all elements of the input vector $\vec{v}$ h positions to the left by updating the encoded plaintext $M(X)$ to $M(X^{J(h)})$

We will use the same example of the input vector $\vec{v}$ used in §D-3.1.1: $\vec{v} = (1.1 + 4.3i, 3.5 - 1.4i)$.
Remember that the encoded plaintext polynomial of $\vec{v}$ is as follows:
$\Delta M(X) = 2355 + 1195X + 1485X^2 + 2933X^3 \in \mathcal{R}_{\langle 4 \rangle} = \mathbb{R}[X]/(X^4 + 1)$
Suppose we want to rotate the input vector $\vec{v}$ by 1 position to the left as follows:
$\vec{v}^{\langle h=1 \rangle} = (3.5 - 1.4i, 1.1 + 4.3i)$

Therefore, we update $\Delta M(X)$ to $\Delta M(X^{J(1)})$ as follows:
$\Delta M(X^{J(1)}) = \Delta M(X^5) = 2355 + 1195(X^5) + 1485(X^5)^2 + 2933(X^5)^3$
$= 2355 + 1195X^5 + 1485X^{10} + 2933X^{15}$
$= 2355 + 1195X \cdot (-1) + 1485X^2 \cdot (-1) \cdot (-1) + 2933X^3 \cdot (-1) \cdot (-1) \cdot (-1)$
$= 2355 - 1195X + 1485X^2 - 2933X^3$

The rotated *forward-ordered* Hermitian input vector is computed as follows:

$$\frac{\tilde{W}^* \cdot \Delta\vec{m}}{\Delta} = \begin{bmatrix} 1, \omega, \omega^2, \omega^3 \\ 1, \omega^5, \omega^{10}, \omega^{15} \\ 1, \overline{\omega}, \overline{\omega^2}, \overline{\omega^3} \\ 1, \overline{\omega^5}, \overline{\omega^{10}}, \overline{\omega^{15}} \end{bmatrix} \cdot \begin{bmatrix} 2355 \\ -1195 \\ 1485 \\ -2933 \end{bmatrix} \cdot \frac{1}{1024}$$

$$= \frac{\tilde{W}^* \cdot \Delta\vec{m}}{\Delta} = \begin{bmatrix} 1, \omega, \omega^2, \omega^3 \\ 1, \omega^5, \omega^2, \omega^7 \\ 1, \overline{\omega}, \overline{\omega^2}, \overline{\omega^3} \\ 1, \overline{\omega^5}, \overline{\omega^2}, \overline{\omega^7} \end{bmatrix} \cdot \begin{bmatrix} 2355 \\ -1195 \\ 1485 \\ -2933 \end{bmatrix} \cdot \frac{1}{1024}$$

$$= \frac{\tilde{W}^* \cdot \Delta\vec{m}}{\Delta} = \begin{bmatrix} 1, \omega, \omega^2, \omega^3 \\ 1, \omega^5, \omega^2, \omega^7 \\ 1, \omega^7, \omega^6, \omega^5 \\ 1, \omega^3, \omega^6, \omega \end{bmatrix} \cdot \begin{bmatrix} 2355 \\ -1195 \\ 1485 \\ -2933 \end{bmatrix} \cdot \frac{1}{1024}$$

$$= \begin{bmatrix} 1, \frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2}, i, -\frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2} \\ 1, -\frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2}, i, \frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2} \\ 1, \frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2}, -i, -\frac{\sqrt{2}}{2} - \frac{i\sqrt{2}}{2} \\ 1, -\frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2}, -i, \frac{\sqrt{2}}{2} + \frac{i\sqrt{2}}{2} \end{bmatrix} \cdot \begin{bmatrix} 2.2998046875 \\ -1.1669921875 \\ 1.4501953125 \\ -2.8642578125 \end{bmatrix}$$

$$\approx (3.500 - 1.4003i,\ 1.0997 + 4.3007i,\ 3.500 + 1.4003i,\ 1.0997 - 4.3007i)$$

Extract the first $\frac{n}{2} = 2$ elements in the above Hermitian vector to recover the input vector:
$(3.500 - 1.4003i,\ 1.0997 + 4.3007i)$
$\approx (3.5 - 1.4i,\ 1.1 + 4.3i) = \vec{v}^{\langle h=1 \rangle}$ ▷ The original input vector $\vec{v}$ rotated by 1 position to the left

In an actual evaluation, apply the automorphism to the ciphertext components, obtaining

$$(A(X^{J(1)}), B(X^{J(1)})).$$

This ciphertext has secret $S(X^{J(1)})$. Key-switch it back to $S(X)$ before decrypting with the original key. The decoded result approximates the rotated vector $\vec{v}^{\langle h=1 \rangle}$, with the original encoding error and the additional key-switching error.

Source Code: Examples of CKKS's homomorphic input vector slot rotation can be executed by running this Python script.

D-3.10 Contemplation on CKKS Encoding

Why CKKS's Encoding Uses the ($\mu = 2n$)-th Cyclotomic Polynomial: Evaluation at the n distinct roots of $X^n + 1$ gives an invertible Vandermonde matrix. Any distinct evaluation points give an invertible Vandermonde matrix over a field; roots of unity additionally make evaluation compatible with multiplication modulo $X^n + 1$, provide conjugate pairing, and allow efficient Fourier transforms. Power-of-two cyclotomic rings are a useful implementation choice, not a consequence of a general claim that only their Vandermonde matrices are invertible.

For the slot order ω^{5^j}, applying $X \mapsto X^{5^h}$ permutes the rows of the evaluation (decoding) matrix. The inverse transform is the encoding matrix; the two matrix orientations must not be interchanged. Conjugate pairing reduces the independent complex coordinates to $n/2$ while keeping n real coefficient coordinates.

D-3.11 Homomorphic Conjugation

For real-coefficient M and $\zeta_j = \omega^{J(j)}$, conjugation satisfies

$$\overline{M(\zeta_j)} = M(\overline{\zeta_j}) = M(\zeta_j^{-1}).$$

The forward-ordered full vector therefore changes from $(\vec{v}, \overline{\vec{v}})$ to $(\overline{\vec{v}}, \vec{v})$. Use the same selected roots $J(j) = 5^j \bmod 2n$ as in encoding, rather than replacing their order with $1, 3, \ldots, n-1$.

⟨Summary D-3.11⟩ CKKS Homomorphic Conjugation

1. Apply the ring automorphism $X \mapsto X^{-1} = X^{2n-1}$ to both ciphertext components, with reduction modulo $X^n + 1$.
2. Key-switch from $S(X^{-1})$ to $S(X)$, accounting for the added key-switching error.

D-3.12 Sparsely Packing Ciphertexts

In §D-3.1, we learned the CKKS encoding scheme, which encodes an input vector with $\frac{n}{2}$ slots (i.e., $\frac{n}{2}$-slot input vector) into an $(n-1)$-degree polynomial. While the polynomial ring's degree n is fixed at the setup stage of CKKS as a security parameter, some applications may only need to use fewer than $\frac{n}{2}$ input vector slots. Suppose we only need to use $\frac{n'}{2}$ slots out of $\frac{n}{2}$ slots, where $n' \geq 2$ divides n. Then, the corresponding input vector and encoded polynomial acquire a special property as described below:

⟨Summary D-3.12⟩ CKKS's Sparsely Packing Polynomial and Ciphertext

Suppose that an $\frac{n}{2}$-slot input vector gets encoded into a polynomial in $\mathbb{R}[X]/(X^n+1)$. And suppose that $n' \geq 2$ divides n. We define polynomial $M(X) \in \mathbb{R}[X]/(X^n+1)$ as the one which has non-zero constants at the terms whose power is a multiple of $\frac{n}{n'}$ and all other terms have zero constants (i.e., $M(X) = c_0 + c_{\frac{n}{n'}} X^{\frac{n}{n'}} + c_{\frac{2n}{n'}} X^{\frac{2n}{n'}} + \cdots + c_{n-\frac{n}{n'}} X^{n-\frac{n}{n'}}$). We can express $M(X)$ as some $M_Y(Y) \in \mathbb{R}[Y]/(Y^{n'}+1)$ where $Y = X^{\frac{n}{n'}}$ and thus $M(X) = M_Y(X^{\frac{n}{n'}})$.

Then, the following are true:

1. Every $M_Y(Y) \in \mathbb{R}[Y]/(Y^{n'}+1)$ is isomorphically mapped to (i.e., decoded into) some $\frac{n}{2}$-slot input vector which comprises $\frac{n}{n'}$ repetitions of $\frac{n'}{2}$ consecutive slot values.
2. Conversely, if an $\frac{n}{2}$-slot input vector comprises $\frac{n}{n'}$ repetitions of the first $\frac{n'}{2}$ consecutive slot values, then the vector gets encoded into some $M_Y(Y) \in \mathbb{R}[Y]/(Y^{n'}+1)$ (i.e., some polynomial in $\mathbb{R}[X]/(X^n+1)$ that has zero constants at the terms whose degree is not a multiple of $\frac{n}{n'}$).

Assume $n' \geq 2$ divides the power-of-two n. The forward inclusion follows below. To infer the converse, also note that both the subring $\mathbb{R}[Y]/(Y^{n'}+1)$ and the space of repeated slot vectors have real dimension n'. The injective canonical embedding maps the first into the second, so equality of dimensions proves surjectivity onto that subspace. An isomorphism alone, without this subspace argument, would not establish the converse.

D-3.12.1 Forward Proof: Decoding of Sparsely Packed Ciphertext

We will prove that for each $M_Y(Y) \in \mathbb{R}[Y]/(Y^{n'}+1)$ (i.e., a polynomial in $\mathbb{R}[X]/(X^n+1)$ that has non-zero constants only at those terms with a power that is a multiple of $\frac{n}{n'}$ and zero constants in all

other terms), the polynomial is decoded into some $\frac{n}{2}$-slot input vector which comprises $\frac{n}{n'}$ repetitions of the first $\frac{n'}{2}$ consecutive slot values.

To decode $M(X)$ into an input vector, evaluate $M(X)$ at the selected $\frac{n}{2}$ roots among the n distinct primitive ($\mu = 2n$)-th roots of $X^n + 1$. The evaluation vector is:

$(\ M(\omega^{J(0)}),\ M(\omega^{J(1)}), \cdots, M(\omega^{J(\frac{n}{2}-1)}))$

▷ where $\omega = e^{\frac{i\pi}{n}}$, the base and generator of the primitive ($\mu = 2n$)-th roots of unity

But since $M(X) = M_Y(X^{\frac{n}{n'}})$, the above evaluation is equivalent to evaluating:

$(\ M_Y((\omega^{J(0)})^{\frac{n}{n'}}),\ M_Y((\omega^{J(1)})^{\frac{n}{n'}}), \cdots, M_Y((\omega^{J(\frac{n}{2}-1)})^{\frac{n}{n'}}))$

$= (\ M_Y((\omega^{\frac{n}{n'}})^{J(0)}),\ M_Y((\omega^{\frac{n}{n'}})^{J(1)}), \cdots, M_Y((\omega^{\frac{n}{n'}})^{J(\frac{n}{2}-1)}))$

$= (\ M_Y(\xi^{J(0)}),\ M_Y(\xi^{J(1)}), \cdots, M_Y(\xi^{J(\frac{n}{2}-1)}))$

▷ where $\xi = e^{\frac{i\pi}{n'}}$, the base and generator of the primitive ($\mu = 2n'$)-th roots of unity

Notice that $\xi = \omega^{\frac{n}{n'}}$. Therefore, the above evaluation of $M_Y(Y)$ outputs $\frac{n}{n'}$ repeated values of $M_Y(Y)$ evaluated at $\frac{n'}{2}$ distinct primitive ($\mu = 2n'$)-th roots of unity.

D-3.13 Modulus Bootstrapping

- Reference: Bootstrapping for Approximate Homomorphic Encryption [20]

In a standard leveled evaluation schedule, rescaling eventually exhausts the available modulus chain. Bootstrapping starts from a small modulus q_0, raises it to a working modulus q_L, and returns a ciphertext at some $q_l > q_0$ after consuming levels internally. The output need not be at the maximum modulus q_L.

Take centered representatives of $(A, B) \in R_{q_0}^2$ and suppose $[B + AS]_{q_0} = \Delta M + E$. Their unreduced phase is

$$B + AS = \Delta M + E + q_0 K,$$

where $K \in R$. This is an integer-lift identity, not a decryption identity with the modulo operation already performed. Bootstrapping approximately removes the $q_0 K$ term while retaining the previous numerical error and introducing a new error.

D-3.13.1 High-level Idea

As the first step of bootstrapping, we forcibly change the modulus of the ciphertext (A, B) from q_0 to q_L. Then, its decryption with reduction modulo q_L would output:

$\mathsf{RLWE}^{-1}(\mathsf{ct} = (A, B)) \bmod q_L$

$= B + A \cdot S \bmod q_L$

$= \Delta M + E + q_0 K \bmod q_L$

This interpretation requires $\|B + AS\|_\infty < q_L/2$. For centered input representatives, $\|B + AS\|_\infty \leq (q_0/2)(1 + \|S\|_1)$, so a sufficiently large q_L can ensure it. Ternary coefficients alone are not a dimension-independent bound; $\|S\|_1$ can be as large as n.

In the $\Delta M + E + q_0 K \bmod q_L$ term, notice that because of the $q_0 K$ term which is not modulo-reduced by q_0 anymore, the ciphertext's decrypted plaintext polynomial's each i-th term would get a corrupted

1	ModRaise: lift ciphertext coefficients to q_L, obtaining phase $\Delta M + E + q_0 K$.
2	CoeffToSlot: homomorphically apply the inverse embedding to put the n real phase coefficients in two ciphertexts.
3	EvalExp: evaluate a nonlinear polynomial approximation to the periodic modular-reduction function, with an explicit input range and error budget.
4	SlotToCoeff: repack the approximate coefficients into one ciphertext at the remaining modulus $q_l > q_0$, including all accumulated errors.

Table 6: High-level Description of CKKS's Bootstrapping Procedure

coefficient $\Delta m_i + e_i + q_0 \cdot k_i \bmod q_L$ instead of $\Delta m_i + e_i \bmod q_L$. So, we now need to eliminate the garbage term $q_0 \cdot k_i \bmod q_L$ in each coefficient and distill the pure plaintext coefficient $\Delta m_i + e_i$.

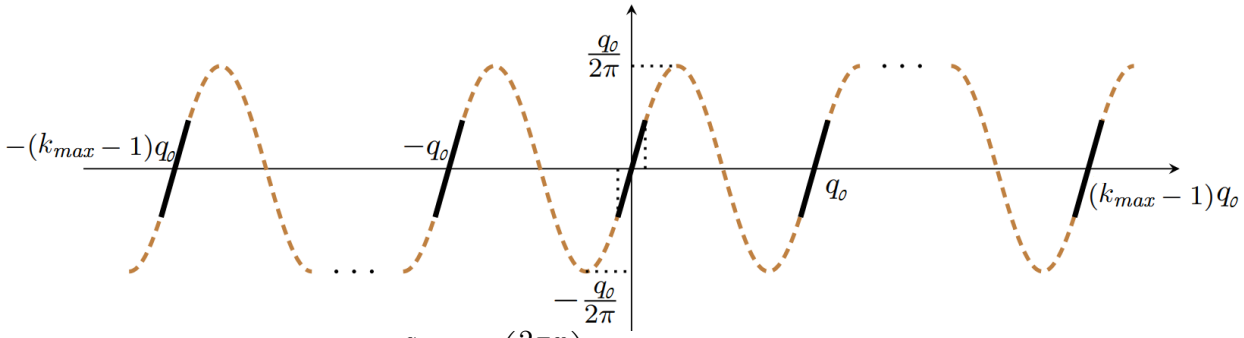


Figure 16: Sine function $f(x) = \frac{q_0}{2\pi} \cdot \sin\left(\frac{2\pi x}{q_0}\right)$ such that $f(\Delta m_i + e_i + q_0 k_i) \approx \Delta m_i + e_i$ (provided $\Delta m_i + e_i \ll q_0$) (Source)

The function is periodic with period q_0. Near $x = kq_0$, it approximates the local centered remainder $x - kq_0$, not the global function x. Its derivative at kq_0 is 1.

Combining these two properties, given input $x = \Delta m_i + e_i + q_0 k_i$,

$$\begin{aligned} f(\Delta m_i + e_i + q_0 k_i) &= \frac{q_0}{2\pi} \sin\left(\frac{2\pi(\Delta m_i + e_i + q_0 k_i)}{q_0}\right) \\ &= \frac{q_0}{2\pi} \sin\left(\frac{2\pi(\Delta m_i + e_i)}{q_0}\right) \\ &\approx \Delta m_i + e_i. \end{aligned}$$

, provided $|\Delta m_i + e_i|/q_0$ is sufficiently small for the target accuracy. This is an additional bootstrapping parameter condition, not a consequence of ordinary CKKS decryption correctness. For $r_i = \Delta m_i + e_i$, the intrinsic sine error satisfies

$$\left| \frac{q_0}{2\pi} \sin(2\pi r_i / q_0) - r_i \right| \leq \frac{2\pi^2}{3q_0^2} |r_i|^3.$$

Polynomial approximation and homomorphic evaluation add further errors. Since ciphertext multiplication acts slot-wise, the algorithm first applies a linear coefficient-to-slot transform, evaluates the nonlinear approximation in slots, and then applies the inverse linear transform, as outlined in Table 6.

D-3.13.2 Mathematical Expansion of the High-level Idea

The following identities describe plaintext values represented by ciphertexts; they are not operations that expose the secret key or decrypt during evaluation. Let $T = \Delta M + E + q_0 K$ be the lifted phase, and use $\mathsf{Enc}(T)$ only as shorthand for a ciphertext representing that phase.

ModRaise: Lift the coefficients of (A, B) to the working modulus q_L. Under the bound above, the lifted ciphertext has raw phase T. It is a valid starting point for homomorphic evaluation without computing AS in the clear.

CoeffToSlot: Let σ be the canonical embedding and let π select the $n/2$ independent slots. The real-linear inverse embedding maps the slots $\pi\sigma(T)$ to the coefficient vector $(T_i)_{i=0}^{n-1}$. Its homomorphic implementation uses rotations, conjugation, additions, and multiplication by encoded matrix constants. Store the n real coefficients in two ciphertexts, with $n/2$ coefficients per ciphertext. Scales and the errors of these transformations must be tracked.

EvalExp: Homomorphically evaluate a polynomial circuit $\widehat{f}$ approximating $f(x) = q_0 \sin(2\pi x/q_0)/(2\pi)$ on the required input range. This is a nonlinear computation, not a linear transformation or multiplication by one encoded polynomial. For each coefficient,

$$\widehat{f}(T_i) = \Delta m_i + e_i + e_{b,i},$$

where $e_{b,i}$ includes sine error, polynomial-approximation error, and all evaluation errors.

SlotToCoeff: Apply the inverse linear packing operation to the two resulting ciphertexts. The final phase is represented as $\Delta M + E + E_b$ at its tracked output scale and modulus q_l; E_b also includes the final transform errors.

Bootstrapping Without a Secret Key: The actual circuit operates on the lifted ciphertext throughout. It does not decrypt and re-encrypt, nor does an identity such as $\mathsf{Enc}(\mathsf{Dec}(\mathsf{ct})) = \mathsf{ct}$ hold for randomized noisy encryption. Public evaluation keys permit the rotations, conjugations, and relinearizations. The output represents an approximation to the original decoded value, with a larger available modulus and additional error.

D-3.13.3 Details: CoeffToSlot

Set $\zeta_j = \omega^{J(j)}$ and define the $n/2 \times n$ evaluation matrix U by $U_{j,k} = \zeta_j^k$. If $\vec{t}$ is the real coefficient vector of T, its independent slots are $\vec{z} = U\vec{t}$. Orthogonality of the roots gives

$$\vec{t} = \frac{1}{n}(\overline{U}^T \vec{z} + U^T \overline{\vec{z}}).$$

Split $U = (U_0\ U_1)$ into two square blocks, according to the first and second halves of the coefficient indices. Then

$$\vec{t}_0 = \frac{1}{n}(\overline{U_0}^T \vec{z} + U_0^T \overline{\vec{z}}), \qquad \vec{t}_1 = \frac{1}{n}(\overline{U_1}^T \vec{z} + U_1^T \overline{\vec{z}}).$$

Applying these two real-linear maps homomorphically produces ciphertexts whose slots contain the two coefficient halves, up to encoding and evaluation errors. Conjugation is computed as in §D-3.11; matrix-vector products use encoded diagonals and rotations as in §D-2.10. They are ciphertext–plaintext multiplications and additions, with level consumption determined by the chosen scale schedule. The notation does not mean multiplying a matrix directly by a ciphertext polynomial's coefficient list in the clear.

D-3.13.4 Details: EvalExp

We will use the sine function $f(x) = \frac{q_0}{2\pi} \cdot \sin\left(\frac{2\pi x}{q_0}\right)$ to approximately eliminate $q_0 k_i$ from $\Delta m_i + e_i + q_0 k_i$ by computing $f(\Delta m_i + e_i + q_0 k_i) \approx \Delta m_i + e_i$. This approximation works if $\Delta m_i + e_i$ is very close to $x = 0$ relative to q_0 (i.e., $\Delta m_i + e_i \ll q_0$). Still, the elimination of $q_0 k_i$ is approximate (i.e., $\approx \Delta m_i + e_i$), because $f(x)$ is $y \approx x$ nearby $x = 0$, not exactly $y = x$.

One issue is that we need to evaluate $f(x)$ homomorphically based on $\mathsf{ct_{s1}}$ and $\mathsf{ct_{s2}}$ as inputs (i.e., $f(\mathsf{ct_{s1}})$ and $f(\mathsf{ct_{s2}})$), but FHE supports only $(+, \cdot)$ operations, whereas the sine graph cannot be formulated by only $(+, \cdot)$. Therefore, we will approximate the sine function $f(x)$ by using the Taylor series (§A-15):

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f'''(a)}{3!}(x-a)^3 + \cdots = \sum_{d=0}^{\infty} \frac{f^{(d)}(a)}{d!}(x-a)^d$$

If we approximate $f(x)$ around $x = 0$, then the approximated polynomial is as follows:

$$\begin{aligned} f(x) &= \frac{q_0}{2\pi} \cdot \sin\left(\frac{2\pi}{q_0} \cdot 0\right) + \frac{q_0}{2\pi} \cdot \frac{2\pi}{q_0} \cdot \frac{\cos\left(\frac{2\pi}{q_0} \cdot 0\right)}{1!} \cdot x + \frac{q_0}{2\pi} \cdot \left(\frac{2\pi}{q_0}\right)^2 \cdot \frac{-\sin\left(\frac{2\pi}{q_0} \cdot 0\right)}{2!} \cdot x^2 + \cdots \\ &= \frac{q_0}{2\pi} \cdot \sum_{j=0}^{\infty} \left(\frac{(-1)^j}{(2j+1)!} \cdot \left(\frac{2\pi x}{q_0}\right)^{2j+1} \right) \\ &\approx \frac{q_0}{2\pi} \cdot \sum_{j=0}^{h} \left(\frac{(-1)^j}{(2j+1)!} \cdot \left(\frac{2\pi x}{q_0}\right)^{2j+1} \right) = \hat{f}(x) \end{aligned}$$

, where $\hat{f}(x)$ is a $(2h+1)$-degree polynomial.

For centered representatives of A, B and a small secret S, the integer phase satisfies $B + AS = \Delta M + E + q_0 K$. Its coefficient norm is at most $(q_0/2)(1 + \|S\|_1)$, and the bound on K also includes $\|\Delta M + E\|_\infty / q_0$. Choose $k_{\max}$ to bound the entire normalized input $|x|/q_0$, including the residual message and error, not just $|k_i|$. Sparse secrets can reduce this range, but require a security analysis with their actual distribution.

The normalized approximation interval is controlled by $\theta = 2\pi x / q_0$, so increasing q_0 alone does not increase its width when $|x|/q_0$ is fixed. For a degree-d polynomial, Horner evaluation uses d multiplications, and standard parallel power constructions have logarithmic multiplicative depth with polynomial work. The cost is not generally exponential in the degree. Scaling and repeated squaring exploit the special exponential-function structure to reduce the required work and depth:

$$\begin{cases} e^{i\cdot\theta} = \cos\theta + i \cdot \sin\theta \\ (e^{i\cdot\theta})^2 = e^{i\cdot 2\theta} \end{cases}$$

Set $\theta = 2\pi x / q_0$. First approximate $e^{i\theta/2^r}$ on the smaller interval $|\theta|/2^r \le 2\pi k_{\max}/2^r$, then square the result r times to approximate $e^{i\theta}$. The required Taylor degree depends on this normalized interval and the desired final error. Squaring amplifies the initial approximation error and adds arithmetic error, so the initial tolerance must account for all r squarings.

Using the Taylor series with degree d_0 around $x = 0$, we can approximate $e^{\frac{2\pi i x}{2^r q_0}}$ as:

$$f_e(x) = e^{\frac{2\pi i x}{2^r q_0}} \approx \sum_{d=0}^{d_0} \frac{1}{d!} \left(\frac{2\pi i x}{2^r q_0}\right)^d = \hat{f}_e(x)$$

Then, we iteratively square $\hat{f}_e$ total r times to get:

$$(\hat{f}_e(x))^{2^r} \approx (f_e(x))^{2^r} = e^{i\frac{2\pi x}{q_0}} = e^{i\theta}$$

Then, based on Euler's formula $e^{i\cdot\theta} = \cos\theta + i\cdot\sin\theta$, we can derive the following relations:

$\overline{e^{i\cdot\theta}} = \cos\theta + \overline{i\cdot\sin\theta}$
$e^{-i\cdot\theta} = \cos\theta - i\cdot\sin\theta$
$e^{i\cdot\theta} - e^{-i\cdot\theta} = (\cos\theta + i\cdot\sin\theta) - (\cos\theta - i\cdot\sin\theta) = 2i\sin\theta$
$\sin\theta = \frac{-i}{2}\cdot(e^{i\cdot\theta} - e^{-i\cdot\theta})$
$\frac{q_0}{2\pi}\cdot\sin\theta = \frac{q_0}{2\pi}\cdot\frac{-i}{2}\cdot(e^{i\cdot\theta} - e^{-i\cdot\theta})$

Substituting $\theta = \frac{2\pi x}{q_0}$, we finally get:
$\frac{q_0}{2\pi}\cdot\sin\left(\frac{2\pi x}{q_0}\right) = \frac{q_0}{2\pi}\cdot\frac{-i}{2}\cdot(e^{i\cdot\frac{2\pi x}{q_0}} - e^{-i\cdot\frac{2\pi x}{q_0}})$

Using the final relation above, the EvalExp step homomorphically evaluates the approximation of $\frac{q_0}{2\pi}\cdot\sin\left(\frac{2\pi x}{q_0}\right)$ where $x = \Delta m_i + e_i + q_0 k_i$ as follows:

1. Homomorphically evaluate the exponential approximation, obtaining $\widehat{z} \approx e^{2\pi i x/q_0}$ with a quantified error.
2. Apply homomorphic conjugation (§D-3.11) to obtain an approximation to $\overline{\widehat{z}}$, accounting for its key-switching error.
3. Subtract the two results and multiply by the encoded constant $-iq_0/(4\pi)$. This approximates $\frac{q_0}{2\pi}\sin(2\pi x/q_0)$. The factor $q_0/(2\pi)$ is essential; multiplying only by $-i/2$ would return the unscaled sine.

The result of EvalExp is two ciphertexts whose input vector slots store the bootstrapped coefficients of M_c, which are modulo-reduced q_0 from $\Delta m_i + e_i + q_0 k_i$ to $\Delta m_i + e_i + e_{bi}$. Note that e_{bi} is a bootstrapping error introduced by the following three factors: (1) the intrinsic homomorphic $(+,\cdot)$ computation noises of the CoeffToSlot, EvalExp, and SlotToCoeff steps; (2) the EvalExp step's Taylor polynomial approximation error of the exponential function $e^{i\theta}$; (3) the EvalExp step's sine graph error, since the graph is not exactly $y = x$ around $x = 0$, but only $y \approx x$.

Note that since the output of the CoeffToSlot step was split into 2 ciphertexts ($\mathsf{ct_{s1}}$ and $\mathsf{ct_{s2}}$), the output of the EvalExp step is also in 2 ciphertexts: ($\mathsf{ct_{b1}}$ and $\mathsf{ct_{b2}}$). The input vector slots of $\mathsf{ct_{b1}}$ store $(\Delta m_i + e_i + e_{bi})_{i=0}^{\frac{n}{2}-1}$, whereas the input vector slots of $\mathsf{ct_{b2}}$ store $(\Delta m_i + e_i + e_{bi})_{i=\frac{n}{2}}^{n-1}$.

D-3.13.5 Details: SlotToCoeff

Using the same blocks U_0, U_1 as in §D-3.13.3, let the two EvalExp output vectors be $\vec{b}_0, \vec{b}_1$, containing approximations to the first and second coefficient halves. Compute

$$\vec{z}_{\text{out}} = U_0\vec{b}_0 + U_1\vec{b}_1.$$

The resulting single ciphertext has $n/2$ complex slots representing the polynomial whose n real coefficients are the concatenation of those halves. Its implicit conjugate slots are $\overline{\vec{z}_{\text{out}}}$; they are not a second encryption of the same coefficient polynomial. The homomorphic transform incurs encoding, key-switching, and scaling errors. Thus its raw output phase is $\Delta_{\text{out}}M + E_{\text{old,out}} + E_b$ at modulus q_l, with all scales and

errors propagated through the chosen implementation. Under a schedule restoring the input scale, this is abbreviated $\Delta M + E + E_b$.

D-3.13.6 Reducing the Bootstrapping Overhead by Sparsely Packing Ciphertext

Assume a proper even divisor $2 \le n' < n$ and set $d = n/n'$. Repeating $n'/2$ independent slots produces a message polynomial in $Y = X^d$, with n' potentially nonzero coefficients. To replicate an initially zero-padded block, use $\log_2 d$ doubling rounds with shifts $-(n'/2)2^j$, for $0 \le j < \log_2 d$.

Message sparsity alone is insufficient after modulus raising: the phase $\Delta M(Y) + E(X) + q_0 K(X)$ generally has nonsparse E and K. The original CKKS bootstrapping paper, Section 5.2, therefore inserts a partial-sum procedure after ModRaise and before CoeffToSlot:

$$\mathsf{ct} \leftarrow \mathsf{ct} + \mathsf{Rotate}(\mathsf{ct}, (n'/2)2^j), \qquad 0 \le j < \log_2 d.$$

This trace-like operation annihilates coefficient positions not divisible by d and multiplies retained coefficients by d, up to evaluation errors. Its normalization must be included in the tracked scales and the modular-reduction period. Omitting it does not justify discarding the other lifted coefficients.

The smaller canonical embedding uses the roots of $Y^{n'} + 1$ and has n' real coefficient coordinates, not n/n'. Since a proper power-of-two divisor satisfies $n' \le n/2$, these coefficient values can fit in one ciphertext. The corresponding smaller coefficient/slot transforms and modular-reduction evaluation reduce cost. Their exact savings depend on the transform algorithm, rotations, and normalization; they are not automatically exactly one half for every parameter choice.

D-3.13.7 Summary

⟨Summary D-3.13.7⟩ CKKS Bootstrapping

1. Start from $(A, B) \in R_{q_0}^2$ with centered phase $\Delta M + E$ and the required input-range bound.
2. Lift the ciphertext coefficients to q_L. The raw phase is $\Delta M + E + q_0 K$, under a no-wrap bound at q_L. RNS implementations must compute any needed basis extension.
3. Apply CoeffToSlot, using two ciphertexts for a fully packed input, or the properly normalized sparse procedure above.
4. Evaluate the nonlinear polynomial approximation to modular reduction, budgeting both approximation error and homomorphic evaluation error.
5. Apply SlotToCoeff. The output is at a remaining modulus $q_l > q_0$, with tracked scale and accumulated error. In equal-scale shorthand, its phase is $\Delta M + E + E_b$, not exactly $\Delta M + E$.

Limitation: Bootstrapping does not recover precision already lost in the input. Further numerical computation must remain within its message, error, and modulus budgets.

Comparison between BFV and CKKS Bootstrapping: Exact-scheme bootstrapping can remove prior encryption noise by evaluating exact decryption, provided its correctness conditions hold. CKKS retains the prior numerical approximation error and adds a new error. Neither fixed-modulus BFV nor a fixed CKKS parameter set supports arbitrary unbounded work without satisfying the respective noise and accuracy conditions. The CKKS procedure here uses the lifted ciphertext directly; the absence of a separate encrypted-secret inner product is not explained by an assumption that all intermediate scales remain fixed.

D-3.13.8 Reducing the Bootstrapping Noise

As explained in §D-3.13.4, the bootstrapping procedure generates three types of noises:

- Type-1 Noise: the intrinsic homomorphic $(+, \cdot)$ computation noises of the CoeffToSlot, EvalExp, and SlotToCoeff steps
- Type-2 Noise: the EvalExp step's approximation error of the exponential function $e^{i\theta}$
- Type-3 Noise: the EvalExp step's sine graph error (i.e., not exactly $y = x$ around $x = 0$, but only $y \approx x$)

All three error sources must be bounded for the chosen input range and parameter set. Finite polynomial evaluation generally retains a nonzero approximation error, while its degree and precision can be increased to reduce it. Improved modular-reduction approximations can reduce the intrinsic sine error; they do not eliminate the need to analyze arithmetic and approximation errors together.

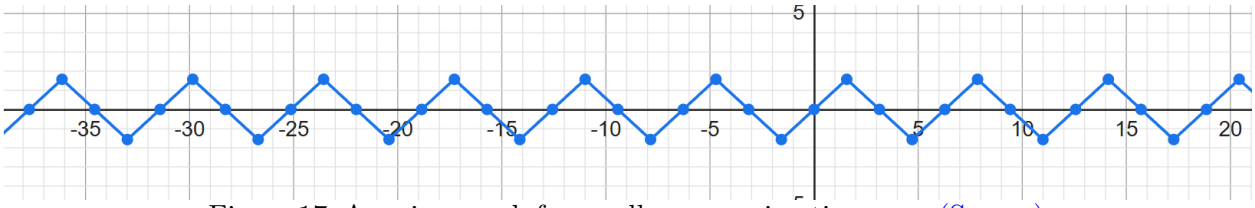


Figure 17: Arc-sine graph for smaller approximation error (Source)

arcsin(sin(x)) Approximation (EUROCRYPT 2021 [21]): The exact composition equals x only on a principal interval such as $[-\pi/2, \pi/2]$ and extends periodically with alternating slopes. Around each required multiple of the period it can represent the desired local remainder. Homomorphic implementations approximate the component functions on specified intervals, for example using minimax polynomials. Their approximation error, interval restrictions, and additional depth must be specified; a universal cost of exactly three levels does not follow from the identity.

Meta-BTS (CCS 2022 [22]): Meta-BTS improves precision by bootstrapping an amplified residual and subtracting it from the preceding result. The following conditional derivation makes the scale and input-range requirements explicit. It is a sufficient construction, not a parameter-independent claim about every implementation of the paper.

Write $v_D(c)$ for the canonical embedding of the centered decryption phase of c, divided by its scale D. All norms below are slot infinity norms unless stated otherwise. Let the base bootstrap take modulus q and scale Δ to the remaining output modulus Q and the same scale Δ. Suppose its contract, for every admissible input with $\|v_\Delta(c)\|_\infty \le B$, is

$$v_\Delta(\mathsf{BTS}(c)) = v_\Delta(c) + b(c), \qquad \|b(c)\|_\infty \le \varepsilon.$$

Admissibility includes the internal lifting and approximation ranges, evaluation keys, and arithmetic bounds. Any output-scale adjustment is included in this contract and its error. Here Q is the modulus remaining after the base bootstrap, not its temporary raised modulus. The error is measured relative to the actual decoded input, including any error that input already contains.

Assume a componentwise rescale ending at scale D has decoded error at most T/D, and put $\rho = T/\Delta$. For nearest-integer component rounding in the degree-n ring, a conservative deterministic choice is

$$T = \frac{n}{2}(1 + \|S\|_1), \qquad \|\sigma(R_B + R_A S)\|_\infty \le T,$$

where the rounding polynomials have coefficient magnitudes at most $1/2$, $\|S\|_1$ is the coefficient one-norm, and σ is the canonical embedding. A tighter established bound may replace this one. The phase must remain within the centered interval at the destination modulus.

For two calls, take an actual integer rescale factor $a > 1$, an input c of modulus aq and scale $a\Delta$, and $z_0 = v_{a\Delta}(c)$. The following operations are componentwise ciphertext operations; brackets mean modulus reduction, without division or rounding:

$$\begin{aligned} u &= \mathsf{RS}_a(c), & v_\Delta(u) &= z_0 + r, \\ y &= \mathsf{BTS}(u), & v_\Delta(y) &= z_0 + r + b_1, \\ h &= ay, & v_{a\Delta}(h) &= z_0 + r + b_1, \\ d &= [h]_q - [c]_q, & v_\Delta(d) &= a(r + b_1), \\ w &= \mathsf{BTS}(d), & v_\Delta(w) &= a(r + b_1) + b_2, \\ c_{\mathrm{out}} &= h - w, & v_{a\Delta}(c_{\mathrm{out}}) &= z_0 - b_2/a. \end{aligned}$$

The moduli are q for u, d and Q for $y, h, w, c_{\mathrm{out}}$. In the last subtraction, the components of w are interpreted at scale $a\Delta$; this intentional scale interpretation divides its decoded value by a. The first rescale error r cancels together with b_1. The residual is bootstrapped with the sign "new ciphertext minus input." In particular, ModDrop must not be replaced by a rescale.

Sufficient input-range conditions are

$$\|z_0\|_\infty + \rho \le B, \qquad a(\rho + \varepsilon) \le B.$$

For example, $aq \mid Q$ and $q > 2\Delta(B + \varepsilon)$ are conservative external modulus conditions ensuring the required reductions and centered phase bounds. The individual reductions of h and c to q may wrap; their difference still has the stated small centered phase because reduction is a ring homomorphism and the residual bound holds. Under these conditions the added output error is at most ε/a. The identity holds for each actual execution, even if r, b_1, b_2 are correlated.

For k calls, let $P_1 = 1$ and $P_{j+1} = a_j P_j$, where every $a_j > 1$ is the actual available integer rescale factor. The j-call construction has input modulus $P_j q$ and input/output scale $P_j\Delta$. To construct the next one, rescale the new input by a_j, apply the j-call construction, multiply its components by a_j, and bootstrap the difference with the original input reduced to q, interpreting that difference at scale Δ. If the first rescale error is r_j and the preceding construction adds e_j, then the last base call receives

$$P_{j+1}(r_j + e_j), \qquad \|r_j\|_\infty \le \rho/P_j, \qquad \|e_j\|_\infty \le \varepsilon/P_j.$$

After subtracting its output, the resulting value is $z_0 - b_{j+1}/P_{j+1}$. Induction therefore gives the sufficient conditions and bound

$$\begin{gathered} a_j(\rho + \varepsilon) \le B \quad (1 \le j < k), \qquad \|z_0\|_\infty + \rho \sum_{j=1}^{k-1} P_j^{-1} \le B, \\ P_k q \mid Q, \qquad q > 2\Delta(B + \varepsilon), \qquad \|v_{P_k\Delta}(c_{\mathrm{out}}) - z_0\|_\infty \le \varepsilon/P_k. \end{gathered}$$

This proof uses k base bootstrap calls and requires their contract on all the derived inputs. Its divisibility conditions are convenient sufficient conditions, not asserted to be necessary for every variant.

In the ideal notation $a_j = 2^b$ and $\varepsilon = 2^{-b}$, the error expression becomes 2^{-kb} and the scale is $2^{(k-1)b}\Delta$. However, with $B = 1$ and any positive ρ, the bound $2^b(\rho + 2^{-b}) > 1$ leaves no residual input margin.

Thus a b-bit bound alone does not establish the advertised iteration: one needs a sharper error bound, a smaller gain, or a larger certified input range. In RNS, a factor close to 2^b must be tracked at its actual value in both P_k and the scale. For instance, if the actual factor is $a+1$ but restoration uses a, the residual contains $-z_0 + a(r+b_1)$ rather than only $a(r+b_1)$; even a small relative mismatch can violate the input range. Changing scale metadata alone does not recover precision.

Limitation in Noise Handling: A quantitative guarantee must combine the approximation domains, arithmetic errors, and the preceding residual-input conditions. Here is one deterministic error budget for the arcsine approach; it also explains which additional bounds a concrete implementation must supply. Let the ideal normalized coefficient values t lie in

$$D_{K,h} = \bigcup_{j=-K}^{K} [j-h, j+h], \qquad 0 < h < 1/4.$$

On these real intervals, $\arcsin(\sin(2\pi t))/(2\pi) = t - j$. Let p_s be the complete inner polynomial, including any cosine/double-angle composition, with uniform real-domain approximation error ϵ_s. Let the CoeffToSlot error be at most E_C, and let H_s bound the total inner circuit evaluation error at its output. Define

$$L_s = \sup_{\operatorname{dist}(z, D_{K,h}) \le E_C} |p_s'(z)|, \qquad E_s = L_s E_C + \epsilon_s + H_s, \qquad s_h = \sin(2\pi h), \qquad Y = s_h + E_s < 1.$$

The supremum is over a complex neighborhood: an error in a nominally real CKKS slot need not be real. Suppose the outer polynomial p_a approximates arcsin on the real interval $[-s_h, s_h]$ with error ϵ_a, and put

$$L_a = \sup_{|z| \le Y} |p_a'(z)|.$$

If its homomorphic evaluation adds at most H_a before division by 2π, the normalized modular-reduction error is bounded by

$$E_R \le \frac{\epsilon_a + L_a E_s + H_a}{2\pi}.$$

Indeed, first compare the evaluated inner polynomial with $\sin(2\pi t)$; then compare p_a at those two arguments along the complex line segment. The real approximation bound is used only at the ideal real argument. Thus this argument does not silently extend a real minimax approximation guarantee to complex inputs. All circuit errors in H_s, H_a must already be propagated to the relevant output, including rescaling and key switching.

Let L be the ideal final linear transform, including the modular-reduction normalizer and all scale conversions, and let E_L bound its evaluation error. A sufficient base-bootstrap budget is

$$\|b(c)\|_\infty \le \varepsilon_{\text{cert}} := \|L\|_\infty E_R + E_L,$$

where the right-hand side may be chosen as the certified ε in the base contract. For example, with an unnormalized coefficient-to-slot evaluation matrix, its norm is at most n; a factor q/Δ in the reconstruction must also appear in L. Actual multiplication and rescaling scales obey $\Delta_{\text{mul}} = \Delta_1 \Delta_2$ and $\Delta_{\text{rs}} = \Delta_{\text{mul}}/w$ for the actual factor w. A scale discrepancy cannot be absorbed into a rounding bound without bounding its message-dependent bias.

These are upper bounds on added error, not proofs of a positive error floor for every input. The error plateau observed for one fixed circuit or parameter set is not a universal lower bound. If $z_0 = m + e_{\text{old}}$,

the corrected output for $k \geq 2$ is $m + e_{\text{old}} - b_k/P_k$; for $k = 1$, the added term is $+b_1$ from the base contract. In both cases its total error is at most $\|e_{\text{old}}\|_\infty + \varepsilon/P_k$, and the old error is not recovered. The finite modulus budget $P_k q \leq Q$ also limits this construction. No independence or zero-mean assumption is used. A probabilistic guarantee must control failure on the actual derived inputs, for example through a simultaneous good event or conditional failure bounds given the preceding history; average precision measured on fresh encryptions is insufficient. If the conditional bounds are δ_j, the union bound gives total failure probability at most $\sum_j \delta_j$ without assuming independence. A particular secure parameter set still needs certified domain, arithmetic, and failure-probability bounds; small numerical examples do not supply them.

D-4 BGV Scheme

BGV supports exact arithmetic in its finite plaintext ring, subject to decryption correctness. BFV uses a large message scale Δ, while BGV places the error in a multiple of its plaintext modulus. The familiar high/low digit picture is an intuition; a literal split into binary bit positions applies only with suitable power-of-two bases, and centered representatives can be negative.

BGV shares exact plaintext-ring arithmetic, tensor multiplication, and key-switching techniques with BFV. BGV commonly uses modulus switching to control noise magnitude along a modulus chain. Fixed-modulus BFV is also limited by its noise budget; it does not support an unlimited number of multiplications without refreshing. Modulus switching and bootstrapping require the scheme-specific correctness conditions described below.

Required Background

- §A-1: Modulo Arithmetic
- §A-2: Group
- §A-3: Field
- §A-4: Order
- §A-5: Polynomial Ring
- §A-6: Decomposition
- §A-7: Roots of Unity
- §A-8: Cyclotomic Polynomial
- §A-9: Roots of Unity and Cyclotomic Polynomial over Ring
- §A-11: Vector and Matrix
- §A-12: Euler's Formula
- §A-13: Modulo Rescaling
- §A-14: Chinese Remainder Theorem
- §A-10: Lagrange's Polynomial Interpolation
- §A-16: Efficient Polynomial Multiplication by FFT and NTT
- §B-1: Lattice-based Cryptography
- §B-3: RLWE Cryptosystem
- §B-4: GLWE Cryptosystem
- §C-1: GLWE Ciphertext-to-Ciphertext Addition
- §C-2: GLWE Ciphertext-to-Plaintext Addition
- §C-3: GLWE Ciphertext-to-Plaintext Multiplication
- §C-4.4: RLWE Modulus Switching
- §C-5: GLWE Key Switching
- §D-2: BFV Scheme
- §D-3: CKKS Scheme

D-4.1 Encoding and Decoding

BGV uses almost the same plaintext encoding scheme as BFV as described in Summary D-2.2.5 in §D-2.2.5, with the only difference that the scaling factor $\Delta = \left\lfloor \frac{q}{t} \right\rfloor$ is not applied to the plaintext polynomial $M(X)$ like BFV does. Instead, BGV applies its own scaling factor $\Delta = t$ to the noise polynomial $E(X)$ whenever it encrypts a new ciphertext (will be explained in §D-4.2).

The following is BGV's encoding and decoding scheme.

⟨Summary D-4.1⟩ BGV's Encoding and Decoding

Input: An n-slot integer modulo t vector $\vec{v} = (v_0, v_1, \cdots, v_{n-1}) \in \mathbb{Z}_t^n$

Encoding:

Convert $\vec{v} \in \mathbb{Z}_t^n$ into $\vec{m} \in \mathbb{Z}_t^n$ by applying the transformation $\vec{m} = \dfrac{\tilde{W} \cdot I_n^R \cdot \vec{v}}{n}$

, where $\tilde{W}$ is a basis of the n-dimensional vector space crafted as follows:

$$\tilde{W} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-2)}) & \cdots & (\omega^{J(0)}) & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-2)}) & \cdots & (\omega^{J_*(0)}) \\ (\omega^{J(\frac{n}{2}-1)})^2 & (\omega^{J(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J(0)})^2 & (\omega^{J_*(\frac{n}{2}-1)})^2 & (\omega^{J_*(\frac{n}{2}-2)})^2 & \cdots & (\omega^{J_*(0)})^2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots & \vdots \\ (\omega^{J(\frac{n}{2}-1)})^{n-1} & (\omega^{J(\frac{n}{2}-2)})^{n-1} & \cdots & (\omega^{J(0)})^{n-1} & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} & (\omega^{J_*(\frac{n}{2}-2)})^{n-1} & \vdots & (\omega^{J_*(0)})^{n-1} \end{bmatrix}$$

, where for this fully split batching construction t is prime and $t \equiv 1 \pmod{2n}$. Choose a generator g of $\mathbb{Z}_t^\times$ and $\omega = g^{(t-1)/(2n)} \bmod t$ (§A-11.6.1). The output is $M = \sum_{i=0}^{n-1} m_i X^i \in \mathbb{Z}_t[X]/(X^n+1)$. Its coefficients are lifted to integer representatives for encryption modulo q; this lift is not an inclusion of rings. More general BGV plaintext moduli do not necessarily provide n scalar slots. Choose a fixed representative of each coefficient modulo t before embedding it into the ciphertext coefficient domain. Correctness is determined by the full phase bound, not just by the sizes of the fresh message coefficients.

Decoding: For the plaintext polynomial $M = \sum_{i=0}^{n-1} m_i X^i$, compute $\vec{v} = \tilde{W}^* \cdot \vec{m}$, where

$$\tilde{W}^* = \begin{bmatrix} 1 & (\omega^{J(0)}) & (\omega^{J(0)})^2 & \cdots & (\omega^{J(0)})^{n-1} \\ 1 & (\omega^{J(1)}) & (\omega^{J(1)})^2 & \cdots & (\omega^{J(1)})^{n-1} \\ 1 & (\omega^{J(2)}) & (\omega^{J(2)})^2 & \cdots & (\omega^{J(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J(\frac{n}{2}-1)}) & (\omega^{J(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J(\frac{n}{2}-1)})^{n-1} \\ 1 & (\omega^{J_*(0)}) & (\omega^{J_*(0)})^2 & \cdots & (\omega^{J_*(0)})^{n-1} \\ 1 & (\omega^{J_*(1)}) & (\omega^{J_*(1)})^2 & \cdots & (\omega^{J_*(1)})^{n-1} \\ 1 & (\omega^{J_*(2)}) & (\omega^{J_*(2)})^2 & \cdots & (\omega^{J_*(2)})^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (\omega^{J_*(\frac{n}{2}-1)}) & (\omega^{J_*(\frac{n}{2}-1)})^2 & \cdots & (\omega^{J_*(\frac{n}{2}-1)})^{n-1} \end{bmatrix}$$

D-4.2 Encryption and Decryption

Compare the phase $\Delta M + E$ of BFV, with $\Delta = \lfloor q/t \rfloor$, to the phase $M + tE$ of BGV (§D-2.3). In BGV, reduction modulo the plaintext modulus removes the divisible error after the centered phase has been recovered correctly. The following symmetric-key presentation uses a prime plaintext modulus and a convenient congruent modulus chain; BGV also has more general plaintext and chain configurations.

Also, in BGV, the ciphertext modulus q is leveled like CKKS's one: $q \in \{q_0, q_1, \cdots, q_L\}$, where each $q_l = \prod_{i=0}^{l} w_i$ (where each w_i is a CRT modulus).

BGV's encryption decryption process is described as follows:

⟨Summary D-4.2⟩ BGV Encryption and Decryption

Initial Setup:

- The plaintext modulus $t = p$ (a prime)
- The ciphertext modulus q is leveled like in CKKS: $q \in \{q_0, q_1, \cdots, q_L\}$, where each $q_l = \prod_{i=0}^{l} w_i$ (each w_i is a CRT modulus), and each $q_l \equiv 1 \bmod t$ (will be explained in §D-4.7)
- The noise scaling factor $\Delta = t$
- The secret key $S \xleftarrow{\$} \mathcal{R}_{\langle n, tern \rangle}$. The coefficients of the polynomial S ternary (i.e., $\{-1, 0, 1\}$).

Encryption Input: $M \in \mathcal{R}_{\langle n,q \rangle}$, $A \xleftarrow{\$} \mathcal{R}_{\langle n,q \rangle}$, $E \xleftarrow{\chi_\sigma} \mathcal{R}_{\langle n,q \rangle}$

1. Compute $B = -A \cdot S + M + \Delta E \ \in \mathcal{R}_{\langle n,q \rangle}$
2. $\mathsf{RLWE}_{S,\sigma}(M + \Delta E) = (A, B) \ \in \mathcal{R}^2_{\langle n,q \rangle}$

Decryption Input: $\mathsf{ct} = (A, B) \ \in \mathcal{R}^2_{\langle n,q \rangle}$

1. $\mathsf{RLWE}^{-1}_{S,\sigma}(\mathsf{ct}) = B + A \cdot S = M + \Delta E \pmod q$
2. Compute $M + \Delta E \bmod t$ to get M. ▷ modulo reduction of $M + \Delta E$ by t

The final output is $M(X) \in \mathbb{Z}_t[X]/(X^n + 1)$.

Conditions for Correct Decryption:

With centered representatives, require $\|M + tE\|_\infty < q_l/2$ at the level where decryption is performed. First compute $[B + AS]_{q_l}$ in the centered interval and then reduce modulo t. The one-sided inequality $m_i + te_i < q_l$ alone is insufficient, and reducing modulo q_l is not generally compatible with subsequent reduction modulo t.

The term eliminated by reduction modulo t is tE, whose coefficients are multiples of t; the coefficients of E need not be multiples of t. The same rule must hold for evaluation-key errors in key switching and relinearization. Throughout this chapter, encryption notation records the phase relation and does not assert that derived errors retain the original distribution.

D-4.3 Ciphertext-to-Ciphertext Addition

BGV's ciphertext-to-ciphertext addition scheme is exactly the same as BFV's scheme (Summary D-2.4 in §D-2.4).

⟨Summary D-4.3⟩ BGV Ciphertext-to-Ciphertext Addition

$$\begin{aligned}
&\mathsf{RLWE}_{S,\sigma}(M^{\langle 1\rangle} + \Delta E^{\langle 1\rangle}) + \mathsf{RLWE}_{S,\sigma}(M^{\langle 2\rangle} + \Delta E^{\langle 2\rangle}) \\
&= (A^{\langle 1\rangle},\ B^{\langle 1\rangle}) + (A^{\langle 2\rangle},\ B^{\langle 2\rangle}) \\
&= (A^{\langle 1\rangle} + A^{\langle 2\rangle},\ B^{\langle 1\rangle} + B^{\langle 2\rangle}) \\
&= \mathsf{RLWE}_{S,\sigma}((M^{\langle 1\rangle} + M^{\langle 2\rangle}) + \Delta E^{\langle 1\rangle} + \Delta E^{\langle 2\rangle})
\end{aligned}$$

D-4.4 Ciphertext-to-Plaintext Addition

BGV's ciphertext-to-plaintext addition scheme is almost the same as BFV's scheme (Summary D-2.5 in §D-2.5). However, one difference is that it's not the case that the plaintext polynomial $\Lambda(X)$ to be added is scaled up by Δ, but it remains as $\Lambda(X)$.

⟨Summary D-4.4⟩ BGV Ciphertext-to-Plaintext Addition

$$\begin{aligned}
&\mathsf{RLWE}_{S,\sigma}(M + \Delta E) + \Lambda \\
&= (A,\ B) + \Lambda \\
&= (A,\ B + \Lambda) \\
&= \mathsf{RLWE}_{S,\sigma}((M + \Lambda) + \Delta E)
\end{aligned}$$

D-4.5 Ciphertext-to-Plaintext Multiplication

BGV's ciphertext-to-plaintext multiplication scheme is exactly the same as BFV's scheme (Summary D-2.6 in §D-2.6).

⟨Summary D-4.5⟩ BGV Ciphertext-to-Plaintext Multiplication

$$\begin{aligned}
&\mathsf{RLWE}_{S,\sigma}(M + \Delta E) \cdot \Lambda \\
&= (A,\ B) \cdot \Lambda \\
&= (A \cdot \Lambda,\ B \cdot \Lambda) \\
&= \mathsf{RLWE}_{S,\sigma}((M \cdot \Lambda) + \Delta E \cdot \Lambda)
\end{aligned}$$

Notice that BGV's ciphertext-to-plaintext multiplication does not consume any multiplicative level.

D-4.6 ModDrop

BGV ModDrop reduces (A, B) coefficient-wise modulo a divisor q_{l-1} of q_l:

$$(A', B') = ([A]_{q_{l-1}}, [B]_{q_{l-1}}).$$

Since $A'S + B' \equiv AS + B \pmod{q_{l-1}}$, it preserves the centered phase $M + tE$ if $\|M + tE\|_\infty < q_{l-1}/2$. Under that condition the plaintext and noise remain unchanged while the modulus headroom decreases. Without the bound, wraparound can alter the plaintext modulo t.

D-4.7 Modulus Switch

- References: Design and implementation of HElib [23]; (Leveled) Fully Homomorphic Encryption without Bootstrapping [24]; Homomorphic Evaluation of the AES Circuit [25].

We describe a convenient special case with $q_l \equiv \widehat{q} \equiv 1 \pmod{t}$ and $\widehat{q} < q_l$. These congruences are sufficient for this presentation, not necessary conditions for every BGV implementation. Let $\rho = \widehat{q}/q_l$ and take centered integer representatives of A, B. Compute the integer polynomials

$$A' = \lceil \rho A \rfloor, \quad B' = \lceil \rho B \rfloor, \quad \epsilon'_A = \widehat{q}A - q_l A', \quad \epsilon'_B = \widehat{q}B - q_l B'.$$

The errors $\epsilon'_A/q_l, \epsilon'_B/q_l$ are real rounding errors of magnitude at most $1/2$, not elements of an integer residue ring. Correct the residues modulo t by

$$H_A = [q_l^{-1}\epsilon'_A]_t, \quad H_B = [q_l^{-1}\epsilon'_B]_t, \quad (\widehat{A}, \widehat{B}) = ([A' + H_A]_{\widehat{q}}, [B' + H_B]_{\widehat{q}}).$$

Here H_A, H_B use centered representatives. Before the final reduction, $\widetilde{A} = A' + H_A$ and $\widetilde{B} = B' + H_B$ satisfy $\widetilde{A} \equiv A \pmod{t}$ and $\widetilde{B} \equiv B \pmod{t}$.

For a proof, write $AS + B = M + tE + Kq_l$ and set $\delta_A = \widetilde{A} - \rho A$, $\delta_B = \widetilde{B} - \rho B$. Then

$$Y = \widetilde{A}S + \widetilde{B} - K\widehat{q} = \rho(M + tE) + \delta_A S + \delta_B, \qquad Y \equiv M \pmod{t}.$$

The latter congruence follows from $q_l \equiv \widehat{q} \pmod{t}$. If $\|Y\|_\infty < \widehat{q}/2$, then $Y = [\widehat{A}S + \widehat{B}]_{\widehat{q}}$ and decryption returns M. A sufficient deterministic condition is

$$\rho\|M + tE\|_\infty + \frac{t+1}{2}(1 + \|S\|_1) < \frac{\widehat{q}}{2}.$$

This follows from $\|\delta_A\|_\infty, \|\delta_B\|_\infty \le (t+1)/2$. The bounded corrections are not automatically harmless when the old phase is close to its boundary.

⟨Summary D-4.7⟩ BGV Modulus Switch

Scale ciphertext representatives approximately by ρ, then apply the congruence correction above. Under the phase bound, the result has phase $M + tE'$ for a new integer error E'. The plaintext modulus and noise divisibility factor remain t; the magnitude of the old phase is reduced, with a new rounding/correction term. This is not a noise reset.

D-4.7.1 Difference between Modulus Switch and ModDrop

CKKS rescaling divides its message scale by q_l/q_{l-1}; its ModDrop leaves the scale unchanged under a phase bound (§D-3.5.4). In BGV, the noise divisibility factor $\Delta = t$ is unchanged by either operation. BGV modulus switching approximately reduces the magnitude of the phase and preserves the plaintext residue through a congruence correction. ModDrop performs only residue reduction and reduces the available phase headroom. Neither operation is restricted to the end of a computation.

Source Code: The Python example illustrates modulus switching; any implementation must enforce the phase and parameter conditions above.

D-4.8 Ciphertext-to-Ciphertext Multiplication

For input phases $M_1 + tE_1$ and $M_2 + tE_2$, tensoring produces

$$D_0 = B_1B_2, \quad D_1 = A_1B_2 + A_2B_1, \quad D_2 = A_1A_2,$$
$$D_0 + D_1S + D_2S^2 \equiv M_1M_2 + t(M_1E_2 + M_2E_1 + tE_1E_2) \pmod{q}.$$

The noise remains divisible by t: it is incorrect to regard the scheme's factor t as successively becoming $t^2, t^4, \ldots$. Reducing M_1M_2 modulo t can add an additional carry to the integer error.

⟨Summary D-4.8⟩ BGV Ciphertext-to-Ciphertext Multiplication

1. Compute (D_0, D_1, D_2) modulo q.
2. Choose an exact integer gadget g_i with $D_2 \equiv \sum_i d_i g_i \pmod{q}$. The BGV evaluation keys must satisfy $B_i + A_iS \equiv g_iS^2 + tE_i \pmod{q}$. Form $(A, B) = (D_1, D_0) + \sum_i d_i(A_i, B_i)$; its added key-switching error is $t\sum_i d_iE_i$.
3. If needed for noise management, apply the BGV modulus switch in §D-4.7. This uses the actual ratio $\widehat{q}/q$ and a congruence correction. It is not division by t with rounding to a multiple of t.

Swapping the Order of Relinearization and Rescaling: One may modulus-switch the three components first, if the correctness bound includes the secret vector $(1, S, S^2)$ and the relinearization key is valid at the new modulus. The order changes the error bound and cost.

Details of the Optional Rescaling: The new error depends on the input messages, both input errors, polynomial convolution, key-switching, and the modulus-switch correction. Merely choosing $w_l = q_l/q_{l-1} > t$ does not guarantee that the output error is smaller than both input errors. Select the modulus chain using a complete bound on the resulting phase. CKKS likewise uses an explicit scale and error budget; noise growth is not a universal monotonic statement about every input.

D-4.9 Homomorphic Key Switching

Choose an exact integer gadget g_i and digits d_i satisfying $A \equiv \sum_i d_i g_i \pmod{q}$. A BGV evaluation key under S' must satisfy $b_i + a_iS' \equiv g_iS + tE_i \pmod{q}$. Then

⟨Summary D-4.9⟩ BGV Key Switching

$$(A', B') = (0, B) + \sum_i d_i(a_i, b_i), \qquad B' + A'S' \equiv M + t\left(E + \sum_i d_iE_i\right) \pmod{q}.$$

This is the gadget technique of §D-2.8, with the BGV-specific factor t in the evaluation-key errors. Check the resulting centered phase bound. Using generic evaluation keys with unscaled errors can change the plaintext residue.

D-4.10 Homomorphic Rotation of Input Vector Slots

For fully split batching and the root order used in §D-2.9, the automorphism $X \mapsto X^{J(h)}$, $J(h) = 5^h \bmod 2n$, rotates the two groups of $n/2$ scalar slots left by h positions within each group.

⟨Summary D-4.10⟩ BGV Homomorphic Rotation of Input Vector Slots

1. Replace $(A(X), B(X))$ by $(A(X^{J(h)}), B(X^{J(h)}))$ modulo X^n+1. The secret becomes $S(X^{J(h)})$ and the phase is $M(X^{J(h)}) + tE(X^{J(h)})$ modulo q.
2. Apply §D-4.9 to switch from $S(X^{J(h)})$ back to $S(X)$, using evaluation keys whose errors are multiples of t.

The automorphism permutes the message exactly, while key switching adds error. Correctness requires the resulting phase to satisfy its centered no-wrap bound. Other batching layouts have their corresponding automorphism-induced permutations.

D-4.11 Modulus Bootstrapping

- Reference: Bootstrapping for BGV and BFV Revisited [15], Sections 3.2–3.4 and 6.

Bootstrapping evaluates a suitable decryption circuit using an encrypted secret key. It produces a ciphertext with a renewed modulus and noise budget, after paying for the operations of the circuit itself. The following is the general bootstrapping outline of the cited paper; it is not an assertion that merely lifting a BGV ciphertext to a larger modulus refreshes it.

The coefficient-to-slot route below also assumes $\gcd(p, \mu) = 1$ for the cyclotomic index μ, as in Section 2.3 of the cited paper. For this book's power-of-two ring with $\mu = 2n$, this requires odd p. The paper's binary digit-extraction case uses compatible cyclotomic parameters; its rounding rule alone does not provide scalar or extension-ring slots in the ramified power-of-two ring.

Let the plaintext modulus be $t = p^r$, where p is prime and $r \geq 1$. After a valid modulus switch, let the ciphertext modulus be q with $q \equiv 1 \pmod{p^\varepsilon}$, where $\varepsilon > r$. Write its centered decryption phase as $M + p^r E$. Choose centered coefficient representatives and compute publicly

$$A' = [p^{\varepsilon-r} A]_q, \qquad B' = [p^{\varepsilon-r} B]_q.$$

There is an integer polynomial K' satisfying

$$u = B' + A'S = p^{\varepsilon-r} M + p^\varepsilon E + qK'.$$

Consequently,

$$w = [u]_{p^\varepsilon} = [p^{\varepsilon-r} M + K']_{p^\varepsilon}, \qquad M = \left[\left\lfloor \frac{w}{p^{\varepsilon-r}} \right\rceil\right]_{p^r}$$

provided that $\|K'\|_\infty < p^{\varepsilon-r}/2$. This is the rounded decryption formula of the cited paper, not an ordinary floor. A sufficient bound must account for all three terms

$$\|K'\|_\infty \leq \left\| \frac{B' + A'S}{q} \right\|_\infty + \frac{p^{\varepsilon-r}\|M\|_\infty}{q} + \frac{p^\varepsilon \|E\|_\infty}{q}.$$

In particular, the bound $\|K'\|_\infty \leq n+1$ does not follow merely from a ternary secret. The input noise and the chosen parameters must also be bounded.

1. **Homomorphic inner product.** Use a bootstrapping key encrypting S under the output secret, at a sufficiently large ciphertext modulus Q and plaintext modulus p^ε. Evaluate $B' + A'S$ homomorphically. This encrypts $w = [u]_{p^\varepsilon}$, with a new encryption error divisible by p^ε. The old ciphertext coefficients A', B' are public inputs to this computation. Correctness requires that this new ciphertext and every later intermediate ciphertext satisfy their respective decryption bounds.

2. **CoeffToSlot.** Homomorphically apply the appropriate linear transformation over the plaintext ring to place the coefficients of w into scalar slots. For the fully split case $p \equiv 1 \pmod{2n}$, the ring has n scalar slots and the transform is the inverse of the encoding matrix over $\mathbb{Z}_{p^\varepsilon}$. For general cyclotomic parameters, slots can be extension rings and multiple ciphertexts are needed, as explained in the cited paper. A complex CKKS matrix cannot simply be reused over this ring.
3. **Digit extraction.** Evaluate the slot-wise rounded map $w_i \mapsto [\lfloor w_i/p^{\varepsilon-r} \rceil]_{p^r}$. For odd p, it can be organized as repeated removal of balanced least significant base-p digits and exact division by p. The digit-extraction polynomials must return the required balanced digit modulo the current plaintext modulus; merely taking an ordinary floor fails for negative K'. Binary digit extraction needs its own rounding convention (including the offset in the cited paper). The input bound above, the digit-polynomial correctness conditions, and the homomorphic error budget are all required.
4. **SlotToCoeff.** Apply the inverse linear transformation to recover the polynomial M in the plaintext ring modulo p^r. The output is a BGV ciphertext at the remaining ciphertext modulus Q_{out}, with phase $M + p^r E_{\text{out}}$. Its error is determined by the bootstrapping circuit; it is not the original error reset to zero.

Handling the Divisions by p: At an intermediate plaintext modulus p^j, suppose digit subtraction gives a phase $pT + p^j F$ and $\gcd(p, Q_j) = 1$. Multiplication of both ciphertext components by $p^{-1} \bmod Q_j$ gives the phase

$$T + p^{j-1}F \pmod{Q_j}.$$

Interpret the result at plaintext modulus p^{j-1}, and verify its centered phase bound. This operation implements exact division only because the preceding plaintext is divisible by p; p^{-1} is not an inverse in $\mathbb{Z}_{p^j}$. After $\varepsilon - r$ such divisions the plaintext modulus is p^r. There is no additional final division by $p^{\varepsilon-r}$.

D-4.11.1 Discussion

The Reason for Modulus Switch from $q_l \to \hat{q}$: The choice $\hat{q} \equiv 1 \pmod{p^\varepsilon}$ makes the overflow $\hat{q}K'$ reduce to K' modulo the bootstrapping plaintext modulus p^ε. It does not by itself bound K' or prove correctness. A valid BGV modulus switch must preserve the plaintext residue and leave the input error within the bound required above. The general switching operation need not require $\hat{q}$ to be a factor of the original modulus.

The Choice of ε: Select ε from a deterministic input bound or an explicit probabilistic noise and secret-key model, together with a target failure probability. Increasing ε also increases the term $p^\varepsilon \|E\|_\infty / q$, so its effect cannot be assessed by the digit gap alone. Larger $\varepsilon - r$ generally makes digit extraction more expensive. The cited paper gives the relevant parameter and failure analysis; no unconditional choice follows from the ring dimension alone.

Generalization to $\Delta = p^r$: The derivation above already includes every positive integer r, with $\varepsilon > r$. The scaling is $p^{\varepsilon-r}$, the rounding gap is $p^{\varepsilon-r}/2$, and digit extraction decreases the plaintext modulus from p^ε to p^r. The BFV variant in §D-2.11.6 uses a different public preprocessing step; the common rounded-decryption structure does not make their ciphertext scaling rules interchangeable.

D-4.11.2 Comparing the Bootstrapping in BFV, BGV, and CKKS

In the BGV and BFV construction of [15], public preprocessing is followed by a homomorphic inner product, linear coefficient/slot transformations, and exact digit extraction under a changing plaintext modulus. Correctness depends on an explicit decryption bound and the homomorphic circuit's error budget. BGV's intermediate exact divisions by p reduce the plaintext modulus as well as the divisible phase; they are not generic inverses in the plaintext ring. BFV has a different scale-and-round preprocessing, as discussed in §D-2.11.6.

CKKS bootstrapping lifts ciphertext coefficients, moves coefficient information into slots, approximately evaluates a periodic modular-reduction function, and applies the inverse linear transform. Its output retains the input approximation error and adds the circuit's approximation and encryption errors. A larger ciphertext modulus restores computational headroom, but does not in general recover precision already lost. Improved precision procedures require their own analysis; neither an unavoidable monotone error increase on every operation nor a universal precision reset follows from these outlines.

D-5 RNS-variant FHE Schemes

BFV, BGV, and CKKS parameter sets can require coefficient moduli larger than a machine word. The residue number system (RNS, §A-14.1) represents such coefficients by residues modulo smaller pairwise coprime moduli. This permits exact modular arithmetic using word-sized operations, without reducing the total modulus or changing the underlying security parameters. The required modulus sizes depend on the scheme, circuit, precision, and security target; no particular bit length is universal.

CRT makes modular addition and multiplication straightforward, but changing bases and implementing scale-and-round require additional algorithms. This section develops those primitives for BFV, BGV, and CKKS. Common TFHE implementations use torus coefficients represented within a machine word and therefore do not need this particular large-integer decomposition; that implementation choice is not a theorem forbidding RNS techniques in other TFHE variants.

In this section, $[x]_q = |x|_q$ denotes the unique integer representative in $[-q/2, q/2)$ (§A-1.5). Unless stated otherwise, RNS moduli are pairwise coprime odd integers; prime moduli are convenient but are not required by CRT. For odd q, the extreme representatives are $-(q-1)/2$ and $(q-1)/2$. Bounds on these integer representatives must be distinguished from congruences. This distinction is essential for both approximate and exact base conversion.

Required Background

- §A-1: Modulo Arithmetic
- §A-2: Group
- §A-3: Field
- §A-5: Polynomial Ring
- §A-6: Decomposition
- §A-13: Modulo Rescaling
- §A-14: Chinese Remainder Theorem
- §A-10: Lagrange's Polynomial Interpolation
- §A-16: Efficient Polynomial Multiplication by FFT and NTT
- §B-1: Lattice-based Cryptography
- §B-3: RLWE Cryptosystem
- §B-4: GLWE Cryptosystem
- §C-1: GLWE Ciphertext-to-Ciphertext Addition
- §C-2: GLWE Ciphertext-to-Plaintext Addition
- §C-3: GLWE Ciphertext-to-Plaintext Multiplication
- §C-4.4: RLWE Modulus Switching
- §C-5: GLWE Key Switching
- §D-2: BFV Scheme
- §D-3: CKKS Scheme
- §D-4: BGV Scheme

D-5.1 Fast Base Conversion: FastBConv

- References: A Full RNS Variant of FV-like Somewhat Homomorphic Encryption Schemes [26]; BASALISC: Programmable Hardware Accelerator for BGV FHE [27].

Let $q = \prod_{i=1}^{k} q_i$, $b = \prod_{j=1}^{l} b_j$, with all base moduli pairwise coprime. The input residues $x_i = [x]_{q_i}$ determine the centered integer $x = [x]_q$. Exact base conversion would output $([x]_{b_j})_{j=1}^{l}$. Fast base conversion omits one large reduction and instead uses the same unreduced CRT sum in every output coordinate:

$$y_i = q/q_i, \qquad z_i = y_i^{-1} \bmod q_i, \qquad T = \sum_{i=1}^{k} [x_i z_i]_{q_i} y_i.$$

⟨Summary D-5.1⟩ Fast Base Conversion: FastBConv

$$\mathsf{FastBConv}(x, q, b) = ([T]_{b_1}, \ldots, [T]_{b_l}).$$

There is an integer u such that $T = x + uq$. For the centered odd-modulus convention above,

$$|T| < kq/2, \quad |x| < q/2, \quad |u| < (k+1)/2, \quad |u| \le \lfloor k/2 \rfloor.$$

Thus the output represents $x + uq \pmod b$, not necessarily $x \pmod b$. Set $U_k = \lfloor k/2 \rfloor$ for this bound.

Proof.

Each summand has absolute value less than $q/2$, and the CRT congruence gives $T \equiv x \pmod q$. The bound on $u = (T - x)/q$ uses both T and x; bounding T alone does not bound uq by the same interval. Every output residue is computed from this common T, so the error is a coherent multiple uq across the destination base. Other residue conventions yield different bounds, including the unsigned convention in [26]. □

This is an approximation to base conversion, not a failure of CRT. Subsequent algorithms must remove the multiple of q, divide it into a controlled error, or explicitly include it in their correctness bound.

D-5.2 Small Montgomery Reduction Algorithm: SmallMont

Small Montgomery reduction limits the multiple of q introduced by a fast base conversion. It does not generally make that conversion exact. The following centered-residue variant makes its assumptions explicit; the original unsigned-residue algorithm and its parameter bound appear in [26], Algorithm 2 and Lemma 4.

Let $x = [x]_q$, choose an odd auxiliary modulus b_α coprime to q and b, and compute the fast CRT lift of $[b_\alpha x]_q$:

$$T = [b_\alpha x]_q + uq = b_\alpha x + (v + u)q, \qquad |u| \le U_k.$$

The input consists of the residues of this same T modulo every b_j and b_α.

⟨Summary D-5.2⟩ Small Montgomery Reduction: SmallMont

1. Compute $a = [Tq^{-1}]_{b_\alpha}$ using the supplied b_α residue.
2. For $j = 1, \ldots, l$, output $r_j = [(T_{b_j} - qa)b_\alpha^{-1}]_{b_j}$.

The result represents $r = (T - qa)/b_\alpha = x + u'q \pmod{b}$. If $b_\alpha > U_k + 1/2$, then $u' \in \{-1, 0, 1\}$.

Proof.

Because $a \equiv v + u \pmod{b_\alpha}$, the quotient $u' = (v + u - a)/b_\alpha$ is integral. Centered representatives give $|v| < (b_\alpha + 1)/2$ and $|a| < b_\alpha/2$. Therefore

$$|u'| < 1 + \frac{U_k + 1/2}{b_\alpha} < 2.$$

This proves the stated three possible integer values. The numerator $T - qa$ is divisible by b_α, and its quotient can be computed modulo each b_j by multiplying by $b_\alpha^{-1} \bmod b_j$. □

The auxiliary residue must remain available throughout the computation. Reconstructing $[T]_b$ and then reducing that value modulo b_α generally loses the original b_α residue. Also, v can grow linearly with b_α; a claim that it grows more slowly is not a proof of the bound.

D-5.2.1 Improving FastBConv by Using SmallMont

With the parameter condition in §D-5.2,

$$\mathsf{SmallMont}(\mathsf{FastBConv}([b_\alpha x]_q, q, bb_\alpha), bb_\alpha, b_\alpha, q) = [x + u'q]_b, \qquad u' \in \{-1, 0, 1\}.$$

The preliminary multiplication by b_α is essential. The result is still an approximate lift of the centered x, and is distinct from FastBConvEx in §D-5.8.

D-5.3 RNS-based ModRaise: $\mathsf{ModRaise}_{\mathsf{RNS}}$

- Reference: A Full RNS Variant of Approximate Homomorphic Encryption [28], Section 3.2.

Base extension from q to qb retains the residues modulo the q_i and appends residues modulo the b_j. With fast base conversion it chooses an approximate integer lift:

⟨Summary D-5.3⟩ $\mathsf{ModRaise}_{\mathsf{RNS}}$

$$\mathsf{ModRaise}_{\mathrm{RNS}}(x, q, qb) = (x_1, \ldots, x_k, \mathsf{FastBConv}(x, q, b)).$$

The combined vector represents $x + uq \pmod{qb}$, with $|u| \le U_k$. Using the Montgomery preprocessing of §D-5.2 reduces this to an appropriate lift $x + u'q$ with $|u'| \le 1$.

This is a choice of integer representative, not a canonical ring embedding from $\mathbb{Z}_q$ into $\mathbb{Z}_{qb}$. If applied coefficient-wise to ciphertext components, write the new representatives as $A + qU_A$ and $B + qU_B$. Their phase changes by

$$q(U_B + U_A S).$$

Thus a component-wise bound $\|U_A\|_\infty, \|U_B\|_\infty \le U_k$ gives the phase bound $qU_k(1 + \|S\|_1)$, not simply qU_k. There can also be an existing multiple of q in the integer phase $B + AS$. These terms must be

handled by the consuming multiplication or bootstrapping algorithm; they are not independent fresh RLWE errors.

D-5.4 RNS-based ModDrop: $\mathsf{ModDrop}_{\mathsf{RNS}}$

$\mathsf{ModDrop}_{\mathsf{RNS}}$ removes the residues for the factors of b from a representation modulo qb:

⟨Summary D-5.4⟩ $\mathsf{ModDrop}_{\mathsf{RNS}}$

$$(x_{q_1}, \ldots, x_{q_k}, x_{b_1}, \ldots, x_{b_l}) \longmapsto (x_{q_1}, \ldots, x_{q_k}).$$

This is exact reduction modulo q; it performs no scaling or rounding.

For CKKS or BGV, the same centered integer phase survives only if it already fits in $[-q/2, q/2)$ coefficient-wise. CKKS then keeps its scale and BGV keeps its plaintext modulus. Otherwise a wrapped phase can change the decoded message. For BFV, even an unchanged phase does not in general preserve the message, because the standard scale changes from $\lfloor qb/t \rfloor$ to $\lfloor q/t \rfloor$. For example, with initial modulus 105, $t = 7$, and noiseless phase 15 encoding $m = 1$, dropping to modulus 35 leaves phase 15 but standard BFV decoding gives 3. Use a BFV-specific scale-and-round operation when message preservation is required.

D-5.5 RNS-based Modulus Switch: $\mathsf{ModSwitch}_{\mathsf{RNS}}$

Let x be an integer given in the combined bases of q and b. For odd b, define $r = [x]_b$, so $(x - r)/b = \lfloor x/b \rceil$. The following operation approximately divides by b and removes its base factors:

⟨Summary D-5.5⟩ $\mathsf{ModSwitch}_{\mathsf{RNS}}$

1. Convert the residues of r from base b to base q using $\mathsf{FastBConv}$. Its common integer lift is $r + ub$, with $|u| \le U_l$.
2. For each $i = 1, \ldots, k$, output

$$y_i = [b^{-1}(x_{q_i} - (r + ub)_{q_i})]_{q_i}.$$

The result represents

$$y = \left\lfloor \frac{x}{b} \right\rceil - u \pmod{q}, \qquad |u| \le U_l.$$

This follows by substituting $x - r = b\lfloor x/b \rceil$. There is no need to assume $b \ll q$ for this congruence. For a single odd dropped modulus, $l = 1$ and $u = 0$, so the result is exact nearest-integer division. With several dropped factors, the additional error $-u$ must be included in the ciphertext phase bound. Applied to (A, B), the total perturbation also includes multiplication of the component errors by S; it is not bounded solely by a single coefficient's u.

CKKS can use this as a rescaling primitive, tracking the actual scale division by b and all rounding errors. BFV and BGV require their own plaintext-preservation rules. In particular, ordinary scale-and-round is not the BGV modulus switch of §D-5.7.

D-5.5.1 Comparing $\mathsf{ModSwitch}_{\mathsf{RNS}}$, $\mathsf{ModRaise}_{\mathsf{RNS}}$, and $\mathsf{ModDrop}_{\mathsf{RNS}}$

$\mathsf{ModRaise}_{\mathsf{RNS}}$ chooses a lift into a larger combined base; $\mathsf{ModDrop}_{\mathsf{RNS}}$ exactly reduces to a smaller sub-base; $\mathsf{ModSwitch}_{\mathsf{RNS}}$ additionally divides the represented integer approximately. These are different operations. Whether any of them preserves a ciphertext's plaintext depends on the scheme, its scale or plaintext modulus, and the full centered phase bound.

D-5.6 RNS-based Decryption

This subsection will explain how to efficiently decrypt RNS-based ciphertexts for BFV, CKKS, and BGV.

D-5.6.1 BFV Decryption: $\mathsf{Dec}^{\mathsf{BFV}}_{\mathsf{RNS}}$

Use the book's ciphertext convention $B = -AS + \Delta M + E \pmod q$. The standard BFV decoding operation on each centered phase coefficient $x = [B + AS]_q$ is $[\lfloor tx/q \rceil]_t$; division by $\Delta = \lfloor q/t \rfloor$ is not algebraically identical. The following redundant-modulus correction implements scale-and-round when its rounding margin is satisfied, as in [26], Section 3.

Let $q = \prod_{i=1}^{k} q_i$ be odd, $\gcd(q, t) = 1$, and choose an odd γ coprime to qt. Put

$$m_0 = \left\lfloor \frac{tx}{q} \right\rceil, \quad \delta = \frac{tx}{q} - m_0, \quad v = [t\gamma x]_q.$$

The fast CRT lift of v is $v + uq$, $|u| \le U_k$. Compute its residues modulo γ and t, and multiply by $-q^{-1}$ in each modulus to obtain (y_γ, y_t). Then

$$y \equiv -q^{-1}(v + uq) \equiv \left\lfloor \frac{t\gamma x}{q} \right\rceil - u = \gamma m_0 + \eta \pmod{\gamma t}, \qquad \eta = \lfloor \gamma\delta \rceil - u.$$

If $|\eta| < \gamma/2$, its centered residue is exactly $y_\gamma = \eta$, and therefore

$$[m_0]_t = [(y_t - y_\gamma)\gamma^{-1}]_t.$$

A sufficient condition is $\gamma|\delta| + 1/2 + U_k < \gamma/2$.

⟨Summary D-5.6.1⟩ $\mathsf{Dec}^{\mathsf{BFV}}_{\mathsf{RNS}}$

1. Evaluate the phase residues $x_i = [B + AS]_{q_i}$.
2. Choose γ satisfying coprimality and the rounding-margin bound, and fast-convert $[t\gamma x]_q$ to the two moduli γ, t.
3. Multiply the two residues by $-q^{-1}$ in their respective moduli, obtaining (y_γ, y_t) with centered y_γ.
4. Output $[(y_t - y_\gamma)\gamma^{-1}]_t$.

Correct BFV decryption additionally requires $[m_0]_t$ to equal the intended plaintext. Merely choosing any coprime γ is insufficient, and the correction threshold is $\gamma/2$, not γ. Values arbitrarily close to a rounding boundary require a correspondingly stronger precision or correction strategy. This is not an unrestricted exact rounding primitive for arbitrary inputs and moduli. The alternative scale-and-round constructions in [29] also specify their precision and error conditions.

D-5.6.2 CKKS and BGV Decryption

Compute $[B + AS]_q$ in the current RNS base. CKKS decodes its centered integer representative with the tracked scale; BGV reduces that representative modulo the plaintext modulus. CRT reconstruction is one valid implementation. Dropping to a single small modulus before decoding is valid only when the entire centered phase fits within half of that smaller modulus. A ciphertext valid at its current level need not satisfy this stronger bound. When necessary, use scheme-correct rescaling or modulus switching with an error analysis, rather than assuming that a bare ModDrop to the lowest level preserves the message.

D-5.7 BGV's RNS-based Modulus Switch: $\mathsf{ModSwitch}^{\mathsf{BGV}}_{\mathsf{RNS}}$

BGV modulus switching must preserve the plaintext residue, in addition to reducing the ciphertext modulus. The ordinary rounding primitive of §D-5.5 does not enforce this condition. Nor can one obtain an unrestricted rounding operation by calling $\mathsf{Dec}^{\mathsf{BFV}}_{\mathsf{RNS}}$ with $t = \hat{q}$: if $\hat{q}$ divides q_l, the required inverse $q_l^{-1} \bmod \hat{q}$ does not exist.

Here is a direct RNS implementation for a common modulus-chain step. Let $q = Qw$, with $\gcd(w, Q) = \gcd(w, t) = 1$ and $w \equiv 1 \pmod{t}$. For each centered coefficient a of either ciphertext component, define

$$d = t[t^{-1}a]_w, \qquad a' = \frac{a - d}{w} \pmod{Q}.$$

Because $d \equiv a \pmod{w}$ and $d \equiv 0 \pmod{t}$, this is an integer division and $a' \equiv a \pmod{t}$ before reduction modulo Q. It is computed in each retained modulus $q_i \mid Q$ as

$$a'_i = [w^{-1}(a_i - d)]_{q_i}.$$

If w has several RNS factors, the centered residue $[t^{-1}a]_w$ must be reconstructed or converted with an exact, bounded procedure; a single-factor step needs no large reconstruction.

Let the input phase be $M + tE$ and use the same overflow polynomial as in §D-4.7. The pre-reduction output phase is

$$Y = \frac{M + tE}{w} - \frac{d_B + d_A S}{w}.$$

Since $q = Qw \equiv Q \pmod{t}$, this phase is congruent to M modulo t. A sufficient no-wrap condition is

$$\frac{\|M + tE\|_\infty}{w} + \frac{t}{2}(1 + \|S\|_1) < \frac{Q}{2}.$$

Under this condition the output is a valid BGV ciphertext for the same plaintext. More general modulus chains can track a plaintext correction factor; the congruence assumption here is a stated special case, not a universal restriction on BGV.

D-5.8 Exact Fast Base Conversion: FastBConvEx

Unlike SmallMont, exact base conversion with a redundant modulus removes the entire CRT ambiguity when an explicit bound is satisfied. This is the Shenoy–Kumaresan style correction used in [26], Section 4.4, Lemma 6. Let $b = \prod_{j=1}^{l} b_j$ and suppose the residues of an integer x are known modulo every b_j and an additional coprime odd modulus b_α. Write

$$\widehat{x} = [x]_b = x + \mu b, \qquad T = \widehat{x} + ub = x + (\mu + u)b,$$

where T is the common unreduced CRT sum used by both fast conversions and $|u| \le U_l$. Require $|\mu + u| < b_\alpha/2$; for a known bound $|\mu| \le \lambda$, the sufficient condition $b_\alpha > 2(\lambda + U_l)$ can be used.

⟨Summary D-5.8⟩ Exact Fast Base Conversion: FastBConvEx

1. Retain the base-b residues of x, and separately its supplied residue $x_\alpha = [x]_{b_\alpha}$.
2. Compute $\gamma = [(\mathsf{FastBConv}(\widehat{x}, b, b_\alpha) - x_\alpha)b^{-1}]_{b_\alpha}$.
3. Output $[\mathsf{FastBConv}(\widehat{x}, b, q) - \gamma b]_q$, coordinate-wise in the target base.

Proof.

Both conversions use T. Hence $\gamma = [\mu + u]_{b_\alpha} = \mu + u$ under the stated bound, and $T - \gamma b = x$. The output is exactly $[x]_q$. This is an algebraic identity; no fresh approximation error remains. If the bound is violated, the correction can instead differ by a multiple of bb_α, which need not vanish modulo q. □

Necessity of the Centered (i.e., Signed) Residue Representation: The correction γ must be interpreted as the signed bounded integer $\mu + u$. The centered convention supplies this interpretation when $|\mu + u| < b_\alpha/2$. An unsigned implementation is also possible if it explicitly recovers the sign; simply treating an unsigned residue as that integer is incorrect. The requirement is a quantified range bound, not the unquantified statement that x must be "much smaller" than its modulus.

D-5.9 Decomposing Multiplication: $\mathsf{DecompMult}_{\mathsf{RNS}}$

RNS supplies an integer gadget for key switching and relinearization. With $q = \prod_i q_i$, define

$$g_i = \frac{q}{q_i}\left[\left(\frac{q}{q_i}\right)^{-1}\right]_{q_i}, \qquad A_i = [A]_{q_i}, \qquad A \equiv \sum_i A_i g_i \pmod{q}.$$

Here each A_i is lifted coefficient-wise to a small integer polynomial. When multiplying it by a ciphertext in R_q, reduce those same small integers in *every* modulus of the RNS base. Keeping only the i-th coordinate and filling the others with zero would give a different multiplier and a different error bound.

⟨Summary D-5.9⟩ $\mathsf{DecompMult}_{\mathsf{RNS}}$

For switching S to S', generate evaluation keys (a_i, b_i) satisfying

$$b_i + a_i S' \equiv g_i S + E_i \pmod{q}.$$

Then $\sum_i A_i(a_i, b_i)$ has phase $AS + \sum_i A_i E_i$ modulo q. Adding $(0, B)$ completes key switching. In BGV use tE_i instead of E_i so that the added error is divisible by the plaintext modulus.
For relinearization, replace $g_i S$ by $g_i S^2$ in the keys, decompose D_2, and add the result to (D_1, D_0) in the book's (A, B) ordering. The resulting phase is

$$D_0 + D_1 S + D_2 S^2 + \sum_i D_{2,i} E_i \pmod{q}.$$

For negacyclic polynomials, $\|A_i E_i\|_\infty \le n\|A_i\|_\infty \|E_i\|_\infty$, so an explicit bound on the added error is $n\sum_i (q_i/2)\|E_i\|_\infty$. One can instead put the modular inverse into the digit $[A_i(q/q_i)^{-1}]_{q_i}$ and use

weight q/q_i, as in [26], Section 4.5. Both choices are valid, but their digits and keys must not be mixed. Encryption symbols in these identities denote phase relations, not equalities of freshly sampled ciphertext distributions.

D-5.10 Applying RNS Techniques to FHE Operations

This subsection will explain how the RNS primitives we have learned so far are used to handle FHE operations for RNS-based ciphertexts in BFV, CKKS, and BGV.

D-5.10.1 Addition and Multiplication of Polynomials

RNS decomposes the coefficient modulus, so scalar addition and scalar multiplication are performed separately modulo each q_i (§A-14.1). Polynomial addition is coefficient-wise, but polynomial multiplication still requires negacyclic convolution modulo $X^n + 1$. It becomes pointwise multiplication only after an appropriate NTT, when the modulus supports that transform. The following scalar identities describe the coefficient arithmetic used inside those polynomial operations.

$$P^{\langle 1 \rangle} = \sum_{a=0}^{n-1} c_a^{\langle 1 \rangle} \cdot X^a \in \mathcal{R}_{\langle n,q \rangle}$$
$$P^{\langle 2 \rangle} = \sum_{b=0}^{n-1} c_b^{\langle 2 \rangle} \cdot X^b \in \mathcal{R}_{\langle n,q \rangle}$$

In the RNS-variant FHE schemes, we express each coefficient of the polynomial as an RNS residue vector as follows:

$$c_a^{\langle 1 \rangle} = (c_{a,1}^{\langle 1 \rangle}, c_{a,2}^{\langle 1 \rangle}, \cdots, c_{a,k}^{\langle 1 \rangle}) \in \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \times \cdots \times \mathbb{Z}_{q_k} \triangleright \text{for } a \in [0, n-1]$$
$$c_b^{\langle 2 \rangle} = (c_{b,1}^{\langle 2 \rangle}, c_{b,2}^{\langle 2 \rangle}, \cdots, c_{b,k}^{\langle 2 \rangle}) \in \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \times \cdots \times \mathbb{Z}_{q_k} \triangleright \text{for } b \in [0, n-1]$$

Given the above RNS setup, when we add or multiply two polynomials, each coefficient-to-coefficient addition is computed as element-wise additions of two RNS residue vectors as follows:

$$c_a^{\langle 1 \rangle} + c_b^{\langle 2 \rangle} \equiv \sum_{i=1}^{k} (c_{a,i}^{\langle 1 \rangle} + c_{b,i}^{\langle 2 \rangle}) y_i z_i \bmod q \triangleright \text{where } y_i = \frac{q}{q_i},\ z_i = |y_i^{-1}|_{q_i}$$
$$\iff (c_{a,1}^{\langle 1 \rangle} + c_{b,1}^{\langle 2 \rangle},\ c_{a,2}^{\langle 1 \rangle} + c_{b,2}^{\langle 2 \rangle}, \cdots,\ c_{a,k}^{\langle 1 \rangle} + c_{b,k}^{\langle 2 \rangle}) \in \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \times \cdots \times \mathbb{Z}_{q_k}$$

Similarly, each coefficient-to-coefficient multiplication is computed as element-wise multiplications of two RNS residue vectors as follows:

$$c_a^{\langle 1 \rangle} \cdot c_b^{\langle 2 \rangle} \equiv \sum_{i=1}^{k} (c_{a,i}^{\langle 1 \rangle} \cdot c_{b,i}^{\langle 2 \rangle}) y_i z_i \bmod q$$
$$\iff (c_{a,1}^{\langle 1 \rangle} \cdot c_{b,1}^{\langle 2 \rangle},\ c_{a,2}^{\langle 1 \rangle} \cdot c_{b,2}^{\langle 2 \rangle}, \cdots,\ c_{a,k}^{\langle 1 \rangle} \cdot c_{b,k}^{\langle 2 \rangle}) \in \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \times \cdots \times \mathbb{Z}_{q_k}$$

Using the above isomorphism, we can efficiently compute ciphertext-to-plaintext addition, ciphertext-to-ciphertext addition, and ciphertext-to-plaintext multiplication of big polynomial coefficients (e.g., 1000 bits big) based on small RNS residues (e.g., 30 bits each).

D-5.10.2 Key Switching

With the evaluation keys and lifted digits of §D-5.9, compute

$$(A', B') = (0, B) + \mathsf{DecompMult}_{\mathrm{RNS}}(A, S_{\langle S', \mathrm{RNS} \rangle}).$$

If the input phase is $B + AS$, the output phase is $B + AS + E_{\text{ks}}$ modulo q, with $E_{\text{ks}} = \sum_i A_i E_i$. BFV and CKKS retain their current message scale while adding this error. BGV uses evaluation keys with errors divisible by t, so its phase remains congruent to the same message modulo t. All operations can be performed in the current RNS base, but the phase bounds remain scheme-specific.

D-5.10.3 Input Slot Rotation

For $J(h) = 5^h \bmod 2n$, apply the automorphism $X \mapsto X^{J(h)}$ to both ciphertext components. Reduce exponents modulo $2n$, with $X^{n+i} = -X^i$. In coefficient representation this permutes entire RNS residue vectors and negates the relevant vectors in every modulus. It introduces no approximation error by itself.

The resulting phase is the original phase evaluated at $X^{J(h)}$, under secret $S(X^{J(h)})$. Then use the RNS key-switching operation in §D-5.10.2 to return to $S(X)$. This step adds key-switching error, with a factor of t for BGV. The induced slot permutation depends on the chosen root ordering: CKKS has $n/2$ independent complex slots, while fully split BFV/BGV batching has two orbits of $n/2$ scalar slots under powers of 5. RNS does not change that permutation.

D-5.10.4 BFV's Ciphertext-to-Ciphertext Multiplication

The BEHZ RNS multiplication in [26], Sections 4.2–4.5, performs base extension, tensor multiplication, approximate scale-and-round, exact conversion back, and relinearization. Its auxiliary-base and error bounds are part of the algorithm. The HPS construction [29], Sections 2.2–2.4 and 4.5, uses different conversion/scaling techniques; the variants should not be conflated.

The following states the centered-residue version of the primitives used here. Let $b = \prod_j b_j$. Use a redundant modulus b_α for the final exact conversion and a *distinct* temporary Montgomery modulus $\widetilde{m}$ for controlling the initial lifts. All base moduli must satisfy the required coprimality conditions.

1. **Base extension.** For each component coefficient x, fast-convert $[\widetilde{m}x]_q$ to the combined auxiliary base $bb_\alpha\widetilde{m}$, then apply $\mathsf{SmallMont}$ with temporary modulus $\widetilde{m}$. This leaves residues modulo bb_α for a lift $\widehat{x} = x + uq$, $|u| \le 1$, and retains the original base-q residues. The auxiliary residue b_α is kept for the later exact conversion; it is not discarded as the Montgomery modulus.
2. **Tensor multiplication.** In the combined base qbb_α, compute the negacyclic integer-polynomial products
$$D_0 = \widehat{B}_1\widehat{B}_2, \quad D_1 = \widehat{A}_1\widehat{B}_2 + \widehat{A}_2\widehat{B}_1, \quad D_2 = \widehat{A}_1\widehat{A}_2.$$
Choose the auxiliary range so that the intended integers tD_j can be represented without wraparound, for example by verifying $\|tD_j\|_\infty < qbb_\alpha/2$. The coefficient convolution, n, t, and the component-lift bounds all matter; $b > \Delta$ alone is insufficient.
3. **Approximate scale-and-round.** Multiply each D_j by t and apply $\mathsf{ModSwitch}_{\mathsf{RNS}}$ dropping the base of q. The output represents
$$Z_j = \left\lfloor \frac{tD_j}{q} \right\rceil - u_j = \frac{tD_j}{q} + \rho_j, \quad \|u_j\|_\infty \le U_k, \quad \|\rho_j\|_\infty \le U_k + \frac{1}{2}.$$
in base bb_α. This is approximate scale-and-round, not exact rounding.
4. **Exact return to base q.** Apply $\mathsf{FastBConvEx}$ to each Z_j using its retained b_α residue. For every coefficient, the bound $|\mu + u| < b_\alpha/2$ in §D-5.8 must hold. An auxiliary-range bound for tD_j does not replace this separate correction bound.
5. **Relinearization.** Compute $(Z_1, Z_0) + \mathsf{DecompMult}_{\text{RNS}}(Z_2, S^2_{\langle S,\text{RNS}\rangle})$ using the (A, B) ordering. This adds the evaluation-key error E_{ks}.

Let $H_i = \widehat{B}_i + \widehat{A}_i S$ denote the lifted integer phases. The output phase satisfies

$$F \equiv \frac{t}{q} H_1 H_2 + \rho_0 + \rho_1 S + \rho_2 S^2 + E_{\mathrm{ks}} \pmod{q}.$$

One cannot replace the three component roundings by a single rounding of $H_1 H_2$. The bound must include the secret-dependent terms, input errors, lift multiples, and the discrepancy between $t\Delta^2/q$ and Δ. For example, writing a centered output phase as $\Delta M_{\mathrm{out}} + E_{\mathrm{out}}$, a sufficient decoding condition is

$$\|\Delta M_{\mathrm{out}} + E_{\mathrm{out}}\|_\infty < q/2, \qquad \left\| \frac{t}{q} E_{\mathrm{out}} + \left(\frac{t\Delta}{q} - 1 \right) M_{\mathrm{out}} \right\|_\infty < 1/2.$$

Here $M_{\mathrm{out}} = [M_1 M_2]_t$. These are absolute error conditions; comparing the error only with the message magnitude is invalid, especially when a message coefficient is zero. See §D-2.7.5 and the cited papers for the corresponding full parameter analysis.

D-5.10.5 CKKS's Ciphertext-to-Ciphertext Multiplication

Compute the tensor product directly in the current base, relinearize using the appropriate RNS evaluation key, and rescale when required by the scale/noise budget (§D-3.5.6). The product phase must fit the current modulus before an approximate interpretation is used; a level number alone does not establish this.

If the input scales are Δ_1, Δ_2 and the dropped modulus is w, the new scale is $\Delta_1 \Delta_2 / w$. A single-prime rescale implements component-wise nearest-integer division exactly, but still contributes phase error $R_B + R_A S$. A multi-factor approximate conversion can contribute additional bounded errors. The BFV-specific factor t, wide tensor-product base, and final exact return to a fixed BFV modulus are not part of this CKKS procedure. Hybrid key-switching may nevertheless use its own auxiliary base.

D-5.10.6 BGV's Ciphertext-to-Ciphertext Multiplication

Tensor multiplication and RNS relinearization give the same plaintext product modulo t, with an error still divisible by t (§D-4.8). Evaluation-key errors must also be multiples of t. If modulus switching is needed, use $\mathsf{ModSwitch}_{\mathrm{RNS}}^{\mathrm{BGV}}$ from §D-5.7, or a more general congruence-preserving variant with an explicitly tracked correction factor. Ordinary CKKS rescaling is not a substitute. The factor t remains fixed; correctness and useful noise reduction depend on the full output-phase bound.

D-5.10.7 BFV's Bootstrapping

RNS is a representation of the ciphertext coefficient modulus; it does not prohibit a plaintext modulus p^ε. A prime power can also be one CRT modulus, provided it is coprime to the other base moduli. It is only invalid to treat repeated factors $p, p, \ldots, p$ as distinct coprime CRT coordinates.

The BFV bootstrapping construction of [15], Sections 3.2–3.4, uses the public preprocessing

$$c_i' = \left[\left\lfloor \frac{p^\varepsilon}{q} c_i \right\rceil \right]_{p^\varepsilon},$$

then homomorphically computes $[c_0' + c_1' S]_{p^\varepsilon}$ with the bootstrapping key. Here $(c_0, c_1) = (B, A)$, and the plaintext modulus for that homomorphic computation is p^ε. Linear transforms and digit extraction

implement the rounded decryption map to plaintext modulus p^r, followed by the inverse transform. See §D-2.11.6 for the scheme-level discussion.

This coefficient-to-slot construction requires $\gcd(p, \mu) = 1$ for the cyclotomic index μ (Section 2.3 of the cited paper). For $\mu = 2n$ with power-of-two n, use odd p. A binary digit-extraction rule cannot by itself supply the missing plaintext-ring CRT in that ring.

Public preprocessing can reconstruct ciphertext coefficients from RNS and round exactly, or use a proven RNS scale-and-round routine with sufficient precision. Ciphertext arithmetic in the homomorphic circuit can then use a separate RNS base for its working ciphertext modulus. If approximate base conversion is used, its error must be included in the rounded-decryption bound before claiming correctness. The BEHZ/HPS multiplication routines alone do not prove a new bootstrapping construction, and the invalid argument about non-coprime factors does not justify replacing this preprocessing by an unproved chain of base extensions and reinterpretations.

For the idealized input phase $(q/p^r)M + E$, component rounding contributes $R_B + R_A S$, and the rounded-decryption error includes $(p^\varepsilon/q)E + R_B + R_A S$. If the actual encryption scale is $\lfloor q/p^r \rfloor$, its discrepancy must be included in E in this expression. Recovering M requires an explicit bound below $p^{\varepsilon-r}/2$, as well as valid intermediate ciphertexts throughout the homomorphic circuit.

D-5.10.8 CKKS's Bootstrapping

Use the CKKS sequence in §D-3.13.7: modulus raising, coefficient-to-slot transformation, approximate modular reduction, and slot-to-coefficient transformation. Sparse bootstrapping additionally needs the partial-sum operation explained in §D-3.13.6. There is no separate explicit homomorphic secret-key inner product in this basic CKKS construction, nor a universal final reinterpretation step.

If approximate RNS lifting changes the components by $q_0 U_A, q_0 U_B$, the lifted phase is

$$\Delta M + E + q_0(K + U_B + U_A S).$$

The additional coefficient-wise lift bound is amplified by the secret polynomial. Even if each coefficient of U_A, U_B lies in $\{-1, 0, 1\}$, the corresponding phase multiple need not. The approximation interval for the periodic modular-reduction circuit must cover this total overflow, and its error budget must include the linear transforms, rescaling, key switching, polynomial approximation, and retained input error. Periodicity alone does not make a finite homomorphic polynomial an exact modular-reduction function.

D-5.10.9 BGV's Bootstrapping

Follow the BGV construction in §D-4.11. First perform a valid modulus switch to $\hat{q} \equiv 1 \pmod{p^\varepsilon}$ and compute centered public coefficients $A' = [p^{\varepsilon-r} A]_{\hat{q}}$, $B' = [p^{\varepsilon-r} B]_{\hat{q}}$. The special modulus need not be a member of the current RNS base: exact public reconstruction and congruence-preserving switching are a valid implementation route.

Homomorphically evaluate $B' + A'S$ using a bootstrapping key at plaintext modulus p^ε and a sufficiently large working ciphertext modulus Q. RNS represents Q; it does not require the plaintext modulus to equal an RNS factor. Then implement the appropriate coefficient/slot transforms and balanced rounded digit extraction, with a verified bound on K' and on every homomorphic intermediate phase.

During digit extraction, each justified exact division by p changes the plaintext modulus $p^j \to p^{j-1}$; after $\varepsilon - r$ such divisions it is p^r. Do not append another multiplication by $p^{-(\varepsilon-r)}$, which would divide the message twice. Also, the base-extension primitive $q \to qb$ does not directly describe an arbitrary

conversion $\hat{q} \to Q$ when $\hat{q}$ does not divide Q. An alternative bootstrapping construction using such lifts needs its own correctness proof; it is not established by the RNS representation alone.

D-5.10.10 Noise Impact of RNS Operations

Track the actual effect of each primitive. FastBConv chooses a lift differing by a bounded multiple of the source modulus; SmallMont limits that multiple but does not generally eliminate it. $\mathsf{ModSwitch}_{\mathsf{RNS}}$ turns a base-conversion discrepancy into a bounded additive error after division. FastBConvEx is exact under its range bound and does not need SmallMont to remove an error. $\mathsf{ModDrop}_{\mathsf{RNS}}$ is exact residue reduction, but can invalidate a ciphertext if the new phase or scale conditions fail.

For component perturbations δ_A, δ_B, the induced phase perturbation is $\delta_B + \delta_A S$, with coefficient bound $\|\delta_B\|_\infty + \|\delta_A\|_\infty \|S\|_1$. Higher-degree ciphertexts similarly involve powers of S. Whether these perturbations are ultimately canceled, approximately removed, or retained as decryption error depends on the consuming scheme and operation; no single blanket noise statement applies to all RNS primitives.

D-5.10.11 Python Source Code of RNS Primitives

We provide a Python script implementing the following exemplary RNS primitives:
FastBConv, $\mathsf{ModRaise}_{\mathsf{RNS}}$, $\mathsf{ModDrop}_{\mathsf{RNS}}$, $\mathsf{ModSwitch}_{\mathsf{RNS}}$, SmallMont, and FastBConvEx.

D-6 FHE Scheme Comparison and Summary

The following tables summarize the ring versions discussed in this book, rather than every variant of these scheme families. Set $R = \mathbb{Z}[X]/(X^n + 1)$ and $R_q = R/qR$, with power-of-two n. Write $[x]_q$ for a centered representative and $\lfloor x \rceil$ for nearest-integer rounding; Rounding, modular reduction, and coefficient bounds are interpreted coefficientwise; polynomial multiplication is in the stated quotient ring.

We use the plus-phase convention $b + \langle \vec{a}, \vec{s} \rangle$ or $B + AS$ throughout these tables. The TFHE chapter uses the equivalent minus convention, obtained by negating the public mask; blind-rotation and key-switching signs must follow that choice. Evaluated ciphertext errors are not assumed to retain the distribution or standard deviation of fresh encryption. A summary formula is subject to its decoding bound, compatible parameters, and evaluation-key assumptions. In particular, publishing encryptions of a secret under a related secret may require a circular/KDM-security assumption or an appropriate key hierarchy.

This requirement depends on the actual auxiliary-message function and ciphertext format. Same-key raw R_q-affine phases admit a mask-shift reduction; that result does not automatically cover secret squares or arbitrary coefficient maps. Independently generated keys in a directed acyclic graph admit a suitable hybrid argument even for such efficiently computable source-key functions, subject to the stated sampling and dependency conditions. Neither proof automatically establishes security for a fixed public-seed expansion. See Section C-5 for the exact conditions.

TFHE	LWE for scalar ciphertexts; GLWE/RLWE for ring ciphertexts and evaluation keys.
BFV / CKKS / BGV	RLWE for the ring versions discussed here. Security depends on the ring, dimensions, moduli, secret/error distributions, and released evaluation keys.

Table 7: Hard Problem Basis

TFHE	Scalar interface: $(\vec{a}, b) \in \mathbb{Z}_q^{k+1}$. Blind rotation uses GLWE/GGSW ring ciphertexts internally.
BFV / CKKS / BGV	A relinearized ciphertext is $(A, B) \in R_q^2$. Multiplication temporarily produces three components.

Table 8: Ciphertext Representation

TFHE	A finite message alphabet, often $m \in \mathbb{Z}_t$, embedded in the torus. Power-of-two t is an encoding choice, not a universal requirement.
BFV / BGV	$M \in R_t$. The modulus t need not be prime. Full scalar batching commonly uses prime $t \equiv 1 \pmod{2n}$; other choices may yield extension-ring slots.
CKKS	Up to $n/2$ complex slots, encoded approximately as an integer polynomial. Real/complex semantic values are not exact finite-ring elements.

Table 9: Plaintext and Slot Domain

TFHE	Here $s_i \in \{0, 1\}$. A separate GLWE secret is used in blind rotation.
BFV / CKKS / BGV	A small polynomial $S \in R$, often with coefficients in $\{-1, 0, 1\}$, embedded into R_q. This is not multiplication in R_3.

Table 10: Secret Key

TFHE	Fresh mask $\vec{a} \leftarrow \mathbb{Z}_q^k$; scalar $b \in \mathbb{Z}_q$. The simple integer encoding below assumes $t \mid q$.
BFV / CKKS / BGV	Fresh mask $A \leftarrow R_q$; $B \in R_q$. An evaluated ciphertext need not have the distribution of a fresh encryption.

Table 11: Ciphertext and Fresh Randomness

All schemes	Fresh error is sampled from a specified small distribution. Discrete Gaussian, bounded, and other distributions require their own parameter analysis.
Evaluation	For independent centered errors, $\mathrm{Var}(e_1+e_2) = \mathrm{Var}(e_1)+\mathrm{Var}(e_2)$. Reusing one ciphertext gives error $2e$, not two independent samples.

Table 12: Noise

TFHE	$\Delta = q/t$ in this integer encoding, with $t \mid q$.
BFV	$\Delta = \lfloor q/t \rfloor$; write $r = q - t\Delta$. The remainder r enters the correctness bounds.
CKKS	Actual scale Δ is metadata and changes during multiplication/rescaling. Require the entire phase, including error, to fit the current modulus.
BGV	Plaintext is unscaled; the error is a multiple of t. Writing $\Delta = t$ for the error factor does not identify it with the BFV/CKKS plaintext scale.

Table 13: Scaling Factor

TFHE	$b = [\Delta m + e - \langle \vec{a}, \vec{s} \rangle]_q$. Sample a fresh mask and error for each encryption.
BFV	$B = [\Delta M + E - AS]_q$.
CKKS	$B = [\lfloor \Delta M \rceil + E_{\mathrm{enc}} - AS]_q$. Below E includes encoding and encryption error.
BGV	$B = [M + tE - AS]_q$.

Table 14: Symmetric Encryption

TFHE	$\phi = b + \langle \vec{a}, \vec{s} \rangle \equiv \Delta m + e \pmod q$.
BFV / CKKS	$\Phi = B + AS \equiv \Delta M + E \pmod q$.
BGV	$\Phi = B + AS \equiv M + tE \pmod q$.

Table 15: Phase Relation

TFHE	$m = \lfloor [\phi]_q / \Delta \rceil \bmod t$ if $\lvert e \rvert < \Delta/2$.
BFV	$M = \lfloor (t/q)[\Phi]_q \rceil \bmod t$. For a chosen integer lift of M, a sufficient condition is $\lVert tE - rM \rVert_\infty < q/2$, where $r = q - t\Delta$.
CKKS	Decode $[\Phi]_q/\Delta$ through the canonical embedding. If $\lVert \Delta M + E \rVert_\infty < q/2$, coefficient error is E/Δ; slot error must be bounded after embedding.
BGV	$M = [\Phi]_q \bmod t$. A sufficient no-wrap condition is $\lVert M + tE \rVert_\infty < q/2$.

Table 16: Decryption and Sufficient Correctness Conditions

TFHE / BFV	A fixed modulus q is sufficient for the basic presentation; RNS and modulus-switching variants also exist.
CKKS	$q_i = \prod_{j=0}^{i} w_j$ with pairwise coprime factors. Dropping $w_l = q_l/q_{l-1}$ by rescaling changes scale by exactly $1/w_l$, even when $w_l \approx \Delta$.
BGV	$q_i = \prod_{j=0}^{i} w_j$ with factors coprime to t. Taking every $w_j \equiv 1 \pmod t$ permits a simple residue-preserving convention; other chains require correction factors.

Table 17: Ciphertext Modulus

All schemes	Add corresponding components: $(A_1 + A_2, B_1 + B_2)$, or $(\vec{a}_1 + \vec{a}_2, b_1 + b_2)$. Keys and moduli must match; CKKS scales must also be aligned.
Error	The lifted phase errors add: $E_{\text{out}} = E_1 + E_2$. For BFV, re-expressing a wrapped sum as a representative in R_t also introduces the $q - t\Delta$ carry correction.

Table 18: Ciphertext-to-Ciphertext Addition

TFHE / BFV	Add Δc to b, or ΔC to B, using the agreed integer representatives.
CKKS	Add the integer encoding $\lfloor \Delta C \rceil$ to B; encoding error is included in the precision budget.
BGV	Add C to B modulo the ciphertext modulus.

Table 19: Ciphertext-to-Plaintext Addition

TFHE / BFV / BGV	Multiply both components by the unscaled plaintext c or C. The phase error becomes ce or CE, plus any representative correction. In R, $\|CE\|_\infty \le \|C\|_1 \|E\|_\infty$.
CKKS	Multiply by an integer encoding at scale Δ_C. If rescaling removes w_l, the output scale is $\Delta_{\text{out}} = \Delta\Delta_C / w_l$. Component rounding contributes $\rho_B + \rho_A S$ with $\|\rho_A\|_\infty, \|\rho_B\|_\infty \le 1/2$.

Table 20: Ciphertext-to-Plaintext Multiplication

TFHE	External products require a compatible GSW/GGSW encryption of one operand. Converting an LWE ciphertext to such a representation uses circuit bootstrapping, not ordinary PBS alone. Noise includes decomposition and evaluation-key errors.
BFV	In the standard FV multiplication, form the integer negacyclic products $(D_2, D_1, D_0) = (A_1A_2, A_1B_2 + A_2B_1, B_1B_2)$, scale components by t/q and round, then relinearize modulo q. Do not reduce products modulo q before scaling. The chapter also analyzes a separately specified $1/\Delta$ teaching variant.
CKKS	Tensor multiplication, relinearization, and (when scheduled) rescaling. With input scales Δ_1, Δ_2, multiplication gives scale $\Delta_1\Delta_2$; after rescaling it is $\Delta_1\Delta_2/w_l$. Relinearization and rounding add error.
BGV	Tensor multiplication and relinearization preserve the message modulo t when the noise bound holds. Optional modulus switching uses the ratio q_{l-1}/q_l and residue correction. It is not division by t followed by rounding to a multiple of t.

Table 21: Ciphertext-to-Ciphertext Multiplication

TFHE	Leveled operations have a finite noise budget. Gate/PBS evaluation performs bootstrapping and therefore does not belong in a claim about computation without bootstrapping.
BFV	Fixed parameters permit a bounded circuit family determined by noise growth. A fixed ciphertext modulus does not imply unlimited multiplication.
CKKS / BGV	The modulus chain and noise/precision budget constrain multiplicative depth and level consumption, not the total number of parallel multiplication gates. Refreshing requires a valid bootstrapping configuration.

Table 22: Computation Budget without Bootstrapping

Ring form	Let $A = \sum_i d_i g_i$ be an exact gadget decomposition. Evaluation-key components K_i have phase $g_i S + \varepsilon_i$ under S'. Then $c' = (0, B) + \sum_i d_i K_i$ has phase $B + AS + \sum_i d_i \varepsilon_i$ under S'.
Conditions	TFHE uses the corresponding vector formula. Approximate decomposition adds a residual term. BGV evaluation-key errors must be multiples of t. A scalar added to a ciphertext means addition to its body component only.

Table 23: Key Switching

ModDrop	For $q' \mid q$, reduce components modulo q' without scaling. CKKS scale metadata stays unchanged; preserving the represented value requires the remaining phase to fit. BGV plaintext is preserved if $\|M + tE\|_\infty < q'/2$; arbitrary wraparound may change it modulo t.
BGV switch	In the convention $q \equiv q' \pmod t$, choose each $c' \equiv c \pmod t$ close to $(q'/q)c$. Scaling changes absolute noise and modulus together, and adds rounding error; it does not sample a fresh noise distribution.

Table 24: Modulus Drop and Modulus Switching

TFHE	Even a scalar requires an encoding such as $m \mapsto \Delta m$ and a decoding/noise margin. LUT padding and the valid input interval must be specified.
BFV / BGV	Batch encoding is a polynomial CRT map over the plaintext ring, under the required factorization assumptions. It is optional when working directly with polynomial messages.
CKKS	Canonical embedding and its inverse relate conjugate-symmetric complex evaluations to real coefficients; scaling and integer rounding then produce a finite encoding.

Table 25: Encoding and Decoding the Plaintext

Ring schemes	Apply the automorphism $X \mapsto X^j$, then key-switch from $S(X^j)$ to $S(X)$. With the usual ordering, $j = 5^h \bmod 2n$ rotates CKKS complex slots or each of the two BFV/BGV rows; it is not one cycle through all n scalar slots.
TFHE	The scalar LWE interface has no SIMD slot rotation. Packed GLWE constructions have other polynomial operations and should be distinguished from this interface.

Table 26: Input Slot Rotation

TFHE / BFV / BGV	Obtain an encryption of the same message (or a specified LUT output) with a usable new noise budget. The output still contains noise and may use another key or parameter set.
CKKS	Refresh usable levels while controlling additional approximation error. Previously accumulated error in the represented value is not generally removed.

Table 27: Bootstrapping Goal

TFHE	Modulus-switch to $2n$. With the plus-phase convention used here, the effective rotation error is $\hat{e} = (2n/q)e + \delta_b + \sum_i s_i \delta_{a_i}$. Initialize a GLWE accumulator encrypting $\Delta_{\text{out}} V$, where V is the unscaled LUT polynomial. Rotate first by $X^{-\hat{b}}$, then use encrypted secret bits to apply $X^{-\hat{a}_i s_i}$. Extract the constant term, which encodes $f(m)$ for the chosen LUT. The extraction key is derived from the GLWE secret; key-switch if needed.
TFHE conditions	Define the unscaled lookup by $F(j) = \text{coeff}_0(VX^{-j})$. It must be constant over the total error interval and respect negacyclic anti-periodicity $F(j+n) = -F(j)$. Arbitrary functions require a suitable restricted/padded input domain or an extended construction.
BFV	For plaintext modulus p^r, transform decryption into rounding modulo p^e: $M = [\lfloor [c'_0 + c'_1 S]_{p^e} / p^{e-r} \rceil]_{p^r}$, under the modulus-switch error bound. Homomorphically evaluate the inner product using an encrypted secret; the constant body must be encoded at the new scale too. CoeffToSlot, digit extraction with exact divisions, and SlotToCoeff implement the remaining map. For odd p use balanced digits; for $p = 2$ include the half-unit rounding offset before deleting low bits. The CRT slot route requires $\gcd(p, 2n) = 1$; a binary bootstrap therefore needs a different compatible cyclotomic ring or construction. Their parameters and extension-slot layout are essential; see [15].

Table 28: Bootstrapping Details: TFHE, BFV

CKKS	Modulus raising exposes a lifted phase $\Delta M + E + Kq_0$. CoeffToSlot is a homomorphic linear transform. EvalMod is a nonlinear approximation to modular reduction on a specified bounded input set. SlotToCoeff returns the result to coefficient form; all transforms consume budget and add error.
CKKS error	The illustrative map $g(x) = q_0 \sin(2\pi x/q_0)/(2\pi)$ satisfies $g(r + Kq_0) = r + \varepsilon_{\sin}$, with $\lvert\varepsilon_{\sin}\rvert \leq (2\pi)^2 \lvert r\rvert^3/(6q_0^2)$. It is not an exact remainder operation; polynomial approximation and homomorphic evaluation add further errors.
BGV	Transform the decryption relation, evaluate the secret-key inner product homomorphically, use CoeffToSlot and digit extraction to remove the unwanted digits, then apply SlotToCoeff. The CRT coprimality condition, odd-prime balanced digits, and binary rounding caveat are as in the BFV row. Exact division by p changes plaintext modulus $p^j \to p^{j-1}$ and requires a divisibility invariant. Merely viewing a small-modulus ciphertext at a larger modulus does not perform this computation; see [15].

Table 29: Bootstrapping Details: CKKS, BGV

TFHE / BFV / BGV	Track a bound on the complete phase error and the probability of exceeding the decoding margin. Bootstrapping creates a new bounded error; it does not set error to zero.
CKKS	Track scale, magnitude, and slot error at each level. Increasing scale can reduce relative rounding error but consumes modulus capacity; it is not a cost-free or universal cure. Error need not increase monotonically in every circuit.
BGV	Modulus switching reduces absolute phase magnitude while lowering the modulus and adding correction error. It is a noise-management step, not an independent refresh operation.

Table 30: Noise and Precision Management

D-7 Python Demo FHE Library

The Python files in `source code/` are educational demonstrations. The maintained command-line examples are `tfhe_example.py`, `bfv_example.py`, `ckks_example.py`, and `bgv_example.py`; the README lists their supported operations.

The upstream source is available at https://github.com/fhetextbook/fhetextbook.github.io, and the reviewed bilingual fork at https://github.com/wang2000star/fhe-textbook-cn.

The implemented operations, simulations, and missing functions must be distinguished. TFHE's `tfhe_mock_pbs` reads a secret key, decrypts, applies a lookup table, and re-encrypts. Its "bootstrapping key" dictionary contains that secret key. It is a client-side functional simulation, not a public evaluation key or a secure server-side implementation of programmable/circuit bootstrapping. The default ciphertext multiplication example instead demonstrates a tensor-product and key-switching construction and does not benchmark standard TFHE bootstrapping. BFV, CKKS, and BGV bootstrapping functions are unimplemented and explicitly raise `NotImplementedError`.

Small toy dimensions, bounded sample errors, and Python/NumPy pseudorandom sampling are used to illustrate algebra. They are not a validated cryptographic parameter set or a production random-number generator. Successful example runs verify only those chosen inputs and do not establish a security level, arbitrary-depth correctness, or the claimed performance of an optimized FHE library. The older standalone encoding and RNS scripts are algebra experiments; see the academic review for their coverage and limitations.

References

[1] Cornell University. Fields and cyclotomic polynomials, 2024. https://e.math.cornell.edu/people/belk/numbertheory/CyclotomicPolynomials.pdf.

[2] Wikipedia. Polynomial ring, 2024. https://en.wikipedia.org/wiki/Polynomial_ring.

[3] Al Doerr, Ken Levasseur. Polynomial rings, 2024. https://math.libretexts.org/Bookshelves/Combinatorics_and_Discrete_Mathematics/Applied_Discrete_Structures_(Doerr_and_Levasseur)/16%3A_An_Introduction_to_Rings_and_Fields/16.03%3A_Polynomial_Rings.

[4] Mei Li, Curtis Clement et al. Chinese remainder theorem, 2024. https://brilliant.org/wiki/chinese-remainder-theorem/.

[5] David Maier. Polynomials and the fast fourier transform (fft), 2011. http://web.cecs.pdx.edu/~maier/cs584/Lectures/lect07b-11-MG.pdf.

[6] StephenHarrigan. Lattice-basedcryptographyandthelearningwith errorsproblem, 2017. https://mysite.science.uottawa.ca/mnevins/papers/StephenHarrigan2017LWE.pdf.

[7] Ilaria Chillotti. Tfhe deep dive - part i - ciphertext types, 2022. https://www.zama.ai/post/tfhe-deep-dive-part-1.

[8] Ilaria Chillotti. Tfhe deep dive - part ii - encodings and linear leveled operation, 2022. https://www.zama.ai/post/tfhe-deep-dive-part-2.

[9] Ilaria Chillotti. Tfhe deep dive - part iii - key switching and leveled multiplications, 2022. https://www.zama.ai/post/tfhe-deep-dive-part-3.

[10] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. TFHE: Fast fully homomorphic encryption over the torus. Cryptology ePrint Archive, Paper 2018/421, 2018. https://eprint.iacr.org/2018/421.

[11] Ilaria Chillotti. Tfhe deep dive - part iv - programmable bootstrapping, 2022. https://www.zama.ai/post/tfhe-deep-dive-part-4.

[12] Marc Joye. Guide to fully homomorphic encryption over the [discretized] torus. Cryptology ePrint Archive, Paper 2021/1402, 2021. https://eprint.iacr.org/2021/1402.

[13] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144, 2012. https://eprint.iacr.org/2012/144.

[14] Inferati Inc. Introduction to the BFV encryption scheme, 2021. https://www.inferati.com/blog/fhe-schemes-bfv.

[15] Robin Geelen and Frederik Vercauteren. Bootstrapping for BGV and BFV revisited. Cryptology ePrint Archive, Paper 2022/1363, 2022. https://eprint.iacr.org/2022/1363.

[16] Shai Halevi and Victor Shoup. Bootstrapping for helib. *J. Cryptol.*, 34(1), January 2021. https://doi.org/10.1007/s00145-020-09368-7.

[17] Mingjia Huo, Kewen Wu, and Qi Ye. A note on lower digits extraction polynomial for bootstrapping, 2019. https://arxiv.org/abs/1906.02867.

[18] Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. Cryptology ePrint Archive, Paper 2024/1587, 2024. https://eprint.iacr.org/2024/1587.

[19] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. Cryptology ePrint Archive, Paper 2016/421, 2016. Published at ASIACRYPT 2017. https://eprint.iacr.org/2016/421.

[20] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 360–384, Cham, 2018. Springer International Publishing. https://eprint.iacr.org/2018/153.pdf.

[21] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. Cryptology ePrint Archive, Paper 2020/552, 2020. https://eprint.iacr.org/2020/552.

[22] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. Cryptology ePrint Archive, Paper 2022/1167, 2022. https://eprint.iacr.org/2022/1167.

[23] Shai Halevi and Victor Shoup. Design and implementation of HElib: a homomorphic encryption library. Cryptology ePrint Archive, Paper 2020/1481, 2020. https://eprint.iacr.org/2020/1481.

[24] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. Cryptology ePrint Archive, Paper 2011/277, 2011. https://eprint.iacr.org/2011/277.

[25] Craig Gentry, Shai Halevi, and Nigel P. Smart. Homomorphic evaluation of the AES circuit. Cryptology ePrint Archive, Paper 2012/099, 2012. https://eprint.iacr.org/2012/099.

[26] Jean-Claude Bajard, Julien Eynard, Anwar Hasan, and Vincent Zucca. A full RNS variant of FV like somewhat homomorphic encryption schemes. Cryptology ePrint Archive, Paper 2016/510, 2016. https://eprint.iacr.org/2016/510.

[27] Robin Geelen, Michiel Van Beirendonck, Hilder V. L. Pereira, Brian Huffman, Tynan McAuley, Ben Selfridge, Daniel Wagner, Georgios Dimou, Ingrid Verbauwhede, Frederik Vercauteren, and David W. Archer. BASALISC: Programmable hardware accelerator for BGV fully homomorphic encryption. Cryptology ePrint Archive, Paper 2022/657, 2022. https://eprint.iacr.org/2022/657.

[28] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full rns variant of approximate homomorphic encryption. In *Selected Areas in Cryptography – SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers*, page 347–368, Berlin, Heidelberg, 2018. Springer-Verlag. https://eprint.iacr.org/2018/931.pdf.

[29] Shai Halevi, Yuriy Polyakov, and Victor Shoup. An improved RNS variant of the BFV homomorphic encryption scheme. Cryptology ePrint Archive, Paper 2018/117, 2018. Published at CT-RSA 2019. https://eprint.iacr.org/2018/117.